

Journal für die reine und angewandte Mathematik, **134**, 198–287.

Part III

Advanced strategies

Chapter 8

Elementary Coupling Strategies

I would rather have questions that can't be answered than answers that can't be questioned.

R. Feynman

The simplest multi-scale or multi-physics coupling strategy is the linear propagation of information from one code to another. In some sense, one code produces the conditions that another code uses to continue the calculation. These couplings are often unidirectional; one code informs another, but there is no back propagation of information. So long as the response from one code to the other is linear, we can refrain from complex code coupling strategies, as described in the next chapter (Chapter 9). Linear coupling strategies lead to new AMUSE applications without increasing your script's complexity, while increasing its applicability.

8.1 Multi-code coupling problems

In this chapter we discuss how one code can produce self-consistent input or boundary conditions for another code without using some intermediate state, such as a file. We encountered such coupled problems in Section 4.5.1, where a gravitational dynamical simulation required interrupts in order to resolve collisions, in Section 5.2.10 for treating stellar explosions, and in Section 6.3.5 where we discussed how a hydrodynamics solver can be stopped to allow a special treatment of high-density clumps to be addressed non-hydrodynamically.

In these examples the physical solvers were not coupled, but instead we adopted a simple linear information propagation strategy ($A \rightarrow B$), in which one code (A) generated the initial conditions for another (B), but there was no back coupling from the other code (B) to the first (A).

In this chapter we focus on weakly coupled problems in which one code generates the data for another code to proceed, after which a third code (or the first one) continues or is (re)started. Towards the end of this chapter, in Section 8.7.2, we will discuss two different strategies of multi-scale and multi-physics coupling. These couplings are relatively simple solutions to the very complex but weakly coupled problem of multi-scale and multi-physics simulations. The numerics becomes considerably more complex

once a code responds non-linearly to small perturbations induced by another code. This more advanced topic of code coupling is discussed in Chapter 9.

A word of caution is in order here. There is probably no unique way to combine numerical simulations of different physical domains. Although many researchers claim self-consistency in their numerical solutions, this rarely is actually the case. The AMUSE approach described here lends itself well to problems in which the physics is weakly coupled, but even in those cases validation is a dark art rather than a well-documented science.

8.2 Advanced data handling

8.2.1 Dedicated channels and copy operations

Key to coupling codes is the ability to transfer data from one code to another, taking the proper unit conversions into account. In AMUSE we achieve this objective by the use of channels. Channels are one of the cornerstones of AMUSE, allowing one code to modify the data on which another code operates, as we now discuss.

Channels are used to communicate data between modules and the driving Python script. We encountered channels previously in Section 2.2.3, where we learned how to copy data from an N -body code to the AMUSE framework, and vice versa. Here we extend that discussion. Recall that channels are dedicated MPI (or sockets) communication links that do not need to be reestablished on every use. While it is possible to access code data directly from the running module (e.g. as `module.particles.mass`), we strongly encourage the use of channels, because they are faster, more efficient, more transparent, and result in cleaner code compared to directly accessing the module's particle set. Perhaps most important, they are independent of specific hardware and software implementations of interprocess communication. For monophysics solvers, channels may seem like overkill, but with multiphysics solvers they are extremely useful, if not essential.

To run stellar evolution and gravity concurrently we require three channels: one to copy data from the gravity solver to the local Python framework, one to copy data from the stellar evolution code to the framework, and one to copy data from the framework to the gravity code. This can be done as follows:

```
channel_from_gravity = gravity.particles.new_channel_to(bodies)
channel_from_stellar = stellar.particles.new_channel_to(bodies)
channel_to_gravity = bodies.new_channel_to(gravity.particles)
```

The local particles (called `bodies` in this example) can contain more information than either of the running codes provide. The first and second channels are used to copy data from the specific modules to the framework, whereas the third channel allows us to write to the gravitational dynamics module (see Section 2.2.3). In this case we do not require a channel from the framework to the stellar evolution code because the gravitational dynamics does not affect the stellar evolution. That would change if we were to incorporate collisions, in which case the mass of a merger product, and therefore the evolution of that star, would be affected by the merger process.


```

31 masses = new_salpeter_mass_distribution(number_of_stars, mass_min, mass_max)
32 mass_total_init = masses.sum()
33 converter = nbody_system.nbody_to_si(mass_total_init, radius_virial)
34 bodies = new_king_model(number_of_stars, W0, convert_nbody=converter)
35 bodies.mass = masses
36 bodies.scale_to_standard(convert_nbody=converter)
37
38 stellar = Sse()
39 stellar.parameters.metallicity = metallicity
40 stellar.particles.add_particle(bodies)
41 bodies.radius = stellar.particles.radius
42
43 gravity = Ph4(converter)
44 gravity.parameters.timestep_parameter = 0.01
45 gravity.particles.add_particles(bodies)
46
47 channel_from_stellar_to_gravity = stellar.particles.new_channel_to(
48     gravity.particles
49 )
50 channel_from_gravity_to_framework = gravity.particles.new_channel_to(bodies)
51

```

Code example 8.1: Initializing a simple solver combining stellar evolution with gravity.

Code example 8.1 presents an initialization sequence for combining stellar evolution with gravitational dynamics, demonstrating that communication through the Python script is often not necessary, as codes can communicate directly with each other through channels. We can arrange for the stellar evolution code to communicate directly with the gravity solver by declaring a direct channel between the two modules and copying the relevant information:

```

channel_from_stellar_to_gravity = stellar.particles.new_channel_to(
    gravity.particles
)

```

The data stream still passes through the AMUSE framework, to ensure proper unit conversion, but in this case the local `bodies` particle set is unaffected. This is a bit tricky, as now the copies of the data in the various running codes are not synchronized with the data in the local Python script.

After the particle set is constructed, the codes initialized, and the channels created, these channels can now be used to communicate between the running modules and the framework. For example, the gravity solver affects stellar positions and velocities, whereas the stellar evolution module affects stellar masses. After taking steps in `stellar`, the new stellar masses must be copied to the `gravity` module before the next gravity step occurs.

Code example 8.2 presents an example of the sequence of calling the codes and the associated copy commands. Here we assume that the channels `channel_stellar_to_gravity` and `channel_gravity_to_framework` are constructed to copy data from `stellar` to `gravity` and back from `gravity` to the framework. Between the calls to `gravity.evolve_model` and `stellar.evolve_model` there is no need to synchronize the data,

because the gravity module does not affect the parameters used by the stellar evolution code.

```

54 energy_total_init = gravity.kinetic_energy + gravity.potential_energy
55 energy_difference_gr = 0 | energy_total_init.unit
56
57 time = 0.0 | time_end.unit
58 while time < time_end:
59     time_step = min(stellar.particles.time_step.amin(), time_end - time)
60
61     stellar.evolve_model(time + time_step / 2)
62     channel_from_stellar_to_gravity.copy()
63
64     energy_total_gr = gravity.kinetic_energy + gravity.potential_energy
65     gravity.evolve_model(time + time_step)
66     energy_difference_gr += (
67         gravity.kinetic_energy + gravity.potential_energy - energy_total_gr
68     )
69
70     stellar.evolve_model(time + time_step)
71     channel_from_stellar_to_gravity.copy()
72     channel_from_gravity_to_framework.copy()
73
74     time += time_step
75     write_set_to_file(bodies, filename, append_to_file=True)
76

```

Code example 8.2: Event loop combining stellar evolution with gravity.

In Code example 8.2 we adopted an interlaced time-stepping strategy, which is a slight improvement over the linear temporal interstepping discussed so far. The more elaborate event-driven approach will be discussed in Section 8.4.1, and illustrated in Code example 8.7.

At the end of the snippet shown here, the `bodies` in the script are up to date and can be used to write a snapshot for later analysis, as we do in the last line of the script, where the `bodies` array is saved to a file called `filename`. Note that the order in which channels are called is important for those parameters that share the same name but may conceivably have different meanings in different codes. For example, both codes here use the particle attribute `mass`, which means essentially the same thing in each code. However, the attribute `radius` can and does mean different things in different modules, requiring some care when managing data transfer (see, e.g. Section 8.7.1).

8.2.1.1 Non-symmetric cross-synchronization of specific attributes

A certain number of attribute names have specific purposes for different modules in AMUSE. The `radius`, used in particle sets, for example, is used in an N -body code as the collision radius that triggers the associated stopping conditions (see Section 4.5.1). In the stellar evolution modules the same attribute refers to the photospheric radius of the star. This is confusing, and at times may even cause the script to behave inappropriately.

To prevent such name confusion, we would like to be able to copy a specific attribute from a running code to another local attribute name in the script. This can be realized through channels, in case the receiver's name does not match that of the sender.

The physics modules tend to use generic terms such as `mass`, `radius`, `position`, etc. In a complicated production script, one may wish to copy the `mass` from the particle set of a stellar evolution code, `stellar.particles`, to a parameter with another name. This might happen, for example, when the gravity solver treats the combined mass of a star and a protoplanetary disk as a single particle, whereas the stellar evolution code uses only the stellar mass to calculate the evolution of the star. In the local particle set `bodies`, the attributes might then be called `bodies.mass_of_star_and_disk` for the total mass of the star and the disk, and `bodies.radius_of_star_and_disk` for the equivalent radius, whereas the mass of the star alone could be called `bodies.stellar_mass`. In the main script we declare the parameter `bodies.mass_star_and_disk = bodies.stellar_mass + bodies.disk_mass`.

We can tune the various channels to copy the right parameters. In this case we can, for example, say

```
channel_to_stellar = bodies.new_channel_to(
    stellar.particles,
    attributes=["radius"],
    target_names=["radius"]
)
channel_to_gravity = bodies.new_channel_to(
    gravity.particles,
    attributes=["mass_of_star_and_disk",
               "radius_of_star_and_disk"],
    target_names=["mass", "radius"]
)
```

When the copy function is called, only the specific named attributes are copied. Similar channels can be declared for the reverse operations, such as copying the mass from the stellar evolution module back to the local particle set

```
channel_from_stellar = stellar.particles.new_channel_to(
    bodies,
    attributes=["mass"],
    target_names=["stellar_mass"]
)
```

In the main event loop we would write

```
channel_to_stellar.copy()
channel_to_gravity.copy()
```

to copy just the masses from the local particle set into the stellar evolution code, and the masses and radii from the local set into the gravity code. Similarly, we would use

```
channel_from_stellar.copy()
```

after the stellar evolution step to copy the updated stellar masses back into local memory. We present an example of how this works in practice in Section [8.4.4](#).

8.2.2 Particle subsets and supersets

Sometimes it is desirable to perform certain tasks on a subset of the particles in a set. This can be accomplished by generating new particle sets, but managing multiple related particle sets can unnecessarily complicate the bookkeeping, increase the chance of mistakes, and reduce readability. We can achieve the same result using channels.

We demonstrate this approach by simulating a system of stars with planetesimal disks around them. All particles should be evolved using the gravitational dynamics module, but only the stars should be affected by stellar evolution. To do this, we define two particle sets, called `planetesimals` and `stars`. Both sets contain masses, positions, and velocities, but the `stars` set also includes stellar data, such as radius, luminosity and temperature.

We send `stars` to the stellar evolution code:

```
stellar.particles.add_particle(stars)
```

after which we can call `evolve_model` to evolve them. But we send both `stars` and `planetesimals` to the gravity module:

```
gravity.particles.add_particle(stars)
gravity.particles.add_particle(planetesimals)
```

We subsequently declare the appropriate channels to copy data from the physics modules to the framework:

```
channel_from_stellar_to_stars = stellar.particles.new_channel_to(stars)
channel_from_gravity_to_stars = gravity.particles.new_channel_to(stars)
channel_from_gravity_to_planetesimals \
    = gravity.particles.new_channel_to(planetesimals)
```

and a similar set of channels from the framework to the physics modules.

In many cases, however, it may be easier to declare a *superset* of particles

```
gravitating_bodies = ParticlesSuperset([planetesimals, stars])
```

After this we can treat `gravitating_bodies` as the particles affected by gravity, keeping the `planetesimals` separate from the `stars` for the stellar evolution. Sending the superset to the gravity module can be accomplished with

```
gravity.particles.add_particles(gravitating_bodies)
```

Only the stars are sent to the stellar evolution module.

In order to guarantee consistency among the various parameters in the particle subsets, we create channels to and from the gravity module, and one channel from the stellar evolution code:

```
channel_to_gravity = gravitating_bodies.new_channel_to(gravity.particles)
channel_from_gravity = gravity.particles.new_channel_to(gravitating_bodies)
channel_from_stellar = stellar.particles.new_channel_to(stars)
```

Now we can easily copy particle attributes after the gravity or stellar evolution steps, e.g.

```
channel_from_stellar.copy_attributes(["mass", "radius"])
```

We can select subset of particles within a particle set by using standard Python tools. For example, suppose we have a superset of stars and planetesimals, as in `gravitating_bodies` just discussed. Filtering out the planetesimals can be realized by selecting on their masses—since the planetesimals have low masses we can write

```
mass_min = 1 | units.MEarth
low_mass = gravitating_bodies[gravitating_bodies.mass < mass_min]
```

to select objects with masses less than 1 Earth mass. In principle, we could use the more arcane but versatile Python `lambda` function. Regrettably, however, as of 27 November 2023 Lambda functions are no longer supported in Python in tandem with the AMUSE `select` operation:

8.2.3 Interrupting codes and stopping conditions

Sometimes it is necessary to have a community code stop and hand control back to the framework, leaving subsequent decisions to the user. As discussed in Section 4.5.1, Section 5.2.10, and Section 6.3.5 (interestingly enough the radiative-transfer module has no stopping conditions) we call these events *stopping conditions*. In Appendix A.5.2 we present a more complete overview of the various stopping conditions. There are many reasons why a community code should stop and return control to the user script, and the circumstances may be quite different for each physical domain. Stopping conditions are intrusive to the community code, and therefore are inconsistent with the AMUSE philosophy of “keep community codes pristine.” Nevertheless, we have taken the step of implementing stopping conditions in most AMUSE community codes. The details vary from one code to another. Barnes–Hut tree codes, for example, are generally not built to detect physical collisions between stars, while in regularized N -body codes, a “collision” may have an entirely different meaning. In all cases, checks for physical (or other) collisions can usually be incorporated into the interface code or at a high level in the community code without entailing sweeping changes in the source.

In an AMUSE script, one can initiate stopping conditions after initializing the worker code. In a gravity module this is realized by

```
detect_collision = gravity.stopping_conditions.collision_detection
detect_collision.enable()
```

Many, but not all, AMUSE N -body codes have collision detection enabled. Those codes that do not will produce a warning message when the user code attempts to set a stopping condition, but subsequently they simply continue without the ability to stop, offering no other indication that stopping is disabled.

In the time-evolution loop you can check for a stopping condition on return from `evolve_model()` and take appropriate action:

```

if detect_collision.is_set():
    for cp in range(len(detect_collision.particles(0))):
        particles_in_code = Particles(
            particles=[detect_collision.particles(0)[cp],
                       detect_collision.particles(1)[cp]])
        local_colliding_particles = \
            particles_in_code.get_intersecting_subset_in(bodies)
        merge_two_particles(gravity, local_colliding_particles)
        gravity.particles.synchronize_to(bodies)

```

The N -body integrator checks for the exception after each internal step and returns to the calling function when the event (in this case, a collision) is detected. The N -body system is synchronized before returning to the framework to allow the calling function to perform further tasks on the data sets without corrupting the integrity of the N -body data.

The flag `detect_collision.is_set()` returns `True` if the stopping condition is met, after which the code for resolving stopping conditions is invoked. Inside this code, the bodies that caused the community code to stop are returned to the Python framework and the user script must deal with the exception. In this example, the gravity solver stops when two bodies approach within a distance smaller than the sum of their radii. The resolution of the exception in this case is to delete the colliding particles and insert a new particle at their center of mass. The following routine accomplishes this as follows:

```

def merge_two_particles(gravity, particles_in_encounter):
    new_particle = Particles(1)
    new_particle.mass = particles_in_encounter.total_mass()
    new_particle.position = particles_in_encounter.center_of_mass()
    new_particle.velocity = particles_in_encounter.center_of_mass_velocity()
    new_particle.radius = particles_in_encounter.radius.sum()
    gravity.particles.add_particles(new_particle)
    gravity.particles.remove_particles(particles_in_encounter)

```

In this routine, a new particle is created and the center-of-mass data of the merging particles are copied to the new particle and added to the `gravity` instantiation. The two merging stars are not needed anymore, so they are removed from `gravity`. Best practice probably requires that we also flag the event, writing the merging particles to a file before removing them from the system (not shown here). Removal is achieved by explicitly removing the particles from the local `bodies`, adding the center of mass, and synchronizing `gravity.particles` with the local data:

```
gravity.particles.synchronize_to(bodies)
```

Of course, for all this to happen, one must set the radii of the particles in advance. For example, you could write

```
bodies.radius = 0.001 | nbody_system.length
```

just after the particles have been initialized, but before they are sent to the community

module. When working with physical units, that line might look as follows

```
bodies.radius = 10 | units.RSun
```

8.2.4 Collision handling

We decide when the point-mass approximation fails based on the sizes of two stars (as determined by the stellar evolution code) and the distance between them (from the N -body code). A collisional stopping condition is triggered when the distance between two stars becomes smaller than the sum of their radii.

Because a stopping condition is an exception, it must be checked for and addressed as soon as possible after it is raised (see Appendix A.5.2). In the case of a collision, once the gravity module reports the exception, we should next evolve the stars to the moment the collision was detected. It is possible (although not common) that the collision may be avoided by a drastic change in stellar radius due to stellar evolution. For example, the gravity module may decide that two giant stars have collided, but when stellar evolution is applied these stars may turn into (non-colliding) compact objects before the collision is resolved. In such a case, the collision should probably not occur, and the dynamical integration can simply continue.

Synchronization between the modules (here gravity and stellar evolution) is very important. Recall that there are usually two copies of the data for each module—one in local framework (Python) memory, the other in the module itself. The module data is up to date at the end of a step, and we must synchronize the modules with the local framework before resolving the stopping condition. To manage a collision between two stars, the framework needs to know the stellar ages, masses, radii, luminosities, and temperatures. Some of this information comes from the gravity module, the rest from the stellar evolution module.

Note that we must keep track of two different “radii”—the natural stellar radius in the stellar evolution module, and the collision radius used by the gravity code to identify close dynamical encounters (see Section 8.7.1). Probably the safest way to manage this is by initializing two separate local variables in the framework particle set. Selective channels can be declared to ensure proper communication between the modules and the framework script:

```
channel_from_stellar = stellar.particles.new_channel_to(bodies,
    attributes=["mass", "radius", "age"],
    target_names=["mass", "radius", "age"])
channel_from_gravity = gravity.particles.new_channel_to(bodies,
    attributes=["x", "y", "z", "vx", "vy", "vz", "mass", "radius"],
    target_names=["x", "y", "z", "vx", "vy", "vz", "mass", "collision_radius"])
channel_to_gravity = bodies.new_channel_to(gravity.particles,
    attributes=["mass", "collision_radius"], target_names=["mass", "radius"])
```

In `channel_from_gravity` we explicitly copy all the positions and velocities to ensure that the particle set in the framework is consistent with the gravity module. Later in the event loop (see Code example 8.3) we will simply call the copy operator for the various channels. We copy data from the stellar evolution to the framework with

```
channel_from_stellar.copy()
```

and copy data from the N -body code to the framework with

```
channel_from_gravity.copy()
```

These lines ensure that the masses and radii used in the gravitational module are consistent with those in the stellar evolution module and in the framework.

Stars are small objects, and collide infrequently. In order to make more stars collide (for debugging or illustrative purposes) we can increase the stellar radii used by the gravitational dynamics, multiplying the radii provided by the stellar evolution code by some factor. This has to be done after the stellar evolution information is copied back into the local particle set. The magnified particle-set data must then be copied into the gravity module's particle set to ensure that the dynamics is aware of the new stellar radii.

```
stellar.evolve_model(time)
channel_from_stellar.copy()
bodies.collision_radius = 1.0e5 * bodies.radius
channel_to_gravity.copy()
gravity.evolve_model(time)
```

This is a pretty dirty trick, but it will guarantee many interesting collisions!

```
82 while time < t_end:
83     time += dt
84
85     bodies.radius *= 1.e+5
86     channel_from_framework_to_gd.copy_attributes(["mass", "radius"])
87     E_dyn = gravity.kinetic_energy + gravity.potential_energy
88     gravity.evolve_model(time)
89     dE_dyn = E_dyn - (gravity.kinetic_energy + gravity.potential_energy)
90
91     Nenc += resolve_stellar_encounter(time, stellar, gravity, bodies,
92     stopping_condition)
93
94     E_stellar = gravity.kinetic_energy + gravity.potential_energy
95     stellar.evolve_model(time)
96     dE_stellar = E_stellar - (gravity.kinetic_energy + gravity.potential_energy)
97
98     channel_from_gd_to_framework.copy()
99     channel_from_se_to_framework.copy_attributes(["age", "mass", "radius"])
100
101     write_set_to_file(bodies.savepoint(time), filename, 'hdf5', append_to_file=
    True)
    print_diagnostics(time, bodies.mass.sum(), E_dyn, dE_dyn, dE_coll,
    dE_stellar)
```

Code example 8.3: Main code declaration and event loop for simulating a star cluster with collisions.

Code example 8.3 presents part of the main routine for simulating the dynamical evolution of a cluster of stars, including stopping conditions for collisions. The function

that resolves the collisions is presented in Code example 8.4. The same routine is also used in Section 8.5.1 where we discuss an event-driven simulation of a star cluster with collisions.

```

28 def resolve_stellar_encounter(time, stellar, gravity, bodies, stopping_condition):
29     Nenc = 0
30     if stopping_condition.is_set():
31         E_coll = gravity.kinetic_energy + gravity.potential_energy
32         print("At time=", gravity.model_time.in_(units.Myr), "number of encounters="
33             , len(stopping_condition.particles(0)))
34         for ci in range(len(stopping_condition.particles(0))):
35             particles_in_encounter = Particles(particles=[stopping_condition.
36                 particles(0)[ci], stopping_condition.particles(1)[ci]])
37             particles_in_encounter = particles_in_encounter.
38             get_intersecting_subset_in(bodies)
39
40             merge_two_stars(bodies, particles_in_encounter)
41             bodies.synchronize_to(gravity.particles)
42             bodies.synchronize_to(stellar.particles)
43             Nenc+=1
44             print("Resolve encounter Number:", Nenc)
45             gravity.evolve_model(time)
46             dE_coll = E_coll - (gravity.kinetic_energy + gravity.potential_energy)
47     return Nenc

```

Code example 8.4: Simple function to resolve stellar collisions.

How do we model the merger product? If we are using a parameterized stellar evolution code, the structure of the merger product cannot easily be determined. We can mimic the mixing of fresh hydrogen into the merger product's core by reducing the stellar age in some mass-dependent way, resulting in an effective rejuvenation of the star (see Section 5.3.12). Afterwards, we place the merged star on a new evolutionary track at the new age and continue the calculation. This can be realized simply by adding a few lines to the linear momentum- (and mass-) conserving routine `merge_two_particles()` discussed in Appendix A.5.2, which then becomes Code example 8.5. In the next subsection we discuss a more sophisticated approach to the problem.

```

13 def merge_two_stars(bodies, particles_in_encounter):
14     com_pos = particles_in_encounter.center_of_mass()
15     com_vel = particles_in_encounter.center_of_mass_velocity()
16     new_particle=Particles(1)
17     new_particle.mass = particles_in_encounter.total_mass()
18     new_particle.age = min(particles_in_encounter.age) * max(particles_in_encounter.
19         mass)/new_particle.mass
20     new_particle.position = com_pos
21     new_particle.velocity = com_vel
22     new_particle.radius = 0 | units.RSun
23     bodies.add_particles(new_particle)
24     bodies.remove_particles(particles_in_encounter)

```

Code example 8.5: Simple function to merge and rejuvenate two stars while conserving linear momentum.

8.3 Runtime analysis and task offloading

AMUSE contains many analysis tools designed to operate on simulation data. Here we describe two modules, `hop` and `Kepler`, that are commonly used in tandem with running simulation codes. There are other codes that are often associated with AMUSE, and their number is growing continuously, but this book can only present a snapshot in time of the development.

8.3.1 The Hop package

`Hop` ([Eisenstein & Hut, 1998](#)) is a module designed to identify and quantify structure in a distribution of points.¹ Although it was originally developed to identify clumps in a 3-dimensional spatial distribution of objects (stars in a cluster or galaxies in the universe), we can easily apply `hop` to any set of particle data. For example, to distinguish planetesimals in a protoplanetary disk from other debris, we might look for clumps in the space of orbital velocity, inclination, and distance from the central star. Figure 8.1 shows a small example of how `hop` can be used to characterize clumpy structure in a distribution of stars.

The code snippet in Code example 8.6 gives the basic calling sequence to run `hop`. Several free parameters guide `hop` in finding structure and substructure; they include `peak_density_threshold`, the lower limit for recognizing a peak, `saddle_density_threshold`, the phase-space density of a saddle point, and `outer_density_threshold`, the minimum density threshold for recognizing a clump. The result in this example is a list of particle sets corresponding to the clumps identified by `hop`. The algorithm is not particularly fast, but `hop` is reliable and works intuitively.

```

58 hop = Hop(unit_converter)
59 hop.particles.add_particles(particles)
60 hop.calculate_densities()
61
62 mean_density = hop.particles.density.mean()
63 hop.parameters.peak_density_threshold = mean_density
64 hop.parameters.saddle_density_threshold = 0.99 * mean_density
65 hop.parameters.outer_density_threshold = 0.01 * mean_density
66
67 hop.do_hop()
68 result = [x.get_intersecting_subset_in(particles) for x in hop.groups()]
69 hop.stop()
70

```

Code example 8.6: Using `hop` to identify substructure in a particle distribution.

¹The name derives from the algorithm used to divide the distribution into equivalence classes representing substructure by “hopping” from each particle to its neighbor with the highest local density.

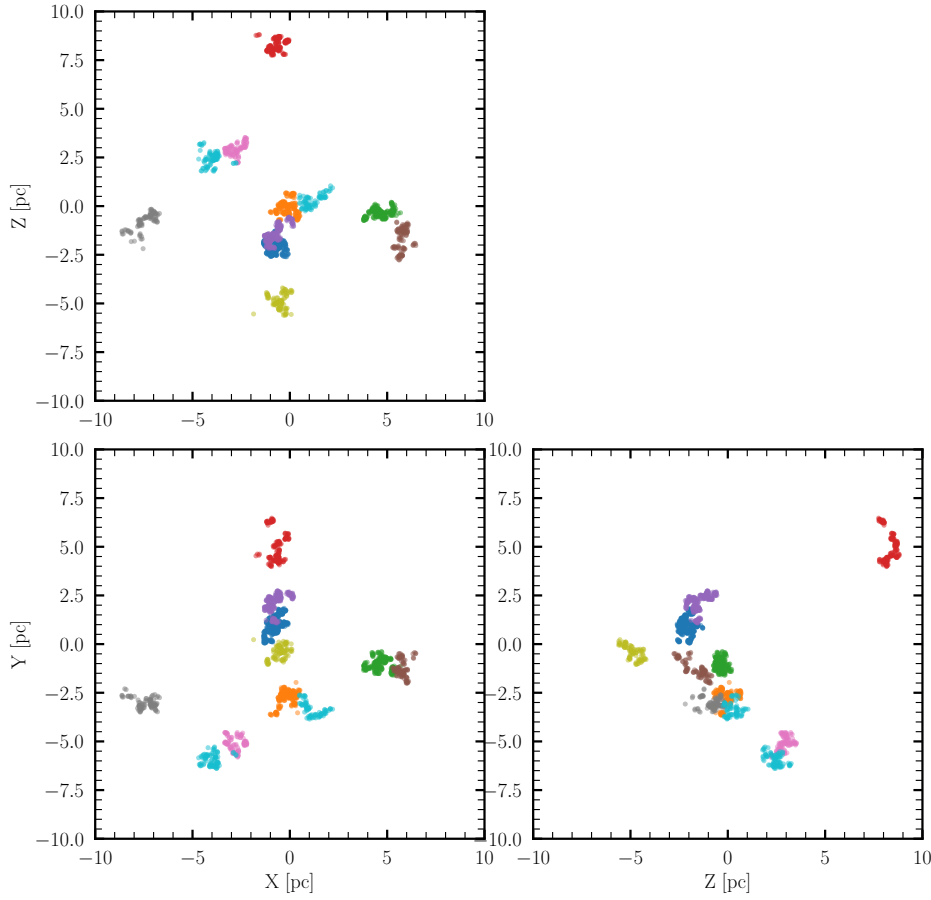


Figure 8.1: Cluster of 1000 stars initially distributed with fractal dimension 1.6 and a half-mass radius 0.5 pc. The 11 different clumps identified by `hop` are indicated by color. The figure was produced using the script `amuse.examples.plot_fractal_clumpy_cluster`

8.3.2 The Kepler package

Plotting positions and animating N -body systems is illustrative and often educational, but not always the most useful for astrophysical analysis. When studying the orbits of planets or multiple systems, the Keplerian orbital elements provide much more quantitative information. These are straightforward to compute from the masses, positions, and velocities of the orbiting bodies, but this is such a common operation that AMUSE contains built-in modules designed to calculate, manipulate, and use Keplerian orbits, and we can simply use one of those to do the work for us. One such module, a legacy of the older `starlab` package², is simply called `Kepler`.

We begin by initializing the `Kepler` package

```
johannes = Kepler(convert_nbody)
```

where `convert_nbody` is a converter (see Section 2.3). We also generate a particle set containing just the two particles for which we want the orbital elements. For example,

²See starlab.sns.ias.edu.

in the earlier example on solar-system orbits (Section 3.4.1 and Section 4.4.1), we could call this local particle set `sun_earth` and have its two particles point to the Sun (particle 0) and to Earth (particle 3).

```
sun_earth = Particles()
sun_earth.add_particle(particles[0])
sun_earth.add_particle(particles[3])
channel_from_to_sun_earth = gravity.particles.new_channel_to(sun_earth)
```

We also created a local channel to these two particles, which we can use in the event loop to synchronize the gravity code data with the local particles in `sun_earth`.

Within the loop we reinitialize `Kepler` with the two particles of interest—Sun and Earth. To print Earth’s semimajor axis, this looks like:

```
johannes.initialize_from_particles(sun_earth)
print(f"Earth's semimajor axis={johannes.get_elements()}")
```

Note that we reinitialize an existing `Kepler` in the event loop, rather than creating a new one each time. This is done for reasons of efficiency. We could simply create and delete a new `Kepler` code every time we require these services, but this can incur a considerable overhead in large runs.

Using the `Kepler` package only for computing simple orbital elements is probably overkill. In AMUSE the same result can be achieved by calling the routine `orbital_elements.orbital_elements_from_binary()`. This function takes two arguments, a particle set (here containing the two stars in a binary) and the gravitational constant `nbody_system.G`, and returns the six orbital elements (see Section 4.4.1). However, the `Kepler` package can also manipulate orbits—returning particle sets corresponding to a specified time, orbital separation, apocenter, or pericenter—making it a useful tool in many applications. For example, the `multiples` module (Section 8.7.1) uses `Kepler` to set up scattering experiments, perform two-body orbit integrations, classify scattering outcomes, and rescale particle data.

8.3.3 Recovering from a code crash

As discussed in Section 4.6, physics modules may crash if they are placed in regimes for which they were not designed. When a physics module in AMUSE crashes, it is undesirable for the framework also to crash. Using Python exception handling we can detect when a code has terminated and take appropriate action. We demonstrate this by using a Henyey code to model the evolution of a $10 M_{\odot}$ star until the code crashes. Another example, where we switch from one stellar evolution code to another, is presented in Section 8.4.3.

Stellar evolution codes tend to crash at the end of the nuclear fusion as the star approaches its death as a supernova explosion. (It may happen sooner, depending on the code!) Evolving a star to this late stage can be expensive because the stellar evolution code has a built-in safety valve: If an evolution step fails to converge, the code iteratively reduces the time step and restarts from the previous self-consistent stellar model. This process is repeated until the code has reached a built-in limiting

number of restarts, at which point it simply gives up.

```
try:
    while True:
        stellar.evolve_model()
except AmuseException as ex:
    print(f"Star evolved to t = {stellar.particles[0].age}")
```

Here we call `evolve_model()` without an argument, which evolves the star using the internal stellar evolution time step, rather than to a specified time. Once we reach the `print` statement the Henyey code has terminated, but the AMUSE interface has not. We can then obtain the last converged stellar evolution model, which is as close to an exploding star as the stellar evolution code can get. In Section 6.4.4 we showed how the post-evolution product can be converted into an SPH particle representation to continue the evolution through the supernova explosion.

Rather than using `MMAMS` it can be desirable to use a hydrodynamical solver to resolve the collision. A first step in this process would be to convert the 1-dimensional stellar evolution structure from a Henyey code to a realization suitable for continuing the calculation with a hydrodynamics solver. One such routine is `convert_stellar_model_to_SPH` from AMUSE's `ext.star_to_sph` package.

In Figure 8.2 we present the density structure for a $1 M_{\odot}$, a $3 M_{\odot}$ and a $10 M_{\odot}$ star, and the associated SPH realization of the same star using 10^4 SPH particles. The SPH realizations nicely follow the density profile calculated using the stellar evolution code for most of the stellar structure. Near the stellar surface the density of individual particles exceeds the results from the stellar evolution code. This is a result of the hydrodynamics solver placing the star in dynamic equilibrium, whereas the stellar evolution code also requires thermal equilibrium in conjunction with the internal energy source, which is absent in the hydro code. Eventually, near the stellar surface, the hydro code is unable to resolve the density structure in the outermost layers. This latter discrepancy is mainly a resolution issue, with only 10^4 SPH particles we cannot resolve densities below $\sim \rho_c / \sqrt{10^4}$.

8.4 Elementary code coupling strategies

A straightforward but very important way to couple codes is by directly propagating information from one code to another at the user script level. In this approach, two or more codes work on portions of the data (typically contained in a particle set), which may also be used by other codes. A classic example can be found in star cluster evolution. Stars move due to gravity and at the same time they evolve due to internal nuclear processes. Such a system can be simulated by a straightforward combination of the methods discussed in Chapters 4 and 5, as we now describe.

8.4.1 Combining gravity with stellar evolution

N -body codes have incorporated treatments of stellar evolution for almost four decades. As a consequence, we have by now a fairly good understanding of the numerical sub-

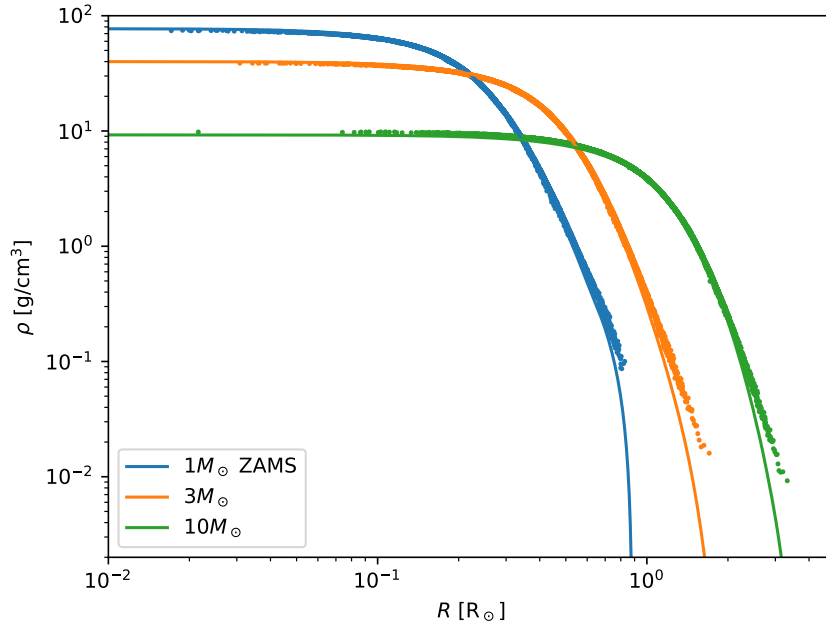


Figure 8.2: Density as a function of distance to the stellar center for three stars of $1M_{\odot}$, $3M_{\odot}$ and $10M_{\odot}$ each at an age of 1 yr. The solid lines show the results from the stellar evolution model (MESA in this case), the dots give the densities attributed to the SPH particles using the `convert_stellar_model_to_sph` routine. This figure was produced using the script `amuse.examples.single_star_to_sph`, which runs in about an hour. For convenience, the hydro data is also available in `amuse.examples.data.fig_star_in_sph_N10000M1.0MSun.npy` for the $1.0M_{\odot}$ star, and equivalently for the $3.0M_{\odot}$ and the $10.0M_{\odot}$ stars, in which case the plot can be generated using `amuse.examples.plot_single_star_to_sph`.

tleties in coupling these pieces of physics. The most common way of solving the combination of stellar evolution and stellar dynamics is the strategy adopted in the seminal work of Terlevich (1987), which updates and synchronizes the two codes at regular time intervals (see also Aarseth, 2003)—update the dynamics to some time, then update the stellar evolution to that time, then update the dynamics to the next time using the new stellar masses, etc. We call this *synchronized temporal discretization*.

In general, both codes will have their own internal time steps, which may be different for each object and may change dynamically with time, but in this prescription the time interval for the exchange of information is generally set by the dynamics—typically some fraction of the crossing time of the self-gravitating system. As discussed in Sections 4.1.2 and 5.1.1, this is appropriate in many cases, but it may not be so for systems in resonance, chaotic dynamics, or young massive star clusters, where stellar evolution time scales may actually be shorter than the relevant dynamical time.

The flexibility afforded by AMUSE allows us to experiment with other time-stepping strategies. One way to improve on the synchronized temporal discretization strategy is to advance the stellar evolution for half a time step, then advance the dynamics for a full step using stellar masses defined at the mid-point, and finally update the stellar evolution to the end of the step. We saw in Section 4.2.3 how evaluating key quantities in the middle of the step leads to improved accuracy. In this case, the stellar evolution

cares nothing for the dynamics, but using centered stellar masses improves the accuracy of the dynamical integration. We call this approach *interlaced temporal discretization*, and we explore its use in the next example. A more complete and elaborate approach to this general problem is presented in Chapter 9.

There are (at least) two complications in coupling stellar evolution with gravitational dynamics: (1) unexpected events in the stellar evolution, and (2) unexpected events in the gravitational dynamics. An example of an unexpected stellar event is a supernova explosion;³ an unexpected dynamical event is a collision between two stars. Neither event can easily be predicted, but they can be handled straightforwardly using interrupts, as discussed in Appendix A.5.2.

8.4.2 Coupling Radiative Transfer with Hydrodynamics

Combining radiative transport with hydrodynamics is not trivial. In principle, we could adopt a linear coupling strategy between two modules, but a bridge-like approach (see Chapter 9) gives superior results, in terms of both better performance and smaller errors. However, as we have not yet discussed bridge, we adopt a simpler but still effective approach similar to the interlaced temporal discretization discussed in Section 8.4.1.

We have defined a small `RadHydro` class to simplify this function, to be incorporated later into a bridge with (for example) a gravity solver. In order for this class to comply with bridge specifications, it must have a few extra functions: the most important is `evolve_model()`. In addition, to make life easier, we include a method to copy data from the two worker codes to the framework, and one to clean up after we are done. For safety (and convenience), we start by synchronizing all relevant data between the two worker codes and the framework, giving us a self-consistent data set to work with. For additional safety, albeit probably unnecessary in this example, we perform a similar synchronization after the two worker codes have reached their objectives.

Between the two synchronization operations, we run the hydrodynamical simulation for half a time step, followed by the radiative transfer simulation for a full time step, and end with a second half time step for the hydrodynamics (see also Section 8.4.1):

```
while t < (tend-timestep/2):
    self.hydro.evolve_model(t+timestep/2.)
    update_rad_from_hydro_channel.copy()
    self.rad.evolve_model(t+timestep)
    update_hydro_from_rad_channel.copy()
    self.hydro.evolve_model(t+timestep)
    t = self.model_time
```

The rest of the radiation–hydrodynamics class is fairly straightforward. In Section 8.6.1, we present an example calculation using this `RadHydro` class using the code example in `amuse.examples.supernova_irradiation`.

³In practice, since the stellar evolution codes actually operate one step ahead of the current time, to permit interpolation of their results on demand, a supernova is not completely unexpected. We ignore this operational reality here.

8.4.3 Switching from one stellar evolution code to another

Stellar evolution code, in particularly the Henyey codes, can be rather fragile. If such a code becomes impossibly slow, or simply crashes, one would not necessarily stop the entire calculation, but escape to a safer (secondary) stellar evolution code or prescription. Ideally, the new stellar evolution code should pick up the failed star from the crashed code and continue it’s evolution from that same point in its evolutionary stage.

This turns out to be non-trivial, as each stellar evolution code will give a different radius, luminosity and temperature for the same mass, age and metallicity. These differences are often small, but can be quite substantial, see for example Section 5.6.1.

To make the transition process simpler, we designed a special module for stellar evolution codes, allowing to start with one code, and switch to another at some arbitrary moment. The module is briefly described in Appendix A. The moment to switch codes can be arbitrary but the evolutionary state of where the new stellar-evolution code starts is chosen as close as possible to the end-point of the previous code.

The handover algorithm performs a root-mean-squares search on the relative differences of mass, radius and luminosity between the last point known state in the primary code, and the entire stellar history of the secondary code (here **Sse** or **SeBa**) until the star turns into a remnant.

In the example (below) we show how a star starts to evolve using **Evtwin**, for 1 Myr, and then we switch (for no particular reason) over to **SSE**. The interface function **EVtwin2SSE()**, is really a small class with two stellar evolution codes, and a collection of other parameters. One of these parameters is **EVtwinAgeAtSwitch**, indicating at what time to switch from one code to the other. Another parameter is **EVtwinException**, which is used if the code-switch is non-voluntary (i.e., the primary stellar evolution code crashed).

```
from amuse.lab import Particle
from amuse.lab import units
from amuse.community.evtwin2sse.interface import EVtwin2SSE

stellar = EVtwin2SSE()
stellar.EVtwinAgeAtSwitch = 1 |units.Myr
star.mass = 10|units.MSun
stellar.particles.add_particle(star)

print(stellar.particles)
stellar.evolve_model(10|units.Myr)
print(stellar.particles)
```

8.4.4 Event-driven simulations

So far our simulations of star clusters have alternately called the gravitational dynamical evolution and stellar evolution solvers at regular time intervals. This gives a somewhat limited perspective on reality—a massive star doesn’t wait until a dynamical encounter is over before blowing up, and a stellar collision doesn’t wait until the

end of a stellar evolution step. Instead, each may interrupt the otherwise orderly joint evolution process.

In conventional simulation languages, taking these interruptions into account at the proper times is difficult—the language semantics simply are not flexible enough to deal with these events. In AMUSE, an event-driven simulation can be easily realized with the proper use of timing, channels, declarations, and calls. In Code example 8.7 we create data structures and declare two codes, a stellar dynamics solver and a stellar evolution code, together with communication channels. In principle, there is nothing really new about these, except that both codes now have stopping conditions enabled.

```

70 masses = new_salpeter_mass_distribution(N, Mmin, Mmax)
71 Mtot_init = masses.sum()
72 converter=nbody_system.nbody_to_si(Mtot_init,Rvir)
73 bodies = new_king_model(N, W0,convert_nbody=converter)
74 bodies.mass = masses
75 bodies.scale_to_standard(convert_nbody=converter)
76
77 gravity = ph4(converter)
78 gravity.parameters.timestep_parameter = 0.01
79 gravity.particles.add_particles(bodies)
80 collision_detection = gravity.stopping_conditions.collision_detection
81 collision_detection.enable()
82
83 stellar = SeBa()
84 stellar.parameters.metallicity = z
85 stellar.particles.add_particle(bodies)
86 stellar.evolve_model(0|units.Myr)
87
88 supernova_detection = stellar.stopping_conditions.supernova_detection
89 supernova_detection.enable()
90
91 channel_from_se = stellar.particles.new_channel_to(bodies)
92 channel_from_gd = gravity.particles.new_channel_to(bodies)
93 channel_to_gd = bodies.new_channel_to(gravity.particles)
94 channel_from_se.copy_attributes(["mass", "radius", "age",
95                                "temperature", "luminosity"])

```

Code example 8.7: Declaration of the solvers and data structures used in Code example 8.8.

Code example 8.8 shows the event loop, which starts by querying the stellar evolution code about the shortest safe time step, the minimum interval during which no star will experience a supernova or significant changes to its `stellar_type` (see Appendix A). The collision handling for the event-driven simulation is handled by a separate function (see the example script Code example 8.4); the supernova handling is illustrated in Code example 8.9. Strictly speaking, this simulation is only event-driven in terms of the collision handling. The stellar evolution is predictable so long as there are no additional complications, for example, due to mass transfer in a close binary system.

We run the gravitational dynamics for that safe time frame. If a collision occurs during this time, the N -body code is interrupted and the encounter is resolved, ending

```

106 while time < t_end:
107
108     dt_min = min(dt, stellar.particles.time_step.min())
109     print("Time steps:", dt.in_(units.Myr), dt_min.in_(units.Myr))
110     time += dt_min
111
112     bodies.radius *= 1.e+5
113     channel_to_gd.copy_attributes(["mass", "radius", "vx", "vy", "vz"])
114     E_dyn = gravity.kinetic_energy + gravity.potential_energy
115     gravity.evolve_model(time)
116     dE_dyn = E_dyn - (gravity.kinetic_energy + gravity.potential_energy)
117
118     resolve_collision(collision_detection, gravity, stellar, bodies)
119     channel_from_gd.copy()
120
121     time = gravity.model_time
122
123     E_stellar = gravity.kinetic_energy + gravity.potential_energy
124     stellar.evolve_model(time)
125     dE_stellar = E_stellar - (gravity.kinetic_energy
126                             + gravity.potential_energy)
127
128     resolve_supernova(supernova_detection, bodies, time)
129     channel_from_se.copy_attributes(["mass", "radius", "age",
130                                   "temperature", "luminosity"])
131
132     write_set_to_file(bodies.savepoint(time), filename, 'hdf5')
133     print_diagnostics(time, bodies.mass.sum(), E_dyn,
134                      dE_dyn, dE_coll, dE_stellar)

```

Code example 8.8: Event-driven simulation with collisions and supernovae.

the dynamics step. In Code example 8.8 we ensure that the stars are evolved to the same time as the N -body integrator. For technical reasons, individual time step codes do not normally guarantee to evolve the system to exactly the time given in the call to the community code. This can be overruled by forcing the N -body code to synchronize to the target time with

```
gravity.parameters.force_sync = True
```

We normally would not recommend synchronizing an N -body code in this way, but it is often necessary in multiscale and multiphysics problems. After the dynamics step, the stellar evolution is updated to the current age of the system, which is the moment of a collision, the moment of a supernova, or some predetermined output time. In Section 10.1.3 we give an example of `force_sync` in the `venice` module in AMUSE.

The results of such an event-driven simulation can be distinctly different from a regular calculation in which both modules are called at fixed time intervals. This is demonstrated in Figure 8.3, where we compare three simulations, one without stellar evolution, and one with constant time steps between stellar dynamics and stellar evolution. The third simulation (yellow curve in Figure 8.3) is event-driven. In this simulation there was no real chance of stellar collisions because the initial cluster was

```

41 def resolve_supernova(supernova_detection, bodies, time):
42     if supernova_detection.is_set():
43         print("At time=", time.in_(units.Myr), \
44               len(supernova_detection.particles(0)), 'supernova(e) detected')
45
46     Nsn = 0
47     for ci in range(len(supernova_detection.particles(0))):
48         print(supernova_detection.particles(0))
49         particles_in_supernova \
50             = Particles(particles=supernova_detection.particles(0))
51         natal_kick_x = particles_in_supernova.natal_kick_x
52         natal_kick_y = particles_in_supernova.natal_kick_y
53         natal_kick_z = particles_in_supernova.natal_kick_z
54
55         particles_in_supernova \
56             = particles_in_supernova.get_intersecting_subset_in(bodies)
57         particles_in_supernova.vx += natal_kick_x
58         particles_in_supernova.vy += natal_kick_y
59         particles_in_supernova.vz += natal_kick_z
60         Nsn += 1
61
62     print('Resolved', Nsn, 'supernova(e)')

```

Code example 8.9: Function for resolving a supernova event.

quite large, so we followed a slightly different event-driven strategy here: we evolved all stars for a minimum stellar evolution step, followed by the dynamics for the same amount of time.

Figure 8.3 shows the evolution of the 25%, 50%, and 75% Lagrangian radii (see Section 4.1.3.1) of three initially identical clusters of 1000 stars over a time interval of 10 Myr. The stars were initially distributed as a $W_0 = 3$ King model (King, 1966; see Section 3.2), with stellar masses drawn from a Salpeter mass function between $1 M_\odot$ and $100 M_\odot$ (see Section 3.3). All runs were performed using the same dynamical integrator. The differences in the Lagrangian radii are therefore a consequence of how stellar mass loss propagates into the dynamical evolution of the cluster.

In the cluster without stellar mass loss, the Lagrangian radii shrink slightly over the 10 Myr period studied, as pure dynamics dictates the evolution. During the first ~ 3 Myr, the red and blue (and yellow) clusters show basically the same dynamical evolution. The differences only become evident later, when the most massive stars start to lose significant mass. After ~ 4 Myr, the Lagrangian radii of the runs with stellar mass loss start to expand.

In the cases with mass loss, about 18% of the initial cluster mass of $3200 M_\odot$ is lost, causing the 25%, 50%, and 75% Lagrangian radii to expand by 30-50%. The outermost Lagrangian radii expand slightly faster than the more inner Lagrangian radii. These expansion factors are significantly larger than would be naively expected based on an adiabatic response of the cluster, which would predict an 18% increase in size (Carathéodory, 1909). The reason is that much of the mass lost is ejected impulsively by the strong winds of the massive Wolf-Rayet stars and (after about 3 Myr) supernova. In addition, the orbits of the mass-losing stars are not circular, as is assumed in the

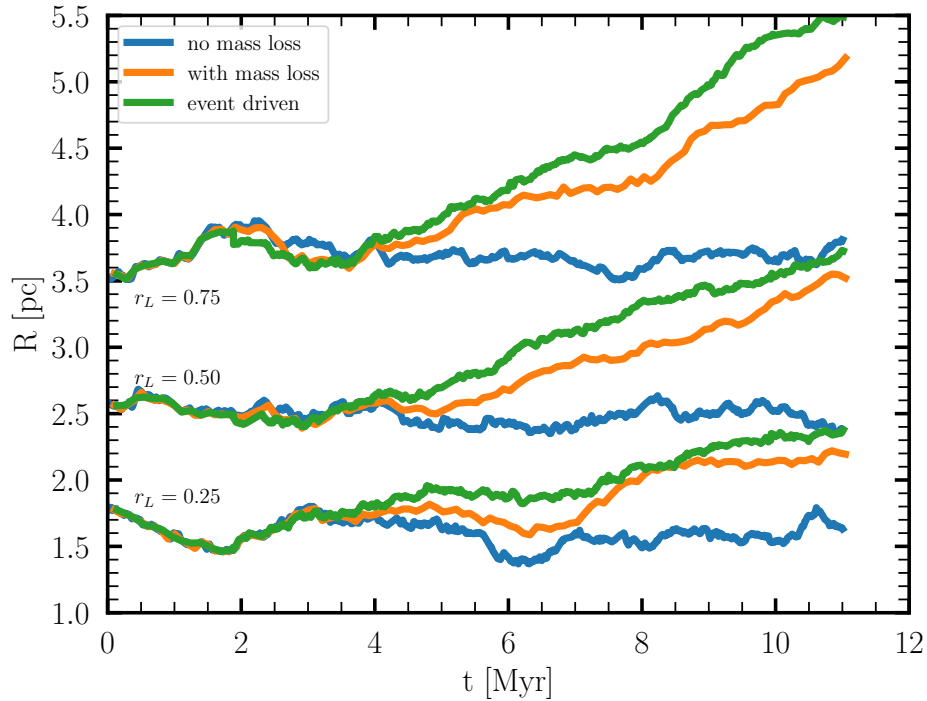


Figure 8.3: Lagrangian radii containing 25%, 50%, and 75% of the total cluster mass for three simulations of a cluster of 1000 stars. Each simulation is performed using exactly the same initial realization. In one of these runs (blue curve) stellar mass loss was ignored. In the second (red curve), mass loss was applied at constant time intervals of 0.1 Myr after the dynamics step (note that in Figure 8.4 we adopted a very small time step of only 10% of the inner orbital period). The yellow curve is the result of an event-driven simulation (see Section 8.4.4), and should, therefore, be considered most reliable. The figure was produced using the script `amuse.examples.gravity_stellar_comparison`, which should run in 5–10 minutes on a laptop.

naive adiabatic argument, but generally quite eccentric.

At the start of these particular simulations, none of the 10 most massive stars ($\gtrsim 20 M_{\odot}$) lay within the inner 25th percentile. By the end of the simulation, at 10 Myr, this number increased to 4 in the run without stellar evolution, and 2 in the other cases. Three massive stars were initialized outside the 75th percentile and had insufficient time to complete an orbit around the cluster center. The more rapid expansion of the outermost Lagrangian radii compared with the innermost radii is therefore in part due to the mass lost by these more massive stars, and not by the segregation of the most massive stars to the cluster center.

Event-driven simulations are generally somewhat slower than fixed-time interaction simulations, but they are probably worth the cost, because the physics is better represented by the numerical model.

8.4.5 Evolution of a hierarchical triple system

In this section we demonstrate an interlaced coupling between stellar evolution and gravitational dynamics by integrating a triple composed of three massive stars, each of them losing mass copiously. The orbital evolution of such a system cannot be described analytically, so the problem must be addressed numerically. This problem turns out to

be rather hard, because the various time scales of the problem are comparable. This example is not much different than presented in Section 8.4.4, but here we apply it to a tightly coupled triple system. Of course, one could consider performing this simulation with the more dedicated triple evolution code **Tres**, discussed in Appendix B.2.1.

The triple system θ Muscae comprises the WC5/6 Wolf-Rayet star WR48 on an inner orbit with period $P_{\text{in}} \simeq 19$ days around an O6/7V star, with an O9.5/B0Iab companion on a much wider outer orbit (Hill *et al.*, 2002; Sugawara *et al.*, 2008). The other orbital parameters are not known, but each of the three stars has a strong stellar wind. We can easily integrate the equations of motion of such a triple system within AMUSE, taking the mass loss of each star into account, by coupling a stellar evolution code with an N -body integrator.

We adopt masses at birth of $60 M_{\odot}$ (for WR48) and $30 M_{\odot}$ for the inner two binary stars, and $20 M_{\odot}$ for the outer companion. The nature of the Wolf-Rayet star suggests that the system is at least a few megayears old. We therefore began the simulation by evolving the three stars to an age of 4.5 Myr using **SeBa**. By that time the masses of the three stars had fallen to $12.48 M_{\odot}$, $28.52 M_{\odot}$, and $19.81 M_{\odot}$. It is difficult to judge whether or not these masses are consistent with the observations, but at least their spectral types are. The mass-loss rates at that age are $-1.6 \times 10^{-5} M_{\odot}/\text{yr}$, $-4.2 \times 10^{-7} M_{\odot}/\text{yr}$, and $-5.2 \times 10^{-8} M_{\odot}/\text{yr}$.

After this initial period of stellar evolution, we initialized the N -body code (in this case **Huayno**) in a configuration consistent with the observed orbit. We assumed that the outer orbit has an inclination of 30° relative to the inner one, with an inner eccentricity of 0.2, and an outer eccentricity of 0.6, although these are not known from observations. For definiteness, we chose an initial mean anomaly and argument of periastron of 180° , and an outer orbital separation of 100 AU.

Figure 8.4 presents the orbital semimajor axes and eccentricities of the inner and outer binaries during an integration from 4.0 Myr to 4.55 Myr using the interlaced scheme just described. The Wolf-Rayet star exploded as a supernova shortly after this, at an age of 4.6 Myr. During the integration, we continued to evolve the stars using **SeBa**, and the mass loss was propagated to the N -body integrator after every $0.1 P_{\text{in}} = 1.9$ days. The evolution of θ Muscae over the relatively small time frame presented in Figure 8.4 barely requires the use of a stellar evolution code, but it allows us to illustrate how stellar evolution can be combined with the dynamics of this complicated triple system.

The orbits of both binaries expanded due to mass loss. On the time scale of the integration, the inner two stars lost $0.18 M_{\odot}$, whereas the outer star lost $0.026 M_{\odot}$. From Figure 8.4 we can see that the outer binary expanded by about 0.8%, which is about a factor of two more than the fractional mass loss from all stars combined. The inner binary was much more strongly affected by mass loss, and expanded by about 2.7%. The variation in the eccentricity of the inner binary is caused by its interaction with the outer star. The classical Lidov-Kozai (von Zeipel, 1910; Lidov, 1962; Kozai, 1962) period is about 0.44 Myr, and corresponds to the large excursions in the eccentricity of the inner binary. The eccentricity of the outer binary remained almost constant.

In this example, and in fact in all other current approaches, mass is lost in small chunks at fixed time intervals. This causes the system to receive a small shock after

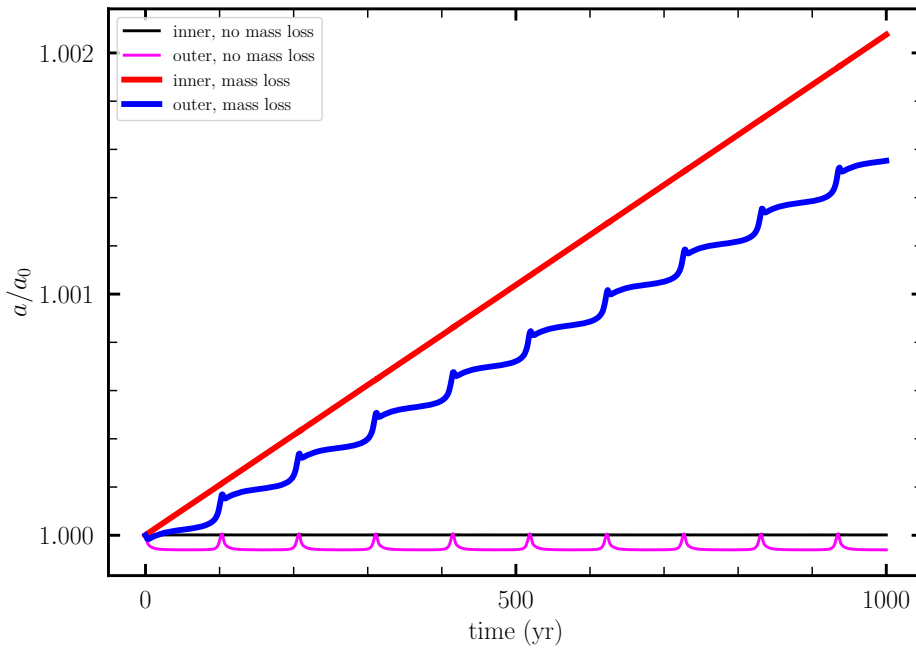


Figure 8.4: Evolution of the orbital semimajor axes of the inner (blue) and outer (red) orbits of the hierarchical triple system θ Muscae. All quantities are expressed relative to their initial values (at $t = 4.5$ Gyr). At the start of the integration, at 4.5 Gyr, the masses were 12.48, 28.52, and 19.81 $M_{\odot}$, respectively. The integration spanned a time interval of 10^3 yr. The script to generate the figure, `amuse.examples.plot_triple_with_wind`, takes a few minutes to run. The code that generated the data is available in `amuse.examples.evolve_triple_with_wind`.

every mass-loss step, and this may have undesirable side effects. In fact, in constructing Figure 8.4 we found that the outcome of the simulation is remarkably sensitive to the choice of integration parameters, especially the time interval on which stellar mass is applied. The script used to perform this calculation can be found in `amuse.examples.evolve_triple_with_wind`, and we invite readers to explore its outcome with different choices for various parameters. It will take between a few minutes and many days to run, depending on the parameter values chosen; the parameters leading to Figure 8.4 took a few hours on a laptop. This sensitivity is a topic of ongoing research, and AMUSE modules capable of incorporating mass loss directly into the dynamical integration are currently under development.⁴ While these are clearly counter to the AMUSE philosophy of not making major changes to the internals of community modules, sometimes code validation requires more intrusive measures.

The relatively poor performance of the code in this particular stongly coupled problem motivated us to develop a new symplectic N -body code with the possibiity of including a mass-loss term in the integrator. We call this code **symple** for its simple structure, its simple implementation, and the symplecticity of the adopted integrator. In fact, **symple** can be run with a wide range of integrators of different order ranging from 1st order to 10th order, each of which is symplectic. For more details about **symple**, we refer to Appendix A.3.4.

⁴See `AMUSE_DIR/examples/textbook/updates/AMUSE_updates.pdf` for ongoing developments.

Table 8.1: Adopted parameters for the stellar wind binary θ Muscae.

parameter	unit	star 1	star 2	star 3
spectral type		WR5	O6V	O9.5III
mass	$M_{\odot}$	12	20	30
radius	$R_{\odot}$	2.2	10	50
luminosity	$L_{\odot}$	2.2×10^5	1.4×10^5	0.55×10^5
temperature	K	7.1×10^4	4.5×10^4	3.9×10^4
mass-loss rate	$M_{\odot}/\text{yr}$	1.8×10^{-5}	1.0×10^{-6}	1.0×10^{-6}
wind velocity	km/s	2000	1000	1000

Although it goes well beyond the scope of this chapter, as a preview of AMUSE capabilities introduced in Chapter 6 and soon in Chapter 9, Figure 8.5 shows an image of the wind produced in θ Muscae, where now we integrate the equations of motion of the individual stars and simultaneously resolve the winds using a hydrodynamical simulation (Jänes *et al.*, 2014). Specifically, the simulation shows the gas density in the orbital plane of the hierarchical triple system. While the winds collide the three stars orbit each other resulting in the tight spiral (left) from the inner binary that interacts with the massive wind of the tertiary outer-most star. The simulation was performed using the SPH code `Fi` using a mass of $10^{-11} M_{\odot}$ for each SPH particle. Both the inner and the outer binaries are initially circular and in the same plane with orbital period of $P_{\text{in}} = 19.1$ days and $P_{\text{out}} = 130$ years.

The wind was resolved in the hydrodynamics solver using the `AcceleratingWind` routines from `amuse.ext.stellar_wind`, which produces a wind acceleration inversely proportional to the distance squared (or $\propto 1/r^2$). The results of the calculations are presented in Figure 8.5.

Evidently our “simple” example of integrating a triple star system has already proved quite challenging. What about an entire star cluster? Generating initial conditions for a star cluster in which we integrate the equations of motion and simultaneously evolve the stars is basically the same issue as discussed in Chapters 4 and 5. Once the initial realization has been created, we can start the stellar evolution code and the gravity solver in much the same way as we did here for θ Muscae.

8.5 Stellar cluster with collisions

8.5.1 Sticky stars

To demonstrate the working of stopping conditions and collision handling in direct N -body codes, we present an example of a simple cluster in which stars are allowed to collide. We opted for an $N = 1000$ body system initially distributed as a $W_0 = 7$ King model with a Salpeter mass distribution between $0.1 M_{\odot}$ and $10 M_{\odot}$ (see Section 3.3). Each star was given a radius of $0.001 N$ -body length units, which resulted in many interesting collisions. The code example is too long to reproduce here, but it is available in `amuse.examples.gravity_collision`.

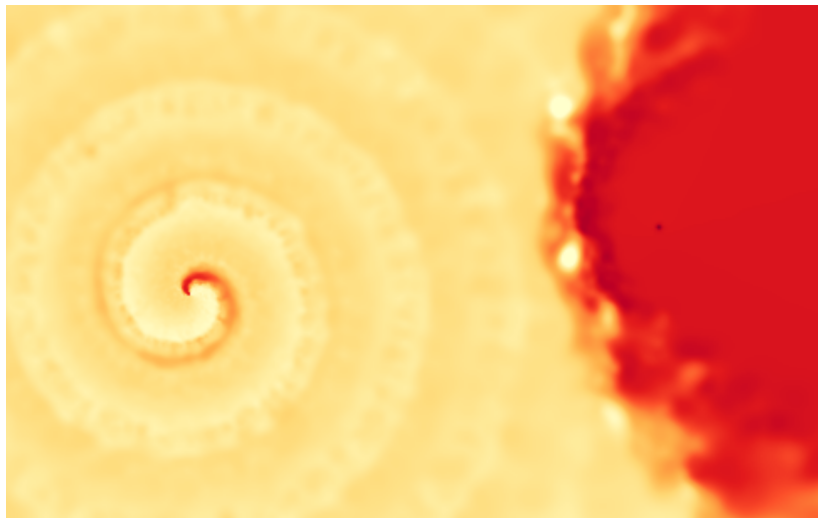


Figure 8.5: Evolution of the Wolf-Rayet triple-star system θ Muscae using *Huayno* (Jänes *et al.*, 2014) as the gravity solver and *fi* for the hydrodynamics (see Chapter 6). The hydrodynamics was resolved using the stellar wind routines in AMUSE (see van der Helm *et al.* (2018)). The script used to produce this figure is too long (and takes too long to run) to reproduce in the text, but its basic principles are described in Chapters 6 and 9.

We ran one simulation with virialized initial conditions. Collisions were resolved using the stopping conditions described in Appendix A.5.2. For collision products, we assumed no mass loss and calculated a new radius such that the mean density remained constant. The integration was performed using the direct integrator *ph4*. Stellar evolution was not included (see Section 5.1). Figure 8.6 presents the evolution of the core and the half-mass radii of the N -body system; the moments at which collisions occur are indicated by bullets. The size of a bullet is proportional to the mass of the collision product.

8.5.2 Using a separate code to manage a collision

If instead of running a parameterized stellar evolution code we are using a full Henyey code, such as MESA or EVtwin, we could in principle use the detailed structural information from the stellar evolution code to construct three-dimensional models of the colliding stars and model the collision as a hydrodynamical simulation, as we will discuss later in Section 8.5.4. This should in principle give the most accurate estimate of mass loss, energy released, etc., but it is typically a computationally expensive calculation—perhaps too expensive to be done routinely.

A useful intermediate solution to the merger problem is provided by the *MMAMS* (short for Make Me A Massive Star) module (Lombardi *et al.*, 2003; Gaburov *et al.*, 2008), as described in Section 5.2.9. The script presented earlier in Code example 5.5 can be used to determine the structure of the merger product. The only additional actions needed are to reposition the merger product in the center of mass frame of the two colliding stars, and to conserve momentum, as in Code example 8.5.

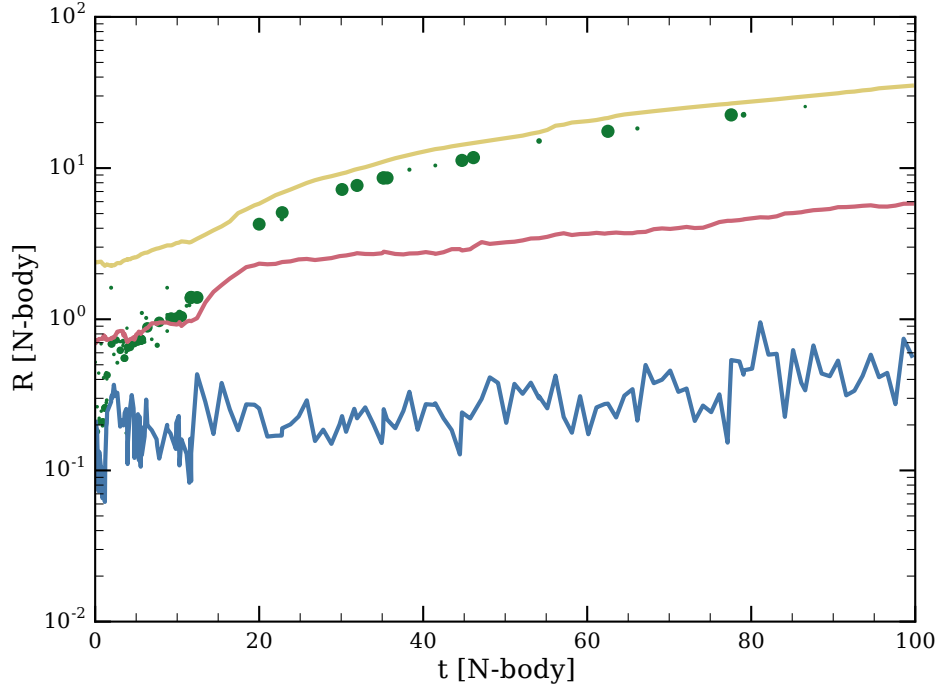


Figure 8.6: Direct N -body simulation (using `ph4`) of a small star cluster over 100 N -body time units. Initially, the cluster contained $N = 1000$ stars with a Salpeter mass distribution spanning two orders of magnitude in mass, distributed in a virialized King model with $W_0 = 7$. The initial radii of all stars were taken to be 0.001 N -body length unit. The bullets indicate the times and locations of stellar collisions; the size of a bullet indicates the mass of the collision product. We assume that no mass is lost and the mean stellar density remains constant. The three curves (bottom to top) illustrate the core radius (blue), the half-mass radius (red), and the 90% Lagrangian radius (yellow) of the cluster. The script that produced this figure is `amuse.examples.gravity_collision`. It should take a few minutes to run on a laptop.

8.5.3 Restarting stellar evolution after a merger

After the collision has been detected and managed, the evolution of the merger product can be continued. For a parameterized code, we simply use a rejuvenation formula to place the merged star on a new track at some evolutionary time (see also Section 5.3.12). We do have a script to transfer the stellar evolution from a parametrized code to a full Henyey code, which is an example in the AMUSE repository called `hr_diagram_cluster`.

However, if we are using a Henyey code, we can continue the detailed evolution of the merger product. Code example 8.10 shows how this can be done. In this example, we evolve two stars to the same time using `MESA`, merge them using `MakeMeAMassiveStar`, and then continue the evolution of the merger product using `MESA`.

```

71 def merge_two_stars_and_evolve(
72     mass_primary, mass_secondary, time_collision, time_end
73 ):
74     stars = Particles(2)
75     stars.mass = [
76         mass_primary.value_in(units.MSun),
77         mass_secondary.value_in(units.MSun)
78     ] | units.MSun

```

```

79 stellar_evolution = Mesa(**MESA_OPTIONS)
80 stellar_evolution.parameters.metallicity = 0.0142
81 stellar_evolution.particles.add_particles(stars)
82 while stellar_evolution.model_time < time_collision:
83     stellar_evolution.evolve_model()
84     print_stars(stellar_evolution)
85 n_shell = min(
86     stellar_evolution.particles[0].get_number_of_zones(),
87     stellar_evolution.particles[1].get_number_of_zones(),
88 )
89 merger_code = Mmams(**MMAMS_OPTIONS)
90 merger_code.parameters.target_n_shells = n_shell
91 merger_code.parameters.dump_mixed_flag = True
92 merger_code.parameters.do_shock_heating_flag = True
93 merger_code.commit_parameters()
97 handler = CollisionHandler(
98     merger_code,
99     stellar_evolution_code=stellar_evolution,
100 )
101 merger_product = handler.handle_collision(
102     stellar_evolution.particles[0],
103     stellar_evolution.particles[1],
104 )
105 merged = stellar_evolution.particles[0]
106 stellar_evolution.evolve_model(keep_synchronous=True)
107 p = stellar_evolution.particles[0]
108 while stellar_evolution.model_time < time_end:
109     stellar_evolution.evolve_model()
110     print(
111         f"Time={stellar_evolution.model_time} {p.stellar_type} "
112         f"{p.mass} {p.radius} {p.temperature} {p.luminosity}"
113     )
114     if p.stellar_type >= 4 | units.stellar_type:
115         break
116 merger_code.stop()
117 stellar_evolution.stop()

```

Code example 8.10: Merge two stars and evolve the merger product. The code example is available in `amuse.examples.merge_two_stars_and_evolve`. Several lines related to plotting Figure 8.7 have been omitted.

Figure 8.7 presents (yellow dashed line) the evolutionary track of the collision product resulting from the merger of stars of masses $3 M_{\odot}$ and $1 M_{\odot}$. The stars collided at 150 Myr, which is half the main-sequence lifetime of the more massive star (determined by first evolving the primary to the end of its nuclear burning lifetime using `EVTwin`). For comparison we overplot the evolutionary tracks of single stars of masses $3 M_{\odot}$ and $4 M_{\odot}$, as well as the track for a main-sequence star of mass $3.70 M_{\odot}$ (the value returned by `MakeMeAMassiveStar`).

The evolutionary track of the merger product makes several rapid excursions in temperature and luminosity before settling after $\sim 10^5$ yr [comparable to its Kelvin–Helmholtz timescale, see Equation (5.6)] onto a new “rejuvenated” track, after which its evolution continues along the main sequence at a slightly lower luminosity than the $4 M_{\odot}$ star. The merger evolutionary track differs significantly from that of a canonical $3.70 M_{\odot}$ star, mainly due to its non-standard abundance profile. Subsequently, on the

subgiant and giant branches, the track is quite similar to that of the $4 M_{\odot}$ star.

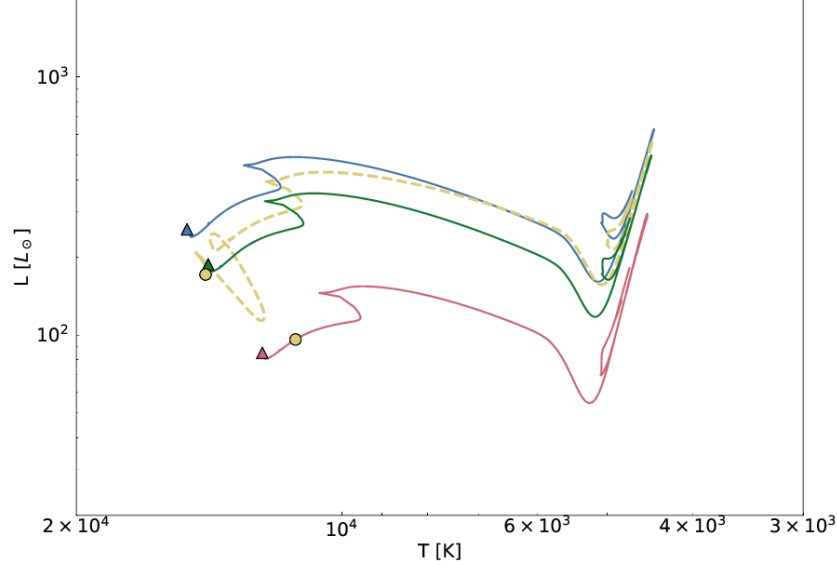


Figure 8.7: Evolutionary tracks for a $3 M_{\odot}$ star (red curve) a $4 M_{\odot}$ star (blue), and a $3.70 M_{\odot}$ star (green) from the main sequence (filled triangles) to the bottom of the AGB. The yellow dashed curve shows the evolutionary track of the merger product of a $3 M_{\odot}$ star and a $1 M_{\odot}$ star that collide half-way along the primary’s main-sequence track, with the merger structure determined by `MakeMeAMassiveStar`. The yellow circles represent the evolutionary states of the primary immediately before the collision and of the merger product immediately afterward. This figure was produced using the script `amuse.examples.merge_two_stars_and_evolve`, which should run in 10–15 minutes on a laptop.

In many cases, a simplified parameterized prescription for the consequences of a stellar merger may be quite satisfactory, although not very accurate. One can ask whether spending significantly more computer time on this complex problem is worth the effort, but many studies have demonstrated that detailed modeling of stellar collisions can give results quite different from the simplified semi-analytic prescriptions described in Section 5.3.12 (Lombardi *et al.*, 1995, 2002, 2003).

8.5.4 Using a hydro code to simulate a stellar merger

In Section 8.5.2, we demonstrated how to use a separate code to resolve a stellar collision using a parameterization of the collision process. Instead of a parameterized code, we could also use a hydrodynamics solver for the collision. Here we do just that, using an SPH code to resolve the merger.

8.5.4.1 Generating stellar structure in a particle set

To start the merger simulation, we first need the structure of two stars. We saw in Chapter 5 how this can be realized. Code example 8.11 demonstrates the process again.

In addition to evolving the star to the desired time, this code also converts the stellar structure provided by the Henyey code to an SPH particle representation:

```

17 def return_evolved_star_hydro(mass, time, number_of_sph_particles):
18     star = Particle(mass=mass)
19     stellar = Evtwin()
20     star = stellar.particles.add_particle(star)
21     stellar.evolve_model(time)
22     number_of_sph_particles = number_of_sph_particles * int(mass.value_in(units.MSun))
23     star_in_sph = convert_stellar_model_to_sph(
24         star, number_of_sph_particles
25     ).gas_particles
26     stellar.stop()
27     return star_in_sph

```

Code example 8.11: Using a Henyey code to construct an SPH realization of a single star of mass `mass` evolved to time `time`. The complete code example is in `amuse.examples.merge_two_stars_sph`.

```
star_in_sph = convert_stellar_model_to_SPH(star, N_sph).gas_particles
```

The method `convert_stellar_model_to_SPH` returns a Python class having `gas_particles` as an attribute. We run the same procedure for the primary and the secondary stars, then add the two SPH realizations together after offsetting the secondary with respect to the primary and giving them some relative velocity. This is illustrated in Code example 8.12. The script looks similar to Code example 5.5, except that here we convert the evolved stars into N smoothed gas particles.

The conversion from Henyey to SPH is done on a grid, which introduces artifacts and generally leaves the SPH star slightly out of equilibrium. We address this problem by relaxing the SPH models (in this case, using a separate SPH code), as shown in Code example 8.13. Here, we slowly reduce the velocities of the SPH particles while running the hydrodynamics code with short steps. This is the same process as for creating a “glass” solution for initial conditions as is done in many simulations (see, for example, Section 3.3 on page 40 in White, 1994). In this example, we have opted to relax the star for 2.5 dynamical times in 250 steps per dynamical time scale, which is rather short. It is better to have the star relax for far longer: J. Lombardi (2010, private communication) recommends at least 25 dynamical time scales.

By default, the output of `convert_stellar_model_to_SPH` represents a star at rest at the origin. Hence, before combining the two SPH realizations of the stars in a hydro solver, their positions and velocities must be changed to reflect the initial condition of the collision. This can be done by shifting each of the stars to its desired center-of-mass position, with all particles within each star being given the same velocity. In Code example 8.12, we position the secondary star such that it is almost touching the primary star, with a transverse velocity of roughly the escape speed in a direction that results in a grazing collision.

In order to prevent the two stars from moving to the left due to the momentum added to the system, one could move all the SPH particles to their center-of-mass frame after merging.

```

all_sph_particles = ParticlesSuperset([primary_in_sph, secondary_in_sph])
all_sph_particles.move_to_center()

```

```

34 def merge_two_stars_sph(
35     mass_primary, mass_secondary, time_collision, number_of_sph_particles
36 ):
37     primary_in_sph = return_evolved_star_hydro(
38         mass_primary, time_collision, number_of_sph_particles
39     )
40     primary_in_sph = relax_sph_realization(primary_in_sph)
41     secondary_in_sph = return_evolved_star_hydro(
42         mass_secondary, time_collision, number_of_sph_particles
43     )
44     secondary_in_sph = relax_sph_realization(secondary_in_sph)
45     radius = primary_in_sph.x.max() + secondary_in_sph.x.max()
46     total_mass = primary_in_sph.mass.sum() + secondary_in_sph.mass.sum()
47     secondary_in_sph.x += 0.8 * radius
48     secondary_in_sph.y += 0.6 * radius
49     secondary_in_sph.vx -= (constants.G * total_mass / radius).sqrt()
50
51     converter = nbody_system.nbody_to_si(mass_primary, 1.0 | units.au)
52     hydro = Gadget2(converter)
53     hydro.gas_particles.add_particles(primary_in_sph)
54     hydro.gas_particles.add_particles(secondary_in_sph)
55     hydro.evolve_model(2.0 | units.hour)
56     hydro.gas_particles.new_channel_to(primary_in_sph).copy()
57     hydro.gas_particles.new_channel_to(secondary_in_sph).copy()
58     hydro.stop()
59     return primary_in_sph, secondary_in_sph

```

Code example 8.12: Using an SPH code to merge two stars. For the full code example see `amuse.examples.merge_two_stars_sph`.

Given that SPH is Galilean-invariant, the shift to the center-of-mass frame is unnecessary, but it makes graphical presentations of the encounter much more convenient.

8.5.4.2 Resolving the collision with the hydrodynamics solver

With the initial conditions established, we instantiate the hydrodynamics solver `Gadget2`

```
hydro = Gadget2()
```

and add the SPH particle sets of the two stars to the community code:

```
hydro.gas_particles.add_particles(primary_in_sph)
hydro.gas_particles.add_particles(secondary_in_sph)
```

Here, we add the particle sets for each of the two stars separately, in order to keep track of which star each SPH particle initially belonged to.

In Code example 8.12 we run the hydrodynamics code for a simulation time of only two hours without saving any diagnostic data. For production simulations, this is not desirable. In many cases, it is better to run the specific code in a loop over time, testing conservation of energy and angular momentum, printing some diagnostics, and writing the raw data to a file at the end of each step. Such a loop might look like

```

66 def relax_sph_realization(sph_star):
67     dynamical_timescale = sph_star.dynamical_timescale()
68     converter = nbody_system.nbody_to_si(dynamical_timescale, 1 | units.RSun)
69     hydro = Gadget2(converter, number_of_workers=2)
70     hydro.gas_particles.add_particles(sph_star)
71
72     to_framework = hydro.gas_particles.new_channel_to(sph_star)
73
74     ts_factor = 2.5
75     time_end = ts_factor * sph_star.dynamical_timescale(mass_fraction=0.9)
76     n_steps = ts_factor * 100
77     velocity_damp_factor = 1.0 - (ts_factor * 2 * np.pi) / n_steps
78     time_step = time_end / float(n_steps)
79     time = 0 | units.day
80     while time < time_end:
81         time += time_step
82         hydro.evolve_model(time)
83         hydro.gas_particles.velocity = (
84             velocity_damp_factor * hydro.gas_particles.velocity
85         )
86     to_framework.copy()
87     hydro.stop()
88     return sph_star

```

Code example 8.13: Relaxing the SPH realization of a star in order to prepare it for a hydrodynamical collision simulation. This code fragment is from `amuse.examples.merge_two_stars_sph`. Here we introduced a scaling factor `ts_factor = 2.5`. This factor indicates how slowly (or how quickly) we reduce the velocities of the particles.

```

time_end = 10 | units.hour
time_step = 1 | units.hour
while hydro.model_time < time_end:
    hydro.evolve_model(hydro.model_time + time_step)
    channel_from_hydro_to_framework.copy()
    print_diagnostics(hydro)
    write_set_to_file(primary_in_sph, "hydro.amuse")
    write_set_to_file(secondary_in_sph, "hydro.amuse")
hydro.stop()

```

Here, `print_diagnostics(hydro)` might print time, energy, angular momentum, and other useful diagnostic information reflecting the progress of the simulation.

8.5.4.3 Recognizing a star in the collision product

It is not completely trivial to recognize the stellar merger product in the messy hydrodynamical blob that results from a collision. Such collisions may be highly asymmetric, rotate rapidly, and have a giant envelope of barely bound low-density gas. Figure 8.8 demonstrates the asymmetries one can expect from a rather violent collision, in this case between a $80 M_{\odot}$ and a $20 M_{\odot}$ star, both at an age of about 0.49 Myr. In this calculation, we adopted `Gadget2` to perform the hydrodynamical simulation using 10^5 equal-mass SPH particles. The amount of mixing seen in Figure 8.8 is somewhat surprising when compared to earlier results, particularly because such collisions tend to lead to relatively little mixing (see the discussion in Lombardi *et al.*, 1996). The use

of more realistic stellar structure models cannot be the reason for the discrepancy (see [Sills & Lombardi, 1997](#)), but it is possible that the very high stellar masses adopted, compared to the initial conditions in [Lombardi *et al.* \(1996\)](#), could lead to more mixing.

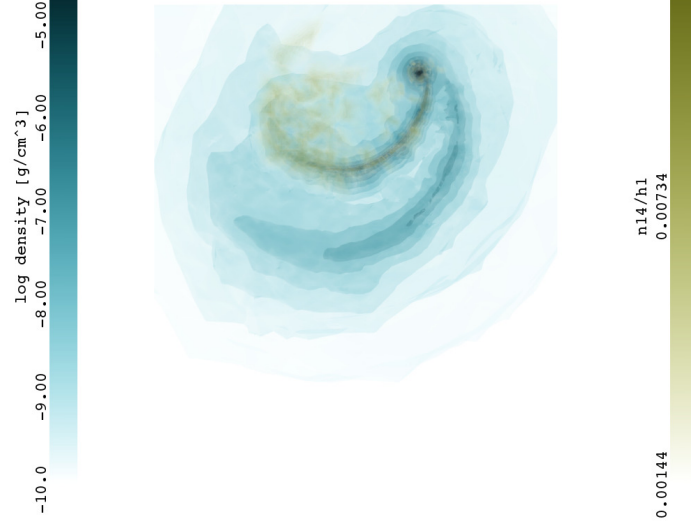


Figure 8.8: Projected density distribution of the gas ejected shortly after the merger between two massive stars. The results of the hydrodynamical simulation are shown 8.8 days after the merger between a $M_{\text{in}} = 80 M_{\odot}$ star and a $m_{\text{in}} = 20 M_{\odot}$ star in an $a_{\text{in}} = 77 R_{\odot}$ orbit with an eccentricity of $e_{\text{in}}^{\text{B}} = 0.87$. Projected density is shown in blue, while the nitrogen/hydrogen ratio is presented in mustard. The origin is at the center of mass of the original two-body system. The two massive loops of ejected gas (at left) are characteristic of a merger. We do not present the plotting routine for this figure, but instead refer to [Portegies Zwart & van den Heuvel \(2016\)](#) for more details.

Here, we adopt a rather simple approach to the problem of identifying the SPH particles making up the merger product. We analyze the results of the SPH simulation using the `hop` clump finder (see Section 8.3.1). The three control parameters for peak density threshold, saddle point density cutoff, and the outer density threshold are specified to help `hop` decide which particles belong to a coherent structure. Each of the groups in the list of particle sets (`hop.groups`) identified by `hop` belongs to an identified structure. For our stellar merger calculation, we are interested in the largest one.

```
def find_clumps_with_hop(sph_particles, unit_converter):
    hop = Hop(unit_converter)
    hop.particles.add_particles(sph_particles)
    hop.calculate_densities()

    mean_density = hop.particles.density.mean()
    hop.parameters.peak_density_threshold = mean_density
    hop.parameters.saddle_density_threshold = 0.99*mean_density
    hop.parameters.outer_density_threshold = 0.05*mean_density
    hop.do_hop()

    blob = [x.get_intersecting_subset_in(sph_particles) for x in hop.groups()]
    hop.stop()
    return blob
```


Finding clumps in an SPH simulation is commonly done in post-processing, but it can also be used to monitor the progress of the merger simulation. In the simulation presented in Figure 8.8, we monitored the clumpiness of the particle distribution at run time, and continued until only a single blob of particles remained. After the detection of this final blob, we ran the hydrodynamics for a further 5.5 years to ensure that it had come into dynamical equilibrium. This final blob is considered to be the merger product, and the residual material is identified as ejecta from the merger process. In this simulation, the merger resulted in a single $83 M_{\odot}$ star.

Once coherent structures have been identified, we often are interested in the original properties of particles that are part of a particular group. For performance and memory optimization, the particles in `hop` are given only the attributes necessary to identify individual clumps, whereas the original particle set (`sph_particles` in the above example) may contain additional information, such as chemical composition. We therefore want to find the intersecting subset of particles with the same `key` among the original `sph_particles` and the particles in `hop`. This is done in the line

```
blob = [x.get_intersecting_subset_in(sph_particles) for x in hop.groups()]
```

Finally, we stop `hop` and return the appropriate subset of particles.

8.5.4.4 Converting from hydro back to stars

In Section 8.5.2, we discussed how a separate code can be used to manage a head-on collision between a $3 M_{\odot}$ and a $1 M_{\odot}$ star, each evolved to an age of 150 Myr, continuing the evolution of the resulting merged star afterwards. There we adopted the output of the **MMAMS** (Gaburov *et al.*, 2008; Lombardi & Sawyer Warren, 2014) package to resolve the collision, but now we can adopt the methods discussed in the previous subsection, using SPH rather than **MMAMS** to determine the structure of the merger product.

As before, we first evolve the stars for 150 Myr, which is half of the primary's main sequence lifetime. Both stars are then converted to SPH realizations and relaxed (see Section 8.5.4.1). The resulting particle sets are combined, with the stellar surfaces just touching and zero relative velocity at infinity. We then use the SPH code **Fi** to resolve the hydrodynamics of the encounter. The free-fall time scale at this moment is just a few minutes, but to ensure that the gas distribution is in equilibrium, we continue the hydrodynamical calculation for 10^3 times longer. After the SPH calculation, we identify the merged star by using `hop` to search for the most massive blob (see Section 8.5.4.3), and then we strip away any low-density unbound splash material. The leftover merger product has a mass of $3.73 M_{\odot}$, very close to the **MMAMS** result for the same encounter (see Section 8.5.2).

After stripping away the envelope and relaxing the SPH system again, we put the star back into the Henyey stellar evolution code. This latter step is a rather delicate process, because the stellar evolution code expects a stellar structure that is in thermal equilibrium. The hydrodynamics blob will be in dynamical equilibrium, but thermal equilibrium cannot be guaranteed. The procedure for converting the SPH star into a Henyey model is

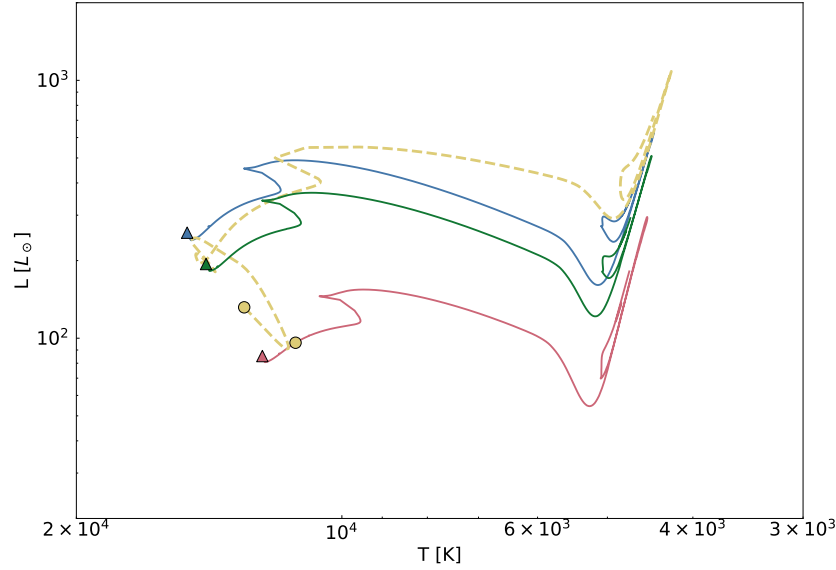


Figure 8.9: Evolutionary tracks for a $3 M_{\odot}$ star (red curve), a $4 M_{\odot}$ star (blue), and a $3.73 M_{\odot}$ star (green) from the main sequence (filled triangles) to the bottom of the AGB. The yellow dashed curve shows the evolutionary track of the merger product of a $3 M_{\odot}$ star and a $1 M_{\odot}$ star that collide halfway along the primary’s main sequence track, with the merger structure determined using SPH and `hop`. The yellow circles represent the evolutionary states of the primary immediately before the collision and of the merger product immediately afterward. This figure was produced using the script `amuse.examples.merge_two_stars_and_evolve`, which should run in 10–15 minutes on a laptop.

```
from amuse.ext.sph_to_star import convert_SPH_to_stellar_model
new_stellar_model = convert_SPH_to_stellar_model(star_in_SPH)
```

This function call can be quite expensive in terms of computer time, and may even fail after an hour or so of crunching. If it succeeds, however, you have a good chance of being able to evolve the star using `MESA` or `EVtwIn`. After this conversion, we continue the evolution of the collision product to study its track in the H–R diagram.

Figure 8.9 displays the same information for this SPH/`hop` merger as was presented in Figure 8.7 for the `MMAMS` merger (see Section 8.5.2). The red, blue, and green tracks show, respectively, the evolution of main sequence stars of masses $3 M_{\odot}$, $4 M_{\odot}$, and $3.73 M_{\odot}$ (the value returned by SPH/`hop`). The yellow dashed line is the evolutionary track of the merged object. As with the merger discussed in Section 8.5.2, the evolutionary track of the merger product makes several rapid excursions in temperature and luminosity before settling [after $\sim 10^5$ yr, again comparable to the Kelvin–Helmholtz time scale; see Equation (5.6)] onto a new stable track. Its subsequent evolution takes it along a significantly more luminous track than either the `MMAMS` equivalent or the $4 M_{\odot}$ star.

Some cautionary notes are in order. First, recognizing the bound blob after the hydrodynamical simulation depends on several parameters in the `hop` algorithm (see Section 8.5.4.3), and may require some experimentation. Second, we have used a relatively small number of SPH particles (100 equal-mass particles per solar mass) here

to model the merging stars. In part, this is to reduce the cost of the SPH simulation, but it also reflects the fact, already noted, that converting from an SPH to a Henyey representation is a very fragile process that may easily fail to produce a workable Henyey model, and whose outcome depends sensitively on the details of how SPH particles are combined into Henyey shells. In particular, the initial location of the merger product on the H–R diagram can vary dramatically with these details, and probably should not be taken too seriously. As with many advanced topics in AMUSE, these details are still under active development: see the updates page `{AMUSE_DIR}/examples/textbook/updates/AMUSE_updates.pdf` for new material.

It is hard to validate the evolution of the merger product. Understanding the causes of these deviations from the standard evolutionary track would require a detailed analysis of the stellar structure, which could easily expand into a PhD project. We will not dwell further on the dark art of stellar astrophysics. It is clear, however, that it is worth exploring the possibility of resolving stellar collisions using hydrodynamical models. At this point however, we consider MMAMS a viable option that at least leads to predictable behavior (see Section 8.5.2).

8.6 Complex linear code-coupling strategies applied

8.6.1 Heating of a protoplanetary disk

The abundance of several species of short-lived radionuclides (SLRs), such as ^{26}Al and ^{60}Fe , in the solar system has puzzled astronomers for some time (MacPherson *et al.*, 1995). Key questions include: How did these volatile elements, particularly ^{26}Al , enter the solar system? How did they come to be part of meteorites? And how did chondrules (the metallic grains containing these elements) form before chondrites (the meteorites that contain them)? There seems to be a lot we do not really understand.

One proposed explanation involves a nearby supernova when the Sun was still part of its parent star cluster. This is a two-stage process. First, the radiation from a bright supernova illuminates and heats the disk. Second, a few decades later, the hot ionized gas from the explosion plows through it. The idea is that it might be possible for the first burst of radiation to heat the disk enough to vaporize the dust, such that the subsequent wave of radioactive material could deposit the SLRs. Later cooling of the disk might subsequently have caused these SLRs to become part of the newly formed dust. This is all speculative, but worthy of further exploration.

Here, we study the first stage of the process: the heating and ionization of a protoplanetary circumstellar disk by the radiation from the supernova. For the gaseous disk, we adopt the protoplanetary disk model discussed in Section 6.4.1. To mimic the thick protoplanetary disk, we adopt a Safronov–Toomre Q parameter of $Q = 25$, rather than $Q = 2$ as in the earlier example of a thin disk (see Section 6.4.1). The disk was constructed as in the earlier example, with 10^4 equal-mass SPH particles at distances between 1 au and 100 au from the Sun, a total mass of $0.01 M_{\odot}$ and a power-law density profile with an exponent of -2. We arbitrarily assume that the disk is oriented at an angle of 15° (almost face-on) to the line of sight of the exploding star. The

hydrodynamical simulation was run using Fi with an isothermal equation of state and an artificial viscosity parameter of $\alpha = 0.1$ (see Section 6.2.4).

For the evolution of the supernova luminosity, we follow Sanders *et al.* (2015) and adopt a simple broken power-law luminosity function with characteristic luminosity and time scales. The script `amuse.examples.supernova_IIP_Lightcurve` presents code from Sanders *et al.* (2015) to reproduce those light curves. We consider two distinct supernovae, with peak luminosities of a few times $\sim 10^9 L_\odot$, reached about 10 days after the onset of the explosion. For the first, we adopt the best-fit parameters for type-IIP supernova SN PS1-11aof, which has a peak luminosity of 2×10^{43} erg/s. The second supernova, PS1-10a, is somewhat dimmer. The light curves for the two supernovae are shown as the blue curves in Figure 8.10.

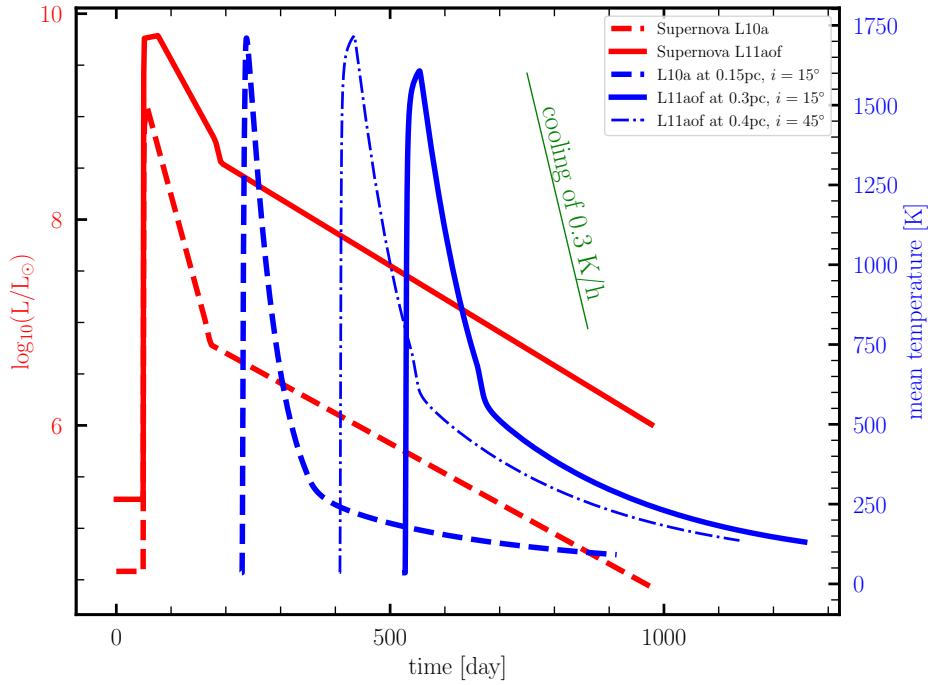


Figure 8.10: Light curves for the type IIP supernovae PS1-10a and PS1-11aof (dashed and solid blue curves, respectively, scale in blue on the left, Sanders *et al.*, 2015). The red curves give the temperature of our model disk at distances of 0.15, 0.3 and 0.4 pc (from left to right). The mean disk temperature (axis scale in red on the right) gives the disk temperature calculated using supernova PS1-10a at $d = 0.15$ pc and $\Theta = 15^\circ$ (the dashed curve) and PS1-11aof at a distance of 0.3 pc with $\Theta = 45^\circ$ and at $d = 0.4$ pc with $\Theta = 15^\circ$ (the solid curves). The thin green line (upper right) indicates a cooling of 0.3 K/hour, which is consistent with cooling after the supernova radiation has heated the disk for SN PS1-11aof, and which is sufficient for the formation of vitreous structures in the chondrules of the circumstellar disk. The calculations are performed at a resolution in the radiative transfer code of $N_{\text{ray}}/dt = 10^7/(1|\text{units}|\text{.hour})$. The figure was made using the script `amuse.examples.supernova_irradiation`. The calculation took several days on a laptop, and for this reason we include the data files and the routine to plot those in `amuse.examples.plot_supernova_IIP_and_disk_temperature`.

We used the radiative transfer code `SPHray`, which is well suited to such simulations, because it does not impose any limits on the spatial geometry or the computational domain. The hot supernova radiation was mimicked by setting some additional parameters in the radiative transfer code.

```
radiative = SphRay(redirection="file", number_of_workers=24)
radiative.parameters.number_of_rays = number_of_rays / time_step
radiative.parameters.default_spectral_type = -3.0
```

Here, `radiative.parameters.number_of_rays = number_of_rays / time_step` defines the number of rays per time step used in the radiative transfer solver. In our case, we adopted `number_of_rays / time_step = 107 / (11 units.hour)`. The parameter `default_spectral_type` is used to indicate the slope of the source's spectral energy distribution. For a supernova, a value of -3 is closer to the actual observed spectral energy distribution than the default value of -1 .

In the radiation transport code, the irradiating source is represented by a particle, called `src_particle`. We can update this particle's parameters at run time by using a separate channel.

```
channel_from_src_to_rad = \
    supernova.particles.new_channel_to(radiative.src_particles)
supernova.evolve_model(time)
channel_from_src_to_rad.copy_attributes(["luminosity"])
```

Note that there is only one source particle in our calculation, even though the code's denotative is plural.

The calling sequence for the radiative transfer and hydrodynamics modules is similar in structure to Code example 8.2 in Section 8.2.1, where we combined stellar evolution with gravitational dynamics. The two codes are called `radiative` and `hydro`. Both codes have appropriate channels defined to and from the local particle sets: `disk` for the SPH particles, `star` for the stellar particle in the hydro code, and `supernova.particles` for the local source particle to the radiative transfer code. The event loop then becomes:

```
while time < time_end:
    time += time_step / 2
    supernova.evolve_model(time)
    channel_from_src_to_rad.copy_attributes(["luminosity"])
    radiative.evolve_model(time)
    channel_rad_to_hydro.copy()

    time += time_step / 2
    hydro.evolve_model(time)
    hydro_to_disk.copy()

    supernova.evolve_model(time)
    channel_from_src_to_rad.copy_attributes(["luminosity"])
    update_source_particle(time)
    radiative.evolve_model(time)
    rad_to_disk.copy()
```

First, we update the supernova luminosity, perform an evolution step with a time step of `time_step / 2` in the radiative transfer code, and copy the irradiated SPH particle data to the hydrodynamics code. Next, we update the time to a full step, evolve the hydrodynamics of the disk, and copy the data back to the framework. Finally, we

repeat the first snippet, updating the supernova luminosity and performing another half radiative time step. In the last line, we copy the data from the radiative transfer code back to the local particle set.

The red curves in Figure 8.10 show the temperature evolution of three identical protoplanetary disks around a $1 M_{\odot}$ star at a distance of 0.15 pc from SN PS1-10a (left dashed curve), and at 0.3 and 0.4 pc from the considerably brighter SN PS1-11aof (right two solid curves). The mean temperature in the disk exceeds 1500 K for each of these supernovae and distances, which is high enough to melt agglomerates with the highest melting points. Carrying out any of these simulations with a lower resolution, using less than 10^7 rays per hour, in the radiative transport calculation would lead to lower temperatures, whereas adopting a larger number of rays hardly affect the results, except that the script takes longer to run.

The shock wave reaches the disk a few tens of years after supernova radiation. This is a more hydrodynamical process, which we will discuss further in Section 10.3.

8.6.2 Integrating a star cluster with interrupts

The close proximity of the Trapezium cluster allows detailed observations of circumstellar disk sizes using HST/WFPC2 (Vicente & Alves, 2005). This size distribution is well described by a power law, but the origin of this distribution is not well understood. Vicente & Alves (2005) argue that “[Despite] the young age of the Trapezium, given that disk destruction is well underway, it is perhaps too late to tell if the present day disk size distribution is primordial or if it is a consequence of the massive star formation environment.”

We can write a small AMUSE script to test this hypothesis, and to study the evolution of the size distribution of circumstellar disks. Such a script should integrate the equations of motions of the stars and keep track of close encounters in order to truncate disks depending on the distance, mass and velocity of a close encounter. Any direct N -body code will be sufficiently accurate for such a study. And since the number of stars is relatively small these calculations can be performed on a laptop. Since the vast majority of close encounters are simple two-body interactions, we can integrate the cluster for a small time frame [smaller than a typical crossing time of ~ 2.4 Myr, see Equation (4.5)], evaluate all close encounters (using the `Kepler` package to find the distances of closest approach), resolve the effect of the encounters on the circumstellar disks, and compare the numerically obtained distribution of disk sizes with the observed distribution.

Code examples 8.14 to 8.16 present the main routine for these runs. The complete script is available in `amuse.examples.gravity_kepler_disks`. It generates the data for Figure 8.11, and should take 5-10 minutes to run on a laptop. This simulation started with 2000 stars drawn from a Kroupa (2001) distribution with masses between $0.01 M_{\odot}$, and $100 M_{\odot}$, with positions drawn from a fractal distribution (Goodwin & Whitworth, 2004) of dimension $F = 1.6$, and velocities scaled so that the cluster was in overall virial equilibrium. At run time we check for close encounters. If such an event is identified we calculate the pericenter distance for the two-body encounter and truncate the disks of both stars.

```

97 while time < t_end:
98     time += dt
99
100    gravity.evolve_model(time)
101    Etot_prev_se = gravity.kinetic_energy + gravity.potential_energy
102    while stopping_condition.is_set():
103        channel_from_gd_to_framework.copy()
104        Ek_enc = gravity.kinetic_energy
105        Ep_enc = gravity.potential_energy
106        for ci in range(len(stopping_condition.particles(0))):
107            particles_in_encounter = Particles(
108                particles=[stopping_condition.particles(0)[ci],
109                          stopping_condition.particles(1)[ci]]
110                local_particles_in_encounter = particles_in_encounter.
111                get_intersecting_subset_in(bodies)
112
113            resolve_close_encounter(gravity.model_time,
114                                   local_particles_in_encounter, Johannes)
115            Nenc+=1
116            print(f"At time= {gravity.model_time} Nenc= {Nenc} ",
117                  (f"Rdisk= {local_particles_in_encounter.disk_radius}"))
118            channel_from_framework_to_gd.copy_attributes(["radius"])
119            dEk_enc += Ek_enc - gravity.kinetic_energy
120            dEp_enc += Ep_enc - gravity.potential_energy
121            gravity.evolve_model(time)
122
123    channel_from_framework_to_gd.copy_attributes(["mass"])

```

Code example 8.14: Main routine of a script to perform an N -body simulation with interrupts to resolve star-disk encounters. The main event loop is presented in Code example 8.15. The star-disk interaction routine is given in Code example 8.16.

The disk truncation radius, r_{trunc} , is calculated from the distance of closest approach between two stars, q , and the masses of the two stars m and M by (Breslau *et al.*, 2014)

$$r_{\text{trunc}} = 0.28q \left(\frac{m}{M} \right)^{0.32}. \quad (8.1)$$

If r_{trunc} is smaller than the current disk size, the disk is truncated at r_{trunc} . The truncation routine is presented in Code example 8.16.

```

8 def resolve_close_encounter(time, bodies, Johannes):
9     Johannes.initialize_from_particles(bodies)
10    rcom = bodies.center_of_mass()
11    vcom = bodies.center_of_mass_velocity()
12
13    a, e = Johannes.get_elements()
14    p = Johannes.get_periastron()
15    print("Close encounter at t=", time.in_(units.Myr), "a=", a.in_(units.AU), "e="
16          , e, "p=", p.in_(units.AU), "M=", bodies.mass.max().in_(units.MSun), bodies.mass
17          .min().in_(units.MSun), "at d=", rcom.in_(units.parsec), "with v=", vcom.in_(
18          units.kms))
19
20    truncate_disks_due_to_encounter(bodies, p)

```

Code example 8.15: Continuation of the script to run an N -body simulation with interrupts to resolve star-disk encounters. The star-disk interaction routine is given in Code example 8.16.

```

21 def truncate_disks_due_to_encounter(bodies, p):
22     q = bodies[1].mass/bodies[0].mass
23     rtr_prim = 0.28*p / q**(0.32)
24     rtr_sec = 0.28*p * q**(0.32)
25     dm0 = truncate_disk_due_to_encounter(bodies[0], rtr_prim)
26     dm1 = truncate_disk_due_to_encounter(bodies[1], rtr_sec)
27
28     mtot = bodies.mass.sum()
29     bodies[0].accreted_mass += dm1 * bodies[0].mass/mtot
30     bodies[1].accreted_mass += dm0 * bodies[1].mass/mtot
31
32     bodies[0].radius = min(bodies[0].radius, 0.5*p)
33     bodies[1].radius = min(bodies[1].radius, 0.5*p)
34
35     bodies[0].mass = bodies[0].stellar_mass + \
36         bodies[0].disk_mass + bodies[0].accreted_mass
37     bodies[1].mass = bodies[1].stellar_mass + \
38         bodies[1].disk_mass + bodies[1].accreted_mass

```

Code example 8.16: Routine to truncate a circumstellar disk during a pericenter passage with another star.

Figure 8.11 compares the size distributions of the circumstellar disks with the observed distribution in the Orion Trapezium cluster. The observed distribution is limited by the telescope’s resolution (Vicente & Alves, 2005). The pixel size in these observations is 45 AU, and disks as small as 1.5 pixels (or 67.5 AU) can be resolved. The authors argue that their sample is complete down to a minimum radius of 100–150 AU. Of the 162 protoplanetary disks, 95 are larger than 100 AU (Vicente & Alves, 2005). Although the observations are limited and the dynamical model is oversimplified, the similarity between the two datasets is striking.

8.7 Multi-scale modules in AMUSE

8.7.1 The multiples module: combining few-body with many-body dynamics

The N -body integrators described in Chapter 4 are designed to move large numbers of stars around a stellar system, and they are extremely efficient in accomplishing that task. However, in reality many stars are born with a companion, in tightly bound binary systems, or they may acquire companions through dynamical interactions. In those circumstances, while many of the direct N -body codes in AMUSE can still follow the motion with reasonable accuracy, the short time steps associated with the tight orbits mean that the integration effectively grinds to a halt once close binaries form.

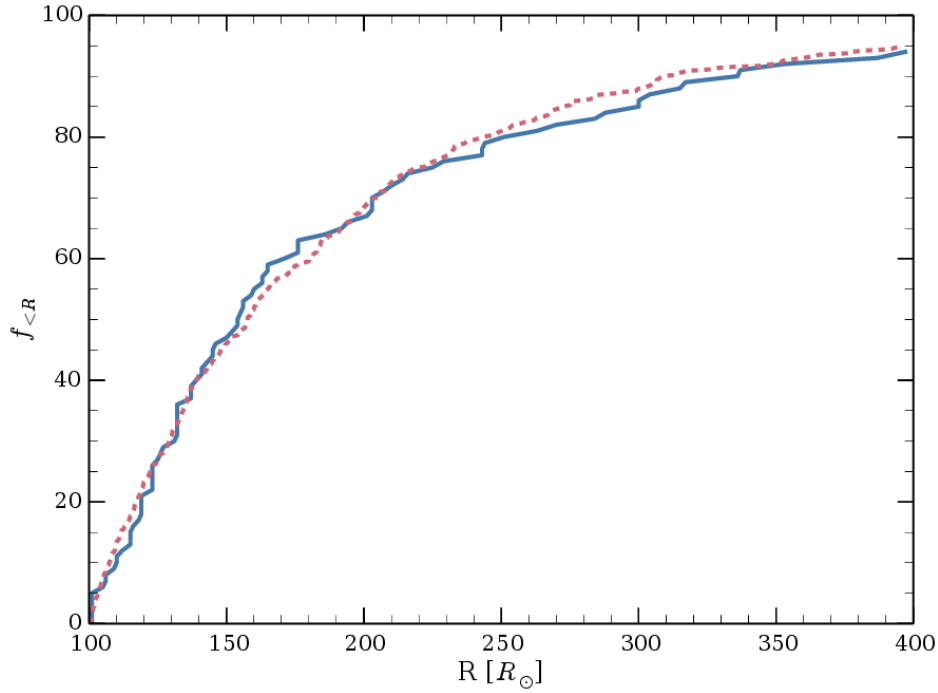


Figure 8.11: Cumulative size distribution of circumstellar disks in the Trapezium cluster. The observations are complete for disk radii exceeding 100 AU. The red dotted line gives the 95 observed disk sizes from the observations. The black solid curve gives the result of our simulations at 1 Myr. A Kolmogorov–Smirnov (KS) comparison (Kolmogorov, 1933; Smirnov, 1948) of the two cumulative distributions results in a KS-statistic of 0.058 with a two-tailed p-value of 0.94, indicating that we cannot reject the hypothesis that both distributions are indistinguishable. The complete script that produced this figure is in `amuse.examples.compare_disk_size_Trapezium`. It simply plots data read from a file and runs in a few seconds on a laptop.

In traditional N -body codes, such as NBODY6/7 and `starlab`, a substantial fraction of the total code base is devoted to following the internal motion of binaries and multiples simultaneously with the large-scale motion of the rest of the system. The algorithms involved have varying degrees of efficiency and accuracy, but all gradually relax the accuracy of the perturbed binary integration until “unperturbed” binaries (usually at a perturbation level of 1 part in 10^{6-7}) are treated as isolated unperturbed objects. The alternative approach, widely used in Monte Carlo schemes, treats multiples (where for simplicity the term will henceforth include binaries) as completely unperturbed until they interact strongly with another object. (A pioneering N -body variant on this approach was the GORILLA code presented by Tanikawa & Fukushige, 2009). Many papers have been devoted to the relative merits of these approaches, but in our opinion there is no convincing evidence that the improved treatment of lightly perturbed binaries inherent in the N -body approach has any significant advantage over the much simpler Monte Carlo treatment.

Accordingly, in the AMUSE `multiples` module, we depart from the N -body norm and adopt the Monte Carlo approach to multiple interactions. The module is designed to handle close encounters between stars and multiples in N -body systems. The basic idea is that the N -body modules manage multiple centers of mass, but are completely ignorant of the internal dynamics of these systems. In operation, the `multiples` module

maintains a database of all stable multiples in the system. A stable multiple is a binary, a hierarchical triple deemed stable by the [Mardling \(2008\)](#) criterion, or a higher-order multiple system in which the Mardling criterion applied to the outermost orbits indicates approximate stability. As illustrated in Figure 8.12, when two centers of mass approach within the sum of their (dynamical) radii, a stopping condition is thrown and control is passed to `multiples`. The radii are typically chosen to be the 90° turnaround distance ($\sim G\langle m \rangle / \langle v^2 \rangle$) for a single star, and a few outer binary semimajor axes for a multiple system. In virial equilibrium and N -body units, this distance is $1/2N$. (All critical `multiples` parameters have reasonable defaults and can be modified by the user script, if desired.)

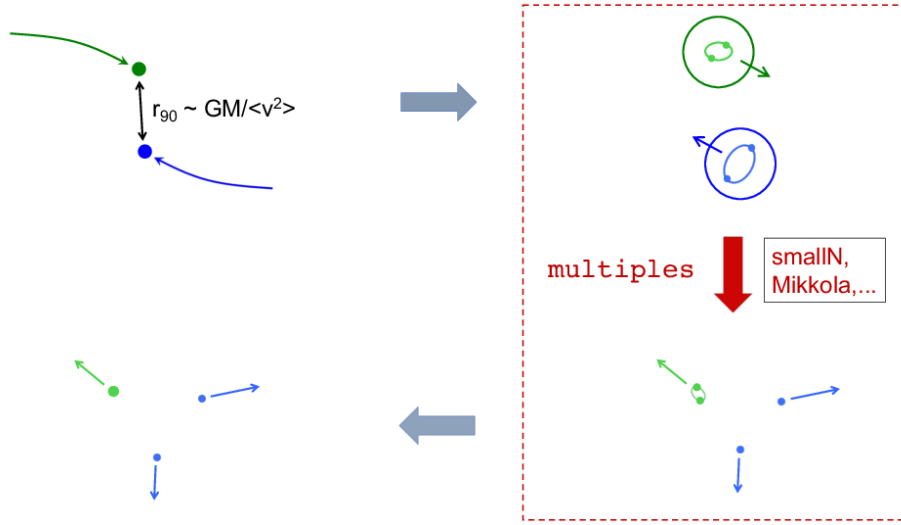


Figure 8.12: Two interacting stars (top left) are identified by the N -body code, expanded into their component stars (top right), integrated to completion by a standalone small- N code (bottom right), and reinstated into the N -body code (bottom left).

Once a dynamical interaction is detected and passes some basic tests (e.g. that the centers of mass are actually approaching), the internal structures of the interacting bodies are reinstated from the `multiples` database and the dynamics of the resulting small- N system followed to completion using a dedicated code. The AMUSE `smallN` module, a robust shared-step time-symmetrized code with analytic extensions ([Hut et al., 1995](#); [McMillan & Hut, 1996](#)) is often used for this task; the `Mikkola` module, which implements Mikkola’s ([Mikkola & Merritt, 2008](#)) algorithmic regularization code is valuable for problems where post-Newtonian gravitational effects must be included. “Completion” in this context means that the system has resolved itself into a collection of mutually receding stable systems (as just defined). We always expect this to be the eventual outcome of a small- N interaction. In principle, the tidal field of the N -body system could be included in the small- N calculation, but this is not currently implemented, although an attempt is made to absorb tidal errors by reorienting the final top-level system. To enforce a degree of isolation, in the current implementation a close perturber (lying within ~ 3 times the initial separation of the interacting centers of mass) will veto the small- N calculation.

A major approximation in this approach is that interactions occur instantaneously—two center of mass particles in the N -body code are replaced by some number of new center of mass particles with no change in the system time. For most interactions, which take only a two-body encounter time ($\sim 1/N$ dynamical times) to complete, this causes a negligible error, but in some long-lived “hierarchical” resonances, as defined by [Hut \(1993\)](#), this may not be acceptable. We address this shortcoming as follows. In a pure scattering experiment we would require that all top-level centers of mass of stable multiples be mutually receding and unbound at the end of the simulation. However, if we also accept centers of mass that form wide bound pairs, hierarchical resonances resulting in long bound intermediate orbits can returned to the N -body code even though the criteria for completion are not fully met. The internal motion is frozen until the N -body code again selects these bodies to continue the interaction. Thus, long resonances are not treated as instantaneous, providing a rudimentary means for the N -body code to interact with the small- N system.

Allowing wide bound systems as an acceptable outcome of a scattering has another important consequence. In practice, it is very convenient to use `multiples` also to manage two-body encounters of single stars, even though most could easily be managed by the N -body integrator. This allows `multiples` to detect wide binaries formed by the N -body code, allowing us to model binary formation as well as binary interactions.

Code example 8.17 illustrates a simple `multiples` integration. Basically, we set up the system and tell `multiples` what N -body, `kepler`, and small- N codes to use and (since `multiples` is written in N -body units) optionally specify a converter). From that point on, we use `multiples` in exactly the same way as any other integrator—its `evolve_model` method invokes the integrator and takes appropriate action if encounters occur. The full script, including diagnostic and other functionality, can be found in `amuse.examples.basic_multiples`.

```

55 def integrate_system(number_of_stars, t_end, seed=None):
56
57     total_mass = number_of_stars | units.MSun
58     length = 1 | units.parsec
59     converter = nbody_system.nbody_to_si(total_mass, length)
60     gravity = ph4(convert_nbody=converter)
61     gravity.initialize_code()
62     gravity.parameters.set_defaults()
63     gravity.parameters.epsilon_squared = (0.0 | units.parsec)**2
64
65     if seed is not None:
66         numpy.random.seed(seed)
67     stars = new_plummer_model(
68         number_of_stars, convert_nbody=converter)
69     stars.mass = total_mass / number_of_stars
70     stars.scale_to_standard(
71         convert_nbody=converter,
72         smoothing_length_squared=gravity.parameters.epsilon_squared
73     )
74     index = numpy.arange(number_of_stars)
75     stars.id = index+1
76     stars.radius = (0.5 / number_of_stars) | units.parsec

```

```

77 gravity.particles.add_particles(stars)
78
79
80 stopping_condition = gravity.stopping_conditions.collision_detection
81 stopping_condition.enable()
82
83 init_smalln(converter)
84 kep = Kepler(unit_converter=converter)
85 kep.initialize_code()
86 multiples_code = multiples.Multiples(gravity, new_smalln, kep,
87                                     constants.G)
88 multiples_code.neighbor_perturbation_limit = 0.05
89 multiples_code.global_debug = 1
110
111 energy_initial = print_diagnostics(multiples_code)
112 multiples_code.evolve_model(t_end)
113 print_diagnostics(multiples_code, energy_initial)

```

Code example 8.17: A simple N -body solver with close-encounter management. The full script can be found in `amuse.examples.basic.multiples`. It should run in a few seconds on a laptop.

Note that the stellar ID defined in the script is important, as it is used for internal bookkeeping within the module. The only slightly unusual aspect of the `multiples` setup is that, instead of providing the small- N module as an argument, the current module expects a function that resets and returns a small- N integrator (see Code example 8.18). This was part of the original design of the module, and is currently under review; it may be eliminated in future versions of AMUSE.

```

13 SMALLN = None
14 def init_smalln(converter):
15     global SMALLN
16     SMALLN = SmallN(convert_nbody=converter)
17 def new_smalln():
18     SMALLN.reset()
19     return SMALLN

```

Code example 8.18: Initializing the small- N part of the `multiples` solver.

Finally, the `multiples` database enables the implementation of treatments of binary evolution (Section 5.3), secular internal evolution of multiple structure (Section 4.4.4), e.g. by the von Zeipel-Lidov-Kozai (von Zeipel, 1910; Lidov, 1962; Kozai, 1962) mechanism, and external perturbations due to wide encounters in the N -body code. These are not yet fully implemented, and represent work in progress.

8.7.2 The Kira module: combining stellar and binary evolution with stellar dynamics

In 1992, with Piet Hut and Jun Makino, we began work on the `starlab` project (Portegies Zwart *et al.*, 1999). Central to this effort were two codes, the stellar evolution code `SeBa` and a 4th-order accelerated Hermite integrator not dissimilar to `ph4`. The combined code was known as `kira`. Many of the fundamental ideas we learned in creating `kira` have percolated into the AMUSE framework.

One aspect in which **kira** was ahead of AMUSE, as described thus far, was in its handling of binary evolution in conjunction with the dynamics. So far we have briefly discussed the evolution of binary stars in Section 5.3, but that was for binaries in isolation and not in a dynamical environment. Here we adopt the same fundamental structure for single stars and binary stars to simulate a cluster with primordial binaries. Binaries and higher-order multiple encounters are managed by the **encounter** module in **amuse**, and it is possible to ionize binaries or make new binaries by dynamical encounters. We call this script **kira.py** in recognition of the original package. It is available in **amuse.examples.kira**. We use the **multiples** module to treat close multiple stellar encounters (see Section 8.7.1). Like many of the more complex AMUSE scripts described in this book, this is a work in progress and likely to change.

There are a few additional details to consider when modeling a star cluster containing binaries (either primordial or dynamically formed). The small orbital periods of binaries tend to dramatically slow down the underlying N -body code, and the many small time steps required to resolve the binaries lead to an steady accumulation of round-off and integration errors. These factors mandate special methods to manage the binaries. A further complication is that binaries are affected both by the stellar evolution of the components and by occasional encounters with other stars (and binaries) in the system. The evolution of single stars can be handled by any of the stellar evolution codes available in AMUSE. Binary evolution, however, is a different matter.

The initial conditions for the simple simulation described here begin with a Plummer sphere of 100 particles with masses randomly selected from a Salpeter mass function between $0.1 M_{\odot}$ and $100 M_{\odot}$. We then select **Nbin=50** stars randomly using

```
binaries = center_of_masses.random_sample(Nbin)
```

These stars are then provided with a secondary companion, with orbital parameters chosen from standard distributions. The mass of the secondary is selected randomly from a uniform distribution between the the minimum stellar mass and that of the primary. The orbital eccentricity is selected between 0 and 1, assuming a thermal distribution ($p(e) = 2e$; Duquennoy & Mayor, 1991), and the semimajor axis is selected using the prescription in Eq. 1.6.3 of Eggleton (2006), with a period between 10 days and 100 years. Other binary orbital parameters are initialized in much the same way as described in Section 5.3.

With stars and binaries initialized, we can start the stellar evolution code and the global gravity solver. We use **SeBa** to resolve the stellar evolution because it has a natural way to handle the evolution of single stars as well as binaries. For the global dynamical evolution of the stars we use **Hermite**. Stable binaries and close 2-body encounters are not integrated, but we follow their dynamical evolution via the **Kepler** package. The global N -body code integrates the center of mass of the close two-body encounters as well as of the center of masses of the stable binary systems. Once another star perturbs a binary (or higher order system) we resolve its interaction using the **multiples** module (Section 8.7.1). The **small Ncode** it relies on is based on the old **Kira_smallN** package from **starlab**.

We start by importing the **encounters** module, which is a general-purpose module designed to handle stellar encounters of all types (collisions, multiple encounters, etc.)

and is still under development

```
from amuse.couple import encounters
```

We then realize `encounters_code` using the `kepler` package to handle stable two-body systems, and `smallN` for multi-body systems

```
encounter_code = encounters.HandleEncounter(
    kepler_code = new_kepler(converter),
    resolve_collision_code = new_smalln(converter)
)
```

Here we use helper functions `new_kepler` and `new_smalln`, which return instantiations of the two codes with common parameters (such as `timestep_parameter` in `smallN`) set to reasonable values. We now start multiples handling using `Multiples`, which is part of the `encounters` module. We give it a global gravity code and the `encounter_code` we just created using the `kepler` and `smallN` modules:

```
multiples_code = encounters.Multiples(
    gravity_code = Hermite(converter),
    handle_encounter_code = encounter_code
)
```

Multiple encounters will now be resolved using `smallN`, whereas two-body stable binaries are addressed using the `kepler` solver, as described in Section 8.7.1.

Having set up all code dependencies, it is time to copy all stellar and binary data to the multiples module and declare channels to the code:

```
multiples_code.particles.add_particles((stars-binary_stars).copy())
multiples_code.singles_in_binaries.add_particles(singles_in_binaries)
multiples_code.binaries.add_particles(binary_stars)
multiples_code.commit_particles()
channel_from_stars_to_particles = (
    stellar.particles.new_channel_to(multiples_code.particles)
)
```

The `multiples` module now takes care of all two-body and higher-order interactions in the star cluster simulation.

After an encounter, binary parameters may have changed. This affects the stellar and binary evolution of these systems, and the stellar evolution code must be informed of these changes. We do this via a stopping condition, in this case `binaries_change_detection`, which stops the `multiples` module when the orbital separation or eccentricity of a binary is significantly affected by the dynamical encounter:

```
stopping_condition \
    = multiples_code.stopping_conditions.binaries_change_detection
stopping_condition.enable()
```

We run the `multiples` module in a (by now) familiar loop:

```

while time < time_end:
    time += time_step
    multiples_code.evolve_model(time)

    if stopping_condition.is_set():
        resolve_changed_binaries(stopping_condition, stellar)

    stellar.evolve_model(ti)
    channel_from_stars_to_particles.copy_attributes(["mass", "radius"])
    update_dynamical_binaries_from_stellar()

```

Newly formed binaries, binaries with orbital elements affected by a dynamical encounter, and binaries that broke up (and must be removed) are handled by the code in Code example 8.19. (Note that we encountered the routine `calculate_orbital_elements` previously in Section 4.4.1.)

```

134
135
136 def resolve_changed_binaries(stopping_condition, stellar, converter):
137     new_binaries = stopping_condition.particles(0)
138     for bi in new_binaries:
139         print(("add new binary:", bi))
140         a, e = calculate_orbital_elements(bi, converter)
141         bi.semi_major_axis = a
142         bi.eccentricity = e
143         stellar.binaries.add_particle(bi)
144         print(("new binary parameters", a, e))
145         print(bi)
146
147     lost_binaries = stopping_condition.particles(1)
148     for bi in lost_binaries:
149         print(("remove old binary:", bi.key))
150         stellar.binaries.remove_particle(bi)
151
152     changed_binaries = stopping_condition.particles(2)
153     for bi in changed_binaries:
154         bs = bi.as_particle_in_set(stellar.binaries)
155         a, e = calculate_orbital_elements(bi, converter)
156         bs.semi_major_axis = a
157         bs.eccentricity = e
158         print(("Modified binary parameters", a, e))
159         print(bs)

```

Code example 8.19: Morphing binaries in `kira.py`.

Similarly, after the stellar evolution step we must update the dynamical information in the binaries. This is realized by reading the binary parameters in the stellar evolution code, calculating the new coordinates of the binary components, and updating the dynamics. Special care is required for binaries that merged during the evolution process, or that were disrupted in a supernova explosion, but those can be handled with separate stopping conditions for `supernova_detection` and `merger_detection` in the stellar evolution module.

```
def resolve_supernova(stellar, stars, multiples_code):
    singles = stars.as_particle_in_set(multiples_code.singles)
    singles_in_stellar = singles.as_particle_in_set(stars)
    singles.velocity += singles_in_stellar.supernova_kick_velocity
```

Figure 8.13 presents the results of a small test simulation using the script `kira` from the `amuse.examples` module. The initial conditions for this calculations are a standard star cluster with 100 stars drawn from a [Salpeter \(1955\)](#) mass function with a minimum mass of $10 M_{\odot}$, distributed in a [Plummer \(1911\)](#) sphere with a characteristic radius of 1 pc. Half of the primary stars are given a secondary companion with mass chosen randomly between $10 M_{\odot}$ and the primary mass, in a orbit with eccentricity taken from a thermal distribution and semimajor axis drawn from the distribution proposed by [Eggleton \(2006\)](#). The other binary parameters and the binaries' orientations in space are random. We integrate the equations of motion using the `Hermite` integrator [see Section 4.3.2; binary evolution is managed using `SeBa` ([Portegies Zwart & Verbunt, 1996](#); [Portegies Zwart & Yungelson, 1998](#); [Toonen et al., 2012](#); [Portegies Zwart & Verbunt, 2012](#))].

Figure 8.13 shows the distributions of orbital semimajor axis and eccentricity of the binaries after integrating for 10 Myr. We integrate for such a short time because that is already sufficient to see differences compared to a calculation without stellar and binary evolution.

The blue bullets in Figure 8.13 indicate the initial conditions (orbital separation and eccentricity) of the primordial binaries in the simulation. The red dots show the orbital parameters after 10 Myr of integration using `kira.py`. For several of the closest binaries the orbital parameters have already been affected by stellar evolution and tidal interactions within the binary. Most of these are affected by tidal circularization, particularly those with short orbital periods and eccentricities $\gtrsim 0.6$. Some of the tightest ($a \lesssim 0.1 R_{\odot}$) circular binaries have disappeared, as they were Roche-lobe overflowing at birth and have merged to form a single star.

8.7.3 The LonelyPlanets module: combining planetary with stellar dynamics

Instead of integrating multiple stars in a cluster, one might be interested in the multi-scale problem of integrating planetary systems. Integrating a single isolated planetary system is usually done with a regular N -body code, but a planetary system in a star cluster is quite challenging, because of the wide range of time scales involved. One way to perform such a calculation is with a divide-and-conquer strategy.

For this purpose we developed `LonelyPlanets` (not to be confused with the popular travel guide books, which deal with one specific rather peculiar planet). We mean to integrate multi-planet systems, possibly with moons, asteroids, and comets together with the dynamical evolution of the stellar cluster.

`LonelyPlanets` works through a two-stage process. In a first simulation, we select a star (in our example randomly), and monitor its most prominent (i.e. strongest) perturbers during the integration of the cluster. The event loop for `LonelyPlanets` is

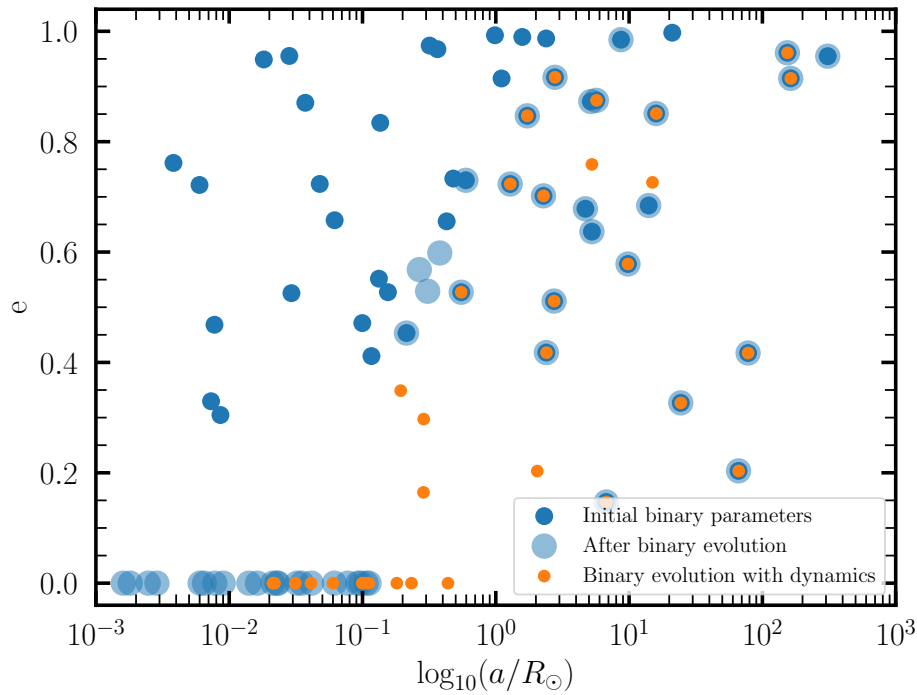


Figure 8.13: Distribution in semimajor axis and eccentricity of a population of binary stars in an N -body system at $t = 0$ Myr (dark blue bullets) and at an age of 10 Myr (big blue and orange bullets). The big blue bullets indicate what happens in stellar evolution but without taking the dynamical evolution into account. The orange bullets show the same binary population but they are affected by stellar evolution, binary evolution and by stellar dynamics. We deliberately plot the blue dots first and larger than the red dots to illustrate how some of the binaries disappear or move within the parameter space. The script that produced this figure is `amuse.examples.kira`. It should run in about a minute on a laptop.

listed in Code example 8.20, and should look quite familiar by now, with the exception that we keep track of the perturbers. In this example, the perturber is any of the 5 nearest stars. In practice, one probably wants to keep track of the strongest perturber, rather than the nearest star, as a massive far away star can perturb a planetary system more severely than a low-mass nearby star.

We find the nearest neighbors simply by sorting stars on the distance to our selected star.

```
def nearest_neighbors(bodies, star, Nnn=3):
    r = ((bodies-star).position-star.position).lengths().value_in(units.pc)
    r, p = (list(t) for t in zip(*sorted(zip(r, bodies-star))))
    b = Particles()
    for pi in p[:Nnn]:
        b.add_particle(pi)
    return b
```

In the second stage, we generate a planetary system around the selected star and integrate it together with the earlier (in stage 1) generated database of perturbers. We further optimize the stage 2 calculation (as illustrated in Code example 8.21) by including the perturbing stars only if their perturbations on the planetary system exceed some pre-determined limit. In our case, we adopt a simple distance criterion of 3000 au.


```

173 while gravity.model_time < time_end:
174     gravity.evolve_model(gravity.model_time + dt)
175     channel_from_gd.copy()
176
177     star.age = gravity.model_time
178
179     target_star.append(star.copy())
180     model_time.append(gravity.model_time)
181     perturbers = find_perturbers(star, bodies, Nnn)
182     nn_distance[str(model_time[-1])] = (
183         star.position - perturbers.position
184     ).lengths()
185     nn_bodies.append(perturbers)
186     print(f"time={model_time[-1].in_(units.Myr)}, x={star.x.in_(units.pc)}")
187

```

Code example 8.20: Snippet of the stage 1 `LonleyPlanets` event loop.

This saves a little computer time, but with only 5 perturbers and 8 planets the gain is rather small.

This is a typical example of optimizing for a rather minor speed-up while taking the risk of creating a complex bug. Fixing the bug may take more time than just integrating for all neighboring stars.

In Figure 8.14 we present the result of the stage 1 calculation of `LonleyPlanets`, where we initialize 1000 stars from a Kroupa mass function (between $0.08 M_{\odot}$ and $30 M_{\odot}$) in a virialized King model (with $W_0 = 5$) 1 pc radius. We randomly selected one star (which happened to have a mass of $\sim 0.11 M_{\odot}$ and was born in the cluster central region [at $(-0.026, -0.072, -0.14)$ pc from the center]. This star acquired planets between 1 au and 100 au in circular orbits in the $x - y$ plane.

In the second stage of the calculation, we integrate the planetary system together with the most strongly perturbing stars. We select the 3 nearest neighbors and the 3 strongest perturbers. Sometimes these are the same stars, but sometimes a low-mass star happens to be near the planetary system, while other, more massive, stars are stronger perturbers. In Figure 8.15 we present the orbital separation and eccentricity of the 15 planets for the first 3 Myr of the evolution of the cluster. Although it is something of a jumble of lines, it is clear that the planet eccentricities rapidly increase with time, and only 4 planets survive until an age of 3 Myr, when we stop the calculation.

It is relatively straightforward to include other effects in to the `LonleyPlanets` scripts, such as test particles (representing asteroids or comets), stellar evolution (representing the cluster stars, but also the host stars of the planetary systems), etc. The method outlined here is a workable test version meant for integrating a single planetary system (composed of some 30 planets) in a cluster composed of 1000 stars among which the planetary system.

Integrating the planetary system requires $\mathcal{O}[(1 + N_p + N_{cc})^2]$ operations per time step. Here $f \leq 1$ indicates the fraction of time steps during which the planetary system is perturbed by the neighboring stars (the gray shaded areas in Figure 8.15). In the simplest case, $f \equiv 1$, indicating that the planetary system is always perturbed, irrespective of the mass or distance to the nearest perturber. This may be the case

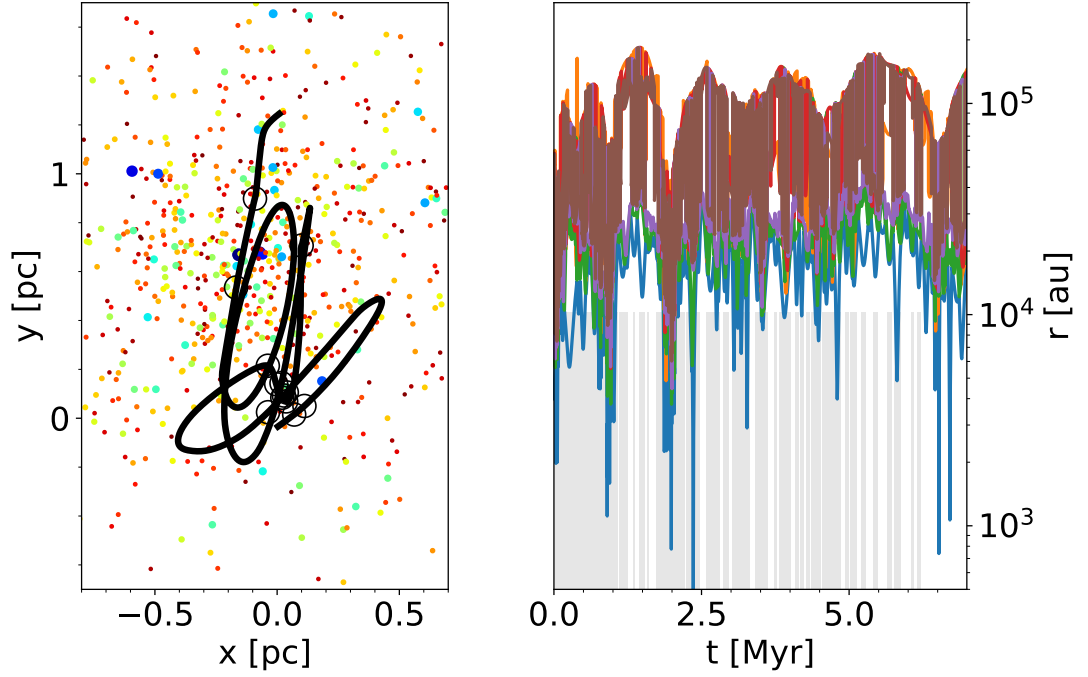


Figure 8.14: First stage calculation for *LonelyPlanets*. Strategy to reduce computing cost without loss of accuracy. It is a two stage process. In the first step (left), we calculate the evolution of the cluster, storing all the close encounter information. The points indicate the initial stellar positions (color and size represent mass). The orbit of a randomly selected star is presented as the solid black curve. The right panel shows over the 3Myr of the evolution in this cluster the distance to the 3 nearest stars from the selected star. In the second step of the calculation, we repeat the close encounters of our selected star with the nearest other stars, but then including planets and asteroids. Left: A representation of the cluster at zero-age, with the selected star's orbit presented as a solid black curve. While integrating, we keep track of the strongest perturbers, and each time such a perturbation exceeds the equivalent of an encounter at around 5000 au a small circle is drawn. Right: The distance from our selected stars to the three strongest perturbers, as a function of time. The identify of these perturbers changes regularly, once their perturbation is overtaken by that of another (more strongly perturbing) star. The vertical dotted lines indicate the moments of strongest perturbation (and identify to the circles in the left-hand panel).

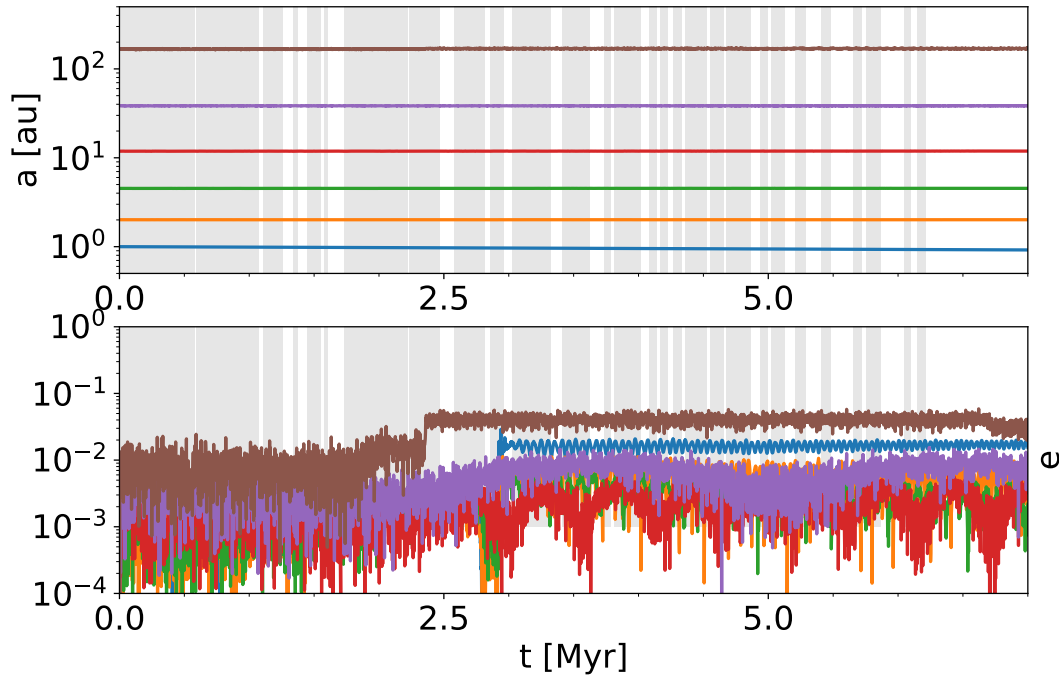
```

286     for ti in range(len(model_time)):
287         star = planetary_system[planetary_system.type == "star"][0]
288         planetary_system.position -= star.position
289         planetary_system.velocity -= star.velocity
290         planetary_system.position += ss_star[ti].position
291         planetary_system.velocity += ss_star[ti].velocity
292         channel_to_gd.copy()
293
294         perturbed = nearest_encounter(
295             ss_star[ti], nn_bodies[ti], maximum_perturber_distance)
296         if perturbed:
297             npert += 1
298             print(f"Integrate with stars: N={len(nn_bodies[ti])} "
299                   f"at time={model_time[ti].value_in(units.Myr)}")
300             gravity.particles.add_particles(nn_bodies[ti])
301         else:
302             nunpert += 1
303             print(f"Time={model_time[ti].in_(units.Myr)}: "
304                   f"N= {len(gravity.particles)} = "
305                   f"{len(planetary_system)}+{len(nn_bodies[ti])}")
306             gravity.evolve_model(model_time[ti])
307
308         if perturbed:
309             gravity.particles.remove_particles(nn_bodies[ti])
310             channel_from_gd.copy()
311
312         star = planetary_system[planetary_system.type == "star"][0]
313         planets = planetary_system - star
314         get_orbital_elements_of_planetary_system(star, planets)
315         amin = (np.min(planets.semimajor_axis.value_in(units.au)) | units.au,)
316         sma.append(planets.semimajor_axis)
317         ecc.append(planets.eccentricity)
318
319         lost_planets = planets[planets.eccentricity > 1]
320         if len(lost_planets) > 0:
321             lp = lost_planets.get_intersecting_subset_in(gravity.particles)
322             gravity.particles.remove_particles(lp)
323         lost_planets = planets[np.isnan(planets.eccentricity)]
324         if len(lost_planets) > 0:
325             lp = lost_planets.get_intersecting_subset_in(gravity.particles)
326             gravity.particles.remove_particles(lp)
327

```

Code example 8.21: Stage 2 *LonelyPlanets* event loop, in which the planetary system is being integrated with nearby stars.

so long the planetary system is a cluster member, but as soon as the star escapes the cluster, $f \rightarrow 0$. This is nicely illustrated in Figure 8.15, where the last strong encounter with the planetary system, at an age of about 5.7 Myr leads to a perturbation in the outermost planet, (see the hiccup in the brown curve in Figure 8.15 at the same time) and causes the star to escape the cluster. The further integration of the planetary system is achieved by a standalone symplectic integrator. We can still include the Galactic potential, in case we are interested in the evolution of the Oort cloud or

Figure 8.15: Second stage calculation for `LonelyPlanets`.

asteroids along in the star’s Galactic orbit.

8.8 Validation

Compared to other coupling strategies, problems with linear coupling are relatively easy to validate, because each of the individual parts can to some degree be validated separately, as described in previous chapters. Energy is generally not strictly conserved in coupled problems, and neither is momentum, but wherever conservation laws do apply it is important to check them. Mass, number of objects, energy, and angular momentum are generally conserved in specific sub-problems, but not throughout the simulation.

A simple validity check for conserving mass during stellar evolution and energy during stellar dynamics is

```

mass = stellar.particles.mass.sum()
stellar.evolve_model(time)
channel_stellar_to_framework.copy()
lost_mass = mass - stars.mass.sum()

channel_framework_to_gravity.copy()
Etot = gravity.kinetic_energy + gravity.potential_energy
gravity.evolve_model(time)
dE = Etot - (gravity.kinetic_energy + gravity.potential_energy)

```

Because mass is lost in stellar evolution, the kinetic and potential energies of the gravitational system are also affected. These can be checked independently, for example with

```
E_stellar = gravity.kinetic_energy + gravity.potential_energy
stellar.evolve_model(time)
channel_stellar_to_gravity.copy()
Etot = gravity.kinetic_energy + gravity.potential_energy
dE_stellar = E_stellar - Etot

gravity.evolve_model(time)
dE_gravity = Etot - (gravity.kinetic_energy + gravity.potential_energy)
```

Printing `dE_gravity` then provides a diagnostic on the quality of the simulation results. Similar strategies can be applied in hydrodynamics, and radiative transfer, as we will see in the next two chapters. We will experience similar issues in Chapter 9, and we postpone further discussion until there.

8.9 Assignments

8.9.1 Interlaced timestepping

In Section 8.4.5 we discussed the evolution of the Wolf-Rayet colliding wind triple system θ Muscae, in which the WC5/6 star WR48 is in a 19 day orbit with an O6 companion, and both are orbited by a distant and relatively low mass O9.5/B0Iab star. In the example presented in Section 8.4.5, stellar evolution and the dynamical evolution of the triple were applied at very short and constant time intervals of only one-tenth of the inner orbital period. It is not clear if such a small time step is actually needed to recover the correct physics. In this assignment we will study this.

In Section 8.4.4 we discussed some alternative time-stepping strategies, but there are other ways to do this. Implement an alternative time stepping scheme by allowing the time step to depend on the rate of mass loss, not allowing the total mass loss during the step to exceed a given value. If the time step becomes too large, the integrator will produce incorrect results, but for time steps that are too small the integration will take too long. Use the interlaced temporal discretization method described in Section 8.4.4 to study if this is really a desirable way of integrating two numerical solvers.

Run a series of simulations, systematically halving the time step from one to the next, and record the semimajor axis and eccentricity of the inner and the outer binary and their relative inclination. Make a plot of the orbital parameters and the wall-clock computer time as a function of the time step size. Based on these curves, can you decide what is the best time step size for simulation a triple system like θ Muscae?

- How does the optimum time step depend on the mass loss rate in the star with the strongest wind?
- How does optimum time step depend on the initial relative inclination between the inner and the outer orbit?

8.9.2 Stellar evolution and dynamics

Generate a Plummer sphere of $N = 1000$ stars with a Salpeter stellar mass function between $M_{\min} = 0.1 M_{\odot}$ and $M_{\max} = 100 M_{\odot}$ (power-law slope $\alpha = -2.3$). Evolve the system using an N -body integrator of your choice, including stellar evolution, and measure the moment when the cluster has lost half of its initial mass. Take, for example, the simple N -body code in Code example 4.1 as a starting point, and expand it with the stellar evolution recipes from Code example 8.1.

Now change the power-law slope from $\alpha = -2.3$ (Salpeter) to $\alpha = -2.0$ and repeat the measurement. Continue this process with increasing slope until the initial mass function is flat. What can you conclude when you plot the time for the cluster to lose half of its initial mass as a function of the slope of the initial mass function?

- Why do clusters with relatively flat mass functions dissolve more quickly than those with a steep mass function?
- What changes when stellar evolution is switched off?
- Why is it not possible to continue the simulation to complete dissolution with a Salpeter mass function without stellar evolution?

8.9.3 Multiple stellar populations

Globular clusters had long been thought to contain single stellar populations—all stars formed at the same time with the same metallicity—but recent discoveries challenge this view. In particular, the discovery of numerous globular clusters [e.g. ω Centauri (Lee *et al.*, 1999), and NGC2808 (Piotto *et al.*, 2007)] with two or more distinct main sequence populations has been puzzling astronomers for at least two decades. One question that keeps cropping up is the rate at which two (or more) distinct stellar populations mix dynamically when their birth distributions differ. In this experiment, we will study this phenomenon by initializing a star cluster with two distinct stellar populations. These might be two populations with different metallicities, or with different ages.

Generate a first population of 500 stars by selecting masses from a Salpeter (or any other) mass distribution. Evolve the population to an age of 1 Gyr. Select positions and velocities for these stars from a Plummer model with a virial radius of 1 pc, and scale the star cluster to virial equilibrium. Now generate a second stellar population at zero age and using the same (Salpeter) initial mass function, but now using a virialized Plummer sphere with a virial radius of 3 pc for the stars. Before starting the N -body integrator you will need to merge the two stellar distributions and scale the whole system to virial equilibrium.

Evolving the two populations dynamically can be done in the same N -body integrator. Use two stellar evolution codes for the two populations. Evolve the entire cluster for 1 Gyr and study how the individual Lagrangian radii of the two populations, and of the cluster as a whole, evolve with time from the start of the N -body integration.

Bibliography

- Aarseth, S. A. 2003. *Gravitational N-body simulations*. Cambridge University press, 2003.
- Breslau, Andreas, Steinhausen, Manuel, Vincke, Kirsten, & Pfalzner, Susanne. 2014. Sizes of protoplanetary discs after star-disc encounters. *A&A* , **565**(May), A130.
- Carathéodory, C. 1909. Untersuchungen über die Grundlagen der Thermodynamik. *Mathematische Annalen*, **67**, 355–386.
- Duquennoy, A., & Mayor, M. 1991. Multiplicity among Solar Type Stars in the Solar Neighbourhood - Part Two - Distribution of the Orbital Elements in an Unbiased Sample. *A&A* , **248**(Aug.), 485.
- Eggleton, Peter. 2006. *Evolutionary Processes in Binary and Multiple Stars*.
- Eisenstein, Daniel J., & Hut, Piet. 1998. HOP: A New Group-Finding Algorithm for N-Body Simulations. *ApJ* , **498**(1), 137–142.
- Gaburov, E., Lombardi, J. C., & Portegies Zwart, S. 2008. Mixing in massive stellar mergers. *MNRAS* , **383**(1), L5–L9.
- Goodwin, S. P., & Whitworth, A. P. 2004. The dynamical evolution of fractal star clusters: The survival of substructure. *A&A* , **413**(Jan.), 929–937.
- Hill, G. M., Moffat, A. F. J., & St-Louis, N. 2002. Modelling the colliding-winds spectra of the 19-d WR + OB binary in the massive triple system θ Muscae. *MNRAS* , **335**(4), 1069–1078.
- Hut, Piet. 1993. Binary–Single-Star Scattering. III. Numerical Experiments for Equal-Mass Hard Binaries. *ApJ* , **403**(Jan.), 256.
- Hut, Piet, Makino, Jun, & McMillan, Steve. 1995. Building a Better Leapfrog. *ApJL* , **443**(Apr.), L93.
- Jänes, Jürgen, Pelupessy, Inti, & Portegies Zwart, Simon. 2014. A connected component-based method for efficiently integrating multi-scale N-body systems. *A&A* , **570**(Oct.), A20.
- King, Ivan R. 1966. The structure of star clusters. III. Some simple dynamical models. *AJ* , **71**(Feb.), 64.
- Kolmogorov, A. N. 1933. Sulla Determinazione Empirica di Una Legge di Distribuzione. *Giornale dell'Istituto Italiano degli Attuari*, **4**. 83–91. 1933., **4**.
- Kozai, Yoshihide. 1962. Secular perturbations of asteroids with high inclination and eccentricity. *AJ* , **67**(Nov.), 591–598.
- Kroupa, Pavel. 2001. On the variation of the initial mass function. *MNRAS* , **322**(2), 231–246.
- Lee, Y. W., Joo, J. M., Sohn, Y. J., Rey, S. C., Lee, H. C., & Walker, A. R. 1999. Multiple stellar populations in the globular cluster ω Centauri as tracers of a merger event. *Nat* , **402**(6757), 55–57.
- Lidov, M. L. 1962. The evolution of orbits of artificial satellites of planets under the action of gravitational perturbations of external bodies. *Planet. Space Sci.*, **9**(10), 719–759.
- Lombardi, James, C. Jr., Rasio, Frederic A., & Shapiro, Stuart L. 1995. On Blue Straggler Formation by Direct Collisions of Main-Sequence Stars. *ApJL* , **445**(June), L117.

- Lombardi, J. C., Thrall, A. P., Deneva, J. S., Fleming, S. W., & Grabowski, P. E. 2003. Modelling collision products of triple-star mergers. *MNRAS* , **345**(3), 762–780.
- Lombardi, James, & Sawyer Warren, Jessica. 2014 (Dec.). *MMAS: Make Me A Star*. Astrophysics Source Code Library, record ascl:1412.010.
- Lombardi, James C. Jr., Rasio, Frederic A., & Shapiro, Stuart L. 1996. Collisions of Main-Sequence Stars and the Formation of Blue Stragglers in Globular Clusters. *ApJ* , **468**(Sept.), 797.
- Lombardi, James C. Jr., Warren, Jessica S., Rasio, Frederic A., Sills, Alison, & Warren, Aaron R. 2002. Stellar Collisions and the Interior Structure of Blue Stragglers. *ApJ* , **568**(2), 939–953.
- MacPherson, G. J., Davis, A. M., & Zinner, E. K. 1995. The distribution of aluminum-26 in the early Solar System - A reappraisal. *Meteoritics*, **30**(July), 365.
- Mardling, R. A. 2008 (May). Resonance, Chaos and Stability in the General Three-Body Problem. *Pages 199–208 of: Vesperini, Enrico, Giersz, Mirek, & Sills, Alison* (eds), *Dynamical Evolution of Dense Stellar Systems*, vol. 246.
- McMillan, Stephen L. W., & Hut, Piet. 1996. Binary–Single-Star Scattering. VI. Automatic Determination of Interaction Cross Sections. *ApJ* , **467**(Aug.), 348.
- Mikkola, Seppo, & Merritt, David. 2008. Implementing Few-Body Algorithmic Regularization with Post-Newtonian Terms. *AJ* , **135**(6), 2398–2405.
- Piotto, G., Bedin, L. R., Anderson, J., King, I. R., Cassisi, S., Milone, A. P., Villanova, S., Pietrinferni, A., & Renzini, A. 2007. A Triple Main Sequence in the Globular Cluster NGC 2808. *ApJL* , **661**(1), L53–L56.
- Plummer, H. C. 1911. On the problem of distribution in globular star clusters. *MNRAS* , **71**(Mar.), 460–470.
- Portegies Zwart, S. F., & van den Heuvel, E. P. J. 2016. Was the nineteenth century giant eruption of Eta Carinae a merger event in a triple system? *MNRAS* , **456**(4), 3401–3412.
- Portegies Zwart, S. F., & Verbunt, F. 1996. Population synthesis of high-mass binaries. *A&A* , **309**(May), 179–196.
- Portegies Zwart, S. F., & Verbunt, F. 2012 (Jan.). *SeBa: Stellar and binary evolution*. Astrophysics Source Code Library, record ascl:1201.003.
- Portegies Zwart, S. F., & Yungelson, L. R. 1998. Formation and evolution of binary neutron stars. *A&A* , **332**(Apr.), 173–188.
- Portegies Zwart, S. F., Makino, J., McMillan, S. L. W., & Hut, P. 1999. Star cluster ecology. III. Runaway collisions in young compact star clusters. *A&A* , **348**(Aug.), 117–126.
- Salpeter, Edwin E. 1955. The Luminosity Function and Stellar Evolution. *ApJ* , **121**(Jan.), 161.
- Sanders, N. E., Soderberg, A. M., Gezari, S., Betancourt, M., Chornock, R., Berger, E., Foley, R. J., Challis, P., Drout, M., Kirshner, R. P., Lunnan, R., Marion, G. H., Margutti, R., McKinnon, R., Milisavljevic, D., Narayan, G., Rest, A., Kankare, E., Mattila, S., Smartt, S. J., Huber, M. E., Burgett, W. S., Draper, P. W., Hodapp, K. W., Kaiser, N., Kudritzki, R. P., Magnier, E. A., Metcalfe, N., Morgan, J. S., Price, P. A., Tonry, J. L., Wainscoat, R. J., & Waters, C. 2015. Toward Characterization of the Type IIP Supernova Progenitor Population: A Statistical Sample of

- Light Curves from Pan-STARRS1. *ApJ* , **799**(2), 208.
- Sills, Alison, & Lombardi, James C. Jr. 1997. The Importance of Realistic Starting Models for Hydrodynamic Simulations of Stellar Collisions. *ApJL* , **484**(1), L51–L54.
- Smirnov, N. N. 1948. Tables for estimating the goodness of fit of empirical distributions. *Annals of Mathematical Statistics*, **19**.
- Sugawara, Y., Tsuboi, Y., & Maeda, Y. 2008. Redshifted emission lines and radiative recombination continuum from the Wolf-Rayet binary θ Muscae: evidence for a triplet system? *A&A* , **490**(1), 259–264.
- Tanikawa, A., & Fukushige, T. 2009. Effects of Hardness of Primordial Binaries on the Evolution of Star Clusters. *Publ. Astr. Soc. Japan* , **61**(Aug.), 721–.
- Terlevich, Elena. 1987. Evolution of N-body open clusters. *MNRAS* , **224**(Jan.), 193–225.
- Toonen, S., Nelemans, G., & Portegies Zwart, S. 2012. Supernova Type Ia progenitors from merging double white dwarfs. Using a new population synthesis model. *A&A* , **546**(Oct.), A70.
- van der Helm, M.I. Saladino, T. P. G., Pelupessy, F. I., Portegies Zwart, S., & Pols, O. R. 2018. Simulating stellar wind in AMUSE. *Submitted to MNRAS*.
- Vicente, S. M., & Alves, J. 2005. Size distribution of circumstellar disks in the Trapezium cluster. *A&A* , **441**(1), 195–205.
- von Zeipel, H. 1910. Sur l’application des séries de M. Lindstedt à l’étude du mouvement des comètes périodiques. *Astronomische Nachrichten*, **183**(22), 345.
- White, Simon D. M. 1994. Formation and Evolution of Galaxies: Les Houches Lectures. *arXiv e-prints*, Oct., astro-ph/9410043.

Chapter 9

Advanced Coupling Strategies

All you really need to know for the moment is that the universe is a lot more complicated than you might think, even if you start from a position of thinking it's pretty damn complicated in the first place.

Douglas Adams

The full potential of AMUSE becomes clear when we couple codes to solve complex multiphysics or multiscale problems. In this chapter we discuss coupling issues and strategies, and present examples of coupled solutions. Computing requirements increase drastically when running multiphysics problems. As a result, some of the examples presented in this chapter may be quite challenging for a laptop or workstation. Non-linear coupling strategies lead to new applications and greatly increase the power of the framework. We are only at the brink of discovering the large number of possibilities and the complexities of the coupling strategies it enables. Many of these, however, may be challenging to validate. We discuss some basic and a few more advanced hierarchical coupling strategies.

9.1 Code-coupling Strategies

Often, two or more processes or physical phenomena are coupled in a single problem. Nature seems to have a good grasp on how to do this, but in the computer it is often far less clear how to achieve an accurate or efficient solution.

Many simulations entail combining physical domains with very different length and time scales. We have seen numerous examples in the preceding chapters. In Section 8.4.1, we introduced an interlaced temporal discretization scheme as a way to couple dynamics and stellar evolution. So far, we primarily symmetrized the interaction between two codes, on the general principle that resolving the effect of code *A* on the time step for code *B*, and vice versa, should produce more accurate results. That approach naturally led us to a drift–kick–drift scheme equivalent to the second-order predictor–corrector or Verlet method (Verlet, 1967; see Sections 4.2.3 and 9.1.1). We will see below that this approach represents much more than just a convenient symmetric coupling; in particular, it provides a symplectic coupling when applied to symplectic

codes (Section 4.2.3.2). We then generalize it to coupling an arbitrary number of codes, even when the modules involved are not symplectic nor even widely separated in scale.

Typically, when two systems interact, they either have comparable scales but different physics, or the same physics but different scales. For clarity, we start with a multi-scale system, in which we identify one or more macroscopic (parent) systems, and one or more microscopic (child) systems. We prefer the parent-child terminology, as it is less confusing in discussions.

A child system is characterised by smaller length and time scales compared to the parent system. The child system then dictates the time resolution (i.e., time step size) and the smallest resolvable distance in the problem. Child systems are numerically expensive because of their short time scales.

The parent system typically has larger length and longer time scales. The longer time scale generally allows for larger numerical time steps, making the parent systems relatively cheap compared to integrating the children. The cost per step, however, can be many times more expensive because of the large extent of the parent and the large number of objects. An example might be a stellar cluster with thousands of stars and a size scale on the order of a parsec. The typical time scale in such a system would be about a megayear. If the cluster is part of the Galaxy, with 100 billion stars, and a multi kpc size, the typical time scale is easily 100 times larger than the child system's.

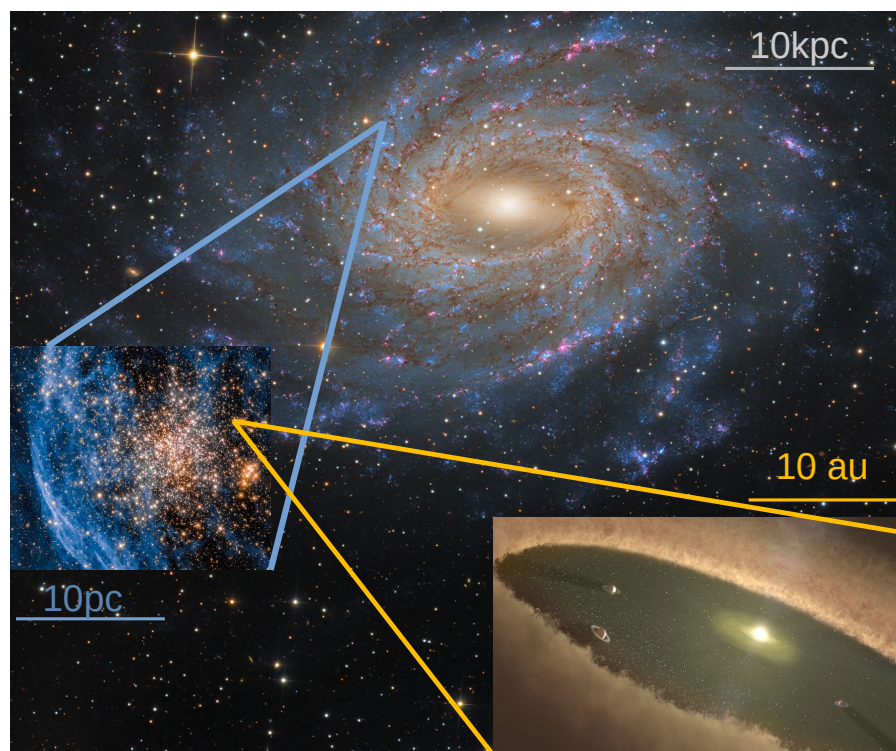


Figure 9.1: Hierarchy of scales illustrated with a hypothetical young planetary system (artist impression of HD 95086 by Spitzer Space Telescope, JPL, NASA, lower right) around a star in a young stellar cluster (illustrated with NGC 1860, NASA, ESA and P. Goudfrooij (STScI); Processing: M. H. Özşaraç (Türkiye Astronomi Dernei), in the galaxy NGC6744 (John Hayes). Images were taken from APOD <https://apod.nasa.gov/apod>.

In Figure 9.1 we illustrate the hierarchy of scales by presenting an artist’s conception of a planetary system, within a young star cluster in a spiral Galaxy. Although the images are real observed systems (the planetary system is an artist’s impression, though), they are not actually associated. The scales (small bars indicated in the image, represent 10 kpc, 10 pc, and 10 au, for the galaxy, the star cluster and the planetary system, respectively. The associated orbital time scales for these systems will be on the order of 100 Myr, 1 Myr, and 100 yr. The Galaxy may contain some 10^{11} stars, the cluster 10^4 stars, and the planetary system several planets.

One scale often dominates in computational expense, memory requirements, communication bandwidth, and storage space. The key to a successful and rewarding multi-scale simulation is to balance these requirements. This scale discrepancy is often addressed by approximating either the parent or the child system, for example using a semi-analytic model. Replacing the child system with an approximate method is often referred to as subgrid physics (whether or not a grid actually exists). Replacing the parent system with an approximate method is often referred to as a background (field) method or external boundary conditions. An example of child subgrid physics is the incorporation of a `multiscales` solver (see Section 8.7.1) or a parameterized stellar evolution solver (see Section 5.2) into a gravitational N -body code. An example of a parent background method is approximating the Galactic tidal field by an analytic potential within which a star cluster evolves. Such approximations are inflexible and prevent feedback between child and parent. Our goal in this chapter is to provide a flexible and self-consistent way to realize feedback between the child and parent systems.

In many dynamical cases of interest, the spatial and temporal scales of the child and parent systems are well separated, and we can split the underlying Hamiltonian for the combined system into rapidly and slowly changing parts. Child and parent evolution can then be followed simultaneously by adopting a kick–drift–kick algorithm, allowing both scales to be integrated separately and the results combined into a consistent solution. If we now choose an accurate but expensive method for the child system and a cheap but less accurate method for the parent system, we may gain the ability to save computer time and tune the accuracy, depending on the scale at which we couple the solvers. The resulting *Bridge method* is flexible and dynamic, in the sense that we can control the degree of coupling between solvers, and if necessary, we can introduce a cascade of bridges to address hierarchical coupling, as we will demonstrate in the matrix of coupled codes in `venice` (see Section 10.1.3).

The method can be generalized, both to higher order and to a variety of physical solvers. In principle, the method can have any even order (the odd orders cancel out), but the second-order coupling scheme is sufficiently accurate for most applications.

9.1.1 The Babylonian integrator

In computational astrophysics, probably the most widely used symplectic integrator is the Verlet (leapfrog) method or Babylonian scheme described in Section 4.2.3. Although only of second order, its simplicity and long-term stability make it the method of choice for many applications. It is often implemented in “standard” symplectic form as a kick–drift–kick scheme:

$$\begin{aligned}
v' &= v_n + \frac{1}{2}a(x_n)\delta t \\
x_{n+1} &= x_n + v'\delta t \\
v_{n+1} &= v' + \frac{1}{2}a(x_{n+1})\delta t.
\end{aligned} \tag{9.1}$$

A moment's reflection reveals that this is equivalent to the second-order predictor–corrector implementation described earlier. We note that, in practice, the acceleration used in the first kick of the next step is the same as that used in the last kick of the current step (see Section 9.1.1), meaning that only one acceleration need be computed per step, a fact of great importance in large N -body calculations.

An alternative (also symplectic) drift–kick–drift scheme is

$$\begin{aligned}
x' &= x_n + \frac{1}{2}v_n\delta t \\
v_{n+1} &= v_n + a(x')\delta t \\
x_{n+1} &= x' + \frac{1}{2}v_{n+1}\delta t.
\end{aligned} \tag{9.2}$$

This approach is much less widely used than its kick–drift–kick cousin, mainly because of its poorer performance in applications where variable time steps are required (Springel, 2005).

Higher-order symplectic schemes also exist (e.g. Yoshida, 1990; Tricarico, 2012; Rein & Spiegel, 2015), and are sometimes employed in astrophysical contexts. The math for these methods goes beyond the scope of this book, but to be complete we explain the higher-order symplectic bridge in Appendix A.4. Finally, we note that time-reversibility in even a non-symplectic scheme is sufficient to ensure that the obtained solution stays close to the true solution and that the error in the energy does not drift. Hut *et al.* (1995) describe an iterative algorithm to make any integrator time-reversible; it is implemented in the AMUSE `smallN` module.

9.1.2 The Bridge Method

In the derivation of the original Bridge scheme, Fujii *et al.* (2007) consider a star cluster (subscript c , indices i and j , total number N_c) orbiting a parent galaxy (subscript g , indices k and l , total number N_g). The internal orbits of stars in the cluster are integrated using a fourth-order Hermite scheme (Makino, 1991) with direct summation of the interstellar gravitational forces. Interactions among the particles comprising the galaxy, and between galaxy particles and cluster stars, are computed using a hierarchical tree force evaluation method (Barnes & Hut, 1986).

To proceed, we need a little dynamics (and in this context, we admit that we exaggerated a little in Section 1.1.3). The Hamiltonian H of the full system can be divided into two parts:

$$H = H_A + H_B, \tag{9.3}$$

where

$$H_A = - \sum_{k < l} \frac{Gm_{gk}m_{gl}}{r_{kl}} - \sum_{k=1}^{N_g} \sum_{i=1}^{N_c} \frac{Gm_{gk}m_{ci}}{r_{ki}} \tag{9.4}$$

is the total gravitational potential energy of the galaxy particles, including their mutual potential energy with the cluster stars, and H_B is the total kinetic energy of the system plus the total internal potential energy of the cluster stars:

$$H_B = \sum_{k=1}^{N_g} \frac{p_{gk}^2}{2m_{gk}} + \sum_{i=1}^{N_c} \frac{p_{ci}^2}{2m_{ci}} - \sum_{i<j} \frac{Gm_{ci}m_{cj}}{r_{ij}}. \quad (9.5)$$

The quantity p is momentum, and r_{ij} is the distance between cluster stars (or galaxy particles) i and j .

Hamilton's equations for a particle can be written as

$$\frac{df}{dt} = D_H f, \quad (9.6)$$

where $f = (\mathbf{x}, \mathbf{p})$ is a six-dimensional phase-space vector representing the particle's dynamical state (position $\mathbf{x}$ and momentum $\mathbf{p} = m\mathbf{v}$), and the differential operator D_H is defined by

$$D_H g \equiv \{g, H\} = \frac{\partial g}{\partial x} \frac{\partial H}{\partial p} - \frac{\partial g}{\partial p} \frac{\partial H}{\partial x} \quad (9.7)$$

for any time-dependent quantity $g(t)$. Here, the syntax $\{.,.\}$ denotes a Poisson bracket (see [Landau & Lifshitz, 1969](#); Section 42). The formal solution to Equation (9.6) is

$$f(t) = e^{tD_H} f(0), \quad (9.8)$$

or, for a single time step,

$$f(t + \delta t) = e^{\delta t D_H} f(t), \quad (9.9)$$

This solution is of little use by itself, as we have no explicit way to evaluate it. However, writing $H = H_A + H_B$, and $D_H = D_A + D_B$, it is generally possible to derive an approximation to the exponential:

$$e^{\delta t (D_A + D_B)} = \prod_{i=1}^K e^{a_i \delta t D_A} e^{b_i \delta t D_B} + O(\delta t^{K+1}). \quad (9.10)$$

The approximate expression on the right side of Equation (9.10) is symplectic—that is, it preserves phase-space volume and hence is time-reversible (see Section 4.2.3.2 and Section 9.1.1)—and formulae for the coefficients a_i and b_i exist for all orders K , giving rise to a family of practical integration schemes known generically as symplectic integrators.

This all looks like rather scary mathematics, but the applications are really quite simple. For example, the second-order ($K = 2$) version of Equation (9.9) is

$$f(t + \delta t) = e^{\frac{1}{2}\delta t D_A} e^{\delta t D_B} e^{\frac{1}{2}\delta t D_A} f(t) \quad (9.11)$$

(that is, $a_1 = \frac{1}{2}$, $b_1 = 1$, $a_2 = \frac{1}{2}$, $b_2 = 0$). In a system in which H_A is the total potential energy and H_B is the total kinetic energy of some dynamical system, it is easily shown that $e^{a\tau D_A}$ advances only the velocity by time $a\tau$:

$$\mathbf{v}(t + a\tau) = \mathbf{v}(t) + a\tau \mathbf{a}(t) \quad (9.12)$$

(a “kick”), where $\mathbf{a}$ is acceleration, while $e^{b\tau D_B}$ advances only the position:

$$\mathbf{x}(t + b\tau) = \mathbf{x}(t) + b\tau \mathbf{v}(t) \quad (9.13)$$

(a “drift”). Thus, the combination of operations in this case is just the kick–drift–kick Verlet–leapfrog scheme discussed in Section 4.2.3.

Applied to the galaxy–cluster system, as discussed in more detail by Fujii *et al.* (2007), this scheme translates to the following algorithm (advancing from time “0” to time “1”):

1. Kick cluster star velocities with time step $\frac{1}{2}\delta t$ using accelerations due to the galaxy; kick galaxy particle velocities with time step $\frac{1}{2}\delta t$ using accelerations due to both galaxy and cluster:

$$\begin{aligned} \mathbf{v}'_c &= \mathbf{v}_{c0} + \frac{1}{2}\delta t \mathbf{a}_{g \rightarrow c0} \\ \mathbf{v}'_g &= \mathbf{v}_{g0} + \frac{1}{2}\delta t \mathbf{a}_{\text{all} \rightarrow g0}. \end{aligned}$$

2. Update (drift) cluster positions and kicked velocities with time step δt , using the internal cluster integration scheme and kicked velocities; update galaxy particle positions with time step δt using kicked velocities:

$$\begin{aligned} \mathbf{x}_{c0} &\rightarrow (\text{high-order scheme}) \rightarrow \mathbf{x}_{c1} \\ \mathbf{v}'_c &\rightarrow (\text{high-order scheme}) \rightarrow \mathbf{v}'_{c1} \\ \mathbf{x}_{g1} &= \mathbf{x}_{g0} + \delta t \mathbf{v}'_g. \end{aligned}$$

3. Recompute accelerations and complete the step by updating (kicking) all cluster and galaxy velocities:

$$\begin{aligned} \mathbf{v}_{c1} &= \mathbf{v}'_{c1} + \frac{1}{2}\delta t \mathbf{a}_{g \rightarrow c1} \\ \mathbf{v}_{g1} &= \mathbf{v}'_g + \frac{1}{2}\delta t \mathbf{a}_{\text{all} \rightarrow g1} \end{aligned}$$

This basic scheme carries over to the generalized Bridge class in AMUSE. Note that the overall scheme will be symplectic if and only if the higher-order scheme is symplectic, but even if it is not, the interlaced symmetry of the algorithm still guarantees second-order accurate results.

9.1.3 Implementation of Bridge

The Bridge scheme can be written in Python as presented in the code example below. The main `evolve_model` method has a loop over time in which three tasks are performed; the system is kicked for half a time step, drifted for the full-time step, then kicked again (we saw a similar approach in Section 8.4.2). The source code is shown

below.

```
def evolve_model(time, time_end, time_step):
    first = True
    while self.time < (time_end-time_step/2):
        if first:
            kick(system, partners, time_step/2)
            first = False
        else:
            kick(system, partners, time_step)
            drift(system, time+time_step)
            time = time+time_step
    if not first:
        kick(system, partners, time_step/2)
    return 0
```

The kick operator is composed of two steps. In the first loop, each system is synchronized, then kicked with the function `get_gravity_at_point()` function in the second loop:

```
def kick(systems, partners, time_step):
    for subsystem in systems:
        if do_sync[subsystem]:
            subsystem.synchronize_model()
    for subsystem in systems:
        for y in partners[subsystem]:
            if subsystem is not y:
                kick(subsystem, y.get_gravity_at_point, time_step)
    return 0
```

Code example 9.1 gives an example of `get_gravity_at_point` for a static background potential.

The drift operator evolves each subsystem for the appropriate time step `time_step`:

```
def drift(system, time_end):
    for subsystem in systems:
        time_step = time_end - subsystem.model_time
        subsystem.evolve_model(time_step)
    return 0
```

9.1.4 Determining the bridge time steps

One of the key elements and uncertainties in bridge is the time step. Considerable research went into methods to automatically determine the bridge time step. It is not some parameter easily set and forgotten about; it is important to carefully choose its proper value.

Over the last several years we have experimented with auto-tuning bridge time steps, but the successes have been marginal. One method entails reinforcement learning, in which a trainable algorithm makes its own decisions on the time step size, considering the computational expense (wall clock time) and energy error produced by the integration ([Saz Ulibarrena & Portegies Zwart, 2025](#)).

In Figure 9.2 we present part of the analysis as explored in [Saz Ulibarrena & Portegies Zwart \(2025\)](#), in which a reinforcement algorithm was used to find the optimum bridge time step. The two axes represent integration (wall-clock) time and energy error. The curve shows the trade-off between the two if we reduce the bridge time step.

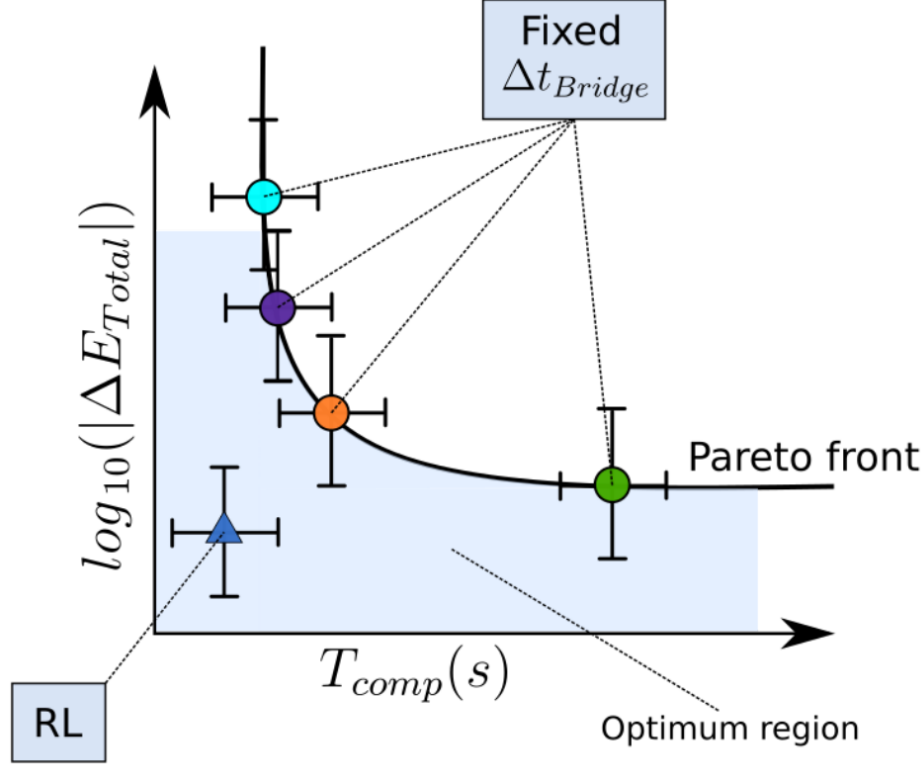


Figure 9.2: Pareto curve for the bridge time-step when tuned using reinforcement learning. The bullet points represent fixed bridge time steps, whereas the reinforcement algorithm allows bridge time steps to change with time.

The automated routine to determine the bridge time step, using reinforcement learning, is interesting and in principle powerful. If fully operational, it would have two major advantages: 1) it would not require any expert knowledge to choose a bridge time step, and 2) the calculation would be faster with a smaller overall energy error. In practice, however, it still turns to be hard to find an optimal choice. For now, we are not really encouraging using reinforcement learning to determine your bridge time step, but in the near future we hope that this method can be made into a workable module.

9.2 Using Bridge

Fortunately, you don't have to code all the intricate details of the Bridge method yourself. AMUSE has a `bridge` class that takes care of the details of implementing the classic and higher-order Bridge schemes just described. We illustrate this by developing a simple model of a star cluster moving in an analytic model of the potential near the

Galactic center (McMillan & Portegies Zwart, 2003; Harfst *et al.*, 2010). Then we show how this model can be extended easily to reproduce the seminal results of Fujii *et al.* (2007), which represented the first fully self-consistent treatment of star clusters near the Galactic center.

9.2.1 Star Cluster in a Static Galactic Potential

Code example 9.1 presents a simple class that provides a power-law analytic approximation to the mass distribution and gravitational field within a few hundred parsecs of the Galactic center:

$$\begin{aligned} M(r) &= M_{\text{gal}} \left(\frac{r}{R_{\text{gal}}} \right)^\alpha \\ a(r) &= -\frac{GM(r)}{r^2}, \end{aligned} \quad (9.14)$$

where $M(r)$ is mass interior to radius r and we take $M_{\text{gal}} = 1.6 \times 10^{10} M_\odot$ within radius $R_{\text{gal}} = 1 \text{ kpc}$, and $\alpha = 1.2$ (Mezger *et al.*, 1996). We initialize the class with

```
galactic_center = GalacticCenterGravityCode(Rgal, Mgal, alpha)
```

```

26 class GalacticCenterGravityCode(object):
27     def __init__(self, R, M, alpha):
28         self.radius=R
29         self.mass=M
30         self.alpha=alpha
31
32     def get_gravity_at_point(self, eps, x, y, z):
33         r2=x**2+y**2+z**2
34         r=r2**0.5
35         m=self.mass*(r/self.radius)**self.alpha
36         fr=constants.G*m/r2
37         ax=-fr*x/r
38         ay=-fr*y/r
39         az=-fr*z/r
40         return ax, ay, az
41
42     def circular_velocity(self, r):
43         m=self.mass*(r/self.radius)**self.alpha
44         vc=(constants.G*m/r)**0.5
45         return vc

```

Code example 9.1: Semi-analytic power-law description of the gravitational field in the inner few hundred parsecs of the Galactic center. The quantity `mass` is the mass within distance `radius` of the center, and `alpha` is the slope of the power-law mass distribution.

Key to this (and any) bridge class is the function `get_gravity_at_point()`, which takes a position in space and a softening length as arguments, then returns the gravitational acceleration at the specified location. This function allows any gravity code to be bridged with any other in AMUSE. (In this particular case, the softening length

is ignored, but the argument is required by bridge because it is used by other codes for which softening may be necessary.)

Next, we initialize the star cluster by starting an N -body code and generating a realization of particle masses, positions and velocities. In the example in Code example 9.2, we initialize a code called `nbodocode`, which could be either a direct N -body code, a tree code, or a gravity-enabled SPH code. The cluster is generated from a [King \(1966\)](#) model with N particles, total mass `Mcluster`, virial radius `Rcluster`, and a dimensionless potential depth `W0` (see also Section 4.1.3.1). Additional parameters for the N -body code can be passed using the argument `parameters`. In this case, we provide the softening parameter as an argument because we will run this script with a tree code, which requires softening.

```

84  Rgal = 1. | units.kpc
85  Mgal = 1.6e10 | units.MSun
86  alpha = 1.2
87  galaxy_code = GalacticCenterGravityCode(Rgal, Mgal, alpha)
88
89  m=galaxy_code.mass*((100|units.pc)/galaxy_code.radius)**galaxy_code.alpha
90  cluster_code = make_king_model_cluster(BHTree, N, W0, M, R,
91                                     parameters=[("epsilon_squared",
92                                                  (0.01 | units.parsec)**2)])
93
94  stars = cluster_code.particles.copy()
95  stars.x += Rinit
96  stars.vy = 0.8*galaxy_code.circular_velocity(Rinit)
97  channel = {"from_framework": stars.new_channel_to(cluster_code.particles),
98            "to_framework": cluster_code.particles.new_channel_to(stars)}
99  channel["from_framework"].copy_attributes(["x", "y", "z", "vx", "vy", "vz"])
100
101  plot_cluster(f, stars.x.value_in(units.pc), stars.y.value_in(units.pc), c='r')
102
103  system = bridge(verbose=False)
104  system.add_system(cluster_code, (galaxy_code,))
105
106  times = quantities.arange(0|units.Myr, tend, timestep)
107  xcom = [] | units.pc
108  ycom = [] | units.pc
109  for i,t in enumerate(times):
110      system.evolve_model(t,timestep=timestep)
111      channel["to_framework"].copy_attributes(["x", "y", "z", "vx", "vy", "vz"])
112      com = stars.center_of_mass()
113      xcom.append(com[0])
114      ycom.append(com[1])
115
116  plt.plot(xcom.value_in(units.pc), ycom.value_in(units.pc), lw=1, c='k')
117
118  x = system.particles.x.value_in(units.parsec)
119  y = system.particles.y.value_in(units.parsec)
120  cluster_code.stop()

```

Code example 9.2: Initializing the gravity code to be coupled via bridge to a background potential.

The bridge is created by constructing a new `gravity` solver that combines the *N*-body code `cluster_code` for the star cluster and the analytic `galaxy_code` for the galactic background. The bridge is declared by

```
gravity = bridge.Bridge()
```

and subsequently we identify the codes to be bridged with

```
gravity.add_system(cluster_code, (galaxy_code,))
```

This line adds `cluster_code` as a code to be bridged, and indicates that it interacts with `galaxy_code`. In general, the second argument is a list of codes with which the first code interacts.¹

In this example, the bridge is unidirectional: the dynamical evolution of the particles in `cluster_code` code is affected by `galaxy_code`, but not the other way around. Multiple codes can be added to a bridge, and their interactions tailored to the problem at hand. This can be very effective if, for example, our galaxy code is composed of several separate parts—for the bulge, bar, and disk, say—and we wanted to model several clusters in the bridge:

```
gravity.add_system(cluster1, (cluster2, cluster3, bulge, bar, disk))
gravity.add_system(cluster2, (cluster1, cluster3, bulge, bar, disk))
gravity.add_system(cluster3, (cluster1, cluster2, bulge, bar, disk))
```

and so on.

Once the bridge is initialized and the integrators added, we set the bridge time step:

```
gravity.timestep = 0.1 | units.Myr
```

It often requires some experimentation to determine what this time step should be. Too long a time step results in inaccurate inclusion of the coupling, whereas too short a time step may make the calculation too expensive. It is possible to change the bridge time step at run time (see Section 10.1.3), but there is currently no ready algorithm to determine its optimum value.

In the event loop over time, we now call `evolve_model` from the bridged `gravity` solver

```
gravity.evolve_model(time)
```

Synchronization between the internal code particles and those in the Python script is realized via channels. The event loop is presented in Code example 9.1.

Figure 9.3 shows the results of a simulation of the orbital evolution of the stars in the Arches cluster in the field of the Galactic center. The 50,000 $M_{\odot}$ star cluster was represented by 1024 equal-mass particles (not a good approximation to the observed mass function of the Arches!) initially represented by a virialized King model (King,

¹A Python aside: the trailing comma after `galaxy_code` turns the single argument into a tuple that can be evaluated. Forgetting the trailing comma will lead to an error with the inscrutable and somewhat intimidating message "object is not iterable". It is only needed when there is only one item in the list.

```

56 def make_king_model_cluster(nbodycode, N, W0, Mcluster,
57                             Rcluster, parameters = []):
58
59     converter=nbody_system.nbody_to_si(Mcluster,Rcluster)
60     bodies=new_king_model(N,W0,convert_nbody=converter)
61
62     code=nbodycode(converter)
63     for name,value in parameters:
64         setattr(code.parameters, name, value)
65     code.particles.add_particles(bodies)
66     return code

```

Code example 9.3: Coupling a gravity code to a background potential using a bridge. The last three lines store the stars x and y positions and terminate the code. In the example we adopted the following parameters: `number_of_particles = 1024`, `king_w0 = 3`, `distance_initial = 50.0 | units.pc`, `mass_cluster = 5.0e4 | units.MSun`, and `radius_cluster = 0.8 | units.pc`.

1966) with a radius of 0.8 pc. The cluster had a non-circular orbit in the Galactic field, starting at a distance of 50 pc. We adopted a Barnes–Hut tree code with a rather large softening of 0.1 pc, to allow us to quickly integrate the motion of the stars in the cluster without getting too distracted by the details of the internal cluster dynamics.

9.2.2 Adding extra functionality with a kick to a bridge

Sometimes one would like to add a functionality to the bridge operator in order to kick the system code. A simple example can be considered with the dynamical friction on a black hole of intermediate mass due to its orbit through a galactic nucleus. We discussed a similar issue in Section 4.1.2.3, but without addressing how to add such a term to the integration of an N-body system.

In principle, this process does not work much different than the bridging of two codes, except that in this case one of the codes has no `particle` attribute and doesn't need to be kicked nor drifting itself. In the example, below, we initiate the `Hermite` integrator two intermediate mass black holes that orbit in a background potential. This example is not very different than what we already discussed in Section 9.2.1.

The difference is that we would like to exert a force on the black holes due to the effect of dynamical friction exerted on the black holes. We simply start the N-body code (here `Hermite`), and add the black holes. Then we start the `DynamicalFriction` code, and bridge them together.

```

code1 = Hermite()
code1.particles.add_particles(cluster)
code2 = DynamicalFriction(Coulomb_logarithm=3.7)
system = Bridge(timestep=0.1 | units.Myr)
system.add_system(code1)
system.add_code(code2)
system.evolve_model( 10 | units.Myr )

```

The dynamical friction solver comprises of two parts, a Plummer potential, and a dynamical friction term. We have created a small class to achieve this, in Figure 9.4.

Since the `Hermite` code is 4th order, we use the symbol S_4 (for stars) for the

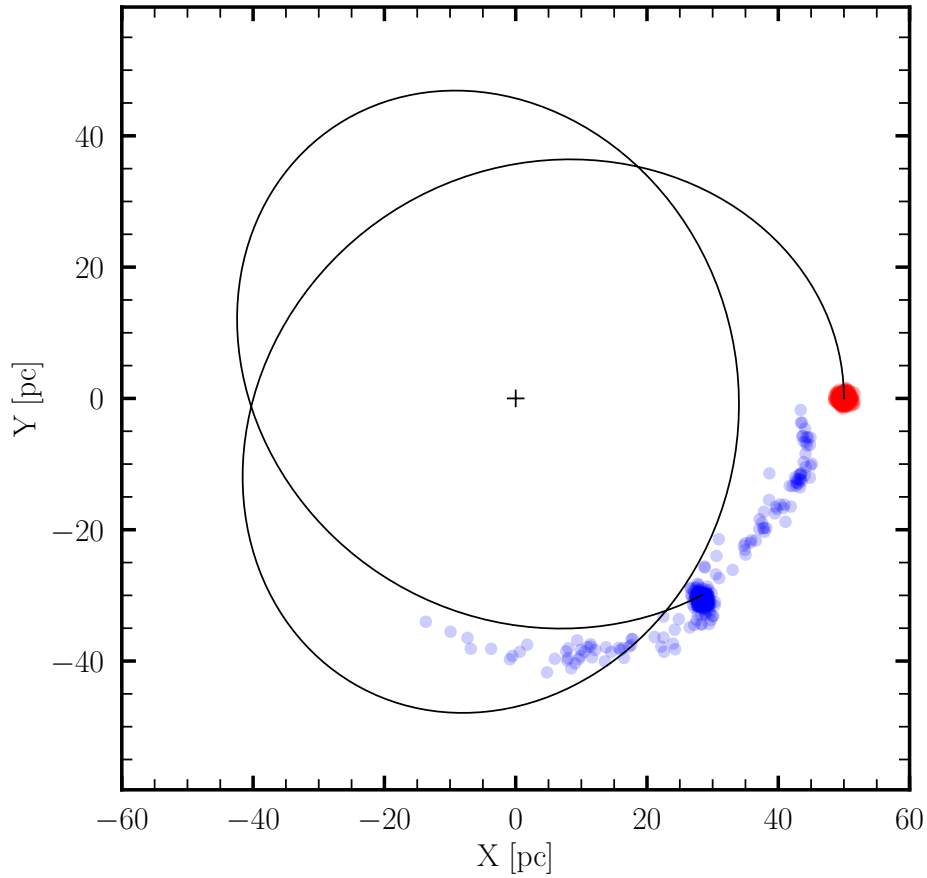


Figure 9.3: Snapshots of an Arches-like cluster orbiting near the Galactic center. Initially (in red) the cluster was composed of 5000 particles taken from a Salpeter mass function over two orders of magnitude with a mean mass of $10 M_{\odot}$ leading to a total mass of $50,000 M_{\odot}$, distributed as a King model in virial equilibrium with $W_0 = 3$ and a virial radius of 0.8 pc. The center of mass of the cluster was placed 50 pc from the Galactic center (along the x-direction) with 80% of the circular velocity (in the y-direction). A softening of 0.1 pc was introduced for all particles in the cluster. The integration was performed using the background galactic potential presented in Code example 9.1. The initialization and main event loop are shown in Code example 9.3 and Code example 9.2, respectively. The equations of motion of the cluster are integrated using the `BHtree` code. The black curve follows the cluster's center of mass, ending in the final snapshot (in blue). The source code for the figure is `amuse.examples.gravity_potential`.

sub-system. The potential is indicated with the letter G (for Galaxy). The bridge is indicated with the left-pointed arrow, and the augmentation is indicated with the letter f (for function) below the bridge arrow. The notation for the augmented bridge in which a 4th order direct N-body code is bridged with a potential including an additional function for (in this case for the dynamical friction term) then has the following notation: $[S_4 \xleftarrow[f]{} G]$. this may look somewhat arcane, at first, but once augmented bridge become hierarchical, multi-layers or more complex in other means, this notation captures these complexities nicely.

A star cluster in the Galactic center will feel a frictional force due to the accumulation of stars in its wake. [McMillan & Portegies Zwart \(2003\)](#) used a direct integration technique to study the time scale on which the Arches star cluster sinks from its birth location at about 30 pc from the Galactic center towards the middle of the Milky Way.

It turns out that the time scale depends quite sensitively on the Coulomb parameter $\ln \Lambda$. In their Fig. 6 they present the results of several calculations for the distance to the Galactic center as a function of time for a black hole (or star cluster) of $50\,000\,M_\odot$. In Figure 9.4 we repeat their calculation but using an external function included in the bridge step

$$[S_4 \xleftarrow{f} G] \quad (9.15)$$

The equation to solve in $f = f(S, G)$ we calculate the acceleration of the individual particles in the N-body code due to dynamical friction (See Equation 7 of [McMillan & Portegies Zwart \(2003\)](#)):

$$\vec{a}_f = -4\pi \ln(\Lambda) G^2 \rho m \frac{\vec{v}_c}{v_c^3} \chi. \quad (9.16)$$

Here G is the gravitational constant, ρ the local stellar density, with mass m , $\chi \simeq v_c/\sqrt{2}\sigma$ (with σ the local 1-dimensional velocity dispersion), and $\ln(\Lambda) \simeq \ln(r)/\ln(R)$ is the Coulomb logarithm. Here r is the distance of the stars to the potential center (which has size R), and $\vec{v}_c$ is the particle's velocity vector. The acceleration ($\vec{a}_f$) is negative, because dynamical friction results in a drag force leading to deceleration.

The class, performing this calculation together with hovering the stars on the background potential is presented in Figure 9.4.

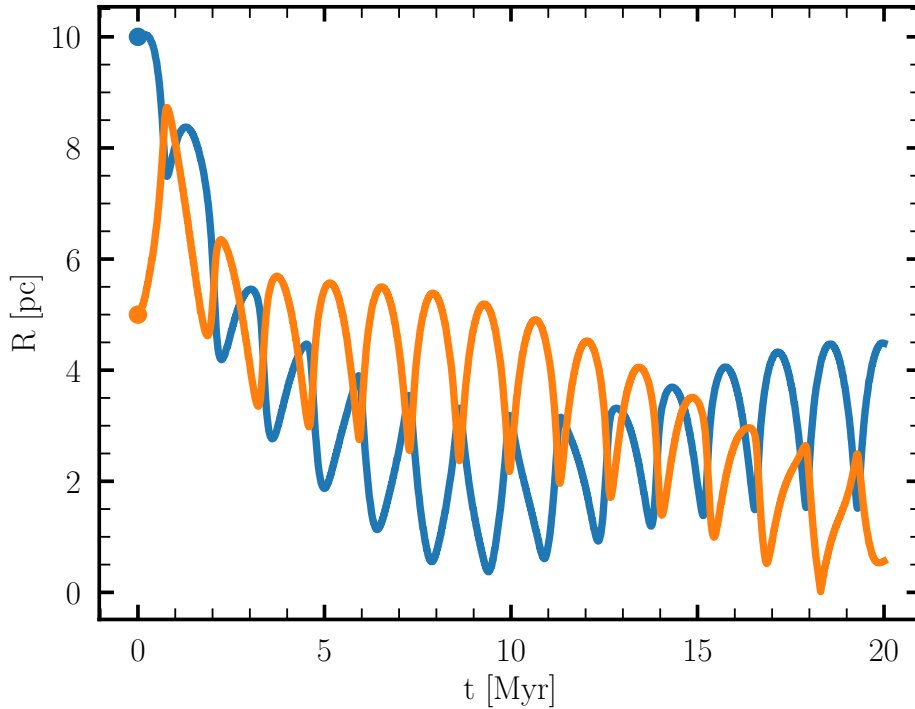


Figure 9.4: Time evolution of Galactocentric distance R of two massive compact objects of $50\,000\,M_\odot$ each from a distance of 10 pc (blue) and 5 pc (orange). Both curves were integrated using a bridge between `ph4` for integrating the black holes and a background potential with dynamical friction using $\ln(\Lambda) = 3.7$. The sinusoidal variations along the curves are the result of the two black holes not being in a perfect circular orbit. The interference between the two black holes is a consequence of their mutual gravitational interactions. The source code, takes a few seconds to run, can be found at `amuse.examples.imbhs_with_friction`

In this example, we included the dynamical friction as an additional drag force in

the kick of the bridging step. Alternatively, we could include a term in the drift-part of the bridge step. Including additional forces to a bridge can be realized in a similar way by adding an extra term to the kick operator. In this way solving constructing an N -body solver that includes the YORP- and Yarkovski-effects², dynamical friction, tidal evolution, gas-drag, or post-Newtonian dynamics becomes relatively straightforward.

```

13 class CodeWithFriction(bridge.GravityCodeInField):
14
15     def kick_with_field_code(self, particles, field_code, dt):
16         self.LnL = 3.7
17
18         R = particles.position.length()
19         vx = particles.vx.mean()
20         vy = particles.vy.mean()
21         vz = particles.vz.mean()
22         rho = field_code.density(R)
23         vc = field_code.circular_velocity(R)
24         X = 0.34
25         m = particles.mass.sum()
26         ax = -4*np.pi*self.LnL*constants.G**2 * rho*m*(vx/vc**3)*X
27         ay = -4*np.pi*self.LnL*constants.G**2 * rho*m*(vy/vc**3)*X
28         az = -4*np.pi*self.LnL*constants.G**2 * rho*m*(vz/vc**3)*X
29         self.update_velocities(particles, dt, ax, ay, az)
30
31     def drift(self, tend):
32         pass
33
34     def get_system_state_relative_to_SMBH(time, black_holes):
35         t = [] | units.Myr
36         r = [] | units.pc
37         SMBH = black_holes[black_holes.name=="SMBH"]
38         for bi in black_holes-SMBH:
39             t.append(time)
40             r.append((bi.position-SMBH.position).length())
41         return t, r
42
43     def get_system_state(time, black_holes):
44         t = [] | units.Myr
45         r = [] | units.pc
46         for bi in black_holes:
47             t.append(time)
48             r.append(bi.position.length())
49         return t, r

```

Code example 9.4: Example class of a code for adding an extra functionality to a bridge. In this case for calculating the dynamical friction term. Source code is available in `amuse.examples.imbhs_with_friction`.

²Named after Ivan Osipovich Yarkovsky (1844-1902), see also [Whipple \(1950\)](#); [Bottke et al. \(2002\)](#).

```
friction_code = CodeWithFriction(
    cluster_gravity,
    (potential,),
    do_sync=True,
    verbose=False,
    radius_is_eps=False,
    h_smooth_is_eps=False,
    zero_smoothing=False,
)
gravity = bridge.Bridge(use_threading=False)
gravity.add_system(cluster_gravity, (potential,))
gravity.add_code(friction_code)
gravity.timestep = dt_bridge
```

9.2.3 The Classic Bridge

Now that we've seen some of the issues involved in setting up and using a simple Bridge system, we can quite easily create a bridge to reproduce the results of [Fujii *et al.* \(2007\)](#) on the dynamical evolution of a star cluster orbiting the Galactic center. We start by creating a classic Bridge combining a fourth-order Hermite predictor–corrector scheme for the cluster and a Barnes–Hut tree code for the Galaxy. After declaring the codes to be bridged—`ph4` and `bhtree`—we create the bridge solver and add the codes.

```
code1 = Ph4()
code2 = Bhtree()
combined_gravity = bridge.Bridge()
combined_gravity.add_system(code1, (code2,))
combined_gravity.add_system(code2, (code1,))
```

In this case, we have created a bidirectional solver, in which each code interacts with the other.

The simulation is then performed much as before:

```
combined_gravity.timestep = 0.1 | units.Myr
combined_gravity.evolve_model(10 | units.Myr)
```

Figure 9.5 presents an example calculation of the Arches cluster. It is similar to the calculation in Figure 9.3, but now we have adopted a particle representation for the Galactic center region and solved the system using the tree code `BHtree`, in much the same way as presented in [Fujii *et al.* \(2007\)](#). The Arches cluster was initialized with a King model with a dimensionless depth of $W_0 = 3$ and a virial radius of 0.8 pc, as in Figure 9.3. The cluster was placed in a almost circular orbit at a distance of 50 pc from the Galactic center. The latter was represented by a [Plummer \(1911\)](#) sphere with a total mass of $1.6 \times 10^{10} M_\odot$ and a scale radius of 100 pc (see Section 3.2).

9.2.3.1 Bridging with a Drifter Code

In many cases, it is overkill to run a direct N -body code or tree code in a background potential, particularly if the particles hardly interact, as is the case with planetesimals

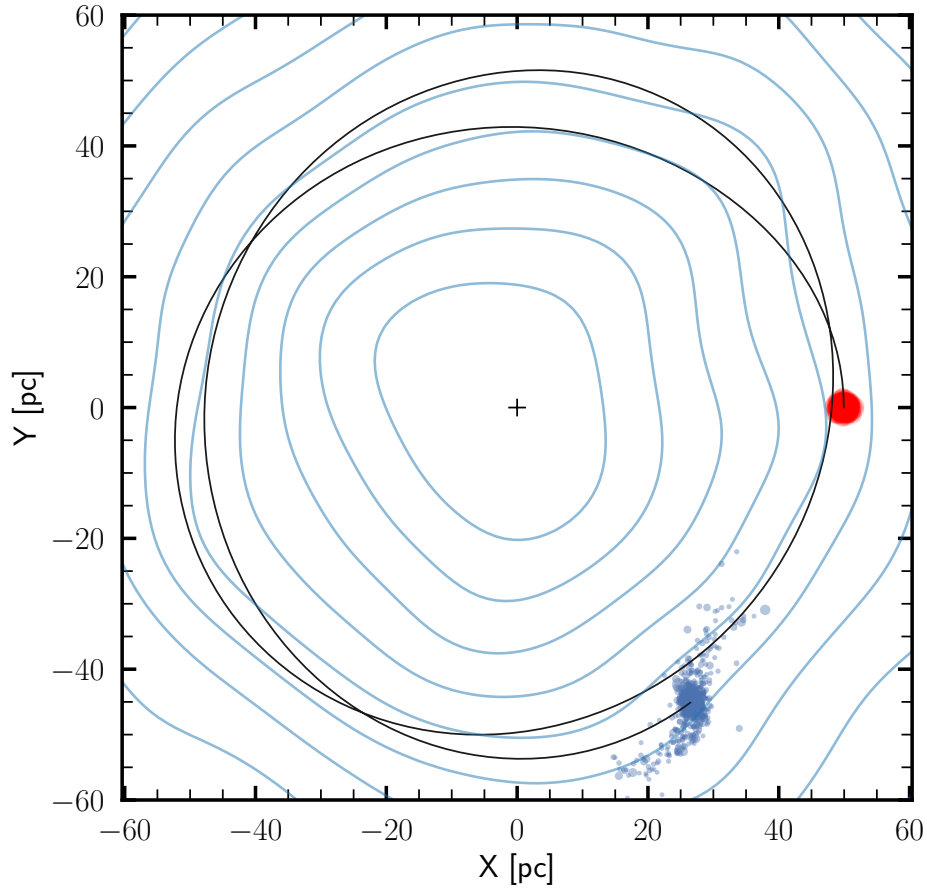


Figure 9.5: Calculation of a star cluster (using `ph4`) in a galaxy (using `BHTree`) for initial conditions comparable to those in Figure 9.3, but now we actively evolve the Galaxy using 10^4 equal mass particles. The star cluster was represented by 1000 point masses drawn from a Salpeter mass function between 0.1 and $100 M_{\odot}$. This star cluster, as in Figure 9.3, was born at a distance of 50 pc from the Galactic center (along the X-axis) and with a velocity of 80% of the circular velocity (in the Y-direction). Evolving to an age of 2.5 Myr took about 3 minutes. The source code for the figure is `amuse.examples.gravity_gravity`.

in a planetary system or individual stars in a background galactic potential. In such cases, it is preferable to use a small Python script that just drifts the particles in the background potential, rather than calculating the (negligible) interparticle forces and integrating the orbits. In Code example 9.5 we present an example of such a simple drift code, which we use here to integrate a low-mass star cluster in a semi-analytic Galactic potential. The drift code itself can be used in the same way as any other gravity code. Note that we use the Python `@property` built-in function which allows our small class to remain compatible with past and future versions (see Appendix C.3).

```

16 class drift_without_gravity:
17     def __init__(self, convert_nbody, time=0 | units.Myr):
18         self.model_time = time
19         self.convert_nbody = convert_nbody
20         self.particles = Particles()
21
22     def evolve_model(self, t_end):
23         dt = t_end - self.model_time
24         self.particles.position += self.particles.velocity * dt
25         self.model_time = t_end
26
27     @property
28     def potential_energy(self):
29         return quantities.zero
30
31     @property
32     def kinetic_energy(self):
33         return (
34             0.5 * self.particles.mass * self.particles.velocity.lengths() ** 2
35         ).sum()
36
37     def stop(self):
38         pass

```

Code example 9.5: Simple drift code to replace the N -body code in a background potential where interactions between particles are unimportant. Source code is available in `amuse.examples.gravity_drifter`.

The example in Code example 9.5 is far from optimal, and the drifting can easily be done in parallel (for an example, see Appendix A.7.4) or using a graphical processing unit. This would require somewhat more elaborate functions, and you might want to explore the possibility of using

```
from amuse.community.fastkick import Fastkick
```

which is a more efficient and optimized implementation of the kick operation using a GPU. An example of the use of `FastKick` is given in Section 9.3.2.1.

Although the script (which is referenced in Code example 9.5) uses a different model for the background Galactic centrer potential adopted for producing Figure 9.3, the results are also different due to the lack of self-gravity among the stars. As a consequence, the stars stay together longer and the cluster disperses more slowly. We do not recommend using this script for simulating star clusters in a background potential, but it is a handy tool for integrating the orbits of individual (non-self-interacting) particles in the galactic potential.

9.3 Bridging Other Codes

Bridging high-precision gravity with low-precision gravity, or gravity with a background field, is a very effective coupling strategy. Bridge is guaranteed to work in these cases because we can write down a common Hamiltonian for the coupled systems, but Bridge is much more versatile than this. Coupled systems for which we cannot write down a

coupled (or any) Hamiltonian can also be integrated using the classic bridge scheme. We could just as easily have replaced the analytic calculation by another N -body code or a hydrodynamical solver, or bridged one hydrodynamical solver with another, or included radiative transfer. One needs a little courage to do this, because we are unaware of any formal proof that this procedure results in a unique or “correct” solution to the coupled problem. On the other hand, resolution and convergence tests indicate that the solutions obtained in this way do appear to be reliable and accurate, and therefore suitable for scientific interpretation. We leave a complete proof to the brave reader, and should one be forthcoming we will be happy to include it in future editions!

The range of applications of Bridge is up to the imagination of the researcher. In the next section, we present a few bridge topologies you might want to experiment with.

9.3.1 Bridge Hierarchies and Hierarchical Bridges

The simplest possible bridge can be realized by integrating two single particles, such as the Sun and the Earth, and following each—not with a regular integrator, but using Bridge directly. Classic Bridge works as a second-order Verlet-leapfrog integrator, which is perfectly adequate for integrating the solar system, or (almost) any N -body system. An example of how to integrate such a two-body system is given in Code example 9.6.

```

23     star_gravity = Ph4(converter)
24     star_gravity.particles.add_particle(ss[0])
25
26     planet_gravity = Ph4(converter)
27     planet_gravity.particles.add_particle(ss[1:])
28
29     channel_from_star_to_framework = star_gravity.particles.new_channel_to(ss)
30     channel_from_planet_to_framework = planet_gravity.particles.new_channel_to(ss)
31
32     gravity = bridge.Bridge(use_threading=False)
33     gravity.add_system(star_gravity, (planet_gravity,))
34     gravity.add_system(planet_gravity, (star_gravity,))

```

Code example 9.6: Bridge construction for integrating the Sun–Earth system.

In this example, we declare a bridge with two components: one for the Sun, which is affected by Earth’s gravity, and one for Earth, which is affected by the gravity of the Sun. The eventual integration is carried out in a loop over time:

```

while time < time_end:
    time += time_step
    gravity.evolve_system(time)

```

in exactly the same way as a more usual integrator or N -body code would be used within the framework. It may seem strange to start an integrator (in this case `ph4`) and have it integrate a single particle (and indeed, some integrators won’t even let you do this), but here the integrator simply performs the drift step and defines `get_gravity_at_point()`,

letting the bridge complete the integration of the equations of motion. It probably makes far more sense to use the drifter code in Code example 9.5 instead of the direct N -body code.

We could make this example even stranger by adding the Moon as a third particle and constructing a bridge for all three elements (Sun, Earth, and Moon) as in Code example 9.7, a simple code to integrate a three-body system using second-order bridge. As you might imagine, this is not a very efficient way to integrate the Sun–Earth–Moon system. In fact, for comparable overall accuracy, our three-body bridge is almost 600 times slower than a direct integration using `ph4`, mainly because it is all done in Python, whereas `ph4` is compiled C++ (Stroustrup, 2013).

```

20  star_gravity = Ph4(converter)
21  star_gravity.particles.add_particle(star)
22
23  planet_gravity = Ph4(converter)
24  planet_gravity.particles.add_particle(planet)
25
26  moon_gravity = Ph4(converter)
27  moon_gravity.particles.add_particle(moon)
28
29  channel_from_star_to_framework = star_gravity.particles.new_channel_to(ss)
30  channel_from_planet_to_framework = planet_gravity.particles.new_channel_to(
31      ss)
32  channel_from_moon_to_framework = moon_gravity.particles.new_channel_to(ss)
33
34  time = 0 | units.yr
35  write_set_to_file(ss, filename,
36                  timestamp=time, overwrite_file=True)
37
38  gravity = bridge.Bridge()
39  gravity.add_system(star_gravity, (planet_gravity, moon_gravity))
40  gravity.add_system(planet_gravity, (star_gravity, moon_gravity))
41  gravity.add_system(moon_gravity, (star_gravity, planet_gravity))

```

Code example 9.7: Bridge construction for integrating the Sun–Earth–Moon system. We call this method the multi-bridge; its topology is at the bottom left of Figure 9.6.

We could extend the list of arguments indefinitely, even hierarchically, making the system quite complex. In Code example 9.8, we demonstrate how a hierarchy of level two can be achieved. The bridge for realizing the interaction between Sun and Earth is again bridged with the Moon, leading to a single composite bridge that integrates the entire system. In this case, the `ph4` instantiations take care of the `get_gravity_at_point()` used in the various bridges. This implementation is even slower than the example in Code example 9.6, but no matter—its purpose is to illustrate the method.

The relative performance of the various Bridge implementations just described is presented in Table 9.1. The three bridged methods use `ph4` in various ways, in combination with a bridge. The hierarchical bridge method uses three nested bridges.

Performance may be much better for other bridge topologies or different problems, but the key message here is that Bridge allows considerable flexibility that may com-

```

37 sp_gravity = bridge.Bridge()
38 sp_gravity.add_system(moon_gravity, (planet_gravity,))
39 sp_gravity.add_system(planet_gravity, (moon_gravity,))
40
41 gravity = bridge.Bridge()
42 gravity.add_system(sp_gravity, (star_gravity,))
43 gravity.add_system(star_gravity, (sp_gravity,))

```

Code example 9.8: Hierarchical bridge for integrating the Sun–Earth–Moon system. The majority of the code is identical to Code example 9.7 except that the last four lines there should be replaced with these lines to initialize the hierarchical bridge. This hierarchical-bridge topology is depicted on the bottom right of Figure 9.6.

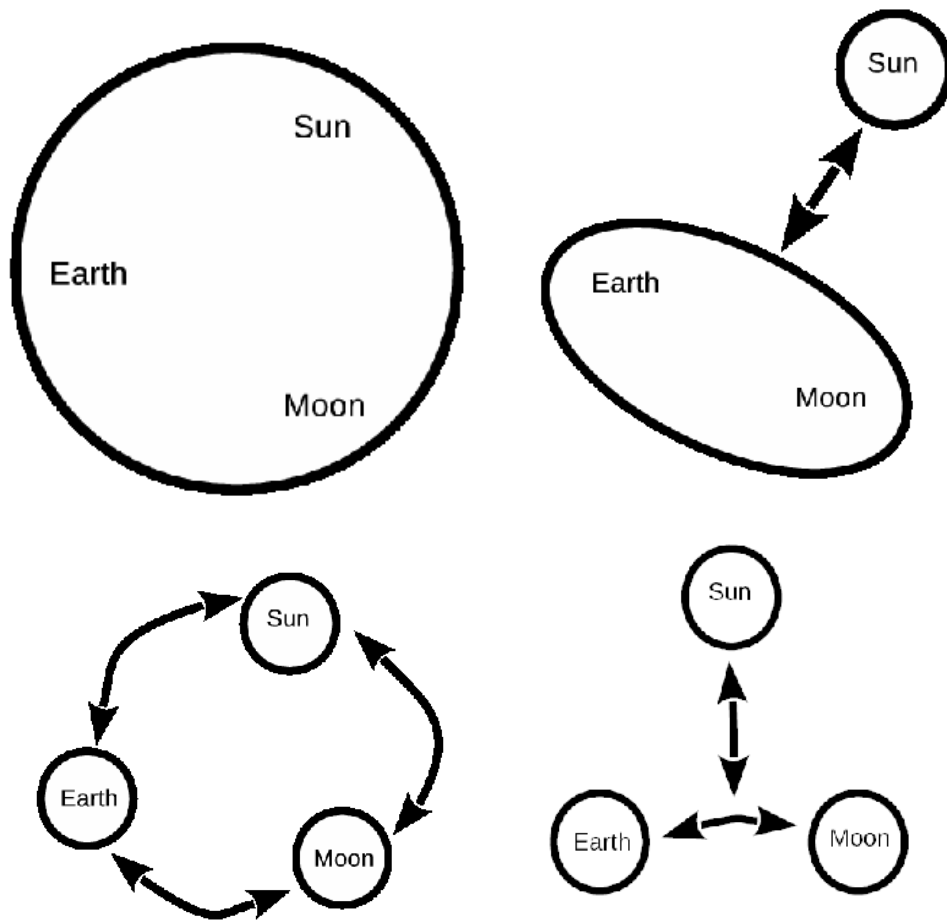


Figure 9.6: The various `Bridge` and `ph4` instantiations in Table 9.1. Top left indicates the use of a single N -body integrator for all three particles. At top right we present a two-way bridge (indicated by the two-pointed arrow) between an N -body realization for only the Sun, and separately for Earth and the Moon. At bottom left we show a three-body bridge, in which each particle is separately handled by an N -body integrator, and each is separately added to a bridge, see Code example 9.7. The bottom right depicts the hierarchical bridge, see Code example 9.8.

pensate for (some) loss of performance in many circumstances. Of course, we don't want performance to drop by a factor of 600, but in practical situations, when running computer-intensive production-quality simulations, the overhead of such complex bridging strategies is actually rather small—generally not more than a few percent.

code	code example	100 yr integration		
		t_{wall} [s]	$d\Delta E_{\text{max}}$	ΔE_{tot}
ph4		4.40	4.21×10^{-5}	1.61×10^{-6}
ph4 normalized	4.1	1	1	1
Bridge with ph4	9.6	4.0	0.52	40
Three-body bridge	9.7	8.4	0.57	36
Hierarchical bridge	9.8	12.5	0.57	36
TBB 4th order	A.4	13	1.13	62
TBB 6th order	A.4	4.4	0.52	31
TBB 8th order	A.4	40	0.78	47

Table 9.1: Performance of various Bridge implementations of the Sun–Earth–Moon problem. See also Figure 9.7. Values are relative to `ph4` as a standard, using a `timestep_parameter` of 0.15 (first line), which gives 4.40 s for the wall clock time, $\Delta E_{\text{max}} = 4.21 \times 10^{-5}$ for the energy error, and $\Delta E_{\text{tot}} = -1.61 \times 10^{-6}$ for the maximum energy error per output time step (0.1 yr) and the total energy error for integration over 20 years, respectively. In the following lines we normalize the performance of various bridged integrators (using `ph4`), including higher order bridges. The latter are generally more expensive, but have a favorable energy error per bridge time step. The various scripts are in `amuse.examples` and are called `no_bridge_ph4_integrator`, `three_body_bridge` (in the table named TBB), `three_body_bridge_hierachical`, and `three_body_bridge_n_order`.

Figure 9.7 compares the evolution of the energy error $(E - E_0)/E_0$ of `ph4` with those of the three bridge topologies listed in Table 9.1. To make the comparison more honest, we degraded the `ph4` time step by using a value of 0.6 for `timestep_parameter`. (This parameter is normally set to 0.1, which would have given a relative energy error better than 10^{-12} per step.) It is interesting to note that the energy error in the non-symplectic fourth-order Hermite integrator `ph4` accumulates, whereas the energy errors in the symplectic bridged systems show no secular evolution. The performance of the fourth- and sixth-order bridge methods is quite poor. At present, these are implemented in Python, which is rather inefficient. Porting these to compiled C++ or FORTRAN has not been a priority because, normally, little time is spent in the bridge, except for these somewhat contrived examples.

The example of a bridge coupling or a hierarchical bridge may appear a bit artificial for integrating the Sun–Earth–Moon system, but for other applications, the strict separation of codes, while combining them either sequentially or hierarchically, creates many interesting possibilities. In that sense, the timings listed in Table 9.1 are not representative of the ultimate performance of a more complex simulation in which each of the individual modules is more compute intensive. We encourage the reader to experiment with bridge until bridging is as familiar an operation as eating chicken pie (Cooder, 1972).

If a higher-order bridge is needed, it can be easily implemented by

```
from amuse.couple import bridge
import amuse.ext.composition_methods as CM
gravity = bridge.Bridge(use_threading=False, method=CM.SPLIT_4TH_S_M4)
```

Here, `composition_methods` provides a library for different bridge-coupling strategies (see Appendix A.4.2). The first free parameter in such complex bridge couplings is the number of evaluations of the same bridge step (m). For an example, see Equa-

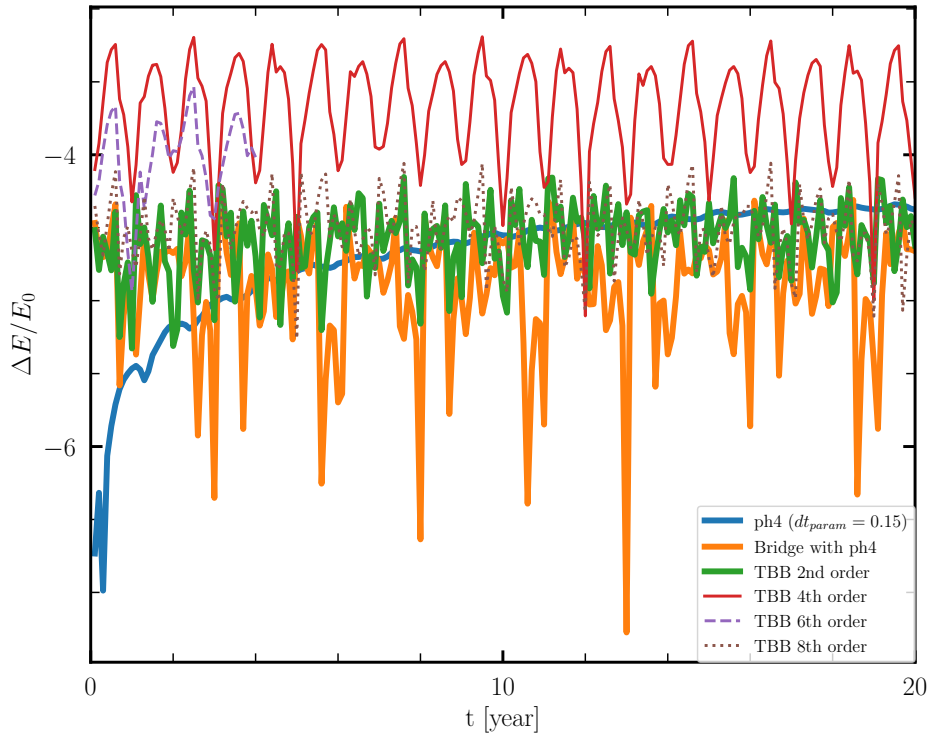


Figure 9.7: Evolution of the energy error while integrating the Sun–Earth–Moon system using a variety of bridge integrators. We use `ph4` with a time-step parameter $\eta = 0.15$ (blue curve) as a reference, which clearly shows a secular increase of the energy error in the calculation. The orange curve gives Earth–moon in `ph4` but subsequently bridged with the Sun. The other curves are calculated with all particles individually bridged with a 2nd, 4th, 6th and 8th order bridge. Interestingly, the 2nd-order bridge performs best for this case. The source code for reproducing this figure is `amuse.examples.plot_sun_and_earth_and_moon`, but the worker codes are presented in Figure 9.6.

tion (A.10), where we describe a sixth-order bridge with $m = 11$ force evaluations per step. The other parameter is the number of symplectic operations (S , see also Appendix A.4). The argument `method` in the `Bridge` declaration indicates the specific bridge scheme to use, in this case a fourth-order method with $m = 4$; for a sixth-order method one might instead adopt `SPLIT_6TH_SS_M13` (see Table A.16 in Appendix A.4.2).

An example of a higher-order bridge is given in `amuse.examples.three_body_bridge_order4m4`. The default `method` is set to `LEAPFROG`, the classic second-order bridge. Note that, as discussed in Appendix A.4, for codes to be bridgeable to an order higher than second, they must be able to run with negative time steps (stepping backward in time). Not all codes in AMUSE comply with this requirement: those that cannot accommodate negative time steps, such as the Barnes–Hut tree code, can only be bridged to second order.

9.3.2 Bridging Gravity with Hydrodynamics

Many hydrodynamics solvers allow the possibility of adding dark matter particles (called `dm_particles` in AMUSE, as supposed to `gas_particles`; see Section 6.3.3). The `dm_particles` have a gravitational effect on the rest of the system, but no hy-

drodynamical interactions. But the numerical solver used to integrate the equations of motion for `dm_particles` is generally identical to the hydrodynamics solver, which typically is a second-order Verlet integrator with a hierarchical tree to evaluate the forces. These integrators are not very accurate, and they are generally not used for collisional N -body systems. Collisional gravitational problems need more accurate N -body integrators, of which many are available in AMUSE. In principle, if we could couple the high-order gravity modules with the hydrodynamics modules, we could have the best of both worlds: the high accuracy of a fourth-order predictor–corrector Hermite integrator (for example `ph4`), along with the hydrodynamics solver of choice (such as `Fi`). In AMUSE we can do this using `Bridge`.

Bridging gravity with hydrodynamics is not much different than bridging gravity with gravity, as discussed in Section 9.1. Essentially the same bridge technique can be adopted, and the script will not look much different from one in which two or more gravitational dynamics codes are coupled. We could replace `ph4` in Code example 9.1 with any of the SPH codes in AMUSE without any further changes. Code example 9.9 presents an example of a coupling between an SPH code and a high-order direct N -body code. Here, we generalize the initialization of the hydro and the gravity codes by making them into classes. (For a brief overview of Python classes, we refer to Appendix C.3.7.) This requires a base class, as presented in Code example 9.10, and two specific classes for the gravity and hydrodynamics solvers. These derived classes are presented in Code examples 9.11 and 9.12. Section 9.4.3 presents an example in which we integrate the equations of motion of three stars using an N -body code, while we address the gas dynamics with an SPH code.

```

84 def gravity_hydro_bridge(Mprim, Msec, a, ecc, t_end, n_steps,
85                          Rgas, Mgas, Ngas):
86
87     stars = new_binary_from_orbital_elements(Mprim, Msec, a, ecc,
88                                              G=constants.G)
89
90     eps = 1 | units.RSun
91     gravity = Gravity(ph4, stars, eps)
92
93     converter = nbody_system.nbody_to_si(1.0|units.MSun, Rgas)
94     ism = new_plummer_gas_model(Ngas, convert_nbody=converter)
95     ism.move_to_center()
96     ism = ism.select(lambda r: r.length()<2*a, ["position"])
97     hydro = Hydro(Fi, ism, eps)
98     model_time = 0 | units.Myr
99     filename = "gravhydro.hdf5"
100    write_set_to_file(stars.savepoint(model_time), filename, 'amuse')
101    write_set_to_file(ism, filename, 'amuse', append_to_file=True)
102
103    gravhydro = bridge.Bridge(use_threading=False)
104    gravhydro.add_system(gravity, (hydro,))
105    gravhydro.add_system(hydro, (gravity,))
106    gravhydro.timestep = 2*hydro.get_timestep()
107
108    while model_time < t_end:
109        orbit = orbital_elements_from_binary(stars, G=constants.G)
110        dE_gravity = gravity.initial_total_energy/gravity.total_energy
111        dE_hydro = hydro.initial_total_energy/hydro.total_energy
112        print(("Time:", model_time.in_(units.yr), \
113              "ae=", orbit[2].in_(units.AU), orbit[3], \
114              "dE=", dE_gravity, dE_hydro))
115
116        model_time += 10*gravhydro.timestep
117        gravhydro.evolve_model(model_time)
118        gravity.copy_to_framework()
119        hydro.copy_to_framework()
120        write_set_to_file(stars.savepoint(model_time), filename, 'amuse',
121                          append_to_file=True)
122        write_set_to_file(ism, filename, 'amuse', append_to_file=True)
123        print("P=", model_time.in_(units.yr), gravity.particles.x.in_(units.au))
124    gravity.stop()
125    hydro.stop()

```

Code example 9.9: Combined solver for gravity and hydrodynamics using the generic class structure presented in Code example 9.10 (see `amuse.examples.gravity_hydro` for the full code example).

```

11 class BaseCode:
12     def __init__(self, code, particles, eps=0|units.RSun):
13
14         self.local_particles = particles
15         m = self.local_particles.mass.sum()
16         l = self.local_particles.position.length()
17         self.converter = nbody_system.nbody_to_si(m, l)
18         self.code = code(self.converter)
19         self.code.parameters.epsilon_squared = eps**2
20
21     def evolve_model(self, time):
22         self.code.evolve_model(time)
23     def copy_to_framework(self):
24         self.channel_to_framework.copy()
25     def get_gravity_at_point(self, r, x, y, z):
26         return self.code.get_gravity_at_point(r, x, y, z)
27     def get_potential_at_point(self, r, x, y, z):
28         return self.code.get_potential_at_point(r, x, y, z)
29     def get_timestep(self):
30         return self.code.parameters.timestep
31     @property
32     def model_time(self):
33         return self.code.model_time
34     @property
35     def particles(self):
36         return self.code.particles
37     @property
38     def total_energy(self):
39         return self.code.kinetic_energy + self.code.potential_energy
40     @property
41     def stop(self):
42         return self.code.stop

```

Code example 9.10: Generic base class for a single code.

```

46 class Gravity(BaseCode):
47     def __init__(self, code, particles, eps=0|units.RSun):
48         BaseCode.__init__(self, code, particles, eps)
49         self.code.particles.add_particles(self.local_particles)
50         self.channel_to_framework \
51             = self.code.particles.new_channel_to(self.local_particles)
52         self.channel_from_framework \
53             = self.local_particles.new_channel_to(self.code.particles)
54         self.initial_total_energy = self.total_energy

```

Code example 9.11: Derived class for gravity includes channels.

```

58 class Hydro(BaseCode):
59     def __init__(self, code, particles, eps=0|units.RSun,
60                 dt=None, Rbound=None):
61         BaseCode.__init__(self, code, particles, eps)
62         self.channel_to_framework \
63             = self.code.gas_particles.new_channel_to(self.local_particles)
64         self.channel_from_framework \

```

```

65         = self.local_particles.new_channel_to(self.code.gas_particles)
66         self.code.gas_particles.add_particles(particles)
67         m = self.local_particles.mass.sum()
68         l = self.code.gas_particles.position.length()
69         if Rbound is None:
70             Rbound = 10*l
71         self.code.parameters.periodic_box_size = Rbound
72         if dt is None:
73             dt = 0.01*numpy.sqrt(l**3/(constants.G*m))
74         self.code.parameters.timestep = dt/8.
75         self.initial_total_energy = self.total_energy
76     @property
77     def total_energy(self):
78         return self.code.kinetic_energy \
79             + self.code.potential_energy \
80             + self.code.thermal_energy

```

Code example 9.12: Derived class for hydrodynamics. Hydro codes often require additional parameters, which can be handled in the derived class.

9.3.2.1 Using bridge to relax multi-component systems

In Section 8.5.4.1 we discussed how to use a Henyey code to generate hydrodynamical (SPH) representations of two stars, which we then smashed into one another. After generating hydrodynamical blobs for each star we relaxed both to a glass-like solution to reduce random noise before we started to resolve the collision using an SPH code. Such a relaxation process can be rather elaborate, particularly when a gaseous body is not just self-gravitating but is also affected by the presence of another potential, such as a semi-analytic background or another self-gravitating system of particles.

In Code example 9.13 we demonstrate the use of **bridge** to relax a body of gas in the background potential of a stellar distribution. Bridge is used implicitly here by importing the **relax** routine

```

from amuse.ext.relax_sph import relax

```

which is designed to generate a glass-like solution for a set of SPH particles by evolving them in a background field while imposing critical damping on the particle velocities. The routine **relax** takes several arguments, including the particle set for the gas, the hydrodynamics code, and the code that generates the background potential.

```

79
80
81 def relax_gas_and_stars(stars, gas):
82     dynamical_timescale = gas.dynamical_timescale()
83     converter = nbody_system.nbody_to_si(dynamical_timescale, 1 | units.parsec)
84     hydro = Fi(converter, mode="openmp", redirection="file", redirect_file="fi.log")
85     hydro.parameters.timestep = dynamical_timescale / 100
86     hydro.parameters.eps_is_h_flag = True
87
88     gravity_field_code = Fastkick(converter, mode="cpu", number_of_workers=2)
89     gravity_field_code.parameters.epsilon_squared = (0.01 | units.parsec) ** 2
90     gravity_field_code.particles.add_particles(stars)
91     relaxed_gas = relax(
92         gas,
93         hydro,
94         gravity_field=gravity_field_code,
95         monitor_func=check_energy_conservation,
96         bridge_options=dict(verbose=True, use_threading=False),
97     )
98     gravity_field_code.stop()
99     return hydro

```

Code example 9.13: Using `bridge` to relax a body of gas in the background potential of a stellar distribution. Source code takes a few minutes to run and is available in `amuse.examples.relax_gas_and_stars`.

The `relax` method uses `bridge` to couple the hydrodynamics with the background potential, and may require a few tuning parameters. These are specified as `bridge_options`, using a dictionary in the instantiation of `relax`. In Code example 9.13 we choose to increase the amount of diagnostic output from the bridge and to switch off threading. In order to monitor the progress of the relaxation process we also include a user-defined call-back function that will be called after each bridge step. In the current example we keep track of energy conservation while relaxing the gas; the monitoring code is in Code example 9.14. The monitoring function takes at least four arguments: the bridge system (or other code which is used to relax the gas), the current step, the current time, and the total number of steps. The last three arguments are only used for diagnostic output.

```

17 def check_energy_conservation(system, i_step, time, n_steps):
18     unit = units.J
19     U = system.potential_energy.value_in(unit)
20     Q = system.thermal_energy.value_in(unit)
21     K = system.kinetic_energy.value_in(unit)
22     print(
23         f"Step {i_step} of {n_steps}, t={time.as_quantity_in(units.yr)}: "
24         f"U={U:.2e}, Q={Q:.2e}, K={K:.2e} {unit}"
25     )

```

Code example 9.14: Call-back function to monitor energy conservation during the `relax` process. Source code is part of available in `amuse.examples.relax_gas_and_stars`.

Code example 9.13 uses another interesting and handy utility in AMUSE, `FastKick`. This routine is actually a rather simple function that computes the kick in the bridge

system, in this case, for the gravity of the stars. Bridge requires such a code, but in this example it would be overkill to use a full N -body code to calculate the kicks between the stars and the gas particles, because we don't need to move the stars and therefore can omit the N^2 operation in the gravity module. Using a direct N -body code, or in some cases even a tree code to resolve the kicks would in fact make the code considerably slower. (You can easily verify this by replacing `FastKick` by any gravity solver from Chapter 4.) `FastKick` is parallel (using the keyword `number_of_workers`), and has a GPU implementation (invoked by setting `mode = "gpu"`):

```
gravity_field_code = FastKick(converter, mode="gpu", number_of_workers=4)
```

The result is a fast and consistent kick implementation.

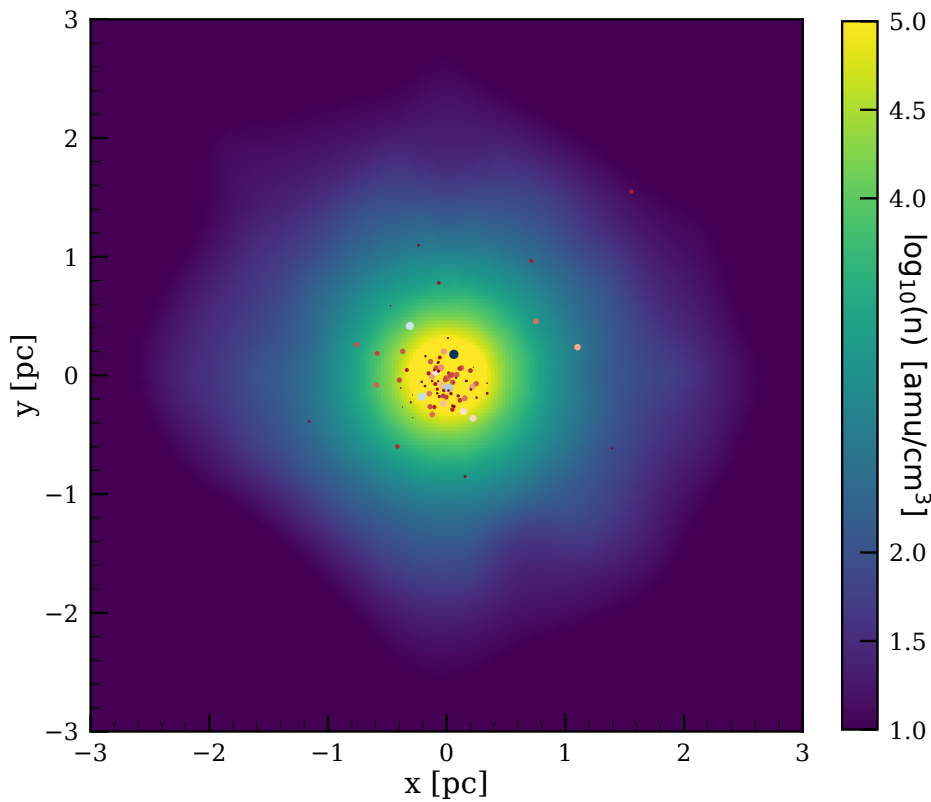


Figure 9.8: Initial conditions for stars and gas in equilibrium. The stars are distributed according to a [Plummer \(1911\)](#) sphere. The gas was initially also generated from a Plummer sphere, but subsequently the gas around each star up to the total mass of that star was removed. The resulting three-dimensional distribution then resembles Tynjetaler cheese. After the gas around the stars was removed we brought the gas distribution into equilibrium with the stars using the `relax` routine in Code example 9.13. The source code to reproduce this figure is `amuse.examples.relax_gas_and_stars`, which takes about 2 hours to run.

Figure 9.8 presents the results of generating initial conditions for 100 stars in a Plummer distribution with a virial radius of 0.33 pc with masses from a [Salpeter \(1955\)](#) function between $1 M_{\odot}$ and $100 M_{\odot}$. The gas was initially distributed identically to the stars with 10^5 equal mass particles but with a mass ten times higher than the total mass in stars. After generating the gas distribution we iteratively removed the gas particles around each star with a mass equal to the mass of that particular star. These

88941 remaining gas particles were then relaxed. This method was used by [Pelupessy & Portegies Zwart \(2012\)](#) to generate the initial conditions for their simulations. In Figure 6.10 we presented the results of a hydrodynamical calculation in which we allowed the gas to clump and form stars through sink particles. The two distributions are quite different, and it is not clear what gives the most satisfactory initial conditions for further simulations.

Generating initial conditions in this way can be expensive in terms of computer time, but that is sometimes the price to pay for a realistic or at least a desirable realization.

9.4 Examples

9.4.1 Dissolving star cluster in the Galactic potential

The Sun was probably born in a star cluster with a mass of 500–10,000 $M_{\odot}$ and a virial radius of about 1 pc ([Portegies Zwart, 2009](#)). In the next few subsections we explore the orbital evolution of the Sun and its parental cluster over the past 4.6 Gyr. This work is motivated by the idea that some of the Sun’s siblings may still be around in the solar neighborhood and may be identifiable ([Portegies Zwart, 2009](#)). As has been pointed out by [Mishurov & Acharova \(2011\)](#), this may not be so easy due to the lumpiness of the Galaxy. In addition, the Galaxy may have changed considerably since the Sun’s birth and reconstruction of the comoving Galactic potential over the last 4.6 Gyr is not trivial. Nevertheless, let’s explore this interesting question for ourselves.

We perform a series of simulations of a small cluster with 1000 stars orbiting in the Galactic disk, for a variety of representations of the Galactic potential. The basic approach is to choose a starting point 4.6 Gyr ago and then follow the orbits of the Sun and its siblings forward in time, to assess their current locations with respect to the Sun.

9.4.1.1 Dissolution of a star cluster in a smooth Galaxy potential

We first adopt a three-component model of the Galactic potential from [Bovy \(2015\)](#), consisting of a bulge, modeled as a power-law density profile with an exponential cut-off, a Miyamoto-Nagai disk ([Miyamoto & Nagai, 1975](#)), and an NFW dark-matter halo ([Navarro *et al.*, 1996](#)). The calling sequence to create this model is

```
from amuse.ext.galactic_potentials import MWpotentialBovy2015
galaxy_model = MWpotentialBovy2015()
```

We initialize the proto-solar cluster by selecting a plausible starting point and investigating how the stars spread out during a 4.6 Gyr integration, without pretending that the starting point represents the actual birthplace of the Sun.

We initialize the cluster with 1000 stars with masses drawn from a Salpeter distribution between 0.1 $M_{\odot}$ and 10 $M_{\odot}$, distributed in a [King \(1966\)](#) model with a dimensionless depth parameter $W_0 = 3$ and a virial radius of 100 pc. We did not add stellar evolution, because with the limited number of stars only a few will be sufficiently

massive to show any noticeable effects. We evolve the cluster to an age of 4.6 Gyr using the `BHtree` direct N -body code which we bridge with the static potential using a bridge time step of 10 Myr. The resulting stellar distribution in the Galaxy is shown in Figure 9.9.

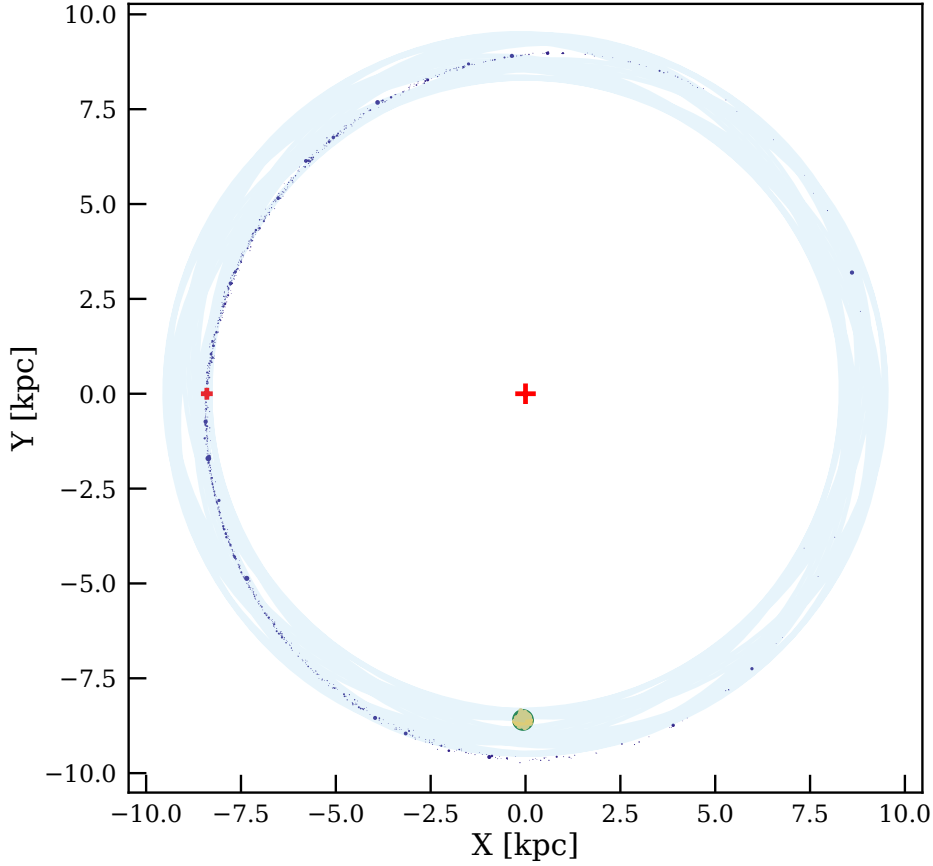


Figure 9.9: Distribution of the solar siblings (blue points) and the Sun (green dot, at left) in the Galaxy, projected in Galactic X and Y coordinates. The Galactic center is indicated by the large red cross, but no Galactic particles are shown because the Galaxy is represented here by a smooth density distribution (in contrast to Figure 9.10 where part of the galaxy is represented by giant molecular clouds and star clusters). The birth location of the Sun is indicated by the green cross at the bottom of the figure, and the initial cluster of 1000 sibling particles by the yellow points. The orbit of the Sun over the last 4.6 Gyr is indicated by the light blue line. The simulation was performed using the script `amuse.examples.solar_cluster_in_galaxy_potential`.

After having orbited the Galactic center some 27 times in 4.6 Gyr, the solar cluster has dispersed along its orbit. Based on these results, [Portegies Zwart \(2009\)](#) concluded that 10 to 100 siblings of the Sun could still be within a distance of 1 kpc of the current location of the Sun in the Galaxy.

9.4.1.2 Use of a rotating bridge

In order to say something substantive about the Sun's past we must adopt a more sophisticated model for the Galactic potential. A rather detailed model of the Galactic potential is available in AMUSE in the `Galaxia` package ([Martínez-Barbosa *et al.*, 2016](#)). This model consists of semi-analytic axisymmetric prescriptions for the bulge,

disk, and halo (Allen & Santillan, 1991), and a parameterization for the bar (Romero-Gómez *et al.*, 2007, 2011) that includes transient spiral arms (Cox & Gómez, 2002). The package has a lot of free parameters for which reasonable choices are needed; some of them will be tuned using data from the Gaia astrometric satellite (Gaia Collaboration *et al.*, 2016) as new data become available. Here we adopt the `Galaxia` potential for the Milky Way, for which parameters are summarized in Table A.15. The current location of the Sun is taken to be $\mathbf{x} = (-8400, 0.0, 17.0)$ pc, $\mathbf{v} = (-11.4, -232, -7.41)$ km/s (Karachentsev & Makarov, 1996). This is a little farther away than the current best estimates of $\sim 8.0 \pm 0.3$ kpc (Camarillo *et al.*, 2018).

We can use the `Galaxia` model in combination with `bridge` to integrate the Sun’s orbit backward in time for 4.6 Gyr to study where it originated. The underlying `Galaxia` galaxy model is rotating, so we take Coriolis forces into account in our simulation by adopting a rotating `bridge` module in `AMUSE` (see Martínez-Barbosa *et al.*, 2016). This model is rather complex, and a simple second-order scheme for the standard `bridge` is insufficient to integrate the equations of motion. In the code example below we adopt a 6th-order `bridge` with $m = 13$ symmetric evaluations per step by setting the method option in `bridge` to `SPLIT_6TH_SS_M13` (see Appendix A.4). This integration scheme does an excellent job and allows us to take rather large time steps without significant loss of accuracy.

The `Galaxia` module and `bridge` are initialized as follows

```
from amuse.community.galaxia.interface import BarAndSpirals3D
from amuse.ext.rotating_bridge import Rotating_Bridge
from amuse.ext.composition_methods import SPLIT_6TH_SS_M13
galaxy = BarAndSpirals3D()
system = Rotating_Bridge(omega, timestep=dt_bridge, method=SPLIT_6TH_SS_M13)
```

We then initialize the Sun at its current position and velocity, and build the `bridge` with two components.

```
drifter = drift_without_gravity(sun_in_rotating_frame)
system.add_system(drifter, (galaxy,))
system.add_system(galaxy, ())
```

The function `drift_without_gravity` is the `drifter` code discussed earlier in Section 9.2.3.1, and `sun_in_rotating_frame` is the particle set that contains the Sun, corrected for the rotating frame of the `Galaxia` model. An example of how this approach works is available in `amuse.examples.integrate_sun_in_galaxy_potential`.

Adopting the current solar coordinates and integrating backward in time for 4.6 Gyr, we find the birthplace of the Sun to be $\mathbf{x} = (6641, -4953, 77.2)$ pc, with velocity $\mathbf{v} = (131.9, 197.4, 2.737)$ km/s (right at the edge of Borg space in the Delta quadrant of the Galaxy³). This example should really be viewed as an illustration of how to use `Bridge` in conjunction with a more complex galactic potential, rather than as the ultimate method of finding the Sun’s birth location. Still, the question in itself remains interesting.

³See <http://www.startrek.com/>.

9.4.1.3 Dissolution of a star cluster in a lumpy Galaxy

The previous two experiments neglected the fact that the Galaxy is lumpy, in the sense that stars and gas tend to form clusters and clouds. In the next study we add a population of clumpy objects to the Galactic potential, in the form of a number of softened point masses representing star clusters and molecular clouds, and have those objects also orbit in the Galactic potential. These objects scatter the proto-solar cluster, causing it to disperse more quickly, spreading the siblings out over a larger volume of space.

We could add clusters and clouds to our Galactic model using any of a number of published references (see e.g. [Heyer & Dame, 2015](#)), but the `GalactICs` module described earlier (see Section 4.4.5) already contains detailed prescriptions for clusters and molecular clouds, so we choose to create these objects using the `GalactICs` model. However, since `GalactICs` does not produce the same disk potential as our adopted [Bovy \(2015\)](#) analytic galaxy potential model (Section 9.4.1.1), we use positions from `GalactICs` but give each molecular cloud a circular velocity in the [Bovy \(2015\)](#) potential. Our adopted `GalactICs` model is generated with a total of 40,000 particles, 3157 of which represent the disk population, the rest follow the distribution for the Galactic halo. We use the former population of particles for the star clusters and molecular clouds, and ignore the rest. We illustrate this operation in `make_giant_molecular_clouds` in the routine `solar_cluster_in_semilive_galaxy`, which is used to generate Figure 9.10. To make the simulation somewhat more realistic we give each of these clusters a mass from a power-law mass distribution between $10^3 M_\odot$ and $10^7 M_\odot$ with a power-law slope of -2.3 (close to Salpeter). The massive objects are softened with a softening length of 100 pc, and are integrated using the `BHtree` tree code ([Barnes & Hut, 1986](#)).

The cluster itself has the same initial conditions and the same Galactic potential as the one adopted for Figure 9.9. Figure 9.10 presents the distribution of the stars for this second calculations at an age of 4.6 Gyr. In comparison with Figure 9.9, the siblings in the Galaxy with the massive objects are considerably more spread out due to encounters than in the case with the smooth galaxy potential. In a realistic Galaxy, the spiral arms should also be taken into account, as was done more properly in [Martínez-Barbosa *et al.* \(2016\)](#), although in that study (using `Galaxia`) the massive objects representing star clusters and molecular clouds were not taken into account.

9.4.1.4 Dissolution of a star cluster in a live Galaxy

We can easily replace the analytic potential used in the previous examples with a live evolving Galaxy by bridging with another gravity solver. For the Galaxy we probably don't want to use a direct N -body solver; a tree code is more appropriate. We start by generating an initial galaxy model with $N = 10000$ particles using `Halogen` ([Zemp *et al.*, 2008](#); [Zemp, 2014](#)) as in the code example below.

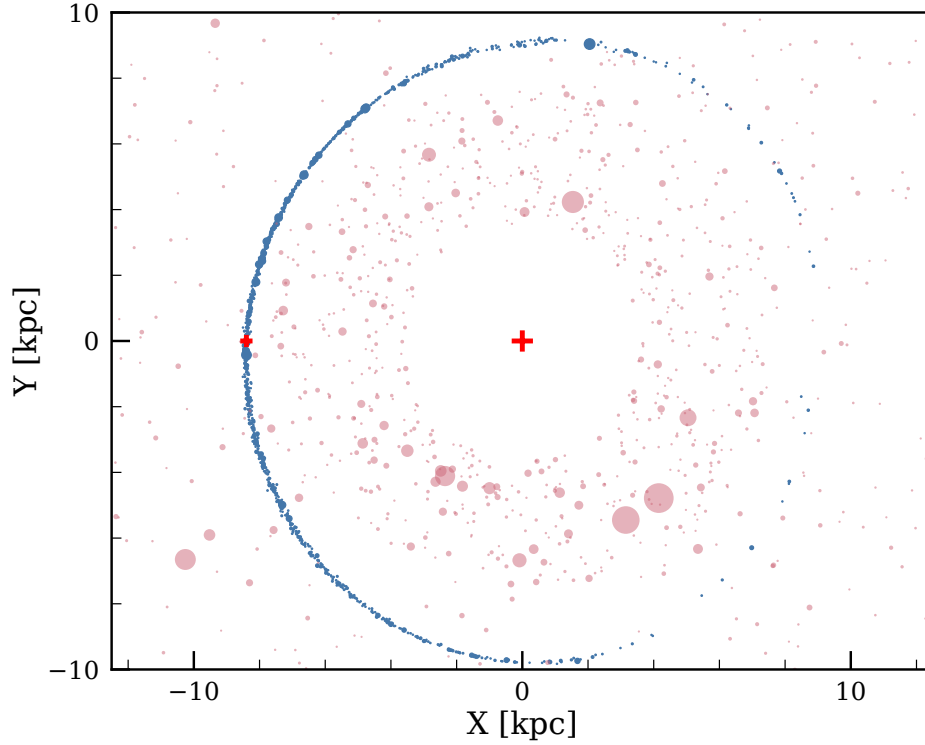


Figure 9.10: Current distribution of the solar siblings (blue bullets) relative to the Sun (green dot). The Galactic center is indicated by the red cross. No galactic particles are shown because the Galaxy is represented as a smooth potential, but we do show the giant molecular clouds included in the calculations (pink circles). The simulation was performed using the script `amuse.examples.solar_cluster_in_semilive_galaxy`, which should take a few minutes to run on a laptop.

```
def initialize_galaxy_model(N, converter):
    instance = Halogen(converter)
    instance.initialize_code()
    instance.parameters.alpha = 2.0
    instance.parameters.beta = 5.0
    instance.parameters.gamma = 0.0
    instance.parameters.number_of_particles = N
    instance.commit_parameters()
    instance.generate_particles()
    galaxy = instance.particles.copy()
    instance.cleanup_code()
    instance.stop()
    return galaxy
```

Note that the parameters used here are comparable, but not identical, to the settings used to generate the `halogen` panel of Figure 3.1.

We create the cluster using a $W_0 = 3$ King model with a virial radius of 3 pc, which we place at the same location as in the previous experiment. In this case we adopted $N = 1000$ stars randomly selected from a Salpeter mass function between $0.1 M_\odot$ and $10 M_\odot$, resulting in a total mass of $\sim 100 M_\odot$. We evolved the galaxy using `BHTree` and the cluster with `ph4`. The results were bridged so that the cluster affects the Galaxy

and the Galaxy the cluster:

```
gravity = bridge.Bridge(use_threading=False)
gravity.add_system(cluster_gravity, (galaxy_gravity,) )
gravity.add_system(galaxy_gravity, (cluster_gravity,) )
```

In this example we have switched off the internal threading of the bridged codes by sending `use_threading=False` as an argument in `Bridge`. Under usual circumstances, threading (or multiprocessing) is enabled in bridge, because the two bridge steps are intrinsically parallel and most computers allow the use of multiple cores simultaneously. On some older computers, however, threading produces memory conflicts which can be prevented by explicitly turning it off.

We set the bridge time step to 1/32 of the initial orbital period of the cluster around the Galactic center

```
gravity.timestep = time_orbit/32.
```

resulting in a bridge time step of ~ 12 Myr. The results of the calculation, after evolving for ~ 4.6 Gyr, are presented in Figure 9.11. We see that the siblings have been dispersed considerably compared to the smooth galaxy (Figure 9.9) or the galaxy in which we take some molecular clouds into account (Figure 9.10). Of course, the granularity of the Galactic potential, which is responsible for the large sibling dispersion evident here, depends on the details of the Galactic model and is likely far smaller than in the simulation described here. The dispersion of the solar cluster is an active field of research, and our lack of understanding the past evolution of the Galaxy seriously hinders this analysis. The consequences of the degree by which the solar cluster dispersed may have profound consequences for our ability to identify and find the lost siblings of the Sun.

9.4.2 Did the Sun originate in M67?

In Section 5.4.2 we discussed the blue stragglers in the star cluster M67. We now return to this cluster in relation to the birthplace of the Sun. In recent years the argument that the Sun originated from M67 has been made, and even though in the previous section we argued that the Sun originated from a lower-mass cluster, we investigate here the arguments for M67 as the parental cluster.

We begin this discussion with the star M67-1194, which is remarkably similar to the Sun in terms of mass and composition (Önehag *et al.*, 2011). Further dynamical analysis led Gustafsson *et al.* (2016) to conclude that the Sun might have originated in this cluster, although according to Pichardo *et al.* (2012), this conclusion is hampered by differences in the orbits of the Sun and the cluster. In addition, 982 of the stars near the cluster's turn-off have a mean age of 3.9 ± 1.2 (Xiang *et al.*, 2017), which is somewhat young compared to the Sun, and M67 is rather massive, making it unlikely that the solar system would have survived in its current form (Portegies Zwart, 2009). Nevertheless, it is interesting to test the hypothesis that the Sun originated from this cluster, and therefore we further analyze this possibility, simply because M67 is the closest massive cluster today with metallicity comparable to the Sun (Pichardo *et al.*,

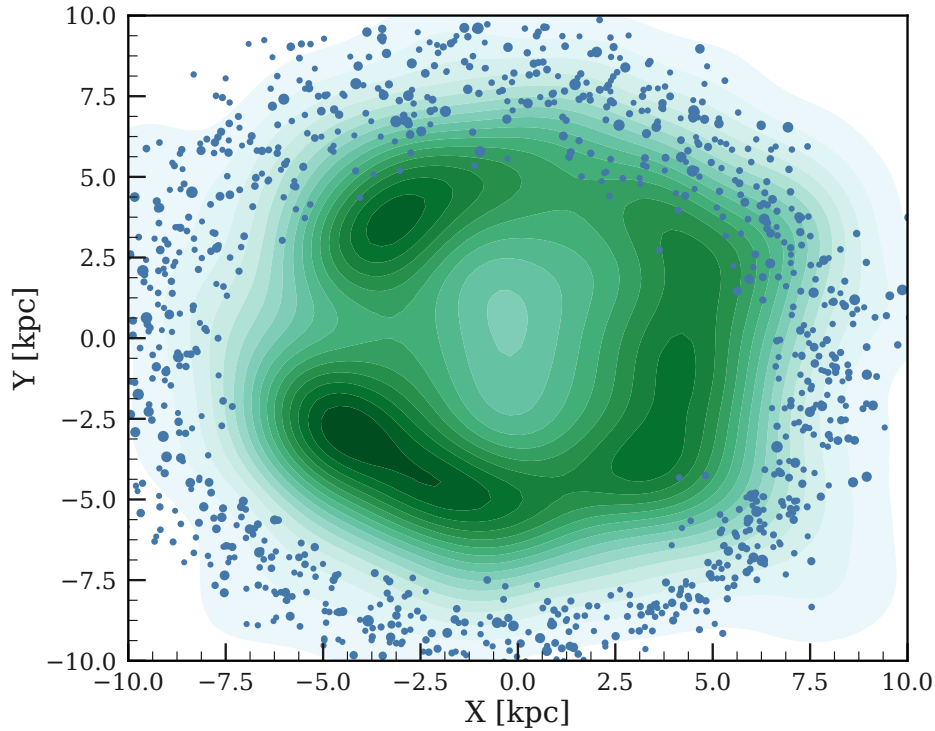


Figure 9.11: Distribution of the Solar siblings (blue bullets) in the Milky Way Galaxy (green shaded contours). We cut out the central 2kpc from the Galaxy to keep contrast in the disk. The Sun in this simulation could be anywhere where you see a blue bullet point. The simulation was performed using the script `amuse.examples.solar_cluster_in_live_galaxy`; the figure was made with `amuse.examples.plot_solar_cluster_in_live_galaxy`.

2012).

The hypothesis that the Sun originated in M67 raises a number of interesting questions. When did the Sun escape from the cluster, and does the solar system still bear any signature of this escape? This question may be linked to the formation and morphology of the Oort cloud and other trans-Neptunian objects.

We perform a simple experiment to study when the Sun and M67 had closest approaches. Possibly a recent close approach might be associated with the moment the Sun escaped the cluster. We integrate the orbits of the Sun and the center of mass of M67 backward in time in a potential representing the Milky Way Galaxy. The last stellar encounter prior to the Sun’s escaping the cluster might be a rather violent event, which could easily affect the orbits in the outer parts of the current solar system. It has therefore been argued that a single encounter, which may also have caused some long-lasting effect on the outer parts of the solar system, initiated the Sun’s escape from the cluster, and the moment of escape may be identified with the closest approach between the Sun and the cluster.

We realize, of course, that the orbit of the Sun or the cluster may have been affected by a passing molecular cloud, but we ignore that in this example (although we have already demonstrated (in Section 9.4.1.3 and Section 9.4.1.4) that this makes a big difference!). We encourage the reader to experiment with a more realistic Galaxy model

and study its consequences in relation to this question in Section 9.5.2. In Section 9.4.1 we present a method by which we might include this effect in the calculation, at least statistically.

We use the potential from `Galaxia` discussed in Section 9.4.1.2 to carry out this calculation. We initialize the main script with two point masses, one for M67 and one for the Sun. For the Sun we adopt the current position and velocity used earlier in Section 9.4.1.2. M67 is located at position (0.77, 0.0, 0.49) kpc and velocity (31.92, -21.66, -8.87) km/s relative to the Sun (Schönrich *et al.*, 2010). Using a bridge containing both the Sun and M67 in the Galactic potential, we integrate the equations of motion backward in time for the age of the Sun, 4.56 Gyr. The age of M67 seems to be somewhat smaller, but if the sun originated from the cluster the ages must of course be about the same (we consider it unlikely that the Sun is a blue straggler). The result of this backward integration is presented in Figure 9.12. According to this calculation, the Sun and M67 passed each other at a distance of ~ 92 pc about 2.59 Myr ago.

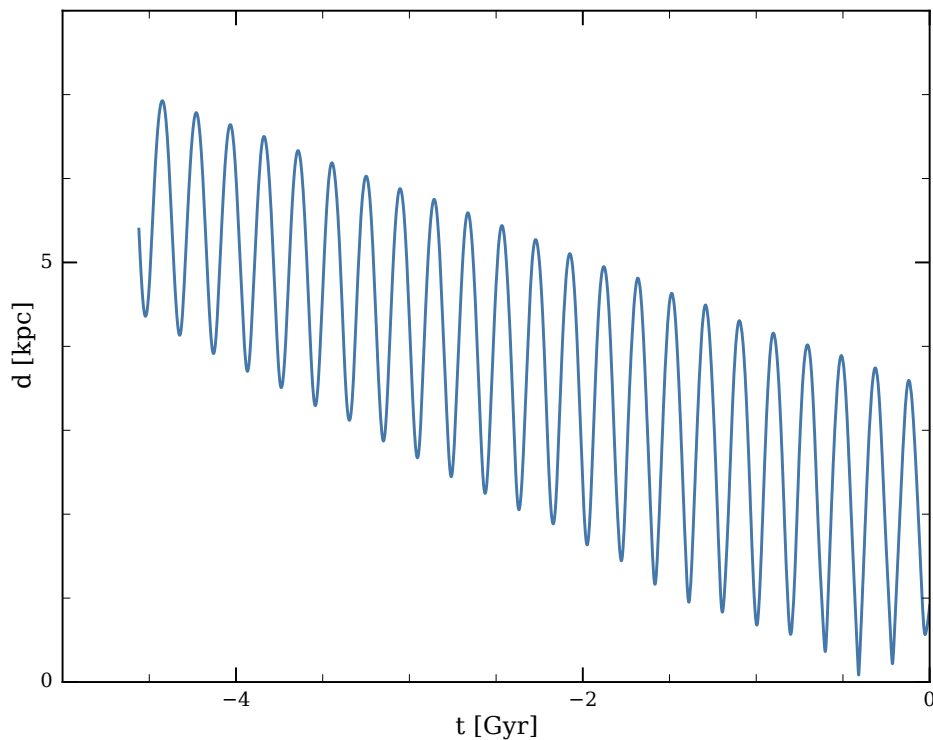


Figure 9.12: Distance between the Sun and the star cluster M67 as a function of time (negative numberz indicate the past). According to this calculation the Sun and M67 had a closest approach of ~ 92 pc some 2.59 Gyr ago. The simulation was performed using the routine in `amuse.examples.sun_and_m67_in_the_galaxy`, and took a few minutes to run.

Our assumed Galactic background potential did not evolve over the last ~ 2.59 Gyr, which is probably not realistic. Including the effects of transient spiral arms, massive molecular clouds, and star clusters surely affected the orbits over that period quite substantially (see for example Section 9.4.1.4). In Section 9.5.2 we explore when and with what velocity the Sun could have been ejected from the cluster and still end up at its current location.

9.4.3 Inspiral of a binary star into a common envelope

When two stars orbit each other, mass may flow from one to the other in a phase of Roche-lobe overflow (RLOF). Binaries are particularly vulnerable to RLOF when the more massive star ascends the giant branch. The stability of mass transfer, in terms of how rapidly mass is transferred to the companion, depends on the structure of the Roche-lobe-filling star, the response of the accreting star, and the resultant change in the orbital parameters of the binary system. RLOF in binary systems has been studied quite exhaustively, both theoretically and numerically (Webbink, 1984; Taam & Sandquist, 2000; Ivanova *et al.*, 2013).

In most triple systems RLOF probably does not proceed much differently than in a binary. For the majority of such systems, the most massive star, which evolves most rapidly, is a member of the inner binary. As a consequence, mass transfer is likely to begin in the inner binary system. The outer orbit is generally relatively wide, so mass lost from the inner binary is felt by the outer star as a slow wind. This affects the orbit of the outer star, and possibly the star itself, but probably not in a way much different than if the inner binary simply were a single star with an excess wind.

In some triples, however, this simple picture is inadequate. In those cases, the outer star is the most massive, and in a subset of these the outer orbital period is sufficiently small that mass transfer will start when the outer star ascends the giant branch. Table 9.2 presents an overview of such triples. The systems listed in Table 9.2 are indicated in Figure 9.13, where we also demonstrate that the most massive outer star experiences RLOF at some moment during the evolution.

Name	m_1	m_2 [$M_\odot$]	m_3	a_{in} [au]	e_{in}	a_{out} [au]	e_{out}	t_{RLOF} [Myr]	m_{RLOF} [$M_\odot$]	r_{Roche} [au]
τ CMa	17.8	17.8	50.0	0.076	0.0	2.49	0.285	4.3	41.5	0.95
V* CQ Dra	0.8	0.26	6.3	0.006	0.0	5.42	0.0	70.1	6.02	2.64
ξ Tau	3.2	3.1	5.5	0.133	0.0	1.23	0.15	83.7	5.50	0.42
V* V1334 Cyg	1.62	1.83	4.4	4.62	0.0	10.7	0.0	168	1.67	3.99
V* DL Vir	2.5	1.4	3.68	0.037	0.0	6.68	0.46	269	3.44	2.34
V* d Ser	2.02	1.94	2.5	0.047	0.0	1.94	0.47	799	2.46	0.62
HD 97131	1.29	0.9	1.5	0.037	0.0	0.80	0.191	2960	1.48	0.26
KIC002856960	0.24	0.22	0.76	0.008	0.0064	0.73	0.612	$> H_0$	0.72	0.12

Table 9.2: Known triples for which the outer (tertiary) star is more massive than either of the two inner stars, and the Roche radius is smaller than the maximum size of the outer star at the tip of the Asymptotic Giant Branch. Column 1 gives the common name of the triple system; columns 2–4 give the masses of the triple components; columns 5–8 give the semi-major axis and eccentricity of the inner and outer orbits; columns 9–11 give the estimated moment at which the outer (most massive) star starts to overfill its Roche lobe, the mass of that star, and its size.

RLOF in these triples is considerably different than mass transfer in an ordinary binary system. One reason is that the mass cannot simply be accreted by the inner object, but may be ejected via a slingshot effect by the rapidly orbiting inner binary. During mass transfer the inner binary also evolves, both stars may accrete some of the material delivered by the outer star, and the leftover material either forms a circumbinary disk or is ejected from the system. At the same time, orbital angular momentum

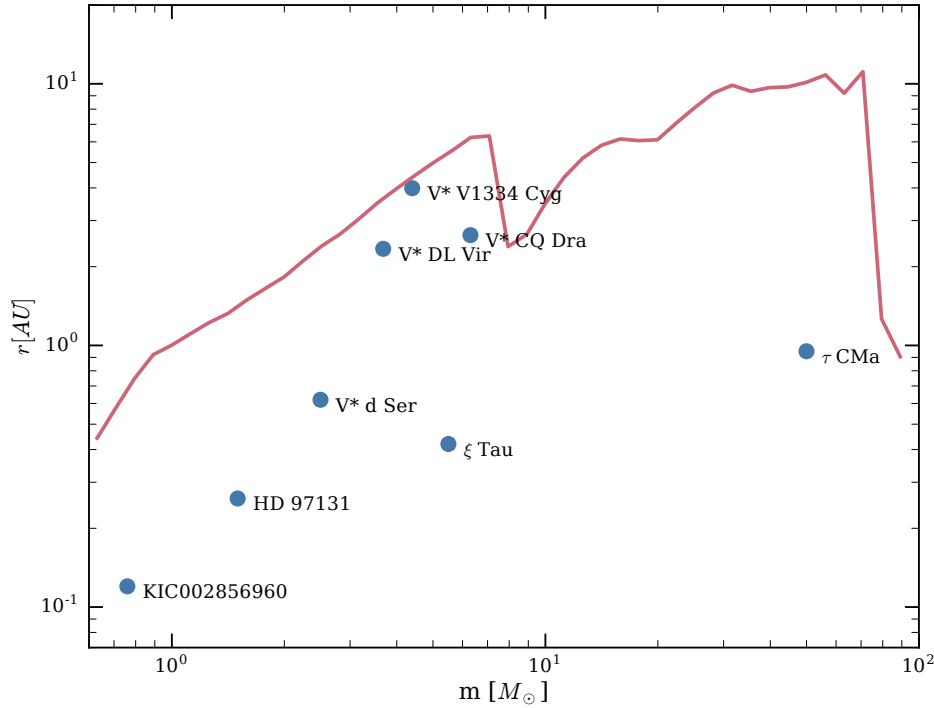


Figure 9.13: Maximum stellar radius at the tip of the asymptotic giant branch as a function of mass (red curve), calculated using `SeBa`. Blue bullets indicate the radius of the (most massive) outer component at the moment of Roche-lobe overflow, for the triple systems listed in Table 9.2. This figure was made using the script `amuse.examples.plot_rlof_triples`.

exchange may affect the eccentricity of the inner binary via to the von Zeipel-Lidov-Kozai (Lidov, 1962; Kozai, 1962) effect.

Simulating all these effects requires both an accurate gravitational solver and some way of taking the hydrodynamics of the system properly into account. The subtleties of von Zeipel-Lidov–Kozai are not well handled by the rather low-accuracy gravity solvers normally used in SPH codes. Instead, we adopt here a 4th-order Hermite solver to integrate the equations of motion of the two inner binary components and the core of the outer most star (see also Section 8.4.5). The giant envelope is modeled using an SPH code, and its core is represented as a dark-matter particle in the hydro code. The coupling between the gravity solver and the hydrodynamics code is realized using Bridge.

We adopt the observed parameters of the triple system ξ Tau as initial conditions for our simulation. We start by evolving the $5.5 M_{\odot}$ primary star using `EVTwin` (Eggleton *et al.*, 2011; Eggleton, 1971, 2006), up to the moment when it fills its Roche lobe, at 0.42 au at an age of 83.7 Myr. The star is then converted to a hydro realization (see Section 6.4.2) with a core mass of $1.4 M_{\odot}$ and placed in the triple system with the orbital parameters listed in Table 9.2. Since the stars lost virtually no mass we do not adjust the orbital parameters to take pre-RLOF mass loss into account. The orbit of the outer star is placed at the relatively low inclination of 9° ; because of the von Zeipel-Lidov–Kozai effect, an inclination initially higher than $\sim 45^{\circ}$ would drive the inner binary into a state of mass transfer before the outer star had a chance to undergo

RLOF.

Figure 9.14 presents the evolution of the semi-major axis of the outer orbit of the ξ Tau triple system. Since the Roche-lobe filling giant hardly loses any mass during its evolution, we simply adopt the sum of the total gas mass plus the core mass as the giant's total mass. The outer orbit shrinks within ~ 10 years from the initial 1.23 au to ~ 1.15 au after the onset of RLOF. During the same period, the eccentricity of the outer orbit decays from 0.15 to ~ 0.1 .

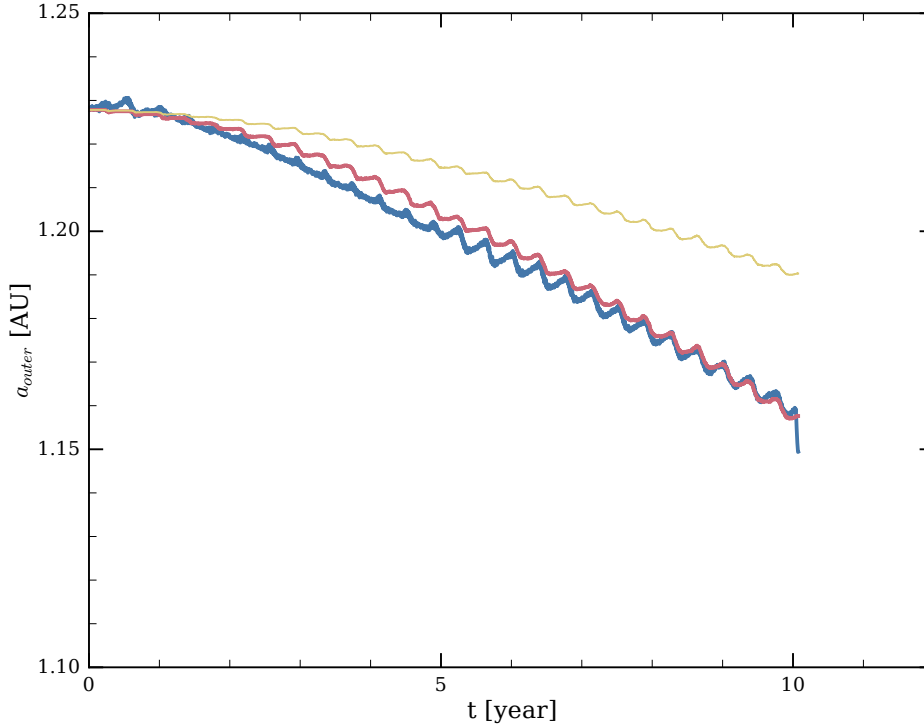


Figure 9.14: Evolution of the semi-major axis of the outer orbit in ξ Tau. In blue, we plot the result of the SPH simulations. Overplotted in red is the semi-analytic solution (Equation (9.17)) using $\eta = 6$. Two additional curves are plotted for $\eta = 4$ (yellow) and $\eta = 8$ (green). The simulation was performed using the script `amuse.examples.xi_tau_rlof`, the plotting was done with `amuse.examples.plot_xi_tau_orbital_evolution`.

We can derive an expression for the effect of mass transfer in the system, starting with a binary of primary mass m_3^0 (note here that the primary is the outer component) and secondary mass $m_{\text{in}}^0 = m_1 + m_2$, and ending after mass transfer with masses m_3 and m_{in} for the primary and secondary stars, respectively. The degree of mass conservation is expressed by the parameter $f = (m_3 + m_{\text{in}})/(m_3^0 + m_{\text{in}}^0)$. Ignoring the internal angular momentum of the outer giant star and the inner binary system, we can express the evolution of the semi-major axis according to the redistribution of mass (Huang & Taam, 1990):

$$\frac{\dot{a}}{a} = -2 \frac{\dot{m}_3}{m_3} \left(1 - \eta \frac{m_3}{m_{\text{in}}} - \frac{1}{2} \frac{(1 - \eta)m_3}{m_3 + m_{\text{in}}} - f \frac{(1 - \eta)m_3}{m_3 + m_{\text{in}}} \right). \quad (9.17)$$

Here η is the specific angular momentum of mass leaving the system, relative to the specific angular momentum of the accreting system (in this case the center of mass of

the inner binary system). Interestingly, this is essentially the same equation (Equation (5.18)), we already saw in relation to the non-conservative evolution of semi-detached binaries in Chapter 5. Overplotted in Figure 9.14 is the evolution of the orbital separation of the outer binary in this semi-analytic model [Equation (9.17)] with $\eta = 6$, a value consistent with losing mass through the second Lagrangian point (L_2) of the outer binary system.

9.4.4 Budding planets in a protoplanetary disk

In an attempt to model the early evolution of a planetary system we simulate a young star surrounded by a gaseous disk and a number of young planets. This is a typical example in which hydrodynamics and gravitational dynamics are both relevant and operate on similar time scales, presenting an outstanding example of the power of AMUSE. The script used to perform these calculations should by now look rather familiar, except perhaps in the technique used to write composite data files (see Appendix A.2.2).

We initialize the protoplanetary system using a central star of mass $m_\star = 1.73 M_\odot$, which happens to be consistent with the mass of β Pic. This system is thought to harbor at least one rather massive planet and a gas disk, but its true composition may be considerably more complex (Winn & Fabrycky, 2015). For this simulation we limit ourselves to a collection of planets with total mass $0.01m_\star$ and a gaseous disk of the same mass. The planets were arranged according to the Oligarchic growth model proposed by Kokubo & Ida (2002), and for which Tremaine (2015) proposed a more general scenario (see also Section 3.4.2). The disk was initialized with a Safronov–Toomre Q-parameter $Q = 1$ (see Section 6.4.1),

Our calculation had 7 planets in circular orbits ranging from a $\sim 0.4 M_{\text{Jup}}$ planet at a separation of 1.0 au to a $\sim 8.1 M_{\text{Jup}}$ planet at ~ 53 au. The gaseous disk was generated using routine `ProtoPlanetaryDisk`, as already discussed in Section 6.4.1, between 1 au and 100 au in the same plane as the planets. The equations of motions were integrated using `mercury` (Chambers & Migliorini, 1997; Chambers, 2012) and the hydrodynamics was resolved using `Fi` (Hernquist & Katz, 1989; Gerritsen & Icke, 1997; Pelupessy *et al.*, 2004). We neglected the evolution of the parent star, since we evolved the system for only 10^4 yr.

Figure 9.15 presents a snapshot of the simulation about 280 yr after the start of the simulation. The planets are represented by various colored bullets with sizes proportional to the logarithm of the planet mass. The thin disk ($Q = 1$) is perturbed by the planets, creating spiral waves. This is particularly visible at middle left, where a slightly inclined planet drives a large spiral arm in the disk. The leading arm (spiral-ing inwards) has its low-density wake on the outside whereas the trailing arm (going outwards) has the low-density wake on the inside. The other planets are barely visible in the image, but their presence can be seen by the structure they induce in the disk.

Although this is a rather simple demonstration of the capabilities of bridge, much more elaborate experiments including moons and other stars can easily be implemented by adjusting the example script in `amuse.examples.planetary_system`.

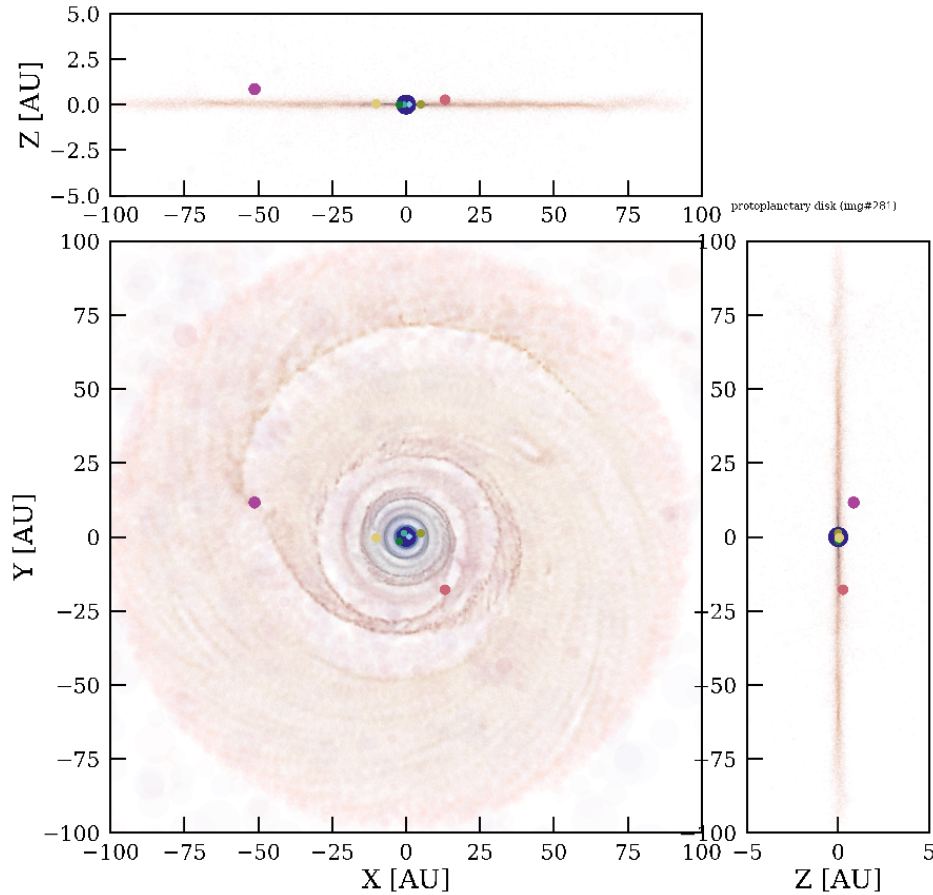


Figure 9.15: Young star with planets (bullets) and a rather flat gaseous disk. This image was made about 280 years after the start of the simulation. The planetary system was generated according to the Oligarchic growth model (see `amuse.examples.make_planets_oligarch`) with a total mass of 1% of the central star’s ($1.73 M_{\odot}$) mass, with the same mass in the gaseous disk (see `ProtoPlanetaryDisk`). The simulation was performed using the routine in `amuse.examples.planetary_system` (which took about a week to run); plotting was done with `amuse.examples.plot_planetary_system`.

9.4.5 Planetary systems in star clusters

Stars in star clusters are often found in pairs, or even triples or higher-order multiples. “Democratic” multiples, in which no pair of objects dominates the binding energy, tend to be unstable and quickly decay into simpler components—ultimately single stars and binaries, the fundamental building blocks of star clusters. Some higher-order hierarchies can survive for a very long time, although eventually most of those also dissolve. The realization that many stars are born with multiple planets (and smaller objects) demands a numerical treatment of hierarchical systems. (We note in passing that a stellar distribution around a supermassive black hole is in many ways a similar numerical problem.)

So far we have only discussed binaries in our discussion of stellar evolution, and how to manage encounters in star clusters in Sections 8.7.1 and 8.7.2. Here we briefly discuss the ongoing development of how to handle rich hierarchical systems of stars, planets and maybe even planetesimals. The module discussed in Section 8.7.2 is intended for

simulating star clusters with a rich population of binaries, whereas the **Nemesis** module described here is meant for integrating planetary systems inside a star cluster.

The relatively short orbital periods of planetary systems with respect to the crossing time of a star cluster gives rise to a number of interesting numerical challenges. One of these is wide range in time scales, which can be days for a planet orbiting a star compared to a million years for a star orbiting the center of mass of a cluster. This wide separation in scales allows us to treat the planetary system differently than the star cluster, often using a different code. The bridge method is ideally suited for such a hierarchical structure.

In **Nemesis** (see Section 10.1.2) each planetary system is integrated with its own individual integrator. When several stars or planetary systems approach each other within a predefined limit, both systems are merged and the integration is continued using a single integrator. If a multi-body system dissolves, single objects are picked up by the global N -body integrator whereas multi-body subsystems continue to be integrated with individual solvers. Integrators in this method are dynamically created and deleted at runtime. **Nemesis** is fast, efficient, parallel, and effective in addressing multi-planetary systems in a star cluster. At the time of writing the **Nemesis** module is not yet available in AMUSE, as we are still in the testing phase (see Figure 9.16). We can say, however, that the method seems to work as expected, and that **Nemesis** will soon become part of the AMUSE code base.

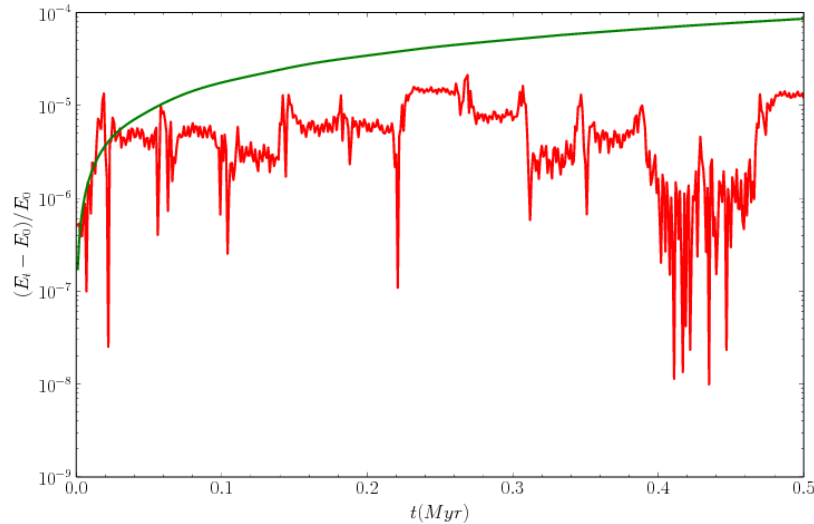


Figure 9.16: Evolution of the numerical integration error in the energy of the solar system when orbited by second $1 M_{\odot}$ star. In this example the other star (allegedly called Nemesis) orbits the solar system in the ecliptic plane with a semi-major axis of 1 pc and an eccentricity of 0.6. The green curve gives the energy error when integrated using **Hermite**; the red curve give the result of the **Nemesis** module.

9.5 Assignments

9.5.1 Drift with gravity code

In Section 9.3.1 we discussed a small experiment in which we integrated the equations of motion for the Sun, Earth, and Moon using **Bridge**. In that example we employed three incarnations of **ph4** to integrate a single Sun, a single Earth and a single Moon, and coupled these three codes using multiple bridges. This was a little excessive, and it would have been much better to use the drifter code in Code example 9.5 instead of the direct N -body codes. Rewrite Code example 9.6, Code example 9.7 and Code example 9.8 using the drifter code from Code example 9.5, and compare them with a small script in which all the particles are integrated using the direct N -body code. Which of the various methods would you prefer, based on reliability, speed, and accuracy?

9.5.2 How did the Sun escape from M67?

If the Sun escaped from the star cluster M67, as discussed in Section 9.4.2, this might have happened quite a while ago, when the cluster and the Sun had a closest approach. In the small calculation performed earlier, we adopted a simple analytic potential for the Galactic background, launched the Sun and M67 from their current locations, and integrated backwards in time for 4.56 Gyr. According to this calculation, at closest approach, the distance between the Sun and the M67 was 92 pc. However, not mentioned in Section 9.4.2 is that the relative velocity between Sun and the cluster at that moment was ~ 48 km/s, which is rather high. An encounter with another cluster member could have ejected the Sun, but this would probably have led to a velocity of the order of the cluster's escape speed, which is an order of magnitude smaller. An encounter that propelled the solar system to a speed of almost 50 km/s would have profound consequences for the further evolution of most of the planets orbiting the Sun.

In this assignment we recognize that the current coordinates of M67 are not precisely known, and ask if we were allowed some freedom, would it be possible to find a better match between the trajectories of the Sun and the cluster? First repeat the calculation in Section 9.4.2 and confirm the earlier result. Then allow the current position and velocity of M67 relative to the solar system to vary within the observed uncertainty, and search for the closest historical distance and relative velocity between the two point masses in the Galactic potential. The underlying assumption is that the Sun was ejected from M67 and that its orbit has not changed appreciably since then.

We could search parameter space by hand, but this is a rather painful process. A better method would be to adopt a Markov-Chain Monte Carlo (MCMC) approach using the Metropolis-Hastings algorithm (Metropolis *et al.*, 1953; Hastings, 1970) to find the optimum solution. This algorithm works quite well for searching a large parameter space. There are more efficient alternatives (Feroz *et al.*, 2009), but MCMC tends to find solutions close to the optimum quite effectively and it is rewarding to implement by yourself.

The Metropolis-Hastings algorithm works as follows.

1. Perform a calculation with some predetermined initial set of parameters, like the Sun's current location and velocity in the Galaxy and that of M67.
2. Integrate both objects backwards in time in the Galactic potential for 4.6 Gyr and measure the minimum “distance” (in position r and velocity v) between the Sun and M67. Let's call this phase-space distance d . In order to be able to add distance and velocity we have to normalize them, in which case one could define d by $d^2 = (r/r_{\min})^2 + (v/v_{\min})^2$, where $r_{\min}$ and $v_{\min}$ are the minimum joint values found for r and v . After the first calculations $d^2 = 2$.
3. Introduce a small change to the initial position and velocity of M67 within the observational uncertainty, or simply adopt a variation of ~ 100 pc in position and ~ 1 km/s in velocity. You could use a more formal approach from the literature, but this procedure will give a reasonable solution; we can later return to this assumption to see if the final solution is reasonable.
4. Repeat the calculation with the introduced variation in the position and velocity of M67, and measure the new minimum phase-space distance, which we call d' . If $d' > d$ the Sun passed the orbit of M67 at a larger phase-space distance, but if $d' < d$ the new solution is better.
5. Now draw a random number X on the interval $[0, 1]$. If $X < \exp(-(d' - d)/d)$ we accept the new initial conditions as the best coordinates for M67, and we update $r_{\min}$ and $v_{\min}$ accordingly. Otherwise we keep the previous best initial conditions.
6. Repeat this procedure (from step 3 to step 5) until you are happy with the results. While performing the iterative procedure you may want to reduce the uncertainty. It is generally best to do this slowly and preferably only when you adopt the new initial realization. You will find that the quality of the calculation improves with the number of iterations. The rate at which it improves depends on your choices for the variations in position and velocity introduced in each new calculation. It may be rewarding to experiment with this to find the most efficiently converging Markov chain.

Instead of writing your own MCMC code, you could adopt a package, such as **emcee** (Foreman-Mackey *et al.*, 2013) which, due to its Python interface, works naturally with AMUSE.

Address the following questions:

- What is the minimum distance and relative velocity you find for M67 and the solar system?
- Is this a realistic solution? Take into account here that the orbital speed of Neptune is ~ 5.5 km/s, that the solar system probably already had the planet by the time it left the cluster, and Neptune still orbits the Sun.

- What difference does it make if you change the potential of the Galaxy, change the number of spiral arms using the `BarAndSpirals3D()` routine from `Galaxia` as discussed in Section 9.4.1.2, or introduce giant molecular clouds (see Section 9.4.1)?
- Instead of searching for the optimal position and velocity of the cluster, you could relax the assumptions about the Galaxy, such as the mass of the bulge and bar, the pitch angle of the spiral arms or their rotation speed. One advantage of the Metropolis-Hastings algorithm is that it is rather agnostic about the parameters you change so long as you define a proper phase-space distance on which to optimize.

9.5.3 Half-treecode, half-direct

We want to study a large system of self-gravitating particles initially distributed in a virialized [Plummer \(1911\)](#) sphere, with a stellar mass function running from $0.1 M_{\odot}$ to $100 M_{\odot}$. We have insufficient computer resources to perform the entire calculation using a direct N^2 method, but we require a precision that exceeds that of a standard treecode. We therefore decide to create a hybrid *split method* in which we combine a treecode with a direct N -body code by interfacing the two codes using `Bridge` within the AMUSE framework.

We divide the stars among the two codes, where the treecode handles the low-mass stars (those with $m < m_{\text{cut}}$) and the direct N -body code takes care of the high-mass stars (with $m \gtrsim m_{\text{cut}}$). In order to optimize the simulation, we will study the behavior of the hybrid code as a function of the mass cut m_{cut} which determines which particles are treated by which code. Although we will perform the numerical experiments with units, we will not take stellar evolution into account because we are currently only interested in validating the proposed hybrid method.

1. Why do we split the particle set in mass rather than, for example, in distance from the center of mass or as a function of local density?
2. Estimate the bridge time step relative to the typical orbital timescale you will use for your calculation.
3. Write the hybrid script and give it an interface so that you can run it from the command line.

Initialize the system with $N = 10^4$ stars with a virial radius of $r = 3 \text{ pc}$, and integrate it for 10 Myr. Perform the following three runs and store the half-mass radius, the core radius, and the total kinetic and potential energies at a time resolution of 0.1 Myr:

- all stars in the treecode;
- all stars in the direct code;
- split code, with $m_{\text{cut}} = 1.0 M_{\odot}$.

Make the following two plots

- the relative energy error compared to the initial energy ($(E - E_0)/E_0$) as a function of time,
- the half-mass and core radii as functions of time.

Also make a table of the wall-clock times of these three runs.

Discuss your findings, and say whether or not running low-mass stars with a tree code and high-mass stars with a direct code is an advantageous strategy.

9.5.4 The accreting black hole in HLX-1

HLX-1 is a variable and bright ($\gtrsim 10^{42}$ erg/s) x-ray source in the direction of the spiral galaxy ESO 243-49 (Farrell *et al.*, 2009). The reasons for its repeated outbursts and the nature of the x-ray source remain elusive (Webb *et al.*, 2017). Figure 9.17 shows a portion of the x-ray counts reported by the SWIFT XRT instrument from Fig. 3 of (Titarchuk & Seifina, 2016).

Observations support the hypothesis that the x-rays are produced by a black hole of mass $10^3 - 10^5 M_\odot$ that is orbited by a massive star (Webb *et al.*, 2012). The companion star is thought to be evolved, so that it fills its Roche lobe near pericenter, but at apocenter no mass is transferred and therefore no x-rays are produced. We can make a very simple model to test this hypothesis by integrating the orbit of a $20 M_\odot$ star around a black hole. In Figure 9.17 we present the results of such a simple calculation in which the giant donor orbits a $10^3 M_\odot$ black hole in a 370 day orbit. To ensure that the donor overflows its Roche lobe near pericenter we adopt an eccentricity of 0.95. The x-ray count rate is assumed to be proportional to the degree to which the $20 M_\odot$ overfills its Roche lobe, which only happens near pericenter.

The counts calculated in the simple model do not align with the observed counts, although a number of features of the observations are shared by the model calculation. Tidal effects, as well as the viscous evolution of the donor star, probably contribute to the variations observed in the x-ray outburst period. In this assignment we will try to improve the simple model used to produce Figure 9.17. We adopt the MESA (Paxton *et al.*, 2010, 2011) stellar evolution package with the Huayno *N*-body integrator (Pelupessy *et al.*, 2012). We evolve a $20 M_\odot$ star for 9.0 Myr, by which time it has ascended the giant branch. We start the simulation when the star has lost $1.54 M_\odot$ as a stellar wind, and its radius is 4.19 au, sufficient to fill its Roche lobe at pericenter for the adopted binary period.

There are two basic approaches we can follow here—model the tidal evolution of the donor star in the black-hole field, or use a smoothed-particle hydrodynamics code to resolve the donor. For tidal evolution we recommend using the `tidal_interaction` routine from the `stellar_tidal_evolution` package in `amuse.ext`. This prescription follows the tidal evolution of planets and stars according to the model by Hansen (2010), which is based on the theory of equilibrium tidal models by Eggleton *et al.* (1998).

To use an SPH code to resolve the donor star in its orbit around the black hole, we convert the outer layers of the donor star to an SPH realization using at least 1000

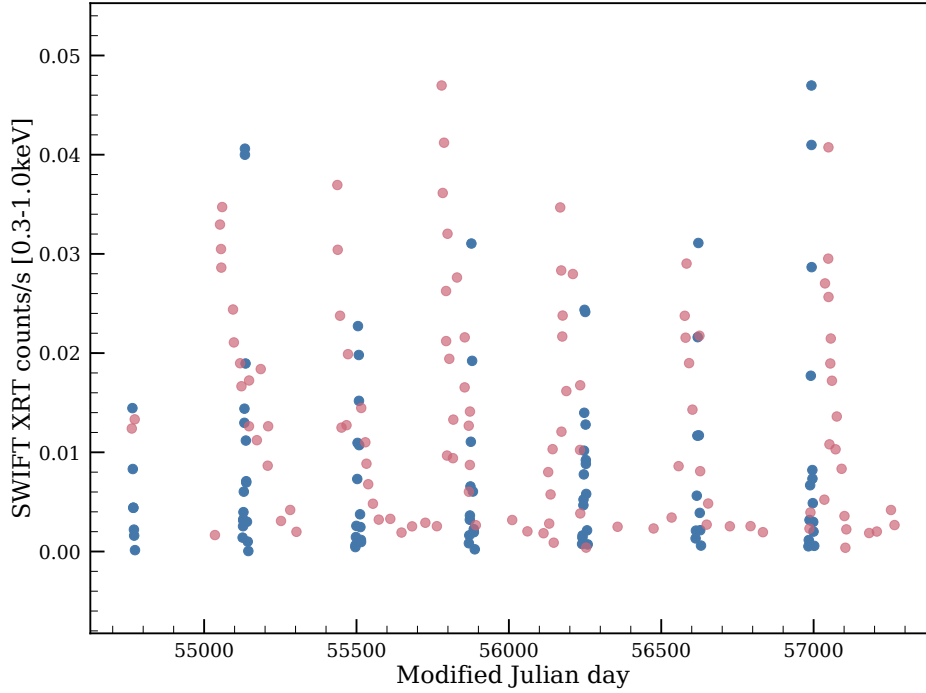


Figure 9.17: X-ray counts of the source HLX-1 as observed by SWIFT XRT in the 0.3–1.0 keV band (Titarchuk & Seifina, 2016) (red bullets). Overplotted are the results of the simple model calculation described in the text (blue bullets).

particles, with the central portion of the star represented by a single massive particle. This can be realized by generating an SPH realization of a star in which a core mass is specified (see Section 8.5.4.1 and in particular Code example 8.11, where we discussed how to convert a stellar evolution model to an SPH realization). Code example 9.15 shows how a star can be evolved to the giant stage and subsequently converted to a SPH realization with a single particle representing its core. We used the same method in Section 9.4.3 to generate the giant star with a core in order to simulate the triple system Xi Tau. In Code example 9.15 we use the argument `with_core_particle` to indicate that the stellar model has a core which should be returned in a separate particle set.

A bridge can then be built with the gravity code to integrate the black hole and the donor’s core, using the SPH code to resolve the stellar envelope. Allow the black hole to accrete using a sink particle, and measure, as a function of time, the amount of material accreted by the black hole. Integrate the system for half a dozen orbits, as demonstrated in Figure 9.17, and plot the accretion rate of the black hole together with the observational data.

Which of the two models, tidal evolution or detailed hydrodynamics, gives a better comparison with observations?

```

26 stellar = Mesa()
27 stellar.particles.add_particle(Particle(mass=o.mass))
28 XiTau = stellar.particles[0]
29 while XiTau.radius <= o.radius:
30     print(
31         f"Time={stellar.model_time.in_(units.Myr)} "
32         f"{stellar.particles.radius.in_(units.AU)} "
33         f"{stellar.particles.stellar_type}"
34     )
35     stellar.evolve_model()
36
37 target_core_mass = XiTau.core_mass
38 print(XiTau)
39 print("Core mass:", XiTau.core_mass.in_(units.MSun))
40 sph_model = convert_stellar_model_to_sph(
41     XiTau,
42     o.Nsph,
43     with_core_particle=True,
44     target_core_mass=target_core_mass,
45     do_store_composition=False,
46     base_grid_options=dict(type="fcc")
47 )
48 core_particle = sph_model.core_particle.as_set()
49 gas_particles = sph_model.gas_particles
50
51 print(f"Ngas={len(gas_particles)} Ncore={core_particle}")
52 write_set_to_file(
53     core_particle, "Hydro_PrimaryStar_XiTau.amuse", "amuse",
54     append_to_file=False
55 )
56 write_set_to_file(gas_particles, "Hydro_PrimaryStar_XiTau.amuse", "amuse")
57 stellar.stop()

```

Code example 9.15: Evolving a star and turning it into a SPH realization with a single particle representing the stellar core.

9.5.5 The formation of the widest binary stars

In 1915 Robert Innes discovered that the binary system α Centauri is orbited by a third single star (Innes, 1915). The discovery was made by examining photographic plates of the region around α Centauri using a blink microscope to search for faint stars with large proper motion (Innes, 1915). It was not until 1917 that Innes (1917) referred to this star as Proxima Centauri: “If this small star had a name it would be convenient—it is therefore suggested that it should be referred to as Proxima Centaurus.”

Today the orbit of Proxima Centauri is well constrained to have semi-major axis $8.7^{+0.7}_{-0.4} \times 10^3$ au and eccentricity $0.50^{+0.08}_{-0.09}$ (Kervella *et al.*, 2017). The inner α Centauri binary has a semi-major axis of about 23.7 au and an eccentricity of 0.524 ± 0.0011 ; the three stars have masses $1.105 \pm 0.0070 M_{\odot}$, $0.934 \pm 0.0061 M_{\odot}$ and $\sim 0.123 M_{\odot}$ (Pourbaix *et al.*, 2002). This very wide orbit is not explained from standard star (and binary) formation theories. An interesting alternative for the formation was proposed by Reipurth & Mikkola (2012). They argue that three stars embedded in a gas cloud can find each other and engage in a triple resonant interaction. The result of the interaction

is the ejection of one of the stars, but due to the presence of the surrounding gas the star does not escape but finds itself parked in a wide orbit around the binary system.

The calculations performed in [Reipurth & Mikkola \(2012\)](#) employed a direct N -body code which in AMUSE is called `Mikkola`. In order to perform the calculations, [Reipurth & Mikkola \(2012\)](#) added a smooth background potential representing the gas. They concluded that the existence of extremely hierarchical systems is a natural consequence of resonant interactions embedded in a primordial gas reservoir. Fundamental to this formation channel is the interaction with the background gas potential, accretion onto the stars, and stellar gravitational interactions. With AMUSE we can relatively easily reproduce their results, but we can also include the actual background gas, rather than a static background potential.

In Section 9.5.3 we had an assignment in which we bridged two gravity codes, one for the low-mass “background” stars and the other for the more massive stellar population. Here we will replace the background population with a gas distribution, simulated using an SPH code.

1. Adopt a mass of $0.1 M_{\odot}$ for each of the three stars and distribute them in a Plummer sphere with a characteristic radius of 400 au. [Reipurth & Mikkola \(2012\)](#) adopted a mean separation between 40 au and 400 au.
2. Add an SPH particle representation to the system, centered on the center of mass of the three-body system, with a mass of $10 M_{\odot}$ and characteristic radius 7500 au.
3. [Reipurth & Mikkola \(2012\)](#) allowed the three stars to accrete from the ambient gas. We can take accretion onto the three stars into account by having a sink particle move along with the stars (see Section 6.3.2). Stop the simulation after 100 Myr and study the resulting triple system.

[Reipurth & Mikkola \(2012\)](#) argued that the resulting projected orbital separation s matches the power law (see their Fig. S1)

$$f(s)ds \sim s^{-1.6}ds. \quad (9.18)$$

Perform a number of simulations ([Reipurth & Mikkola, 2012](#); performed a total of 180,000) with randomized initial realizations from the above initial conditions, and compare their projected separation with the distribution suggested by [Reipurth & Mikkola \(2012\)](#).

A key aspect of this model is that the three embryonic stars are initially identical, but are forced to compete for gas to accrete through dynamical interactions. The consequential change in mass subsequently feeds back into their orbital evolution, resulting in the power-law distribution in orbital separation. Note however, that the obtained $s^{1.6}$ power-law is in reasonable agreement with observations (or maybe a bit steeper, see [Reipurth & Mikkola, 2012](#)).

Repeat the experiment using 4 stars rather than 3, each with a mass of $0.1 M_{\odot}$, adopting the same initial conditions as before, and including the sink particles. Answer the following two questions:

- What is the fate of the fourth star?

- What are the distributions of the final masses of the four stars?
- Do you expect that the inner binary contains the two most massive stars, as in the observed Alpha-Proxima Centauri triple system.

9.5.6 Photoevaporating disk in binary system

In Section 10.1.3 we discuss the possibility of coupling multiple codes into a self-scheduling organized fashion. The described method, **venice**, allows multiple codes to be coupled on their own internal temporal resolved and spatially resolved scales. This leads to AMUSE to address intricate multi-scale and multi-physics problems.

One such a problem is the photoevaporation and gravitational perturbation of a circumstellar disk around a low-mass star in an eccentric orbit around a massive companion star. Such a disk is hydrodynamically affected by the gravity of the companion, while being photoevaporated. In AMUSE two methods can be effective in addressing this problem. It is possible to resolve the disk using a hydrodynamics solver together with the gravity using an N -body solver and the radiation with a radiative transfer code. The other way would be using the combination of **Vader** (see Appendix B.2.3), as a disk-evolution code, with a Kepler solver together with the FRIED grid (see Appendix B.2.4). Both methods allow you to study various aspects of such a problem. In this assignment we are going to try both methods.

The key aspect for the evolution of an irradiated and gravitationally influenced disk around a low-mass star is the truncation, evaporation, and structural changes in the disk due to the other star. The first method, in principle, incorporates all these effects, whereas the more parameterized method only informs us about the disk's photo-evaporation.

Write a script to address the coupled problem by bridging an SPH code for the disk with a radiative transfer code, a stellar evolution code, and a direct N -body solver for the mutual gravity.

Write a second script, using **venice**, where the gravity is addressed with a Kepler solver, the disk using **vader**, the stellar evolution with the appropriate solver, and the radiative interaction with the FRIED grid.

Answer the following questions:

- To what extent is the disk gravitationally affected by the companion star?
- To what extent is the disk gravitationally affected by the radiative transfer?
- What can you say about the difference in speed when comparing the two scripts?
- Which of the two approaches is most satisfactory for addressing the problem's key questions?

Bibliography

Adams, D.N. 1979. *The Hitchhiker's Guide to the Galaxy*. Pan Books.

- Allen, Christine, & Santillan, Alfredo. 1991. An improved model of the galactic mass distribution for orbit computations. *Revista Mexicana de Astronomia y Astrofisica*, **22**(Oct.), 255.
- Barnes, Josh, & Hut, Piet. 1986. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nat*, **324**(6096), 446–449.
- Bottke, Jr., W. F., Vokrouhlický, D., Rubincam, D. P., & Broz, M. 2002. The Effect of Yarkovsky Thermal Forces on the Dynamical Evolution of Asteroids and Meteoroids. *Pages 395–408 of: Bottke, Jr., W. F., Cellino, A., Paolicchi, P., & Binzel, R. P. (eds), Asteroids III.*
- Bovy, Jo. 2015. galpy: A python Library for Galactic Dynamics. *ApJS*, **216**(2), 29.
- Camarillo, Tia, Mathur, Varun, Mitchell, Tyler, & Ratra, Bharat. 2018. Median Statistics Estimate of the Distance to the Galactic Center. *PASP*, **130**(984), 024101.
- Chambers, J. E., & Migliorini, F. 1997 (July). Mercury - A New Software Package for Orbital Integrations. *Page 27.06 of: AAS/Division for Planetary Sciences Meeting Abstracts #29.* AAS/Division for Planetary Sciences Meeting Abstracts, vol. 29.
- Chambers, John E. 2012 (Jan.). *Mercury: A software package for orbital dynamics.* Astrophysics Source Code Library, record ascl:1201.008.
- Cooder, R. 1972. *Crow Black Chicken.*
- Cox, Donald P., & Gómez, Gilberto C. 2002. Analytical Expressions for Spiral Arm Gravitational Potential and Density. *ApJS*, **142**(2), 261–267.
- Eggleton, P. P., Tout, Christopher, Pols, Onno, Izzard, Rob, Eldridge, John, Lesaffre, Pierre, Stancliffe, Richard, Church, Ross, & Lau, Herbert. 2011 (July). *STARS: A Stellar Evolution Code.* Astrophysics Source Code Library, record ascl:1107.008.
- Eggleton, Peter. 2006. *Evolutionary Processes in Binary and Multiple Stars.*
- Eggleton, Peter P. 1971. The evolution of low mass stars. *MNRAS*, **151**(Jan.), 351.
- Eggleton, Peter P., Kiseleva, Ludmila G., & Hut, Piet. 1998. The Equilibrium Tide Model for Tidal Friction. *ApJ*, **499**(2), 853–870.
- Farrell, Sean A., Webb, Natalie A., Barret, Didier, Godet, Olivier, & Rodrigues, Joana M. 2009. An intermediate-mass black hole of over 500 solar masses in the galaxy ESO243-49. *Nat*, **460**(7251), 73–75.
- Feroz, F., Hobson, M. P., & Bridges, M. 2009. MULTINEST: an efficient and robust Bayesian inference tool for cosmology and particle physics. *MNRAS*, **398**(4), 1601–1614.
- Foreman-Mackey, Daniel, Hogg, David W., Lang, Dustin, & Goodman, Jonathan. 2013. emcee: The MCMC Hammer. *PASP*, **125**(925), 306.
- Fujii, Michiko, Iwasawa, Masaki, Funato, Yoko, & Makino, Junichiro. 2007. BRIDGE: A Direct-Tree Hybrid N-Body Algorithm for Fully Self-Consistent Simulations of Star Clusters and Their Parent Galaxies. *Publ. Astr. Soc. Japan*, **59**(Dec.), 1095.
- Gaia Collaboration, Brown, A. G. A., Vallenari, A., Prusti, T., de Bruijne, J. H. J., Mignard, F., Drimmel, R., Babusiaux, C., Bailer-Jones, C. A. L., Bastian, U., & et al. 2016. Gaia Data Release 1. Summary of the astrometric, photometric, and survey properties. *A&A*, **595**(Nov.), A2.
- Gerritsen, J. P. E., & Icke, V. 1997. Star formation in N-body simulations. I. The impact of the stellar ultraviolet radiation on star formation. *A&A*, **325**(Sept.), 972–986.

- Gustafsson, Bengt, Church, Ross P., Davies, Melvyn B., & Rickman, Hans. 2016. Gravitational scattering of stars and clusters and the heating of the Galactic disk. *A&A* , **593**(Sept.), A85.
- Hansen, Brad M. S. 2010. Calibration of Equilibrium Tide Theory for Extrasolar Planet Systems. *ApJ* , **723**(1), 285–299.
- Harfst, S., Portegies Zwart, S., & Stolte, A. 2010. Reconstructing the Arches cluster - I. Constraining the initial conditions. *MNRAS* , **409**(2), 628–638.
- Hastings, W. K. 1970. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, **57**(1), 97–109.
- Hernquist, Lars, & Katz, Neal. 1989. TREESPH: A Unification of SPH with the Hierarchical Tree Method. *ApJS* , **70**(June), 419.
- Heyer, Mark, & Dame, T. M. 2015. Molecular Clouds in the Milky Way. *ARA&A* , **53**(Aug.), 583–629.
- Huang, R. Q., & Taam, R. E. 1990. The non-conservative evolution of massive binary systems. *A&A* , **236**(1), 107–116.
- Hut, Piet, Makino, Jun, & McMillan, Steve. 1995. Building a Better Leapfrog. *ApJL* , **443**(Apr.), L93.
- Innes, R. T. A. 1915. A Faint Star of Large Proper Motion. *Circular of the Union Observatory Johannesburg*, **30**(Oct.), 235–236.
- Innes, R. T. A. 1917. Parallax of the Faint Proper Motion Star Near Alpha of Centaurus. 1900. R.A. 14 h 22m 55s.-0s 6t. Dec-62° 15'2 0'8 t. *Circular of the Union Observatory Johannesburg*, **40**(Sept.), 331–336.
- Ivanova, N., Justham, S., Chen, X., De Marco, O., Fryer, C. L., Gaburov, E., Ge, H., Glebbeek, E., Han, Z., Li, X. D., Lu, G., Marsh, T., Podsiadlowski, P., Potter, A., Soker, N., Taam, R., Tauris, T. M., van den Heuvel, E. P. J., & Webbink, R. F. 2013. Common envelope evolution: where we stand and how we can move forward. *A&A Review* , **21**(Feb.), 59.
- Karachentsev, I. D., & Makarov, D. I. 1996. Orbital velocity of the Sun and the apex of the Galactic center. *Astronomy Letters*, **22**(4), 455–458.
- Kervella, P., Thévenin, F., & Lovis, C. 2017. Proxima's orbit around α Centauri. *A&A* , **598**(Feb.), L7.
- King, Ivan R. 1966. The structure of star clusters. III. Some simple dynamical models. *AJ* , **71**(Feb.), 64.
- Kokubo, Eiichiro, & Ida, Shigeru. 2002. Formation of Protoplanet Systems and Diversity of Planetary Systems. *The Astrophysical Journal*, **581**(1), 666.
- Kozai, Yoshihide. 1962. Secular perturbations of asteroids with high inclination and eccentricity. *AJ* , **67**(Nov.), 591–598.
- Landau, Lev Davidovich, & Lifshitz, E. M. 1969. *Mechanics*.
- Lidov, M. L. 1962. The evolution of orbits of artificial satellites of planets under the action of gravitational perturbations of external bodies. *Planet. Space Sci.*, **9**(10), 719–759.
- Makino, Junichiro. 1991. Optimal Order and Time-Step Criterion for Aarseth-Type N-Body Integrators. *ApJ* , **369**(Mar.), 200.
- Martínez-Barbosa, C. A., Brown, A. G. A., Boekholt, T., Portegies Zwart, S., Antiche, E., & Antoja, T. 2016. The evolution of the Sun's birth cluster and the search for

- the solar siblings with Gaia. *MNRAS* , **457**(1), 1062–1075.
- McMillan, Stephen L. W., & Portegies Zwart, Simon F. 2003. The Fate of Star Clusters near the Galactic Center. I. Analytic Considerations. *ApJ* , **596**(1), 314–322.
- Metropolis, Nicholas, Rosenbluth, Arianna W., Rosenbluth, Marshall N., Teller, Augusta H., & Teller, Edward. 1953. Equation of State Calculations by Fast Computing Machines. *J. Comp. phys.* , **21**(6), 1087–1092.
- Mezger, Peter G., Duschl, Wolfgang J., & Zylka, Robert. 1996. The Galactic Center: a laboratory for AGN? *A&A Review* , **7**(4), 289–388.
- Mishurov, Yu. N., & Acharova, I. A. 2011. Is it possible to reveal the lost siblings of the Sun? *MNRAS* , **412**(3), 1771–1777.
- Miyamoto, M., & Nagai, R. 1975. Three-dimensional models for the distribution of mass in galaxies. *Publ. Astr. Soc. Japan* , **27**(Jan.), 533–543.
- Navarro, Julio F., Frenk, Carlos S., & White, Simon D. M. 1996. The Structure of Cold Dark Matter Halos. *ApJ* , **462**(May), 563.
- Önehag, A., Korn, A., Gustafsson, B., Stempels, E., & Vandenberg, D. A. 2011. M67-1194, an unusually Sun-like solar twin in M67. *A&A* , **528**(Apr.), A85.
- Paxton, Bill, Bildsten, Lars, Dotter, Aaron, Herwig, Falk, Lesaffre, Pierre, & Timmes, Frank. 2010 (Oct.). *MESA: Modules for Experiments in Stellar Astrophysics*. Astrophysics Source Code Library, record ascl:1010.083.
- Paxton, Bill, Bildsten, Lars, Dotter, Aaron, Herwig, Falk, Lesaffre, Pierre, & Timmes, Frank. 2011. Modules for Experiments in Stellar Astrophysics (MESA). *ApJS* , **192**(1), 3.
- Pelupessy, F. I., & Portegies Zwart, S. 2012. The evolution of embedded star clusters. *MNRAS* , **420**(2), 1503–1517.
- Pelupessy, F. I., van der Werf, P. P., & Icke, V. 2004. Periodic bursts of star formation in irregular galaxies. *A&A* , **422**(July), 55–64.
- Pelupessy, Federico I., Jänes, Jürgen, & Portegies Zwart, Simon. 2012. N-body integrators with individual time steps from Hierarchical splitting. *New Astron.* , **17**(8), 711–719.
- Pichardo, Bárbara, Moreno, Edmundo, Allen, Christine, Bedin, Luigi R., Bellini, Andrea, & Pasquini, Luca. 2012. The Sun was Not Born in M67. *AJ* , **143**(3), 73.
- Plummer, H. C. 1911. On the problem of distribution in globular star clusters. *MNRAS* , **71**(Mar.), 460–470.
- Portegies Zwart, Simon F. 2009. The Lost Siblings of the Sun. *ApJL* , **696**(1), L13–L16.
- Pourbaix, D., Nidever, D., McCarthy, C., Butler, R. P., Tinney, C. G., Marcy, G. W., Jones, H. R. A., Penny, A. J., Carter, B. D., Bouchy, F., Pepe, F., Hearnshaw, J. B., Skuljan, J., Ramm, D., & Kent, D. 2002. Constraining the difference in convective blueshift between the components of alpha Centauri with precise radial velocities. *A&A* , **386**(Apr.), 280–285.
- Rein, Hanno, & Spiegel, David S. 2015. IAS15: a fast, adaptive, high-order integrator for gravitational dynamics, accurate to machine precision over a billion orbits. *MNRAS* , **446**(2), 1424–1437.
- Reipurth, Bo, & Mikkola, Seppo. 2012. Formation of the widest binary stars from dynamical unfolding of triple systems. *Nat* , **492**(7428), 221–224.

- Romero-Gómez, M., Athanassoula, E., Masdemont, J. J., & García-Gómez, C. 2007. The formation of spiral arms and rings in barred galaxies. *A&A* , **472**(1), 63–75.
- Romero-Gómez, M., Athanassoula, E., Antoja, T., & Figueras, F. 2011. Modelling the inner disc of the Milky Way with manifolds - I. A first step. *MNRAS* , **418**(2), 1176–1193.
- Salpeter, Edwin E. 1955. The Luminosity Function and Stellar Evolution. *ApJ* , **121**(Jan.), 161.
- Saz Ulibarrena, Veronica, & Portegies Zwart, Simon. 2025. Reinforcement learning for adaptive time-stepping in the chaotic gravitational three-body problem. *Communications in Nonlinear Science and Numerical Simulations*, **145**(June), 108723.
- Schönrich, Ralph, Binney, James, & Dehnen, Walter. 2010. Local kinematics and the local standard of rest. *MNRAS* , **403**(4), 1829–1833.
- Springel, Volker. 2005. The cosmological simulation code GADGET-2. *MNRAS* , **364**(4), 1105–1134.
- Stroustrup, B. 2013. *The C++ Programming Language*. 4th edn. Addison-Wesley Professional.
- Taam, Ronald E., & Sandquist, Eric L. 2000. Common Envelope Evolution of Massive Binary Stars. *ARA&A* , **38**(Jan.), 113–141.
- Titarchuk, Lev, & Seifina, Elena. 2016. ESO 243-49 HLX-1: scaling of X-ray spectral properties and black hole mass determination. *A&A* , **595**(Nov.), A101.
- Tremaine, Scott. 2015. The Statistical Mechanics of Planet Orbits. *The Astrophysical Journal*, **807**(2), 157.
- Tricarico, Pasquale. 2012 (Apr.). *ORSA: Orbit Reconstruction, Simulation and Analysis*. Astrophysics Source Code Library, record ascl:1204.013.
- Verlet, Loup. 1967. Computer "Experiments" on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules. *Phys. Rev.*, **159**(Jul), 98–103.
- Webb, N. A., Guérou, A., Ciambur, B., Detoeuf, A., Coriat, M., Godet, O., Barret, D., Combes, F., Contini, T., Graham, Alister W., Maccarone, T. J., Mrkalj, M., Servillat, M., Schroetter, I., & Wiersema, K. 2017. Understanding the environment around the intermediate mass black hole candidate ESO 243-49 HLX-1. *A&A* , **602**(June), A103.
- Webb, Natalie, Cseh, David, Lenc, Emil, Godet, Olivier, Barret, Didier, Corbel, Stephane, Farrell, Sean, Fender, Robert, Gehrels, Neil, & Heywood, Ian. 2012. Radio Detections During Two State Transitions of the Intermediate-Mass Black Hole HLX-1. *Science*, **337**(6094), 554.
- Webbink, R. F. 1984. Double white dwarfs as progenitors of R Coronae Borealis stars and type I supernovae. *ApJ* , **277**(Feb.), 355–360.
- Whipple, F. L. 1950. A comet model. I. The acceleration of Comet Encke. *ApJ* , **111**(Mar.), 375–394.
- Winn, Joshua N., & Fabrycky, Daniel C. 2015. The Occurrence and Architecture of Exoplanetary Systems. *ARA&A* , **53**(Aug.), 409–447.
- Xiang, Maosheng, Liu, Xiaowei, Shi, Jianrong, Yuan, Haibo, Huang, Yang, Chen, Bingqiu, Wang, Chun, Tian, Zhijia, Wu, Yaqian, Yang, Yong, Zhang, Huawei, Huo, Zhiying, & Ren, Juanjuan. 2017. The Ages and Masses of a Million Galactic-disk Main-sequence Turnoff and Subgiant Stars from the LAMOST Galactic Spectro-

- scopic Surveys. *ApJS*, **232**(1), 2.
- Yoshida, Haruo. 1990. Construction of higher order symplectic integrators. *Physics Letters A*, **150**(5-7), 262–268.
- Zemp, Marcel. 2014 (July). *Halogen: Multimass spherical structure models for N-body simulations*. Astrophysics Source Code Library, record ascl:1407.020.
- Zemp, Marcel, Moore, Ben, Stadel, Joachim, Carollo, C. Marcella, & Madau, Piero. 2008. Multimass spherical structure models for N-body simulations. *MNRAS*, **386**(3), 1543–1556.

Chapter 10

Expanding and Extending AMUSE

No limit may be set to art,
neither is there any craftsman that is fully master of his craft.

The Instruction of Ptahhotep

Congratulations on reaching the final chapter of this book! In the last several chapters we have demonstrated how AMUSE can be used to run individual codes and, more importantly, how to couple codes to simulate more complex systems. In this final chapter we present several examples of advanced strategies for coupling multiple codes trading performance for accuracy, and vice versa. In addition, we explore two tightly coupled physical problems in more depth. In the first example we explore accretion onto a supermassive black hole from a population of massive stars in orbit around it. In the second we study the effect of a nearby supernova on a young planetary system with a circumstellar disk. The problems in this chapter require significant computer resources for high-resolution results, but the examples can be run at lower but acceptable resolution with some patience on a good workstation.

10.1 Intricate multi-scale coupling strategies

The stars in a cluster are often found in pairs, triples or even higher-order hierarchies. Democratic hierarchies are notoriously unstable and fall apart into smaller components. In some rare cases higher-order hierarchies can be stable for a longer time, but eventually most of them dissolve. The fundamental building blocks of a star cluster are ultimately single stars and binaries.

So far we have only discussed binaries in our discussion on stellar evolution (Chapter 5, Sections 8.7.1 and 8.7.2, and Section 9.4.5). In this chapter we will discuss some of the numerical complications with binaries. The terms binaries and higher-order hierarchies should be read here in their broadest context, because we will use these terms to address planetary systems, stars around a supermassive black hole, and ordinary stellar multiples.

Within the AMUSE framework we have two fundamental quite different methods to deal with hierarchical systems as part of a large star cluster. We discuss them separately in Section 10.1.1 and Section 10.1.2.

In each case we make slightly different assumptions which help us to resolve the dynamics of the multiple systems in a larger stellar environment. As a rule of thumb, we could argue that the `encounters` module (see Section 10.1.1), is meant for simulating star clusters with lots of binaries, whereas the `Nemesis` module (see Section 10.1.2) is used for integrating planetary systems including moons and asteroids inside a star cluster.

Although we present here two methods in which hierarchical systems can be resolved in a large environment, there are other strategies possible. One such possibility, effectively employed in several studies, is the use of Kepler’s 2nd law (Kepler, 1609) to resolve close binaries in a star cluster. The interaction with other stars can then be incorporated as perturbations applied externally onto the binary system. With the various bridging strategies and the information from this chapter it is quite easy to construct a hybrid integrator in which Kepler solutions replace some of the orbital integrations.

In the final two sections we discuss various modules developed within the framework. These include the `Encounters` (Section 10.1.1), `Nemesis` (Section 10.1.2), `Venice` (Section 10.1.3), `Torch` (Section 10.2.1), and `Ekster` (Section 10.2.2), `Fresco` (Section 10.5.1) modules.

10.1.1 Encounters: multiples in stellar clusters

The `encounters` module is based on the principle that encounters between a number (>2) of stars are short-lived compared to the global evolution time scale of the cluster. And that, if such an encounter is long-lived, it can be separated into a number of shorter encounters. The underlying assumption of the `encounters` module then is that any encounter is instantaneous with respect to the cluster’s dynamical evolution.

The crossing time for a typical stellar cluster is on the order of a few million years (see Section 4.1.2), whereas a 100 au encounter between two solar-mass stars lasts a few thousand years. A democratic resonance encounter does not necessarily last much longer, unless it is associated with wide excursions to large distance in a hierarchical resonance. A star in such an excursion-orbit may then be perturbed again by another star in the cluster, and we argue that such encounters can be separated into individual encounters.

Binaries and multiples can form dynamically in the cluster, but in many cases we want to start with a population of primordial binaries. In Code example 10.1 we demonstrate how binary stars are set up in the `multiples` module. It encompasses three particle sets, one for the single stars (not in binaries), one for the binary centers of mass, and one for the stars in binaries. Note that the latter are replicated (as pointers) in the binary center of mass particle set as `child1` and `child2`. The particle set provided (called `binaries`) contains three attributes which are relevant for setting up the binaries, and for the operation of the multiples module. These include the `semimajor_axis`, `eccentricity`, and the `radius`. The first two are used to set up the binary, and could equally well be provided externally. The radius, however, is an important parameter that tells the `multiples` module within what distance a binary should be considered isolated. Here we adopt three times the apocenter distance, but

this parameter is problem-dependent, and probably should evolve at run time.

Once the stars and binaries are generated, we can set up the code structure. When in an evolving star cluster, two or more stars approach each other within this `radius` we stop the global cluster integration. A new N -body code is started within which the encountering stars are integrated until the system resolves again. The resolved system is subsequently put back in the global N -body integrator and the interim integrator is deleted again.

In Code example 10.2 we present the fundamental setup for the `multiples` module. This starts with generating the converter, and starting the N -body code. We subsequently start the encounters handling module, and the `multiples` module. The `encounters` module has two important arguments: the code used for resolving two-

```

137     for bi in binaries:
138         mp = bi.mass
139         ms = np.random.uniform(mmin.value_in(units.MSun),
140                                mp.value_in(units.MSun)) | units.MSun
141
142         nb = new_binary_orbit(mp, ms, bi.semi_major_axis, bi.eccentricity)
143         nb.position += bi.position
144         nb.velocity += bi.velocity
145         nb = singles_in_binaries.add_particles(nb)
146         nb.radius = ZAMS_radius(bi.mass)
147         nb.type = "planet"
148         nb.name = "planet"
149         binary_particle = bi.copy()
150         binary_particle.child1 = nb[0]
151         binary_particle.child2 = nb[1]
152         resulting_binaries.add_particle(binary_particle)
153
154     single_stars = center_of_masses-binaries
155     return single_stars, resulting_binaries, singles_in_binaries

```

Code example 10.1: Setting up binaries for the `multiples` module. (The full code example is in `amuse.examples.multiples_minimal`.)

```

201     converter=nbody_system.nbody_to_si(stars.mass.sum(), t_end)
202     gravity = Ph4(converter)
203     gravity.parameters.force_sync = True
204
205     encounter_code = encounters.HandleEncounter(
206         kepler_code=new_kepler(converter),
207         resolve_collision_code=new_smalln(converter),
208         interaction_over_code=None,
209         G=constants.G
210     )
211     multiples_code = encounters.Multiples(
212         gravity_code=gravity,
213         handle_encounter_code=encounter_code,
214         G=constants.G
215     )

```

Code example 10.2: Setting up the `multiples` module.

body encounters, and the code for resolving multi- (> 3) body encounters. For the former we use the standard Kepler solver, and the latter is the `SmallN` solver in AMUSE.

The `multiples` module also has two handling codes, one is the gravity code for integrating the global system, and the other is the encounters module, we just defined earlier. For the gravity module we use in `ph4` in this example, but it could be any N -body code.

In Code example 10.3 we demonstrate how to add the various particles to the `multiples` module. Here three types of particles are included: the `stars`, the `planets_in_JuMBOs`, and the `JuMBOs`. The `stars` are all the gravitational objects handed to the code. The `planets_in_JuMBOs` include the particles which are the member of a binary system, and the `JuMBOs`, in this example, represent the centers of mass of the binaries. The term `JuMBO`, may be a bit arcane here, but is related to the recent discovery of Jupiter-mass binary objects in the direction of Orion's Trapezium cluster (Pearson & McCaughrean, 2023).

One advantage of the `multiples` module is the strict separation of binaries (and multiples) from the single stars. This allows us to invoke binary and triple evolution by treating those substructures with separate methods compared to single stellar evolution. An example of this is presented in the `kira` code in Code example 10.4 and Section 8.7.2.

Although the `encounters` module is quite powerful and speeds up N -body codes with a large number of binaries or multiples, it is best suited to the rather specific problem of relatively separated multiple encounters in a stellar cluster. The assumption that the rest of the cluster is not evolving while the subsystem experiences an interaction may break down when an encounter lasts for a substantial amount of time, and when many encounters occur simultaneously. An example would be a star cluster in which the majority of stars are orbited by planets, maybe including some binary stars. For addressing such a complex multi-scale but rather narrow range of physical processes, we developed the more general `Nemesis` module, which we discuss in the following section.

10.1.2 Nemesis: planetary systems in stellar clusters

What the `multiples` module does for binary stars or more hierarchical multiples, `Nemesis` does for systems with one or more dominating body and several minor bodies, such as a planetary system. The main advantage of `Nemesis` over `Encounters` is the consis-

```

219 multiples_code.particles.add_particles(stars.copy())
220 multiples_code.singles_in_binaries.add_particles(planets_in_JuMBOs)
221 multiples_code.binaries.add_particles(JuMBOs)
222 multiples_code.commit_particles()
223
224 stopping_condition \
225     = multiples_code.stopping_conditions.binaries_change_detection
226 stopping_condition.enable()
```

Code example 10.3: providing the particles to the multiples module.)

```

1  resulting_binaries = Particles()
2  singles_in_binaries = Particles()
3  binaries = center_of_masses.random_sample(Nbin)
4  mmin = center_of_masses.mass.min()
5  for bi in binaries:
6      mp = bi.mass
7      ms = (
8          np.random.uniform(mmin.value_in(units.MSun), mp.value_in(units.MSun))
9          | units.MSun
10     )
11     a = random_semimajor_axis_PPE(mp, ms)
12     e = np.sqrt(np.random.random())
13
14     nb = new_binary_orbit(mp, ms, a, e)
15     nb.position += bi.position
16     nb.velocity += bi.velocity
17     nb = singles_in_binaries.add_particles(nb)
18     nb.radius = 0.01 * a

```

Code example 10.4: Example of how to use the encounter module.

tent time integration. **Encounters** assumes interactions between multiple subsystems to be instantaneous, whereas in **Nemesis** time progresses gradually and continuously, although objects can switch from child to parent system, and vice versa, on a local crossing time scale.

The realization that many stars are potentially born with multiple planets (and smaller objects) begs for a numerical treatment of such hierarchical systems. Stellar distributions around supermassive black holes are quite similar to planetary systems, except that the distributions of orbital eccentricity and inclinations of stars around a supermassive black hole tend to have a wider range compared to planetary systems.

The relatively short orbital periods of planetary systems with respect to the crossing time of a star cluster gives rise to a number of interesting numerical challenges. One of these is the time to solution, which, due to the short time steps for integrating planets, calls for multi-time-step algorithms. A disadvantage of these algorithms, however, is their lack of symplecticity.

In **Nemesis** each planetary system is integrated with its own individual integrator. When several stars or planetary systems approach each other within a pre-specified limit both systems are merged and further integration is realized by a single integrator. If a multi-body system dissolves, single objects are picked up by the global N -body integrator whereas the multi-body subsystems continue to be integrated with their individual solvers. Integrators in this method are dynamically created and deleted at runtime. **Nemesis** is fast, efficient, parallel, and effective in addressing multi-planetary systems in a star cluster (van Elteren *et al.*, 2019). In addition, it is possible for each subsystem to be addressed with its own private integrator. This flexibility makes **Nemesis** quite unique, and applicable to a wide range of problems. One could even address one system with a hydro disk, while around another star the disk has already coagulated into a planetary system. In principle, a child system could itself be a version of **venice** or **multiples**, or even a complete **Torch** simulation. Here, however, we focus

the the dynamical aspects of *Nemesis*.

In *Nemesis* we integrate planetary systems and stars together, but not entirely self consistently. Planets or planetesimals on wide orbits may move between their host child (the planetary system) and the parent system (star cluster). As a consequence, that particular particle will be integrated for parts of its orbit with different integrators. An example would be an Oort-cloud object orbiting a star which is a cluster member. The planetesimal then will be integrated in the child system when near pericenter, say within 1000 au, but beyond a certain point it will be considered part of the parent system. The logic associated with this child-to-parent transition is, despite being parallelized, expensive in the current implementation of AMUSE. In addition, the child system will lose its symplectic qualities by regularly adding and removing particles. The errors associated with these changes are small, but noticeable for long time-scale integrations.

Child systems are composed of small numbers of stars, planets, and other minor bodies. These stars and planets have a relative interaction time step smaller than some fraction η of the free-fall time. The parent system (including the collective of child system centers of mass) is typically integrated using a 4th-order Hermite code, or maybe a treecode. Each child system is integrated using a high-order N -body code for small- N systems, such as *rebound* (Rein & Liu, 2012), *Mercury* (Chambers & Migliorini, 1997), or *Huayno* (Pelupessy *et al.*, 2012).

In calculating the force on the center of mass of a subsystem, we take the substructure of the other subsystems self-consistently into account. This procedure ignores the gravity of the child system’s minor bodies on the internal evolution of each of the other child systems’ minor bodies, and vice versa. While integrating the Alpha Centauri system, we would then, for example, ignore the gravitational effect of planetesimals in the Solar system. The errors that result from this procedure can be estimated, and turn out to be negligible, as expected.

Nemesis is implemented in a number of bridge schemes (see Chapter 9). These bridges integrate the equations of motion of the individual components (collective and subsystems) self-consistently via a second-order kick-drift-kick method. One advantage is that the entire method is independent of the integrators used for either the collective or of the subsystems. Individual bridges are needed between the cluster and the center-of-mass subsystems, and between all subsystems and the cluster. These bridges connect the split parts by employing a second-order kick-drift-kick (KDK) method. Kicks are calculated from the gravitational field of the kicking cluster, and drifts are the evolution of the models in the separate N -body solvers (Portegies Zwart *et al.*, 2020).

The bridge between the cluster and each subsystem needs to take into account that the subsystem is represented in the cluster by a center-of-mass particle. Two adaptations to a vanilla KDK method are needed for this. First, to avoid an extra self-gravity force, the kick has to be calculated with all particles except the center-of-mass particle of the subsystem in question. Next, to avoid doubly calculating the gravitational field of the cluster on the ensemble of the particles, once during the evolution of the particles in the direct N -body code and once with this kick, the kick on the central particle from the cluster must be subtracted from all kicks on the particles in the subsystem.

10.1. INTRICAT

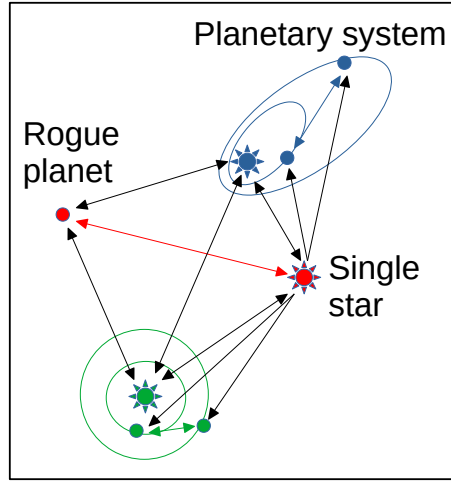


Figure 10.1: Strategy to reduce computing cost without loss of accuracy in *Nemesis*. The planets in one system (blue) interact with the center of mass of another system (green) but not with its planets. Single stars and free-floating planets are marked in red.

The bridge between the subsystems and the cluster needs similar adaptations. Now everything is reversed; the particles in each subsystem kick the center-of-mass particles in the cluster, except the one representing the subsystem. The kick of the center-of-mass particle of each subsystem is subtracted from the sum of the kicks of each particle in the subsystem. Figure 10.1 presents a schema of the method. Of all the possible interactions between the particles we ignore the forces between the particles of one subsystem and another.

Although we have accounted for the interactions between the cluster and the subsystems we do not yet have a working method as each kick on a subsystem will result in a velocity component of the center of mass of that subsystem. The same is true of each drift phase: the N -body solvers should conserve the center-of-mass position and velocity but small errors will cumulatively result in an increasing position offset and velocity component. After the first kick and for each drift of a KDK step we need to absorb the translation of the center-of-mass velocity and position into the center-of-mass particle in the cluster. This is not necessary for the second kick as the kick is independent of the velocity component and this component will be resolved in the first kick of the next step.

Dividing the cluster in this way introduces a new problem: the gravitational evolution of each subsystem can disintegrate or subdivide the subsystem. The subsystem would grow in size and no longer evolve on a separate space scale as the cluster. A clustering algorithm is used to determine for each subsystem if it needs to be restructured in one or more new subsystems. This algorithm is applied after each KDK step.

Similar to subsystem disintegration, subsystems can merge to form a new subsystem. We detect these mergers by defining a radius on each center-of-mass particle and interrupting the drift phase of the center-of-mass system whenever the distance between two of these particles is smaller than the sum of the individual radii. Before merging the subsystems we need to bring the state of each system to the merge time, by applying a kick and drift on each subsystem. The kick will be subtracted if the time is before half the step time and added otherwise. Next each system is added to the new subsystem and the new center-of-mass particle is added to the cluster. The drift of the cluster can then continue as normal.

10.1.3 Venice: a generalized multi-scale bridge

Bridge is powerful, but limited by the rigidity of its implementation. These limitations include the order of the integration and the constant time step. Both can be relaxed. Here we show how these limitations can be mitigated by using a generalization we call **Venice** (Wilhelm & Portegies Zwart, 2024).

Venice is an operator-splitting algorithm designed to address the challenges of integrating numerical models across multiple time scales in multi-physics simulations. As in bridge, **Venice** is a multi-physics coupling strategy for combining a diverse collection of physical processes operating on vastly different timescales.

The traditional bridge approach becomes computationally expensive and complex due to the need to synchronize all processes on the shortest common timestep. In **Venice**, we overcome these limitations by enabling adaptive coupling of independent components on their individually relevant timescales, thereby improving both performance and maintainability.

In **Venice** we connect all available codes in a matrix of bridges. Each bridge coupling is associated with an interaction time scale, resulting in a matrix of time scales. The coupling is 2nd-order, as in bridge, but in principle this is easily generalized. Dynamically changing the order of individual bridges, however, is currently not supported within **Venice**. The coupling strategy, however, is considerably more flexible than in the traditional bridge. In searching for a name for an elegant method representing a matrix of bridges, we settled on *La Serenissima*: the Italian city of canals.

The adaptive time-stepping allows all components to operate on an individual natural timescale. As in Equation (9.3), we split the general operator and solve each part separately. After solving the individual parts, we combine them into a unique, self-consistent solution. The mathematics is a bit daunting, for which we refer to Wilhelm & Portegies Zwart (2024). With **Venice** in mind, one could perceive bridge as a simpler bidirectional strategy.

The main advantages of **Venice** include its temporal flexibility and the modular design, making it easier to maintain and extend simulation frameworks and optimize multi-scale and multi-physics solvers. The algorithm supports an arbitrary number of operators and coupling strategies, making it suitable for a wide range of multi-physics problems.

To make **Venice** work properly in AMUSE, we use channels to copy the data from one code to another. These are defined directly between codes and allow the

copying any subset of data from one code to the other. They must be explicitly called, which is done between operations. In a more relaxed approach, the copying could be automatically realized after the call to the specific code, but this would result in an end state where the data of different codes is not synchronized, in which case it is non-trivial to determine which code contains the most recent state. The explicit approach, where we copy all data from a code after its evolution step, introduces some unnecessary copy operations, but is less prone to mistakes. Of course, this can be optimized, in particular if one takes the multi-scale modeling paradigms (Borgdorff *et al.*, 2011) properly into account.

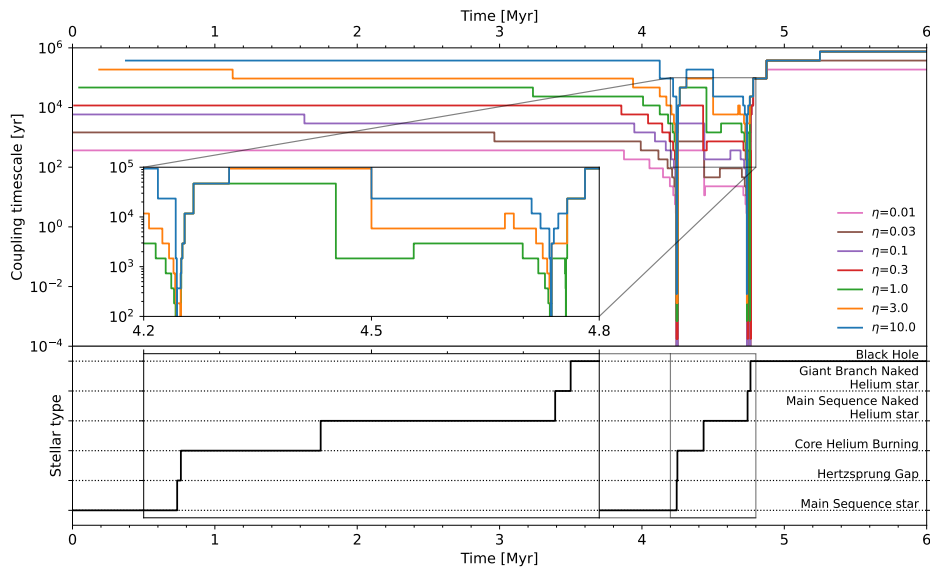


Figure 10.2: Illustration of the dynamic coupling timescale between gravitational dynamics and stellar evolution (top panel). The calculation is performed for a range of coupling time scales η from 0.01 to 10. The bottom panel illustrates the type of the most massive star in the cluster (of $50 M_{\odot}$) star in time. We illustrate how different coupling time scales lead to slightly different moments the most massive star in the cluster is updated (see (Wilhelm & Portegies Zwart, 2024)). `amuse.examples.venice_minimal`. It should take about 10 minutes to run on a laptop.

As an example of how `Venice` operates, we dynamically evolve a stellar cluster with a few very massive stars. The fundamental setup can be written as follows

```
from venice import Venice
def make_gravity_stellar (converter, timestep, verbose=False):
    stellar = Seba()
    gravity = Ph4(converter)
    gravity.parameters.force_sync = True

    system = Venice()
    system.verbose = verbose
    system.add_code(stellar)
    system.add_code(gravity)

    system.add_channel(0, 1, from_attributes=["mass"], to_attributes=["mass"])
    system.timestep_matrix[0,1] = timestep
    return stellar, gravity, system
```

First we initialize both codes for stellar evolution and dynamics. Note that we added the line

```
gravity.parameters.force_sync = True
```

to guarantee that the N -body code properly synchronizes to the requested time (see also Section 8.4.4). Some N -body codes do not naturally synchronize to precisely the time requested, because it introduces extra numerical errors, slows down the code, and is often not necessary. For **Venice**, however, we do require the codes to be synchronized at the end of a step.

The actual **Venice** system is initiated much in the same way we initialize a bridge in the lines

```
system = Venice()
system.add_code(stellar)
system.add_code(gravity)
```

Here the script is started, and the (in this case) two codes are added. Running the code is not very different than a usual code bridge.

```
timesteps = [3., 1., 0.3, 0.1, 0.03, 0.01, 0.003, 0.001] | units.kyr
stellar_l, gravity_l, system_l = make_gravity_stellar(converter, timesteps[i])

stellar_l.particles.add_particles(cluster_l_copy)
gravity_l.particles.add_particles(cluster_l_copy)
channel_l = gravity_l.particles.new_channel_to(cluster_l_copy)

stellar_i, gravity_i, system_i = make_gravity_stellar(
    converter,
    timesteps[i]
)
system_i.interlaced_drift = True

stellar_i.particles.add_particles(cluster_i_copy)
gravity_i.particles.add_particles(cluster_i_copy)
channel_i = gravity_i.particles.new_channel_to(cluster_i_copy)

system_l.evolve_model(end_time)
channel_l.copy()
system_l.stop()
```

Subsequently, running the code is not much different than any other code in AMUSE.

With **Venice**, we can couple an arbitrary number of codes and run them on their own preferred time scales. If you encounter astrophysical problem in which two codes are coupled, **Venice** is probably overkill, but once more than two codes require coupling you should consider using this strategy.

10.2 Advanced coupling projects

10.2.1 Torch: forming star clusters from molecular clouds

One of the objectives of the MODEST community is to model the formation and evolution of a star cluster from the moment a giant molecular cloud starts to collapse. This has led to the **Torch** project¹, in which AMUSE is used as coupling framework to model star cluster formation by integrating multiple simulation codes.

Installing **Torch** is non-trivial. After AMUSE is properly installed, the magnetohydrodynamics code FLASH needs to be installed, after which the **Torch** package can be added. You can download an academic version of FLASH from <http://flash.uchicago.edu/site/flashcode>.

If not all packages are installed in a conda environment, you may have to set some environment variables in your shell

```
export AMUSE_DIR=/path/to/amuse
export FLASH_DIR=/path/to/FLASH4.5
export TORCH_DIR=/path/to/torch
export PYTHONPATH=$PYTHONPATH:$TORCH_DIR/src
```

Then run the installation run script:

```
./install.sh
```

You will be prompted to confirm your environment variables, and continue with the installation. The installer will then automatically copy interface code from `TORCH_DIR/src` to their correct locations in your AMUSE and FLASH repositories. It will also ensure that the FLASH4.5-a patch for HDF5 >1.10.x compatibility is applied in your FLASH repository. The main script is `torch_user.py`, but we encourage you to make a copy in some local directory and change it to your liking.

In Figure 10.3 we show the analytic energy injection of a pressure-dominated snowplow phase of a supernova explosion in a gaseous medium using the **Torch** simulation package. The various vertical lines give the Sedov-Taylor (t_{ST}), thermal (t_{TR}), and snowplow (t_{PDS}) transition times. Figure 10.3 demonstrates how in **Torch** these high-energy phenomena are satisfactorily represented.

The **Torch** module uses three codes coupled using the AMUSE framework. These codes are the MHD-radiative-transfer hydrodynamics code FLASH (Fryxell *et al.*, 2000), the direct N -body code **PeTar** (Wang *et al.*, 2020), and the binary evolution code **SeBa** (Portegies Zwart & Verbunt, 1996). Although each of these codes could in principle be replaced by any other in AMUSE, the **Torch** script has become quite intricate, and by now it may not be trivial to replace, for example, FLASH for Ramses (Teyssier, 2002). On the other hand, it is somewhat easier to replace **PeTar** for the **multiples** module, and **SeBa** for **Mesa**, for example.

The performance and accuracy of **PeTar** are tuned using the parameter `r_out` which is a boundary radius between the tree- and direct N -body gravity. The default is the tuned for simulating star clusters, but by all means, test this, and modify it for your

¹see <https://torch-sf.bitbucket.io/quickstart.pdf>

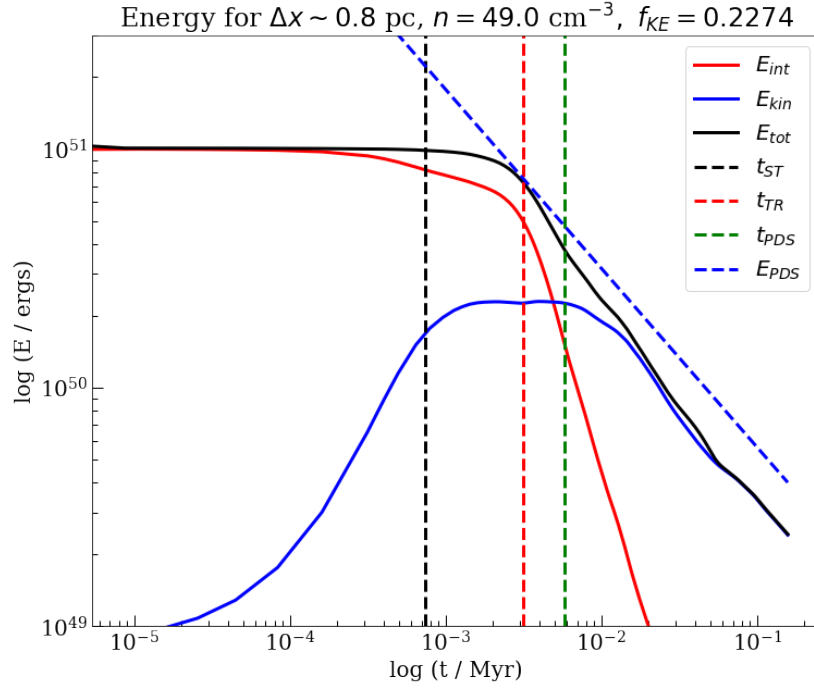


Figure 10.3: The Sedov-Taylor (t_{ST}), thermal (t_{TR}), and snow plow (t_{PDS}) transition times (Draine, 2011) are shown as vertical dashed lines, and the analytic form of the energy during the pressure dominated snow plow phase is shown as a blue dashed line above the energy during this phase. (Reproduced from Wall *et al.* (2020), their figure 2.)

particular use case. **Torch** can also use **VorAMR** (Lewis *et al.*, 2023), a utility developed within the **Torch** framework that allows for the conversion of output data from **AREPO** to be converted to an AMR grid. An example is provided in Figure 2.1 of Section 2.2.2.

A **Torch** calculation starts with a giant molecular cloud that collapses under its own gravity. Star formation is mediated through sink particles. We use AMUSE’s sink-particles module to handling the sinks (see Section 6.3.2). Stars are spawned from these sinks according to a pre-determined mass function. Stellar feedback is facilitated by the transfer of stellar parameters (luminosity, mass-loss rates) from **SeBa** to **FLASH**, where these parameters are used to calculate the effects of radiation, winds, and supernovae on the surrounding gas.

Recently, we added anisotropic winds, or jets to simulate the formation of low-mass clusters (Appel *et al.*, 2025). Protostellar jets and outflows are critical phenomena associated with star formation, launched by the interaction of magnetic fields and accretion disks.

10.2.2 Ekster: forming clusters from gas to star

Somewhat similar to the **Torch** endeavor is the **Ekster**² module in AMUSE (Rieder, 2019; Rieder & Liow, 2021). **Ekster** has a similar philosophy as **Torch**, except that it uses an SPH method for the hydrodynamics. **Ekster** was developed with the SPH code

²Ekster is the Dutch name for the European magpie. Also, it is a homophone for Exeter, where the module was primarily developed. **Ekster** is available from: <https://github.com/amusecode/amuse-ekster>.

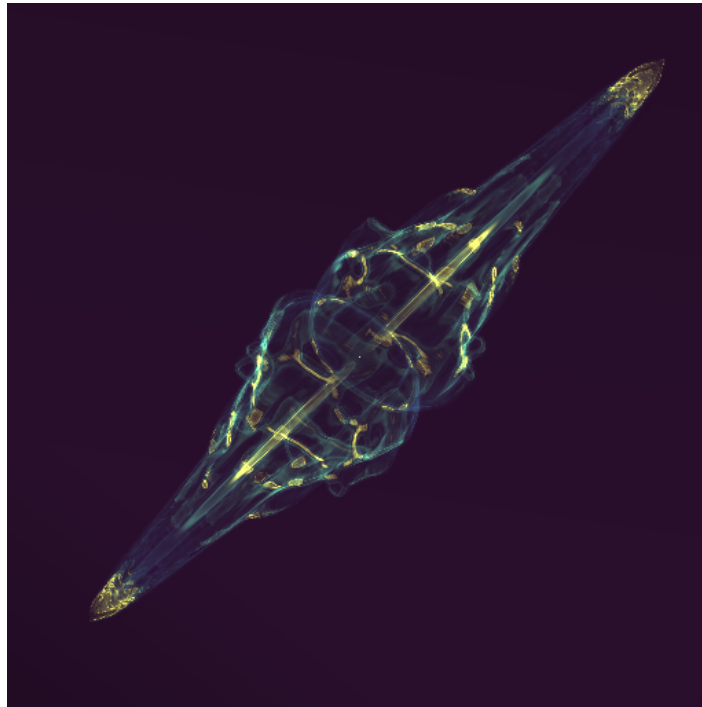


Figure 10.4: A volume rendering of a protostellar jet produced by a single star particle in a uniform density box. This volume rendering was generated from a simulation run using the jets branch of the Torch code. Note that although the injection weighting is smooth, as described in [Appel et al. \(2025\)](#), hydrodynamical instabilities develop as the injected material propagates into the denser background material.

Phantom ([Price et al., 2018](#)) as its primary hydrodynamics code, although following the AMUSE philosophy it is possible to use any other SPH code, provided it supports the high-density stopping condition Appendix A.5.2. Like **Torch**, **Ekster** combines hydrodynamics with gravity and stellar evolution, by default using the same codes as in **Torch** (**PeTar** and **SeBa**).

The application domain for **Ekster** is quite similar to that for **Torch**, as was illustrated in [Sills et al. \(2023\)](#). **Ekster** was developed with relatively low mass resolution in mind ($\sim 10^5 M_\odot$ per gas particle), allowing it to run on modest-sized machines (anything from a laptop to a small cluster). When gas reaches a sufficiently high density, star formation starts through the formation of sink particles, each of which acts as a star forming region. The mass in the sinks is converted into stars, which are spatially distributed within the sink’s accretion radius using a gaussian distribution. The mass of the stars is sampled from a pre-determined mass function. After star formation, the sink’s accretion radius shrinks so that its “density” remains constant. This prevents the sink moving towards and accreting low-density gas regions.

As an alternative for simulations with a slightly higher mass resolution, **Ekster** also has a mode where multiple sinks can be grouped according to their positions, velocities, and ages. These groups can then “borrow” mass from each other for star formation. This allows massive stars to form even when individual sinks are unable to accrete enough mass for such stars ([Liow et al., 2022](#)).

In Figure 10.5 we show a small section of a simulated galaxy ([Dobbs & Pringle, 2013](#)) for which the simulation was continued using **Ekster** in AMUSE ([Rieder et al.,](#)

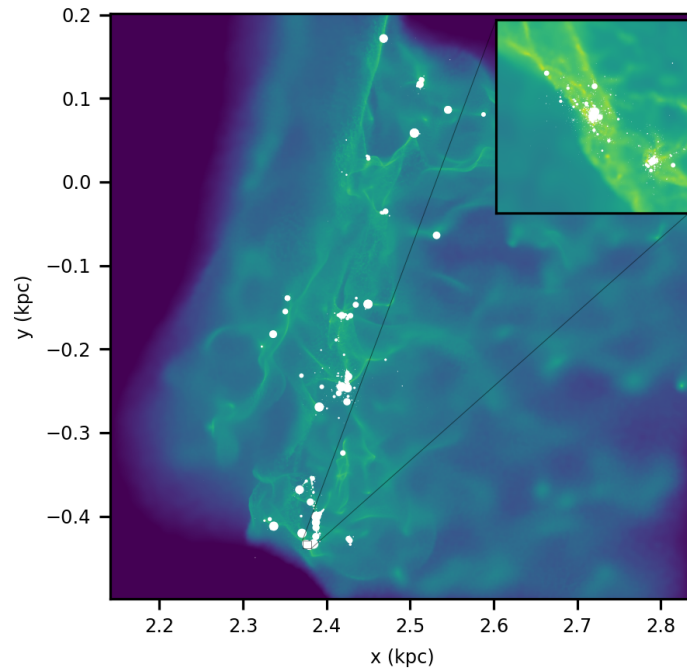


Figure 10.5: One of the regions CRC0060 at an age of 3 Myr (Rieder *et al.*, 2022). The inset shows a 10 parsec zoom of the largest star cluster. Symbol sizes are proportional to the stellar radius. (Figure is reproduced from lower-left panel in Figure 7 of Rieder *et al.* (2022)).

2022). It shows one of the active star-forming regions in the simulated galaxy at an age of 3 Myr after the collapse of the giant molecular cloud.

10.3 Supernova impact on the early solar system

Meteoritic evidence suggests that short-lived radionuclides—specifically ^{26}Al and ^{60}Fe , with half lives of 0.717 and 1.49 Myr, respectively (Schiller *et al.*, 2015)—were incorporated into grains in the primitive solar nebula about 2 Myr after those grains formed (Luu *et al.*, 2015; Scott, 2007). As discussed in Section 8.6.1, the origin of these unstable elements is uncertain, but one hypothesis is that a nearby supernova might have enriched the solar circumstellar disk with these radionuclides shortly after the Sun formed. The process would have required high temperatures (1500–2000 K) to melt the grain material. In principle, the same supernova could have produced the ^{26}Al and ^{60}Fe radionuclides, delivered them to the disk, and heated the disk enough to melt the hardest grains. For this to work, the supernova must have exploded within about 0.5 pc and within the first 1.5 Myr of the budding solar system. The proximity is necessary to guarantee a sufficient amount of short-lived radionuclides, and the time scale stems from the half-life of the volatile elements.

It might seem extraordinary to posit a type II supernova so close to the Sun just 2 Myr after it formed. However, considering that the Sun was probably born in a star cluster, the probability of a such an event is actually quite high. Looney *et al.* (2006) argue that a $23 M_{\odot}$ star could deliver a sufficient amount of short-lived radionuclides when it explodes at an age of about 8.4 Myr. The time difference can plausibly be

explained by an age spread in the star formation process, in which the Sun formed somewhat later than the most massive stars (Stahler & Palla, 2005).

The possibility of a nearby supernova was first suggested by Chevalier (2000). Subsequent hydrodynamical calculations have been performed to study the amount of supernova remnant material that could be accreted by the protoplanetary disk (Ouellette *et al.*, 2007). These simulations suggest that the supernova might not be able to deposit sufficient mass on the disk to match the observations, but made some simplifying assumptions and were not conclusive. One of those assumptions was homogeneous density throughout the slab of supernova material hitting the disk. Here we take the opportunity to revisit the problem using AMUSE, combining gravitational dynamics, hydrodynamics, and radiative transfer codes to explore this scenario. Unlike previous studies, we investigate both the initial irradiation of the disk by the supernova flash and the direct effects of the subsequent blast wave, both of which could have contributed to the heating of the disk. Most of the ingredients of the scripts described below have already been discussed in previous chapters.

We start by generating a protoplanetary system with a star, some gas-giant planets, a gas disk, and debris. This system is evolved using a bridged hydrodynamics–gravity solver (see Section 9.3.2) in which we integrate the debris disk as test particles in the combined potentials of the planets and the gas disk. We ignite a supernova at a distance of $d_{\text{sn}} = 0.1$ pc and assume that the normal to the disk is inclined at an angle of 60° to the direction to the supernova, so the radiation and the blast wave impinge on the disk almost edge-on. A head-on blast wave would be less effective in tilting the disk, and would damage the disk more severely.

The supernova explosion is modeled in two stages. First, a radiative transfer code is started to irradiate the protoplanetary disk roughly $d_{\text{sn}}/c \sim 100$ days after the explosion. Then, approximately 10 years later, the blast wave arrives at a speed of $\sim 10,000$ km/s. At the adopted small distance, the supernova shell is still in its free-expansion phase and the hot gas has not been significantly slowed by the interstellar medium. The short distance and therefore short travel time guarantees that the short-lived radionuclides are still abundant in the blast wave. We study the effect of the supernova irradiation and the subsequent blast wave on the disk, and determine if the disk survives and if a sufficient amount of short-lived radionuclides can be deposited.

10.3.1 Initial conditions and model parameters

We start the simulation by generating a young solar system with four giant planets using the method discussed in Section 4.4.1. The underlying assumption is that the rocky planets have not yet emerged, so instead we incorporate a debris disk. The solar system was generated at an arbitrary Julian date of 2474649.5 days³ After initialization we recenter the system to place the center of mass at the origin.

The gas disk around the star is generated using $n_{\text{disk}} = 10000$ equal-mass SPH particles between $r_{\text{min}} = 1$ AU and $r_{\text{max}} = 100$ AU from the Sun, with total mass of $0.01 M_\odot$. We adopt a flat density profile with Safronov-Toomre Q-parameter $q_{\text{out}} = 25$

³Of course the precise Julian date is irrelevant, but it was this book’s 2nd edition submission deadline.

(see Section 6.4.1). You might wish to experiment with higher resolution, but note that the calculation is quite expensive in terms of computer time, and parallelization is limited to the OpenMP implementation of the adopted hydrodynamics solver (see below).

```
converter = nbody_system.nbody_to_si(mass_star, radius_min)
disk = ProtoPlanetaryDisk(
    n_disk, converter, radius_min, radius_max, q_out=25.0, discfraction=0.01
).result
n_planets = int(planets.mass.sum() / disk[0].mass)
disk = disk.random_sample(n_disk - n_planets)
```

Since we generate the particle set `planets` independently of the gas, we randomly remove gas particles from the disk until the total mass is $0.01 M_{\odot}$. We then generate a debris disk composed of $n_{\text{debris}} = 1000$ planetesimals orbiting the star. This is a small number, but sufficient to trace the potential. The debris disk density distribution is generated in much the same way as the gas disk, except that we do not correct for the planetesimal masses, which are assumed to be negligible.

As in Section 8.6.1, we model the supernova light curve using the multiple power-law fit of Sanders *et al.* (2015) to supernova PS1-10a. We use parameters from their Table 3, with a peak luminosity of 2.3×10^{42} erg/s ($10^{8.8} L_{\odot}$) a few days after the explosion.

10.3.2 The combined solver

The first part of the calculation combines a hydrodynamics solver for the circumstellar disk, a radiative transfer code, a gravity solver for the giant planets, and a test-particle solver for the massless debris. The radiative transfer and the hydrodynamics are coupled as described in Section 8.6.1. In fact, the script for the combined hydro and radiative transfer is quite similar to the one presented in that section. It differs mainly in that the Sun is no longer a dark-matter particle in the hydro code; now it is integrated together with the giant planets using the N -body code.

The debris disk is resolved using the Kepler solver in the `Huayno` integrator:

```
debris_gravity = Huayno(converter, number_of_workers=12)
debris_gravity.inttype = KEPLER
debris_gravity.particles.add_particles(debris)
debris_gravity.timestep = 1 | units.hour
```

Here we start `Huayno` with a Kepler solver (`KEPLER = 14`) and run it on 12 cores concurrently (see Appendix A.3.3 for some of the other options available in `Huayno`). The debris disk of massless particles is subsequently sent to the N -body code, with a fixed time step of 1 hour. This is very short, but the time step in this experiment is set not by the dynamics, but by the radiative transfer. Later we could increase the time step again.

For the N -body code to integrate the planets and the Sun we adopt `ph4`, which is initialized with

```
planet_gravity = Ph4(converter)
planet_gravity.particles.add_particles(star_and_planets)
```

After declaring the various solvers we build the bridge, in this case composed of two gravity solvers and the radiation-hydrodynamics solver discussed in Section 8.4.2:

```
gravity_hydro = bridge.Bridge(use_threading=True)
gravity_hydro.add_system(debris_gravity, (planet_gravity, radhydro,))
gravity_hydro.add_system(planet_gravity, (radhydro,))
gravity_hydro.add_system(radhydro, (planet_gravity,))
gravity_hydro.timestep = min(1 | units.hour, time_step / 4.)
```

In the first line we start the bridge, setting threading to `True` to speed up the calculation. In the second line we add the `debris_gravity` code to the bridge, specifying that it is affected by `planet_gravity` and the radiative-hydrodynamics solver `radhydro`. In the third and fourth lines we add the gravity code for the planets and the radiative-hydrodynamics solver. These are affected by each other but not by the debris disk. In the last line we set the bridge timestep to 1 hour (or even smaller), similar to the internal timestep for the Kepler solver.

10.3.3 Radiative hydrodynamics with cooling and heating

The simulation starts when the bright irradiation of the supernova reaches the young solar system. During this period the heating and cooling of the disk is calculated self-consistently with the radiative transfer code. Figure 10.6 presents side and top views of the solar system 6 years after the ignition of the supernova. In the radiative-hydro calculation we assume that the gas is isothermal and free of dust. The lack of dust reduces the cooling efficiency of the disk material. With a dust mass fraction of 0.1, the equilibrium temperature in the radiation field drops by about 20%.

Once the luminosity of the supernova drops below $10^6 L_{\odot}$ (after about a year of irradiation), we switch off the radiative transfer code, because it is rather expensive to keep using it just for the cooling of the gas, and turn on the external cooling functions for the gas (see Section 6.3.4). The remaining codes continue to run as before.

Figure 10.7 shows top view images of the surface temperature of the circumstellar disk at four moments in time. Although the flash of radiation does briefly raise the temperature of the disk to well over 2000 K (second image from the left in Figure 10.7), the subsequent cooling is so efficient that, by the time the supernova shock wave reaches the young solar system ten years later, the disk has cooled back to its “normal” (pre-supernova) temperature of 20–200 K. In addition, the heating and cooling are so sudden that the disk hardly has time to respond, and as a result its morphology is only slightly affected by the supernova flash. The upshot is that, by the time the blast wave arrives, the disk structure is essentially the same as its initial state.

10.3.4 Injection of the supernova blast wave

The hot radioactive outflow from the supernova reaches the solar system about 9.8 yr after the explosion. We inject the hot gas at a distance of 10 AU from the solar system.

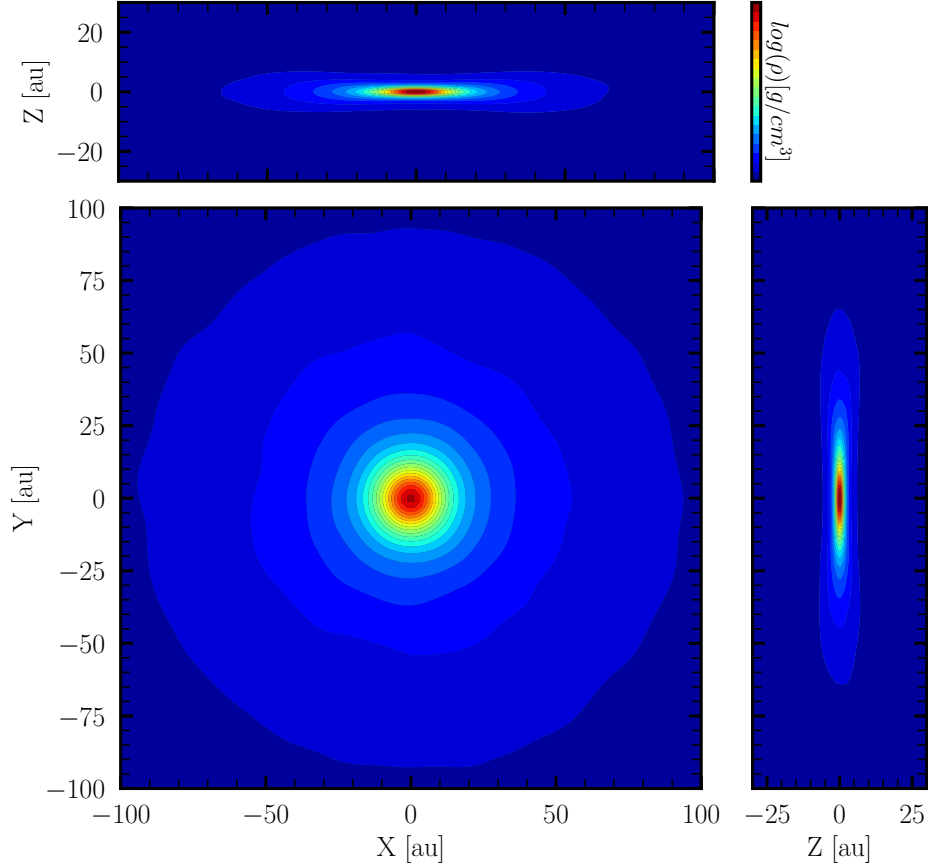


Figure 10.6: View of the solar system 6.4 years after the supernova explosion. By this time the supernova has dimmed to such a low luminosity that there is no reason to continue the radiative transfer, but the blast wave has not yet arrived. The color bar at the top right of the figure presents a scaling of ρ ranging from $\sim 10^{-13}$ to $\sim 10^{-10}$ g/cm³. The code to reproduce this result is `amuse.examples.supernova_irradiation`, which took a few days to run on a fast workstation. The figure was made using `amuse.examples.plot_projected_disk_density_from_top`

The hot shell is initialized in a cylinder directed at the solar system. This cylinder has a radius of $r_{\text{shell}} = 150$ AU, to ensure that the entire solar system is engulfed, and the gas moves parallel to its axis. The density is uniform across the cylinder, but along the axis the density varies according to

$$\rho \propto \frac{1}{z + 10 \text{ AU}}, \quad (10.1)$$

where z is distance along the axis, with $z = 0$ at the shock front. The total mass in the cylinder is

$$m_{\text{shell}} = \frac{1}{4} m_{\text{eject}} \left(\frac{r_{\text{shell}}}{d_{\text{sn}}} \right)^2 \quad (10.2)$$

For SN PS1-10a, the estimated total mass of the ejecta is $m_{\text{eject}} = 6 M_{\odot}$.

The initial temperature of the expanding supernova shell is about $T_{\text{eject}} = 9 \times 10^7$ K at a shell radius of 10 AU, but by the time the gas reaches the solar system it has

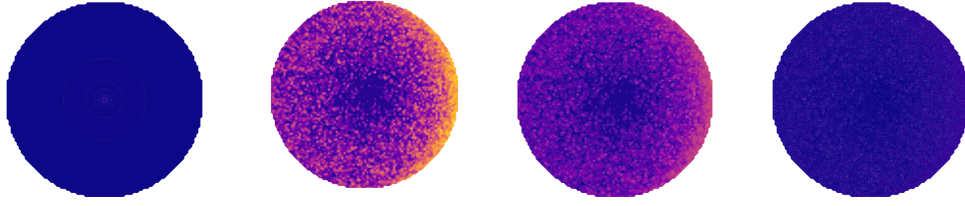


Figure 10.7: Top view of the mid-plane temperature of the circumstellar disk at four epochs: the start of the simulation (leftmost image); 26 days after the onset of supernova PS1-11aof; about 100 days after the supernova; and about 2 years later, when the disk has almost completely cooled down again (rightmost image). Each image was taken from the simulation where the supernova exploded at a distance of 0.2 pc and at an angle of $\theta = 60^\circ$. The figure was made using `amuse.examples.plot_projected_disk_temp_from_top`

cooled to (Spicer *et al.*, 1990)

$$T_{\text{shell}} = T_{\text{eject}} \left(\frac{d_{\text{sn}}}{10 \text{ AU}} \right)^{-3(\gamma-1)} \quad (10.3)$$

so $T_{\text{shell}} \simeq 9500 \text{ K}$. Here we have adopted $\gamma = 7/5$. The sound speed in the shell is about $c_s \simeq 5.8 \text{ km/s}$, negligible compared to the speed of the shell at impact.

10.3.5 The supernova blast wave hits the disk

In Figure 10.8 we present the surface density profile of the disk at three moments in time: the moment the blast wave hits the disk, 55 days later, and 3 years after the impact. The last curve (dark purple) shows the Solar system’s debris disk, if we spread out all planets and planetesimals over a narrow ring. As we have just seen, the disk structure just before the blast wave hits is almost indistinguishable from the initial disk profile, as the disk has cooled back to its pre-irradiation state. Shortly after the blast wave hits, the disk is most strongly affected near the edges by the ram pressure of the shock front, but the disk has little time to respond—the shock front takes only a few weeks to cross the disk. Afterwards, the disk erodes more rapidly. Fifty-five days after the supernova blast wave hit the disk, the disk mass has dropped to half its initial value ($\sim 0.0052 M_\odot$, from an initial $0.01 M_\odot$). Most of the mass is lost from the outermost parts of the disk. After 1 year the disk mass has dropped by an order of magnitude (to less than $0.001 M_\odot$).

From that point on the disk structure remains largely unchanged, mainly due to the low density of the supernova material at late times. The final state of the disk is shown as the red curve in Figure 10.8, $\sim 3 \text{ yr}$ after first contact. We note that the planets and planetesimals are barely affected by the supernova, remaining in basically unchanged orbits around the central star. For comparison, Figure 10.8 also presents an estimate of the density profile of the current solar system’s planetesimal disk. Here we simply assume that each planetesimal contributes equally to the local density and that the solar system disk has a total mass of $0.01 M_\odot$, equivalent to the initially adopted disk mass. If the planetesimals had already formed by the time of the supernova shell impact, they would hardly be affected by the blast wave. It is interesting to note in Figure 10.8 that the disk becomes considerably depleted beyond 6–7 AU, where we also find a huge drop in the current planetesimal distribution. One might wonder if

the current depletion beyond this range can in part be explained by such a supernova blast wave.

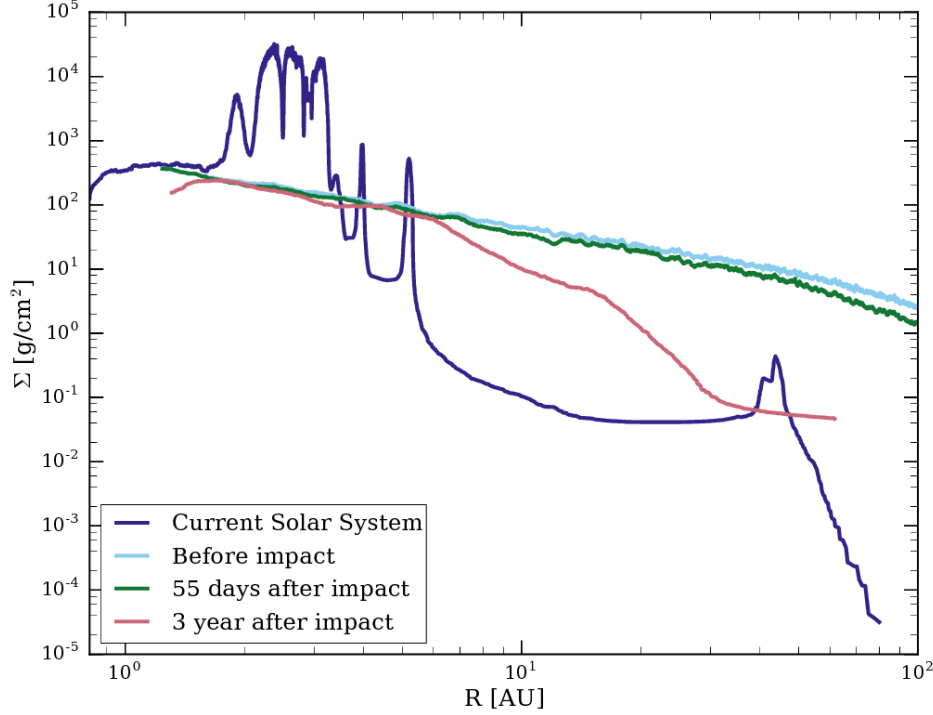


Figure 10.8: Surface density profile of the protoplanetary disk at three moments in time: just before the blast wave hits the disk (blue), 55 days later (green), and 3 years later (red). After about a year the disk has a mass of $\lesssim 0.001 M_{\odot}$. Also shown (dark purple) is the current mass distribution of planetesimals in the solar system.

Figure 10.9 presents the temperature evolution of the protoplanetary disk. The temperature rises about 100 days after the supernova explosion due to the supernova flash. During this first irradiation phase, the disk becomes sufficiently hot to melt the hardest chondrules. One can wonder however, if this initial melting of the disk has any long-term consequences for the evolution of the solar system. The problem as initially posed sought to explain the enrichment by radioactive elements, which are delivered with the blast wave, but clearly the time between the irradiation and the blast wave is sufficiently long for the disk to cool down again before the shock arrives.

The nuclear blast wave hits the disk about 10 years later, heating it again to high temperatures. The blast wave does deposit some material in the Sun's protoplanetary disk, although the total amount of mass is small, less than 1% of the current disk mass. This is probably too small to explain the measured enrichment of radioactive elements in the early solar system, although these melting periods might have been important for other aspects of the formation and evolution of the solar system, such as the formation of molybdenum isotopic elements in carbonaceous and non-carbonaceous meteorites (Budde *et al.*, 2016). A supernova explosion therefore could have had important consequences for the formation and evolution of the early solar system, but it is debatable whether it could have deposited a sufficient amount of short-lived radionuclides to explain the abundances of their decay products in primitive chondrules.

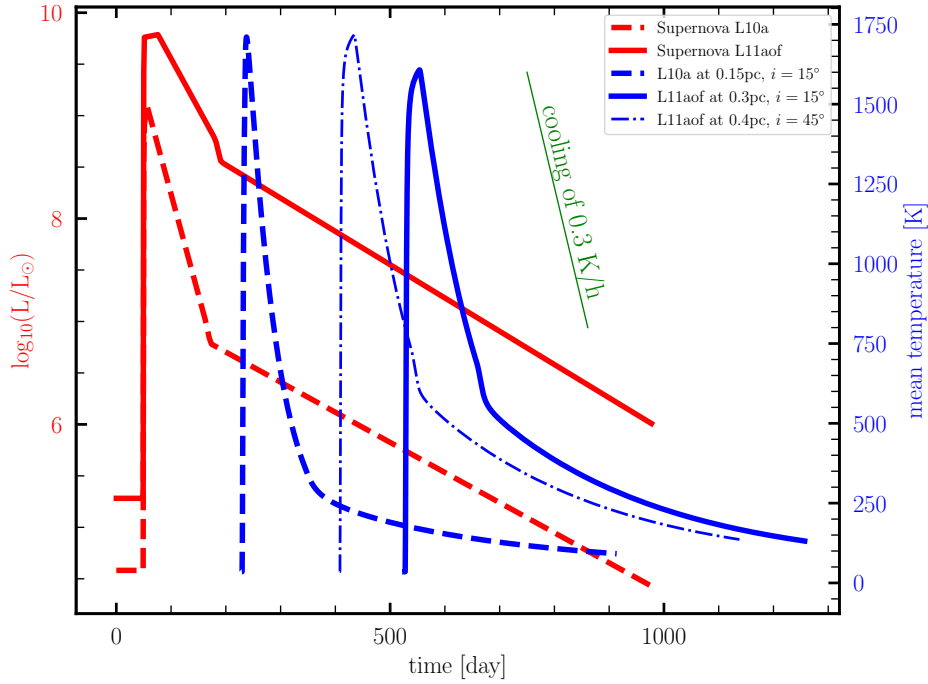


Figure 10.9: Evolution of the mean temperature of the disk due to the supernova explosion. The supernova radiation starts to heat the disk some 100 days after the explosion, and a decade later the hot nuclear blast wave starts to heat the disk. The two horizontal lines indicates a temperatures of 1500 and 2000 K, which are sufficient to melt hard chondrules.

Finally, we note that the continuous flow of hot gas causes the gas in the surviving disk to become eccentric, and the disk starts to tilt (see also [Wijnen *et al.*, 2017](#)) to an average inclination of about $5^\circ \pm 3$. In Figure 10.10 we show the evolution of the inclination of the protoplanetary disk (relative to its initial value). This was measured by calculating the orbital elements for each of the gas particles and binning them in inclination using $10\sqrt{n}$ bins between zero and the maximum measured inclination. We subsequently adopted the median for display in the figure.

During the first 100 days the median inclination of the disk grows to a maximum of about 6° before settling down to about 4° throughout the remainder of the period studied. The bulk of the disk gas has inclination between 1° and 9° . The inclination of the Sun with respect to the ecliptic plane in the solar system is $5.6^\circ \pm 1.2^\circ$, which is not inconsistent with the results from these simulations, illustrating that external events such as supernovae can plausibly account for the Sun's current inclination with respect to the ecliptic. As noted earlier, the planets were unaffected by the blast wave. If the calculation had been continued for a longer time, both the gaseous disk and the planets would probably have realigned to form a new combined disk of gas, debris and planets. This would have led to an averaging of the angular-momentum axis. However, we did not follow this process, but stopped the calculations after the the impact shell traverses the disk.

The model provides a plausible explanation for the tilt of the ecliptic with respect to the Sun's equator, and also of the truncation of the outer edge of the solar system.

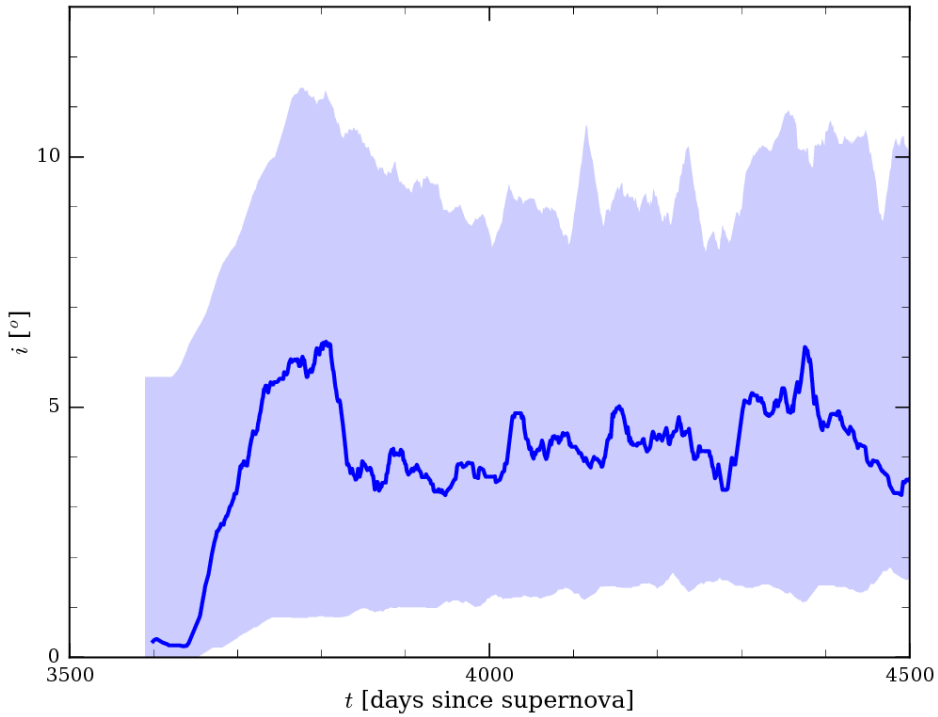


Figure 10.10: Evolution of the inclination of the planetary disk as a function of time since the disk was hit by the hot gas of the supernova blast wave. The median of the disk material inclination distribution is presented as the blue curve, with the shaded region giving the 5th and 95th percent percentiles.

Solving two problems with a single model is very satisfactory for researchers, particularly when complex models have trouble explaining even one of them. In this way, simulations can be used to guide the researcher’s intuition, or to quantify a qualitative description of a physical process.

10.4 Accretion in the Galactic center from S-star winds

The supermassive black hole in the Galactic center, Sgr A*, is orbited by a few dozen young (7–11 Myr), massive (19–25 $M_{\odot}$) stars (Schödel *et al.*, 2002; Ghez *et al.*, 2005), often called the “S stars.” Because their orbits are dominated by the mass of the central black hole, these stars allow an accurate measurement of the black hole mass (Ghez *et al.*, 2003; Genzel *et al.*, 2010) and give us unique insights into resonant and relativistic effects in its vicinity (see Merritt *et al.*, 2010). The evolution of the S stars also has important implications for the local environment. With relatively high masses and advanced ages, these stars have strong stellar winds. Some of this wind material may find its way into the black hole, possibly giving rise to the X-ray emission observed by the Chandra X-ray observatory (Baganoff *et al.*, 2003). The Chandra observations constrain the emitting region in the vicinity of Sgr A* to lie within $\sim 12,000$ AU of the central black hole (Baganoff *et al.*, 2003). We can ask whether the winds from the

S stars can account for the X-ray luminosity, or if the accreting material must come from some other population farther out.

The relatively small amount of accretion (just $\sim 10^{-7} M_{\odot} \text{ yr}^{-1}$) onto the black hole during the pericenter passage of the nearest star S2 led [Nayakshin \(2005\)](#) to argue that the S stars cannot be responsible for the general accretion onto the black hole. However, the accretion process is probably more complicated than just the wind of a single star, as was demonstrated in [Cuadra *et al.* \(2006\)](#), where the winds of the dozen S stars nearest to the black hole were resolved by hydrodynamical simulations. Recently [Lützgendorf *et al.* \(2016\)](#) repeated this calculation with 27 S stars using the AMUSE framework. This calculation combined SPH simulations of the gas dynamics with direct N-body integration of the orbits of the S stars, including the effects of the stellar winds and accretion onto both the black hole and the S stars themselves. In this section we show how the script used to perform these calculations was constructed.

We have encountered all of these ingredients in earlier chapters (specifically, Chapters 4 to 6). The coupling strategy was discussed in detail in Chapter 9. We repeat the [Lützgendorf *et al.* \(2016\)](#) calculation here without radiative transfer, as this would put rather severe constraints on available computer resources. However, following the discussion on code-coupling strategies between hydrodynamics and radiative transfer in Chapter 7, it would be straightforward to expand the discussion to include radiative transfer if desired.

10.4.1 Initial conditions

We start the simulation by constructing a realization of the 27 innermost S stars. The S stars are relatively massive and have most likely lost significant mass to stellar winds since their formation. Their current mass-loss rates are observed to be on the order of $10^{-5} M_{\odot}/\text{yr}$. In principle, knowing the masses and ages of these stars, we could simply model them using a stellar evolution code, feeding the mass loss directly into the hydro calculation. However, in practice, the masses and ages are quite poorly known. In this experiment we adopt a simpler description that illustrates the general approach.

Figure 10.11 presents the results of a series of stellar evolution calculations using the parameterized stellar evolution code SSE (see Section 5.2.2). Reconstructing this figure should by now be relatively straightforward, given the examples in Chapter 5, and we do not provide the source code here. Note that the observed mass loss rate for the S stars (grey bar) is consistent with winds from either 7–11 Myr old giants with initial masses between $19 M_{\odot}$ and $25 M_{\odot}$, or $18 M_{\odot}$ supergiants at an age of about 14 Myr. Accordingly, we simply assume that our model S stars were all born at the same time, each with a mass of $20 M_{\odot}$ and solar metallicity ($z = 0.02$), and are now 9 Myr old. These masses, ages, and metallicities are broadly consistent with the observed stellar parameters in the Galactic center. We evolve our model S stars to this age before integrating the equations of motion. At the start of the simulation, their masses are $18.1 M_{\odot}$, radii $890 R_{\odot}$, effective temperatures 3630 K , and luminosities $1.25 \times 10^5 L_{\odot}$. Using the theoretical wind model of [Kudritzki *et al.* \(1989\)](#) (see [Kudritzki & Puls, 2000](#); for a review), we find a mass loss rate of $\sim 7.9 \times 10^{-6} M_{\odot}/\text{yr}$ with a relatively low wind velocity of about 80 km/s .

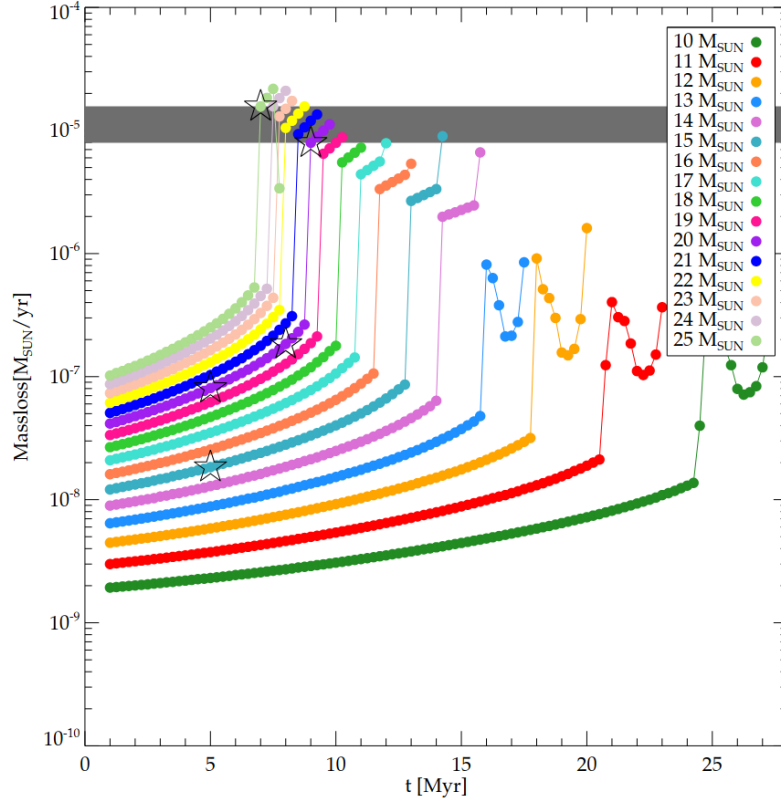


Figure 10.11: Mass-loss rates of high-mass stars, as functions of age and mass. The observed mass loss range for the S stars, between 8×10^{-6} and $2 \times 10^{-5} M_{\odot} \text{ yr}^{-1}$ is indicated by the gray horizontal bar. To make the mass-loss rates of the stars in our simulations consistent with the observations, we opted to use $20 M_{\odot}$ stars at an age of 9 Myr (marked with an open star). (This is Figure 9 from [Lützgendorf et al., 2016](#); copyright AAS; reproduced with permission.)

Once their physical properties are established, the stars are placed in orbit around the central supermassive black hole using the orbital parameters provided in Tab. 7 of [Gillessen et al. \(2009b\)](#), calculated using the methods discussed in Section 4.4.1. Our simulation starts with the positions and velocities of the S stars on January 1, 2001. We adopt a black hole of mass $4.45 \times 10^6 M_{\odot}$ located at the origin of the coordinate system, with zero velocity. The initial coordinates of the S stars are generated with the script `amuse.examples.initialize_sstars.py` (see also Figure 10.12).

10.4.2 The combined solver

We start the simulation with no interstellar gas yet present; we assume that all such gas will be created by the winds of the S stars as they orbit the central supermassive black hole. We follow the hydrodynamics of the stellar winds while integrating the orbits of the S stars in the black hole potential. Although we do not expect the stellar parameters to change much during the integration, we run a parameterized stellar evolution code in tandem with the other codes to determine the stellar wind parameters. This makes the script more flexible, homogeneous, and self-contained.

10.4.2.1 Bridging gravity with hydrodynamics

For the gravitational dynamics we use the direct N -body integrator **Mercury**. Since we are integrating only a few dozen stars we use the single-core non-GPU enabled version of this code (see Chapter 4 for details). We adopted the initial conditions from [Gillessen *et al.* \(2009a\)](#) and converted them to Cartesian coordinates as described in Section 4.4.1. The result of integrating the initial conditions for 500 yr is presented in Figure 10.12.

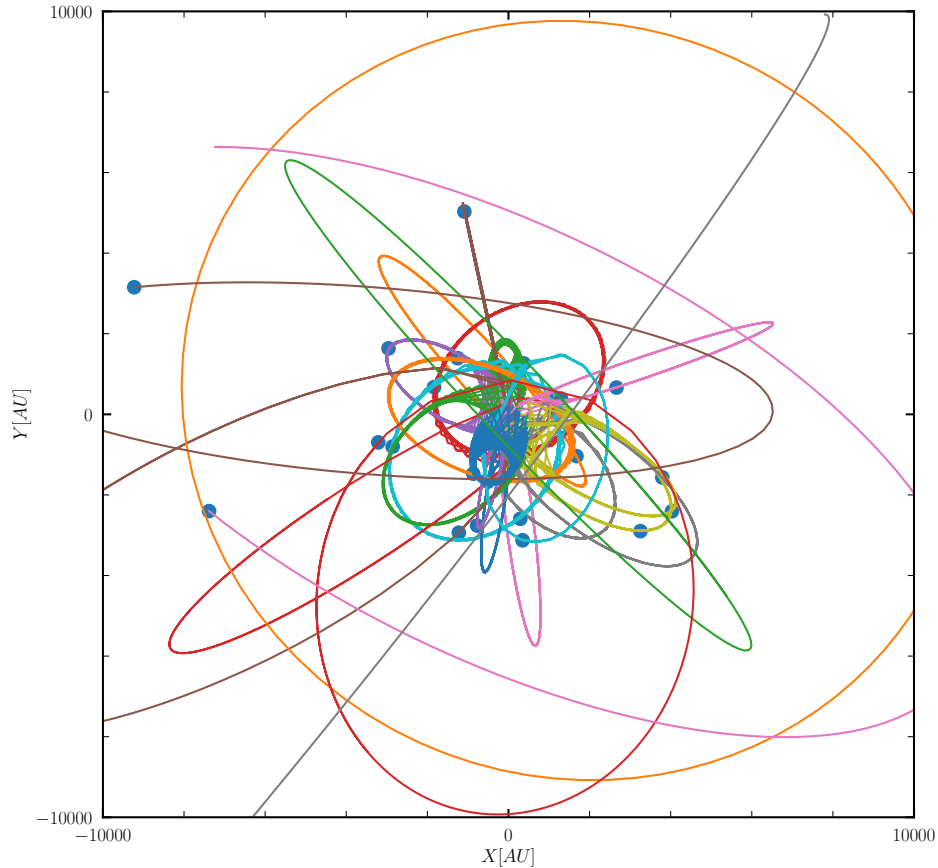


Figure 10.12: Orbital integration of the S stars over an interval of 500 yr, corresponding to the time frame of the simulation. The integration was performed using the symplectic N -body integrator **Mercury** (see Section 4.3). This figure was made using the script `amuse.examples.plot_sstar_orbits`.

The complicated boundary conditions, the winds and accretion processes, and the lack of intrinsic symmetries makes a grid-based code impractical for this application. With the SPH formalism, we can relatively easily tune the stellar winds, incorporate cooling, and monitor the accretion processes, we therefore use the SPH code **Fi** to resolve the hydrodynamics in the system. Shocks (see Section 6.1.2) are an issue in these calculations, and internal temperatures can rise high enough to emit X-rays. We deal with these rising temperatures by incorporating the cooling mechanism introduced in Section 6.3.4.

The combined solver for the hydrodynamics and the gravity is constructed using **bridge** (see Chapter 9). In this case, the gravity is affected by the hydrodynamics and

the hydrodynamics by the gravity, much as discussed in Code example 9.6:

```
gravity_hydro = bridge.Bridge()
gravity_hydro.add_system(gravity, (hydro,))
gravity_hydro.add_system(hydro, (gravity,))
gravity_hydro.timestep = 2.0 * hydro.parameters.timestep
```

The final line initializes the bridge time step to a conservative value.

10.4.2.2 Stellar winds

New SPH particles are generated at run time when stars lose sufficient mass in their stellar winds. Instead of using the simple wind description discussed in Section 6.4.3, we adopt the built-in AMUSE wind module, which is imported via

```
from amuse.ext.stellar_wind import new_stellar_wind
```

This wind module is a class, similar to but more sophisticated than the method presented in Code example 6.2 in Section 6.3.1. This built-in wind module can be started by initializing the mass of the SPH particles modeling the wind (`mass_gas`), the target particle set to which this gas should be added (here called `gas`), and the time step (`dt`). We also set an additional parameter called `derive_from_evolution`, which indicates that the winds of the stars should be derived from the running stellar evolution module.

```
wind = new_stellar_wind(
    mass_gas, target_gas=gas, timestep=time_step / 2., derive_from_evolution=True
)
```

After declaring the stellar wind class, we add the particles, in this case the particle set of $20 M_{\odot}$ stars at an age of 9 Myr (see `amuse.examples.initialize_sstars.py`). The channel from the stars to the wind particles ensures that the wind parameters calculated in the stellar evolution code are properly copied to the particles in the wind module.

```
wind.particles.add_particles(stars)
channel_stars_to_wind = stars.new_channel_to(wind.particles)
```

In the eventual script we update the wind module as for any other code:

```
wind.evolve_model(time)
```

10.4.2.3 The accretion process

During the integration of the equations of motion, we allow the black hole and the S stars to accrete from the ambient gas. As with the wind module, we use the internal sink particle solution in AMUSE (see Section 6.3.2) which can be imported by:

```
from amuse.ext.sink import new_sink_particles
```

after which we initiate the sink particles for the S stars and have them accrete:


```
sink_stars = new_sink_particles(stars)
sink_stars.accrete(gas)
```

The stellar parameters are provided by the stellar evolution code, and the stellar radius is used as the sink radius.

For the black hole, the sink is initialized with an accretion radius smaller than the pericenter distance of the innermost S star.

```
sink_BH = new_sink_particles(BH, sink_radius=accretion_radius)
sink_BH.accrete(gas)
```

Choosing the proper value for this accretion radius is non-trivial and affects the accretion rate onto the black hole. A small accretion radius is inefficient in terms of computer time, whereas a large radius causes too much mass to accrete onto the black hole. First-order estimates of the accretion radius are given by [Bondi & Hoyle \(1944\)](#) and [Hoyle & Lyttleton \(1939\)](#):

$$r_{\text{BHL}} = \frac{2GM_{\text{BH}}}{(v_{\text{wind}}^2 + c_s^2)} \quad (10.4)$$

Here c_s is the local sound speed, which depends on the temperature of the gas (see Equation (6.17)). Although the temperature near the black hole is very high (it is an X-ray emitting source), the sound speed is still small compared to the speeds of the S star winds. For a stellar wind velocity of ~ 1800 km/s we estimate the accretion radius to be ~ 2400 AU. For our calculation, we adopt 100 AU as the radius at which accretion actually occurred—considerably smaller than the Bondi-Hoyle-Lyttleton radius, but large enough to avoid a dramatic breakdown or inefficiency of the hydrodynamics solver.

10.4.2.4 Removing escaping gas

Some of the gas liberated by the stars will escape the computational domain. If a gas particle drifts too far from the Galactic center, it will find that the Galactic center is isolated instead of embedded in a larger environment. Thus, if we want to resolve the hydrodynamics on the scale of the S stars, we cannot also resolve the motion of the gas on kiloparsec scales. To err on the safe side, we strip gas as soon as it passes a distance of twice the apocenter distance of the farthest S star, if its speed exceeds the local escape speed from the black hole. We calculate these two parameters once for the system:

```
system_size = (2 * stars.a * (1 + stars.ecc)).amax()
system_escape_velocity = (
    2 * constants.G * (stars.mass.sum() + BH.mass) / system_size
).sqrt()
```

For our system, these values are ~ 0.7 parsec and ~ 760 km/s, respectively. Gas particles meeting these criteria are identified and removed from the simulation:

```

out_of_box = gas[gas.position.lengths() > system_size]
unbound = out_of_box[escaping.velocity.lengths() > system_escape_velocity]
escaping = unbound.select(
    lambda pos, vel: pos.dot(vel) > quantities.zero, ["position", "velocity"]
)
if escaping.is_empty():
    return
escaped_gas.add_particles(escaping)
gas.remove_particles(escaping)

```

Every time the amount of gas in the simulation changes (increases due to stellar winds, decreases by accretion or escape), the hydrodynamics solver is updated. We realize this by synchronizing the local gas particles with the particles in the hydrodynamics solver:

```
gas.synchronize_to(hydro.gas_particles)
```

10.4.2.5 The event loop

After declaring and initializing all the separate ingredients, we can write the event loop for this simulation. It is not complicated, but rather long due to the accumulation of the separate pieces. We have removed the diagnostics, validation, and data-writing operations for brevity and to clarify the basic functionality. But of course, for a production simulation these checks are of the utmost importance.

```

while time < time_end:
    time = time + time_step / 2
    stellar.evolve_model(pretime+time)
    time = time + time_step / 2
    gravity_hydro.evolve_model(time)
    stellar.evolve_model(pretime+time)

    channel_stars_to_wind.copy()
    wind.evolve_model(time)
    gas.synchronize_to(hydro.gas_particles)

    sink_BH.accrete(gas)
    gas.synchronize_to(hydro.gas_particles)
    sink_stars = new_sink_particles(stars)

    sink_stars.accrete(gas)
    gas.synchronize_to(hydro.gas_particles)
    remove_escaping_gas(gas, system_size, system_escape_velocity, escaped_gas)
    gas.synchronize_to(hydro.gas_particles)

```

The sinks keep track of the amount of angular momentum accreted by the black hole particle in order to estimate the size of the accretion disk.

```

m_accreted = BH_sink.particles.mass.sum()
diskrad = (
    sink_BH.angular_momentum.length()**2
    / (constants.G * m_accreted**2 * BH.mass[0])
)

```

Running this script for any length of time and at a reasonable resolution of at least $5 \times 10^{-8} M_{\odot}$ per gas particle requires a substantial amount of computer time, really requiring the use of a large computing cluster or supercomputer rather than a laptop or workstation. Figure 10.13 shows the simulation results as published in Lützgendorf *et al.* (2016).

10.4.3 Results of the simulation

The highest resolution run performed by Lützgendorf *et al.* (2016) was for a gas-particle mass of $4.8 \times 10^{-8} M_{\odot}$, with new SPH particles generated during each time step of 1 yr. It lasted for 500 yr, which is about 3 times the orbital period of the star S8. Figure 10.13 presents several snapshots of this simulation. As discussed previously, the initial snapshot (upper left) contains no gas; it takes a few hundred years for the system to reach a richer gas distribution. The simulation was stopped at 500 yr because, although the gas distribution continues to change, the accretion onto the central black hole stabilizes at $\sim 1.4 \times 10^{-7} M_{\odot}/\text{yr}$. This slightly exceeds the observed accretion rate of 10^{-8} – $10^{-7} M_{\odot}/\text{yr}$ (Baganoff *et al.*, 2003; Xu *et al.*, 2006).

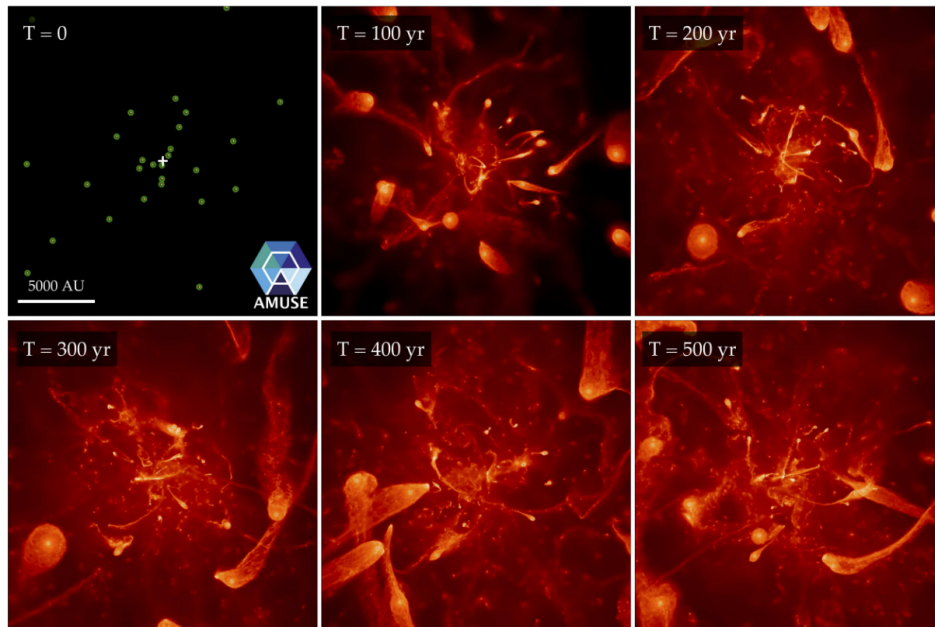


Figure 10.13: Evolution of the S stars in the Galactic center. This is Figure 3 from Lützgendorf *et al.* (2016); a spectacular animation of this simulation can be found at <https://youtu.be/soGFgzRso3c>. (Copyright AAS. Reproduced with permission.)

Figure 10.14 presents the total amount of mass accreted by the black hole during

several simulations. The various curves show the result of different calculations in which we varied the ages of the S stars. As discussed earlier, the ages are important for determining the S star winds and therefore accretion onto the black hole. If the S stars are young ($\lesssim 8$ Gyr) they are unable to account for the observed accretion onto the black hole, but if they are giants with ages $\gtrsim 9$ Myr, the observed accretion rate onto the black hole could be explained by the S star winds.

In every simulation, the contribution of star S2 provides at most $\sim 2\%$ of the total accretion rate onto the black hole. The idea that S2, being the nearest star, contributes most to the accretion onto the black hole is clearly not valid. During our 500 yr simulation, the star S14 contributed most mass to the black hole—almost an order of magnitude more than S2—even though S2 has the smallest pericenter distance. Of course, our assumption that all of the S stars have the same mass and age is artificial and somewhat contrived. When improved observations of the S stars provide better constraints on their masses and evolutionary states, we can easily improve our treatment of accretion onto the black hole.

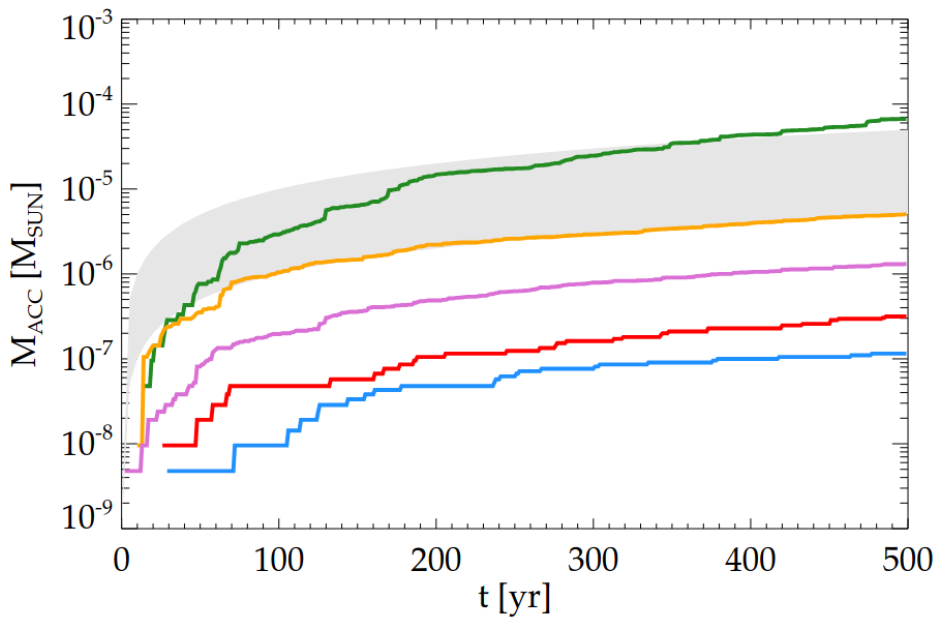


Figure 10.14: Total amount of mass accreted by the central supermassive black hole, as a function of time. The gray shaded area is an estimate of the total amount of mass accreted by the black hole, using observed mass accretion rates (Baganoff *et al.*, 2003; Xu *et al.*, 2006). The curves represent calculations for $20 M_{\odot}$ stars at ages of 5 Myr (red curve), 8 Myr, (purple curve) and 9 Myr (green curve). The other two curves are for $15 M_{\odot}$ stars at 5 Myr (blue) and $25 M_{\odot}$ at 7 Myr (yellow).

10.5 Visualisation

10.5.1 Fresco: visualising gas-rich dusty star clusters

Fresco is a visualization package specifically for making “observational” images of

simulations, focused on simulations in which stellar evolution, stellar dynamics, and possibly gas dynamics are incorporated. It takes the luminosities and temperatures from stars, and uses these to calculate their brightness in a series of colour bands (by default the Bessell UBVRI bands). Then, using the positions of the stars, it calculates a base image for each colour band, which is then convolved with a point-spread function (PSF). Finally, the colour bands are combined to create an RGB image, by default mimicking the colours as the human eye would see them.

An additional feature of **Fresco** is the ability to include extinction from interstellar dust, using gas particles as a proxy. In this case, it creates a 3D extinction map using the **FiMap** module (part of **Fi**), by default using a fixed gas-to-dust ratio, though this can be customised. After the brightness of each star is calculated in each colour band, the extinction is then applied before convolving with the PSF. Additionally, the dust can reflect a fraction of the nearby starlight, using a tweakable albedo parameter.

Fresco is not in the main **amuse** GitHub repository, but can be downloaded from <https://github.com/amusecode/amuse-fresco>. An example of an image made using **Fresco** is presented in Figure 10.15, which was generated with exactly the same data as were used to generate Figure 6.10.

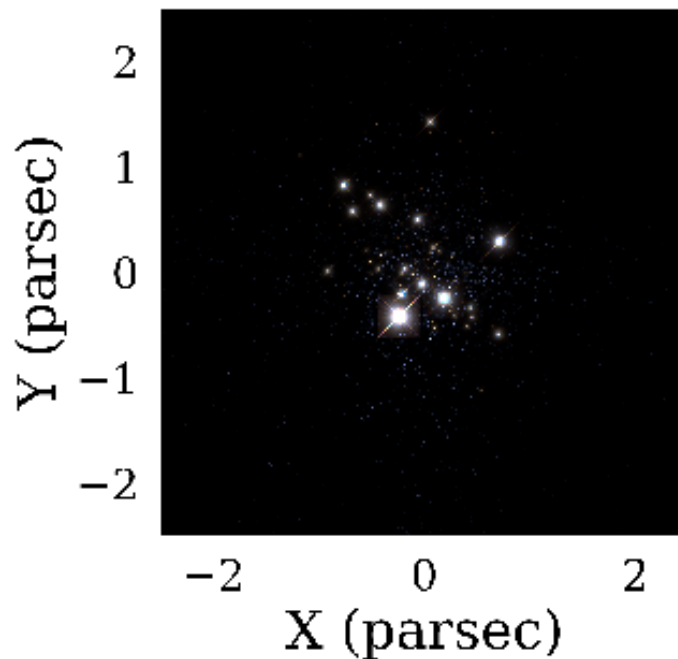


Figure 10.15: Projection of a collapsing molecular cloud at an age of 4.5 Myr. The stars are represented with the point-spread function of the Hubble Space Telescope, and the underlying gas distribution is given in contours. The data from which this figure is made is identical to those used to generate Figure 6.10.

10.6 Closing

You have reached the end of the final chapter of this book. Congratulations! We have discussed several examples on the use of AMUSE, and we gave a number of pointers to

other projects in which the framework is used. Several of these examples are published in peer-reviewed journals. They generally address somewhat more complex problems, but the fundamental working remains unchanged. What is left now, are the appendices and some philosophical concluding remarks in in Part [III](#).

Bibliography

- Appel, Sabrina M., Andersson, Eric P., Burkhart, Blakesley, Cournoyer-Cloutier, Claude, Lewis, Sean, Mac Low, Mordecai-Mark, McMillan, Stephen L. W., Polak, Brooke, Portegies Zwart, Simon, Tran, Aaron, & Wilhelm, Maite J. C. 2025 (Jan.). Exploring the Role of Protostellar Jets in Simulations of Star Cluster Formation using Torch. *Page 343.08 of: American Astronomical Society Meeting Abstracts*. American Astronomical Society Meeting Abstracts, vol. 245.
- Baganoff, F. K., Maeda, Y., Morris, M., Bautz, M. W., Brandt, W. N., Cui, W., Doty, J. P., Feigelson, E. D., Garmire, G. P., Pravdo, S. H., Ricker, G. R., & Townsley, L. K. 2003. Chandra X-Ray Spectroscopic Imaging of Sagittarius A* and the Central Parsec of the Galaxy. *ApJ* , **591**(2), 891–915.
- Bondi, H., & Hoyle, F. 1944. *MNRAS* , **104**, 273.
- Borgdorff, Joris, Lorenz, Eric, Hoekstra, Alfons G., Falcone, Jean-Luc, & Chopard, Bastien. 2011. A Principled Approach to Distributed Multiscale Computing, from Formalization to Execution. *Pages 97–104 of: IEEE 7th International Conference on E-Science, e-Science 2011, Workshop Proceedings, Stockholm, Sweden, December 5-8, 2011*. IEEE Computer Society.
- Budde, Gerrit, Burkhardt, Christoph, Brennecka, Gregory A., Fischer-Gödde, Mario, Kruijer, Thomas S., & Kleine, Thorsten. 2016. Molybdenum isotopic evidence for the origin of chondrules and a distinct genetic heritage of carbonaceous and non-carbonaceous meteorites. *Earth and Planetary Science Letters*, **454**, 293 – 303.
- Chambers, J. E., & Migliorini, F. 1997 (July). Mercury - A New Software Package for Orbital Integrations. *Page 27.06 of: AAS/Division for Planetary Sciences Meeting Abstracts #29*. AAS/Division for Planetary Sciences Meeting Abstracts, vol. 29.
- Chevalier, Roger A. 2000. Young Circumstellar Disks near Evolved Massive Stars and Supernovae. *ApJL* , **538**(2), L151–L154.
- Cuadra, Jorge, Nayakshin, Sergei, Springel, Volker, & Di Matteo, Tiziana. 2006. Galactic Centre stellar winds and Sgr A* accretion. *MNRAS* , **366**(2), 358–372.
- Dobbs, C. L., & Pringle, J. E. 2013. The exciting lives of giant molecular clouds. *MNRAS* , **432**(1), 653–667.
- Draine, B. T. 2011. On Radiation Pressure in Static, Dusty H II Regions. *ApJ* , **732**(2), 100.
- Fryxell, B., Olson, K., Ricker, P., Timmes, F. X., Zingale, M., Lamb, D. Q., MacNeice, P., Rosner, R., Truran, J. W., & Tufo, H. 2000. FLASH: An Adaptive Mesh Hydrodynamics Code for Modeling Astrophysical Thermonuclear Flashes. *ApJS* , **131**(1), 273–334.
- Genzel, Reinhard, Eisenhauer, Frank, & Gillessen, Stefan. 2010. The Galactic Center massive black hole and nuclear star cluster. *Reviews of Modern Physics*, **82**(4),

- 3121–3195.
- Ghez, A. M., Duchêne, G., Matthews, K., Hornstein, S. D., Tanner, A., Larkin, J., Morris, M., Becklin, E. E., Salim, S., Kremenek, T., Thompson, D., Soifer, B. T., Neugebauer, G., & McLean, I. 2003. The First Measurement of Spectral Lines in a Short-Period Star Bound to the Galaxy’s Central Black Hole: A Paradox of Youth. *ApJL* , **586**(2), L127–L131.
- Ghez, A. M., Salim, S., Hornstein, S. D., Tanner, A., Lu, J. R., Morris, M., Becklin, E. E., & Duchêne, G. 2005. Stellar Orbits around the Galactic Center Black Hole. *ApJ* , **620**(2), 744–757.
- Gillessen, S., Eisenhauer, F., Trippe, S., Alexander, T., Genzel, R., Martins, F., & Ott, T. 2009a. Monitoring Stellar Orbits Around the Massive Black Hole in the Galactic Center. *ApJ* , **692**(2), 1075–1109.
- Gillessen, S., Eisenhauer, F., Fritz, T. K., Bartko, H., Dodds-Eden, K., Pfuhl, O., Ott, T., & Genzel, R. 2009b. The Orbit of the Star S2 Around SGR A* from Very Large Telescope and Keck Data. *ApJL* , **707**(2), L114–L117.
- Hoyle, F., & Lyttleton, R. A. 1939. The effect of interstellar matter on climatic variation. *Proceedings of the Cambridge Philosophical Society*, **35**(3), 405.
- Kepler, J. 1609. *Astronomia nova*. Vol. 1.
- Kudritzki, R. P., Pauldrach, A., Puls, J., & Abbott, D. C. 1989. Radiation-driven winds of hot stars. VI. Analytical solutions for wind models including the finite cone angle effect. *A&A* , **219**(July), 205–218.
- Kudritzki, Rolf-Peter, & Puls, Joachim. 2000. Winds from Hot Stars. *ARA&A* , **38**(Jan.), 613–666.
- Lewis, Sean, McMillan, Stephen, Mac Low, Mordecai-Mark, Cournoyer-Cloutier, Claude, Polak, Brooke, Tran, Aaron, Wilhelm, Maite, Sills, Alison, Klessen, Ralf, & Wall, Joshua. 2023 (Jan.). Star Cluster Formation: The effects of early forming massive stars and building a bridge between Voronoi mesh and block-structured codes. *Page 109.03D of: American Astronomical Society Meeting Abstracts #241*. American Astronomical Society Meeting Abstracts, vol. 241.
- Liow, Kong You, Rieder, Steven, Dobbs, Clare L., & Jaffa, Sarah E. 2022. Grouped star formation: converting sink particles to stars in hydrodynamical simulations. *MNRAS* , **510**(2), 2657–2670.
- Looney, Leslie W., Tobin, John J., & Fields, Brian D. 2006. Radioactive Probes of the Supernova-contaminated Solar Nebula: Evidence that the Sun Was Born in a Cluster. *ApJ* , **652**(2), 1755–1762.
- Lützgendorf, N., Helm, E. van der, Pelupessy, F. I., & Portegies Zwart, S. 2016. Stellar winds near massive black holes - the case of the S-stars. *MNRAS* , **456**(4), 3645–3654.
- Luu, Tu-Han, Young, Edward D., Gounelle, Matthieu, & Chaussidon, Marc. 2015. Short time interval for condensation of high-temperature silicates in the solar accretion disk. *Proceedings of the National Academy of Sciences*, **112**(5), 1298–1303.
- Merritt, David, Alexander, Tal, Mikkola, Seppo, & Will, Clifford M. 2010. Testing properties of the Galactic center black hole using stellar orbits. *Phys. Rev. D.* , **81**(6), 062002.
- Nayakshin, S. 2005. Using close stars as probes of hot accretion flow in Sgr A*. *A&A*

- , **429**(Jan.), L33–L36.
- Ouellette, N., Desch, S. J., & Hester, J. J. 2007. Interaction of Supernova Ejecta with Nearby Protoplanetary Disks. *ApJ*, **662**(2), 1268–1281.
- Pearson, Samuel G., & McCaughrean, Mark J. 2023. Jupiter Mass Binary Objects in the Trapezium Cluster. *arXiv e-prints*, Oct., arXiv:2310.01231.
- Pelupessy, Federico I., Jänes, Jürgen, & Portegies Zwart, Simon. 2012. N-body integrators with individual time steps from Hierarchical splitting. *New Astron.*, **17**(8), 711–719.
- Portegies Zwart, S. F., & Verbunt, F. 1996. Population synthesis of high-mass binaries. *A&A*, **309**(May), 179–196.
- Portegies Zwart, Simon, Pelupessy, Inti, Martínez-Barbosa, Carmen, van Elteren, Arjen, & McMillan, Steve. 2020. Non-intrusive hierarchical coupling strategies for multi-scale simulations in gravitational dynamics. *Communications in Nonlinear Science and Numerical Simulations*, **85**(June), 105240.
- Price, Daniel J., Wurster, James, Tricco, Terrence S., Nixon, Chris, Toupin, Stéven, Pettitt, Alex, Chan, Conrad, Mentiplay, Daniel, Laibe, Guillaume, Glover, Simon, Dobbs, Clare, Nealon, Rebecca, Liptai, David, Worpel, Hauke, Bonnerot, Clément, Dipierro, Giovanni, Ballabio, Giulia, Ragusa, Enrico, Federrath, Christoph, Iaconi, Roberto, Reichardt, Thomas, Forgan, Duncan, Hutchison, Mark, Constantino, Thomas, Ayliffe, Ben, Hirsh, Kieran, & Lodato, Giuseppe. 2018. Phantom: A Smoothed Particle Hydrodynamics and Magnetohydrodynamics Code for Astrophysics. *Proc. Astron. Soc. Aust.*, **35**(Sept.), e031.
- Rein, H., & Liu, S. F. 2012. REBOUND: an open-source multi-purpose N-body code for collisional dynamics. *A&A*, **537**(Jan.), A128.
- Rieder, Steven. 2019 (July). *rieder/ekster: IAUS 351*.
- Rieder, Steven, & Liow, Kong You. 2021 (Sept.). *Ekster*.
- Rieder, Steven, Dobbs, Clare, Bending, Thomas, Liow, Kong You, & Wurster, James. 2022. The formation and early evolution of embedded star clusters in spiral galaxies. *MNRAS*, **509**(4), 6155–6168.
- Sanders, N. E., Soderberg, A. M., Gezari, S., Betancourt, M., Chornock, R., Berger, E., Foley, R. J., Challis, P., Drout, M., Kirshner, R. P., Lunnan, R., Marion, G. H., Margutti, R., McKinnon, R., Milisavljevic, D., Narayan, G., Rest, A., Kankare, E., Mattila, S., Smartt, S. J., Huber, M. E., Burgett, W. S., Draper, P. W., Hodapp, K. W., Kaiser, N., Kudritzki, R. P., Magnier, E. A., Metcalfe, N., Morgan, J. S., Price, P. A., Tonry, J. L., Wainscoat, R. J., & Waters, C. 2015. Toward Characterization of the Type IIP Supernova Progenitor Population: A Statistical Sample of Light Curves from Pan-STARRS1. *ApJ*, **799**(2), 208.
- Schiller, Martin, Paton, Chad, & Bizzarro, Martin. 2015. Evidence for nucleosynthetic enrichment of the protosolar molecular cloud core by multiple supernova events. *Geochimica et Cosmochimica Acta*, **149**(Jan.), 88–102.
- Schödel, R., Ott, T., Genzel, R., Hofmann, R., Lehnert, M., Eckart, A., Mouawad, N., Alexander, T., Reid, M. J., Lenzen, R., Hartung, M., Lacombe, F., Rouan, D., Gendron, E., Rousset, G., Lagrange, A. M., Brandner, W., Ageorges, N., Lidman, C., Moorwood, A. F. M., Spyromilio, J., Hubin, N., & Menten, K. M. 2002. A star in a 15.2-year orbit around the supermassive black hole at the centre of the Milky

- Way. *Nat* , **419**(6908), 694–696.
- Scott, Edward R. D. 2007. Chondrites and the Protoplanetary Disk. *Annual Review of Earth and Planetary Sciences*, **35**(1), 577–620.
- Sills, Alison, Rieder, Steven, Buckner, Anne S. M., Hacar, Alvaro, Portegies Zwart, Simon, & Teixeira, Paula S. 2023. Using molecular gas observations to guide initial conditions for star cluster simulations. *MNRAS* , **519**(3), 4142–4151.
- Spicer, D. S., Clark, R. W., & Maran, S. P. 1990. A Model of the Pre-Sedov Expansion Phase of Supernova Remnant–Ambient Plasma Coupling and X-Ray Emission from SN 1987A. *ApJ* , **356**(June), 549.
- Stahler, S. W., & Palla, F. 2005. *The Formation of Stars*.
- Teyssier, R. 2002. Cosmological hydrodynamics with adaptive mesh refinement. A new high resolution code called RAMSES. *A&A* , **385**(Apr.), 337–364.
- van Elteren, A., Portegies Zwart, S., Pelupessy, I., Cai, M. X., & McMillan, S. L. W. 2019. Survivability of planetary systems in young and dense star clusters. *A&A* , **624**(Apr.), A120.
- Wall, Joshua E., Mac Low, Mordecai-Mark, McMillan, Stephen L. W., Klessen, Ralf S., Portegies Zwart, Simon, & Pellegrino, Andrew. 2020. Modeling of the Effects of Stellar Feedback during Star Cluster Formation Using a Hybrid Gas and N-Body Method. *ApJ* , **904**(2), 192.
- Wang, Long, Iwasawa, Masaki, Nitadori, Keigo, & Makino, Junichiro. 2020. PETAR: a high-performance N-body code for modelling massive collisional stellar systems. *MNRAS* , **497**(1), 536–555.
- Wijnen, T. P. G., Pelupessy, F. I., Pols, O. R., & Portegies Zwart, S. 2017. Changes in orientation and shape of protoplanetary discs moving through an ambient medium. *A&A* , **604**(Aug.), A88.
- Wilhelm, Maite J. C., & Portegies Zwart, S. 2024. VENICE: A multi-scale operator-splitting algorithm for multi-physics simulations. *A&A* , **691**(Nov.), A71.
- Xu, Ya-Di, Narayan, Ramesh, Quataert, Eliot, Yuan, Feng, & Baganoff, Frederick K. 2006. Thermal X-Ray Iron Line Emission from the Galactic Center Black Hole Sagittarius A*. *ApJ* , **640**(1), 319–326.